Snakebite-Holoruminant-MetaG

A handbook of the pipeline, its tools and the workflow system behind it

Daniel Fischer

2026-11-04

Table of contents

Welcome 1
Who this book is for . . . 1
How the book is organised . . . 1
What you can trust in this book . . . 2
How this book was written . . . 2

I. Part I: Foundations 3

1. Linux on a cluster 4
1.1. What a cluster is . . . 4
1.1.1. Login nodes and compute nodes . . . 4
1.1.2. Shared disks, with different purposes . . . 5
1.1.3. You ask for resources, and you get exactly those . . . 5
1.2. Connecting . . . 6
1.2.1. Keeping things running after you disconnect . . . 6
1.3. Moving around . . . 6
1.3.1. Paths . . . 7
1.3.2. Symbolic links . . . 8
1.4. Looking at files . . . 8
1.5. Compressed files and archives . . . 10
1.6. Pipes and redirection . . . 10
1.6.1. Process substitution . . . 11
1.7. Exit codes: how programs report failure . . . 12
1.8. Variables and quoting . . . 13
1.9. Loops and arrays . . . 14
1.10. Scripts . . . 14
1.11. Permissions . . . 15
1.12. Software modules . . . 15
1.13. File formats you will meet . . . 15
1.14. What's next . . . 16

2. Asking for resources with SLURM 17
2.1. What the cluster looks like . . . 17
2.2. Your first job script . . . 17
2.3. Watching your jobs . . . 19

2.4. After the job: `sacct` tells you what happened . . . 20
2.4.1. What a time limit looks like . . . 21
2.4.2. What running out of memory looks like . . . 21
2.5. Many similar jobs: arrays . . . 21
2.6. Working interactively . . . 22
2.7. How to choose your resources . . . 23
2.8. Summary . . . 23

3. Containers **24**
3.1. What a container is . . . 24
3.2. Docker and Apptainer . . . 24
3.3. Running a tool from a container . . . 25
3.4. Bind mounts: what a container can see . . . 26
3.4.1. The `TMPDIR` trap . . . 26
3.5. Containers in Snakemake . . . 27
3.6. What about conda? . . . 28
3.7. Summary . . . 28

4. Why a workflow system? **29**
4.1. The script that grows . . . 29
4.2. Describe the result, not the steps . . . 30
4.3. A few words of vocabulary . . . 30
4.4. Which Snakemake, and how to get it . . . 31
4.5. Running the examples of this book . . . 31
4.6. What's next . . . 32

5. Your first rule **33**
5.1. The Snakefile . . . 33
5.2. How `{input}` and `{output}` work . . . 33
5.3. The dry run: always look first . . . 34
5.4. Running it . . . 35
5.5. Running it again . . . 36
5.6. When does a rule run again? . . . 36
5.7. Useful options to know now . . . 37
5.8. What Snakemake wrote to your folder . . . 37
5.9. Summary . . . 38

6. Wildcards: one rule for many files **39**
6.1. Replacing names with placeholders . . . 39
6.2. Snakemake can't guess what you want . . . 39
6.3. Two errors worth recognising . . . 41
6.4. How matching really works . . . 42
6.5. Using wildcard values in the command . . . 42
6.6. Naming conventions in this pipeline . . . 43

6.7. Summary 43

7. Chaining rules **44**
7.1. Four rules 44
7.1.1. A docstring 45
7.1.2. Commands over several lines 45
7.1.3. Named inputs 46
7.2. What Snakemake plans 46
7.3. Seeing the DAG 48
7.4. Checking on what happened 49
7.5. Why output files matter so much 49
7.6. Summary 50

8. Targets: `rule all` and `expand()` **51**
8.1. A rule that only asks for files 51
8.2. `expand()` writes the list for you 52
8.3. Target rules for parts of a workflow 53
8.4. Using another rule's inputs 54
8.5. Finding samples from file names 55
8.6. Summary 55

9. Just enough Python **56**
9.1. Python in one page 56
9.1.1. Lists and loops 57
9.1.2. f-strings 57
9.1.3. List comprehensions 58
9.1.4. Dictionaries 58
9.1.5. Functions and lambdas 58
9.1.6. Paths 58
9.2. Reading a sample sheet with pandas 59
9.3. Input functions: an input that depends on the wildcards 60
9.3.1. Functions that return a function 62
9.4. Python that runs at parse time 63
9.5. Summary 63

10. Configuration **64**
10.1. A config file 64
10.2. Why `params:` and not just `{config[...]}`? 65
10.3. How this pipeline organises its configuration 66
10.3.1. The default is a copy, and copies can grow old 68
10.4. Summary 68

11. Logs, benchmarks, threads and resources **69**
11.1. A complete rule 69

11.2. `log:` is where messages go . . . 70
11.3. `benchmark:` measures what a job used . . . 70
11.4. `threads:` says how many cores a job can use . . . 71
11.5. `resources:` says what else it needs . . . 72
11.5.1. Resources that depend on the attempt . . . 72
11.6. The anatomy of a rule in this pipeline . . . 73
11.7. Summary . . . 74

12. Special kinds of output files **75**
12.1. The example . . . 75
12.2. `temp()`: delete when no longer needed . . . 77
12.3. `directory()`: the output is a folder . . . 78
12.4. `touch()`: a marker file for a step with no file output . . . 78
12.5. `protected()`: guard against accidents . . . 79
12.6. `ancient()`: don't rerun because of the date . . . 79
12.7. `pipe()`: a stream between two jobs . . . 79
12.8. `multiext()`: one prefix with several endings . . . 79
12.9. Summary . . . 79

13. When things go wrong **81**
13.1. A way of reading errors . . . 81
13.2. Errors in the Snakefile . . . 81
13.2.1. A typo in a keyword . . . 81
13.2.2. Single braces in a command . . . 82
13.3. Errors in the plan . . . 82
13.3.1. `MissingInputException` . . . 82
13.3.2. `MissingRuleException` . . . 83
13.3.3. `AmbiguousRuleException` . . . 83
13.4. Errors while a job runs . . . 83
13.4.1. The command failed . . . 83
13.4.2. The command worked, but the output is missing . . . 85
13.5. After a crash: incomplete files and locks . . . 86
13.6. When nothing goes wrong, but something did: strict mode . . . 88
13.7. What strict mode doesn't see: `>(...)` . . . 89
13.8. Why did this job run again? . . . 90
13.9. A checklist for a failed run . . . 91
13.10Summary . . . 92

14. Structuring a large workflow **93**
14.1. The layout . . . 93
14.2. `include:` puts files together . . . 94
14.3. A module is a folder with a `__main__.smk` . . . 95
14.4. Rule names say where a rule lives . . . 95
14.5. Paths in one place . . . 96

14.6. Where to run from: the project folder . . . 97
14.7. Summary . . . 98

15. Running on the cluster **99**
15.1. How it works . . . 99
15.2. Profiles: settings for the cluster . . . 99
15.3. The rules stay the same . . . 100
15.4. Where to look: three kinds of logs . . . 102
15.5. When a job fails: retries and attempts . . . 103
15.6. A trap: `retries:` in a rule behaves differently . . . 104
15.7. A few more options for the cluster . . . 106
15.8. Where Snakemake itself should run . . . 107
15.9. Summary . . . 107

16. Checkpoints: when the plan depends on results **108**
16.1. The example . . . 108
16.2. What it looks like . . . 109
16.3. Costs and alternatives . . . 111
16.4. Summary . . . 111

17. How this pipeline uses Snakemake **112**
17.1. The run script . . . 112
17.2. What happens when Snakemake reads the workflow . . . 114
17.3. Resource sets and escalation . . . 114
17.3.1. Retries are set in one place . . . 116
17.3.2. Escalation in action . . . 116
17.4. Debug messages . . . 118
17.5. Conventions at a glance . . . 119
17.6. How to read a rule of this pipeline . . . 119
17.7. Summary . . . 120

18. Testing a workflow **121**
18.1. Level 1: the dry run . . . 121
18.2. Level 2: the linter . . . 121
18.3. Level 3: unit tests for rules . . . 122
18.4. Level 4: running with stand-in tools . . . 123
18.5. Level 5: real runs with real data . . . 124
18.6. What you should run when . . . 124
18.7. Summary . . . 124

II. Part II: The tools **125**

19. The tools at a glance **126**
19.1. How to read these chapters . . . 126

19.2. Please cite the tools . . . 126
19.3. Some terms you will meet . . . 127
19.4. The file formats . . . 127
19.5. The containers . . . 128
19.6. Versions of the tools in the containers . . . 129

20. FastQC **131**
20.1. What it does . . . 131
20.2. Input . . . 131
20.3. Output . . . 131
20.4. When to use it . . . 132
20.5. Reading it for a metagenome . . . 132
20.6. In this pipeline . . . 133
20.7. Pitfalls . . . 133
20.8. Reference . . . 133

21. fastp **134**
21.1. What it does . . . 134
21.2. How it works . . . 134
21.3. Input . . . 135
21.4. Output . . . 135
21.5. When to use it . . . 135
21.6. In this pipeline . . . 136
21.7. Pitfalls . . . 136
21.8. Reference . . . 136

22. Bowtie2 **137**
22.1. What it does . . . 137
22.2. How it works . . . 137
22.3. Input . . . 137
22.4. Output . . . 138
22.5. When to use it . . . 138
22.6. In this pipeline . . . 138
22.7. Pitfalls . . . 139
22.8. Reference . . . 139

23. SAMtools and friends **140**
23.1. What it does . . . 140
23.2. The formats . . . 140
23.2.1. SAM . . . 140
23.2.2. The flag . . . 140
23.2.3. BAM and CRAM . . . 141
23.3. The subcommands used here . . . 141
23.4. Reading the summaries . . . 142

23.5. The compressors . . . 142
23.5.1. bgzip . . . 142
23.5.2. pigz . . . 142
23.5.3. GNU parallel . . . 143
23.6. Pitfalls . . . 143
23.7. Reference . . . 143

24. Kraken2 **144**
24.1. What it does . . . 144
24.2. How it works . . . 144
24.3. Input . . . 144
24.4. Output . . . 145
24.5. When to use it . . . 145
24.6. In this pipeline . . . 146
24.7. Pitfalls . . . 146
24.8. Reference . . . 146

25. Krona **147**
25.1. What it does . . . 147
25.2. Input . . . 147
25.3. Output . . . 147
25.4. When to use it . . . 148
25.5. In this pipeline . . . 148
25.6. Pitfalls . . . 148
25.7. Reference . . . 149

26. SingleM **150**
26.1. What it does . . . 150
26.2. How it works . . . 150
26.3. Input . . . 150
26.4. Output . . . 151
26.5. When to use it . . . 151
26.6. In this pipeline . . . 151
26.7. Pitfalls . . . 152
26.8. Reference . . . 152

27. sylph **153**
27.1. What it does . . . 153
27.2. How it works . . . 153
27.3. Input . . . 153
27.4. Output . . . 154
27.5. When to use it . . . 154
27.6. In this pipeline . . . 154
27.7. Pitfalls . . . 155

27.8. Reference . . . 155

28. MetaPhlAn **156**
28.1. What it does . . . 156
28.2. How it works . . . 156
28.3. Input . . . 156
28.4. Output . . . 156
28.5. When to use it . . . 157
28.6. In this pipeline . . . 157
28.7. Pitfalls . . . 158
28.8. Reference . . . 158

29. HUMAnN **159**
29.1. What it does . . . 159
29.2. How it works . . . 159
29.3. Input . . . 159
29.4. Output . . . 160
29.5. When to use it . . . 160
29.6. In this pipeline . . . 160
29.7. Pitfalls . . . 161
29.8. Reference . . . 161

30. phyloFlash **162**
30.1. What it does . . . 162
30.2. How it works . . . 162
30.3. Input . . . 162
30.4. Output . . . 162
30.5. When to use it . . . 163
30.6. In this pipeline . . . 163
30.7. Pitfalls . . . 164
30.8. Reference . . . 164

31. DIAMOND **165**
31.1. What it does . . . 165
31.2. Why compare at the protein level . . . 165
31.3. How it works . . . 165
31.4. Input . . . 165
31.5. Output . . . 166
31.6. When to use it . . . 166
31.7. In this pipeline . . . 166
31.8. Pitfalls . . . 167
31.9. Reference . . . 167

32. NCycDB **168**
32.1. What it does . . . 168
32.2. How it works . . . 168
32.3. Input . . . 168
32.4. Output . . . 169
32.5. When to use it . . . 169
32.6. In this pipeline . . . 169
32.7. Pitfalls . . . 169
32.8. Reference . . . 170

33. Nonpareil **171**
33.1. What it does . . . 171
33.2. How it works . . . 171
33.3. Input . . . 172
33.4. Output . . . 172
33.5. When to use it . . . 172
33.6. In this pipeline . . . 172
33.7. Pitfalls . . . 173
33.8. References . . . 173

34. Assembly: the idea **174**
34.1. The de Bruijn graph . . . 174
34.2. The choice of k . . . 174
34.3. Why metagenomes are hard . . . 175
34.4. Which to choose . . . 175

35. MEGAHIT **176**
35.1. What it does . . . 176
35.2. How it works . . . 176
35.3. Input . . . 176
35.4. Output . . . 176
35.5. When to use it . . . 177
35.6. In this pipeline . . . 177
35.7. Pitfalls . . . 177
35.8. Reference . . . 177

36. metaSPAdes **178**
36.1. What it does . . . 178
36.2. How it works . . . 178
36.3. Input . . . 178
36.4. Output . . . 178
36.5. When to use it . . . 179
36.6. In this pipeline . . . 179
36.7. Pitfalls . . . 179

36.8. Reference . . . 180

37. Binning: the idea **181**
37.1. What can go wrong . . . 181

38. CONCOCT **182**
38.1. What it does . . . 182
38.2. How it works . . . 182
38.3. Input . . . 182
38.4. Output . . . 182
38.5. When to use it . . . 183
38.6. In this pipeline . . . 183
38.7. Pitfalls . . . 183
38.8. Reference . . . 183

39. MaxBin2 **184**
39.1. What it does . . . 184
39.2. How it works . . . 184
39.3. Input . . . 184
39.4. Output . . . 184
39.5. When to use it . . . 185
39.6. In this pipeline . . . 185
39.7. Pitfalls . . . 185
39.8. Reference . . . 186

40. MetaBAT2 **187**
40.1. What it does . . . 187
40.2. How it works . . . 187
40.3. Input . . . 187
40.4. Output . . . 187
40.5. When to use it . . . 188
40.6. In this pipeline . . . 188
40.7. Pitfalls . . . 188
40.8. Reference . . . 188

41. Prodigal **189**
41.1. What it does . . . 189
41.2. How it works . . . 189
41.3. Input . . . 189
41.4. Output . . . 189
41.5. When to use it . . . 190
41.6. In this pipeline . . . 190
41.7. Pitfalls . . . 191
41.8. Reference . . . 191

42. HMMER **192**
42.1. What it does . . . 192
42.2. How it works . . . 192
42.3. Input . . . 192
42.4. Output . . . 192
42.5. When to use it . . . 193
42.6. In this pipeline . . . 193
42.7. Pitfalls . . . 193
42.8. Reference . . . 194

43. MAGScoT **195**
43.1. What it does . . . 195
43.2. How it works . . . 195
43.3. Input . . . 195
43.4. Output . . . 196
43.5. When to use it . . . 196
43.6. In this pipeline . . . 196
43.7. Pitfalls . . . 197
43.8. Reference . . . 197

44. dRep **198**
44.1. What it does . . . 198
44.2. How it works . . . 198
44.3. Input . . . 198
44.4. Output . . . 199
44.5. When to use it . . . 199
44.6. In this pipeline . . . 199
44.7. Pitfalls . . . 200
44.8. Reference . . . 200

45. CoverM **201**
45.1. What it does . . . 201
45.2. How it works . . . 201
45.3. Input . . . 201
45.4. Output . . . 201
45.5. When to use it . . . 202
45.6. In this pipeline . . . 202
45.7. Pitfalls . . . 203
45.8. Reference . . . 203

46. featureCounts **204**
46.1. What it does . . . 204
46.2. How it works . . . 204
46.3. Input . . . 204

46.4. Output . . . 205
46.5. When to use it . . . 205
46.6. In this pipeline . . . 205
46.7. Pitfalls . . . 206
46.8. Reference . . . 206

47. QUAST **207**
47.1. What it does . . . 207
47.2. Understanding the numbers . . . 207
47.3. Input . . . 207
47.4. Output . . . 208
47.5. When to use it . . . 208
47.6. In this pipeline . . . 208
47.7. Reference . . . 208

48. CheckM2 **209**
48.1. What it does . . . 209
48.2. How it works . . . 209
48.3. Input . . . 209
48.4. Output . . . 209
48.5. What's a good genome? . . . 210
48.6. When to use it . . . 210
48.7. In this pipeline . . . 210
48.8. Pitfalls . . . 211
48.9. References . . . 211

49. GTDB-Tk **212**
49.1. What it does . . . 212
49.2. What's GTDB? . . . 212
49.3. How it works . . . 212
49.4. Input . . . 212
49.5. Output . . . 213
49.6. When to use it . . . 213
49.7. In this pipeline . . . 213
49.8. Pitfalls . . . 214
49.9. References . . . 214

50. PhyloPhlAn **215**
50.1. What it does . . . 215
50.2. How it works . . . 215
50.3. Input . . . 215
50.4. Output . . . 215
50.5. When to use it . . . 216
50.6. In this pipeline . . . 216

50.7. Reference . . . 216

51. DRAM **217**
51.1. What it does . . . 217
51.2. How it works . . . 217
51.3. Input . . . 217
51.4. Output . . . 218
51.5. When to use it . . . 218
51.6. In this pipeline . . . 218
51.7. Pitfalls . . . 219
51.8. Reference . . . 219

52. CAMPER **220**
52.1. What it does . . . 220
52.2. How it works . . . 220
52.3. Input . . . 220
52.4. Output . . . 220
52.5. When to use it . . . 221
52.6. In this pipeline . . . 221
52.7. Pitfalls . . . 221
52.8. Reference . . . 221

53. Bakta **222**
53.1. What it does . . . 222
53.2. How it works . . . 222
53.3. Input . . . 222
53.4. Output . . . 222
53.5. When to use it . . . 223
53.6. In this pipeline . . . 223
53.7. Pitfalls . . . 223
53.8. Reference . . . 223

54. eggNOG-mapper **224**
54.1. What it does . . . 224
54.2. How it works . . . 224
54.3. Input . . . 224
54.4. Output . . . 225
54.5. When to use it . . . 225
54.6. In this pipeline . . . 225
54.6.1. eggNOG version 7 . . . 226
54.7. Pitfalls . . . 226
54.8. Reference . . . 226

55. Proteinortho **227**
55.1. What it does . . . 227
55.2. How it works . . . 227
55.3. Input . . . 227
55.4. Output . . . 227
55.5. When to use it . . . 228
55.6. In this pipeline . . . 228
55.7. Reference . . . 228

56. Tiara **229**
56.1. What it does . . . 229
56.2. How it works . . . 229
56.3. Input . . . 229
56.4. Output . . . 229
56.5. When to use it . . . 230
56.6. In this pipeline . . . 230
56.7. Pitfalls . . . 230
56.8. Reference . . . 230

57. 4CAC **231**
57.1. What it does . . . 231
57.2. How it works . . . 231
57.3. Input . . . 231
57.4. Output . . . 231
57.5. When to use it . . . 232
57.6. In this pipeline . . . 232
57.7. Pitfalls . . . 232
57.8. Reference . . . 232

58. REMAG **233**
58.1. What it does . . . 233
58.2. How it works . . . 233
58.3. Input . . . 233
58.4. Output . . . 233
58.5. When to use it . . . 234
58.6. In this pipeline . . . 234
58.7. Pitfalls . . . 234
58.8. Reference . . . 235

III. Part III: The modules **236**

59. Setting up a project **237**
59.1. Two folders . . . 237

59.2. Getting the databases . . . 238
59.3. The run script . . . 238
59.4. `config/config.yaml` . . . 239
59.4.1. Paths to the other files . . . 239
59.4.2. Fast storage . . . 240
59.4.3. The assembler . . . 241
59.4.4. Limits and resource sets . . . 241
59.5. `config/escalation.yaml` . . . 241
59.6. `config/features.yaml` . . . 242
59.6.1. `hosts` . . . 242
59.6.2. `magscot` . . . 242
59.6.3. `databases` . . . 243
59.7. `config/params.yaml` . . . 243
59.8. `config/.docker.yml` . . . 243
59.9. `config/samples.tsv` . . . 243
59.10 `config/profiles/` . . . 244
59.11 Checking the setup . . . 245
59.12 Summary . . . 245

60. Modules `reads` and `reference` 246
60.1. What they are for . . . 246
60.2. The module `reads` . . . 246
60.2.1. `reads__link__run`: one consistent name . . . 246
60.2.2. `reads__fastqc`: first look at the quality . . . 247
60.2.3. What you get . . . 247
60.3. The module `reference` . . . 248
60.3.1. `reference__hosts__recompress` . . . 248
60.4. Summary . . . 248

61. Module `preprocess` 249
61.1. The flow . . . 249
61.2. Trimming: `preprocess__fastp__run` . . . 250
61.2.1. What you get . . . 251
61.3. Removing host reads . . . 251
61.3.1. `preprocess__bowtie2__build` . . . 251
61.3.2. `preprocess__bowtie2__map` . . . 252
61.3.3. `preprocess__bowtie2__extract_nonhost__run` . . . 253
61.3.4. `preprocess__store_final_fastq` . . . 253
61.4. Checking the result . . . 254
61.4.1. `preprocess__samtools__stats_cram` . . . 254
61.4.2. `preprocess__fastqc__fastp` and `preprocess__fastqc__nonhost` 254
61.5. Things to look at after a run . . . 254
61.6. Summary . . . 254

62. Module `read_annotate` 255
62.1. The tools at a glance . . . 255
62.2. Fast storage for big databases . . . 256
62.3. Kraken2 . . . 257
62.3.1. `read_annotate__kraken2__assign` . . . 257
62.3.2. `read_annotate__krona__visualize` . . . 258
62.4. SingleM . . . 258
62.4.1. `read_annotate__singlem__pipe` . . . 258
62.4.2. `read_annotate__singlem__condense` . . . 259
62.4.3. `read_annotate__singlem__microbial_fraction` . . . 259
62.4.4. `read_annotate__singlem__aggregate_microbial_fraction` . . . 259
62.5. sylph . . . 259
62.5.1. `read_annotate__sylph_profile` . . . 259
62.6. MetaPhlAn . . . 260
62.6.1. `read_annotate__metaphlan__run` . . . 260
62.6.2. `read_annotate__metaphlan__condense` . . . 260
62.7. HUMAnN . . . 260
62.7.1. `read_annotate__humann__run` . . . 260
62.7.2. `read_annotate__humann__condense` . . . 261
62.8. phyloFlash . . . 261
62.8.1. `read_annotate__phyloflash__run` . . . 261
62.8.2. `read_annotate__phyloflash__condense` . . . 261
62.9. DIAMOND . . . 262
62.9.1. `read_annotate__diamond__assign` . . . 262
62.9.2. `read_annotate__diamond__summarise` . . . 262
62.10Nonpareil . . . 262
62.10.1`read_annotate__nonpareil__run` . . . 262
62.10.2`read_annotate__nonpareil__aggregate` . . . 263
62.11nCycDB . . . 263
62.11.1`read_annotate__ncyc__prepare_input` . . . 263
62.11.2`read_annotate__ncyc__run` . . . 264
62.12The options in one place . . . 264
62.13What to look at after a run . . . 264
62.14Summary . . . 264

63. Module `assemble` 265
63.1. The idea . . . 265
63.2. The flow . . . 265
63.3. Which reads go into which assembly . . . 266
63.4. Assembling . . . 266
63.4.1. `assemble__metaspades__run` . . . 267
63.4.2. `assemble__megahit__run` . . . 268
63.4.3. The contig names . . . 268
63.4.4. Your own assemblies . . . 269

63.5. Mapping the reads back . . . 269
63.5.1. assemble__bowtie2__build__run . . . 269
63.5.2. assemble__bowtie2__map . . . 269
63.6. Binning . . . 270
63.6.1. assemble__concoct__run . . . 270
63.6.2. assemble__maxbin2__run . . . 271
63.6.3. assemble__metabat2__run . . . 271
63.7. Refining: MAGScoT . . . 271
63.7.1. assemble__magscot__prodigal . . . 272
63.7.2. assemble__magscot__hmmsearch . . . 272
63.7.3. assemble__magscot__marker_table . . . 272
63.7.4. assemble__magscot__merge_contig_to_bin . . . 272
63.7.5. assemble__magscot__run . . . 273
63.7.6. assemble__magscot__label_contigs . . . 273
63.8. Dereplicating: dRep . . . 274
63.8.1. assemble__drep__split_bins . . . 274
63.8.2. assemble__drep__run . . . 274
63.8.3. assemble__drep__catalogue . . . 275
63.9. What you get . . . 275
63.10What to look at after a run . . . 275
63.11Summary . . . 275

64. Module quantify **277**
64.1. Why map to a catalogue . . . 277
64.2. The flow . . . 277
64.3. Mapping . . . 278
64.3.1. quantify__bowtie2__build . . . 278
64.3.2. quantify__bowtie2__map . . . 278
64.4. Counting . . . 278
64.4.1. quantify__coverm__genome__run . . . 279
64.4.2. quantify__coverm__genome_aggregate . . . 279
64.4.3. quantify__coverm__contig_one . . . 279
64.4.4. quantify__coverm__contig_aggregate and quantify__coverm__contig . . . 279
64.5. The options . . . 280
64.6. What you get . . . 280
64.7. Summary . . . 281

65. Module mag_annotate **282**
65.1. The tools at a glance . . . 282
65.2. Quality . . . 283
65.2.1. mag_annotate__quast . . . 283
65.2.2. mag_annotate__checkm2__predict . . . 283

65.3. Taxonomy . . . 284
65.3.1. mag_annotate__gtdbtk__classify . . . 284
65.3.2. mag_annotate__phylophlan . . . 284
65.4. Function . . . 285
65.4.1. DRAM on the catalogue . . . 285
65.4.2. DRAM per assembly . . . 285
65.4.3. mag_annotate__camper__annotate . . . 286
65.4.4. Bakta . . . 286
65.4.5. mag_annotate__eggnog . . . 287
65.4.6. mag_annotate__proteinortho . . . 287
65.5. What to look at after a run . . . 287
65.6. Summary . . . 288

66. Module contig_annotate **289**
66.1. The flow . . . 289
66.2. Finding the genes . . . 290
66.2.1. contig_annotate__prodigal__run . . . 290
66.2.2. contig_annotate__cut_prodigal . . . 290
66.3. eggNOG-mapper . . . 290
66.3.1. contig_annotate__eggnog_find_homology . . . 291
66.3.2. contig_annotate__eggnog_orthology_chunk . . . 291
66.3.3. contig_annotate__eggnog_merge_annotations . . . 291
66.3.4. contig_annotate__eggnog7__run . . . 292
66.4. Counting reads per gene . . . 292
66.4.1. contig_annotate__cram_to_bam . . . 292
66.4.2. contig_annotate__featurecounts__run . . . 292
66.5. Other annotations . . . 292
66.5.1. contig_annotate__camper__annotate . . . 292
66.5.2. contig_annotate__diamond__assign . . . 293
66.5.3. contig_annotate__hmmer__assign . . . 293
66.5.4. nCycDB . . . 294
66.6. What you get . . . 294
66.7. Summary . . . 294

67. Module eukaryote **295**
67.1. Why three tools . . . 295
67.2. Tiara . . . 296
67.2.1. eukaryote__tiara__run . . . 296
67.3. 4CAC . . . 296
67.3.1. eukaryote__4cac__extract_graph . . . 296
67.3.2. eukaryote__4cac__run . . . 297
67.4. REMAG . . . 297
67.4.1. eukaryote__remag__filter . . . 297
67.4.2. eukaryote__remag__bin . . . 298

67.5. Reading the results together . . . 299
67.6. Summary . . . 299

68. Modules `report` and `helpers` 300
68.1. The module `report` . . . 300
68.2. The module `helpers` . . . 301
68.3. The rule `test_error` . . . 302

69. Common problems 303
69.1. How to find out what happened . . . 303
69.2. Quick table . . . 303
69.3. Memory . . . 305
69.4. Time . . . 305
69.5. Containers can't see a file . . . 305
69.6. Disk space . . . 306
69.7. Starting again after a problem . . . 306
69.8. Problems of single rules . . . 307
69.8.1. Kraken2 or DIAMOND: "database too large" . . . 307
69.8.2. eggNOG-mapper: waiting forever . . . 307
69.8.3. MAGScoT or dRep: no bins . . . 307
69.8.4. DRAM: a confusing error in `distill` . . . 307
69.8.5. `mag_annotate__proteinortho` . . . 307
69.8.6. `read_annotate__diamond__summarise` . . . 308
69.8.7. Pandas errors at the start . . . 308
69.9. Failures that didn't exist before . . . 308
69.10Asking for help . . . 308
69.11Summary . . . 308

IV. Part IV: Tutorial 309

70. Tutorial with real data 310
70.1. The data . . . 310
70.2. Running it yourself . . . 311
70.2.1. On a cluster . . . 311
70.2.2. With only Docker . . . 311
70.2.3. The samples . . . 312
70.3. Stage `reads` . . . 312
70.4. Stage `reference` . . . 313
70.5. Stage `preprocess` . . . 313
70.5.1. What fastp did . . . 313
70.5.2. What host removal did . . . 315
70.5.3. The clean reads . . . 316

70.6. Stage `read_annotate` . . . 317
70.6.1. Nonpareil . . . 317
70.6.2. nCycDB . . . 318
70.7. Stage `assemble` . . . 318
70.7.1. Assembly . . . 318
70.7.2. Mapping back . . . 319
70.7.3. Binning . . . 320
70.7.4. MAGScoT . . . 321
70.7.5. dRep, and two things that went wrong . . . 323
70.8. Stage `quantify` . . . 324
70.9. Stage `contig_annotate` . . . 326
70.9.1. Genes . . . 326
70.9.2. Counting reads per gene . . . 326
70.9.3. Hydrogenases . . . 327
70.9.4. CAMPER . . . 328
70.9.5. nCycDB on contigs . . . 328
70.10Stage `mag_annotate` . . . 328
70.11Stage `eukaryote` . . . 329
70.12What this run found . . . 329
70.13What's still waiting for the complete run . . . 330

71. The tests in the repository **332**
71.1. The escalation test . . . 332
71.2. The DIAMOND test . . . 332
71.3. The Krona test . . . 332
71.4. The real results . . . 333
71.5. The complete run . . . 333

Appendices **334**

A. Rule reference **334**
A.1. Module `reads` . . . 334
A.1.1. `reads/__main__.smk` . . . 334
A.1.2. `reads/fastqc.smk` . . . 334
A.1.3. `reads/link.smk` . . . 335
A.2. Module `reference` . . . 335
A.2.1. `reference/__main__.smk` . . . 335
A.2.2. `reference/hosts.smk` . . . 335
A.3. Module `preprocess` . . . 336
A.3.1. `preprocess/__main__.smk` . . . 336
A.3.2. `preprocess/bowtie2.smk` . . . 336
A.3.3. `preprocess/fastp.smk` . . . 338
A.3.4. `preprocess/fastqc.smk` . . . 339

A.3.5. preprocess/samtools.smk . . . 339
A.4. Module read_annotate . . . 340
A.4.1. read_annotate/__main__.smk . . . 340
A.4.2. read_annotate/diamond.smk . . . 341
A.4.3. read_annotate/humann.smk . . . 342
A.4.4. read_annotate/kraken2.smk . . . 343
A.4.5. read_annotate/krona.smk . . . 344
A.4.6. read_annotate/metaphlan.smk . . . 345
A.4.7. read_annotate/ncyc.smk . . . 345
A.4.8. read_annotate/nonpareil.smk . . . 346
A.4.9. read_annotate/phyloflash.smk . . . 347
A.4.10. read_annotate/singlem.smk . . . 348
A.4.11. read_annotate/sylph.smk . . . 350
A.5. Module assemble . . . 350
A.5.1. assemble/__main__.smk . . . 350
A.5.2. assemble/binners/concoct.smk . . . 350
A.5.3. assemble/binners/maxbin2.smk . . . 351
A.5.4. assemble/binners/metabat2.smk . . . 351
A.5.5. assemble/bowtie2.smk . . . 352
A.5.6. assemble/drep.smk . . . 353
A.5.7. assemble/magscot.smk . . . 354
A.5.8. assemble/megahit.smk . . . 356
A.5.9. assemble/metaspades.smk . . . 357
A.6. Module quantify . . . 357
A.6.1. quantify/__main__.smk . . . 357
A.6.2. quantify/bowtie2.smk . . . 358
A.6.3. quantify/coverm.smk . . . 359
A.6.4. quantify/samtools.smk . . . 360
A.7. Module mag_annotate . . . 361
A.7.1. mag_annotate/__main__.smk . . . 361
A.7.2. mag_annotate/bakta.smk . . . 361
A.7.3. mag_annotate/camper.smk . . . 362
A.7.4. mag_annotate/checkm2.smk . . . 363
A.7.5. mag_annotate/dram.smk . . . 363
A.7.6. mag_annotate/dram_mag.smk . . . 365
A.7.7. mag_annotate/eggnog.smk . . . 366
A.7.8. mag_annotate/gtdbtk.smk . . . 366
A.7.9. mag_annotate/phylophlan.smk . . . 367
A.7.10. mag_annotate/proteinortho.smk . . . 367
A.7.11. mag_annotate/quast.smk . . . 368
A.8. Module contig_annotate . . . 368
A.8.1. contig_annotate/__main__.smk . . . 368
A.8.2. contig_annotate/camper.smk . . . 369
A.8.3. contig_annotate/diamond.smk . . . 369

A.8.4. `contig_annotate/eggnog.smk` . . . 370
A.8.5. `contig_annotate/eggnog7.smk` . . . 372
A.8.6. `contig_annotate/featurecounts.smk` . . . 372
A.8.7. `contig_annotate/hmmer.smk` . . . 373
A.8.8. `contig_annotate/ncyc.smk` . . . 374
A.8.9. `contig_annotate/prodigal.smk` . . . 374
A.9. Module `eukaryote` . . . 375
A.9.1. `eukaryote/4cac.smk` . . . 375
A.9.2. `eukaryote/__main__.smk` . . . 376
A.9.3. `eukaryote/remag.smk` . . . 377
A.9.4. `eukaryote/tiara.smk` . . . 378
A.10. Module `report` . . . 379
A.10.1. `report/__main__.smk` . . . 379
A.10.2. `report/assemble.smk` . . . 381
A.10.3. `report/contig_annotate.smk` . . . 381
A.10.4. `report/mag_annotate.smk` . . . 381
A.10.5. `report/preprocess.smk` . . . 382
A.10.6. `report/quantify.smk` . . . 382
A.10.7. `report/read_annotate.smk` . . . 383
A.10.8. `report/reads.smk` . . . 383
A.10.9. `report/reference.smk` . . . 384
A.11. Module `helpers` . . . 384
A.11.1. `helpers/fastqc.smk` . . . 384
A.11.2. `helpers/samtools.smk` . . . 384
A.12. Module `devel` . . . 386
A.12.1. `devel/throw_error.smk` . . . 386

Welcome

This book explains Snakebite-Holoruminant-MetaG from the ground up: the Linux and cluster knowledge you need, the workflow system Snakemake, every tool the pipeline runs, every module and rule, and a tutorial with real data. You don't need any programming background to start, and you can stop reading at the point where you know enough for your work.

Who this book is for

- **Newcomers to metagenomics or to clusters** should read from the beginning. Part I assumes that you have never written a workflow, and explains the Linux commands it uses.
- **Users of the pipeline** who already know Snakemake can start at Part II for the tools, or Part III for what a module does.
- **Developers of the pipeline** will mostly use Part III and the rule reference in the appendix.

How the book is organised

Part	Content
Part I: Foundations	Linux on a cluster, SLURM, containers and a thorough introduction to Snakemake
Part II: The tools	Every tool in the pipeline: what it does, its input and output, when to use it and why we do
Part III: The modules	Every module, rule by rule, and how the rules work together
Part IV: Tutorial	A run with real data, stage by stage, with the results and how to read them
Appendices	The generated reference of all rules

What you can trust in this book

Everything printed as terminal output comes from a real run, and each chapter says where it was run. The examples of Part I are in `book/examples/`, so you can run them yourself and compare. When something could not be run, for example a real SLURM submission from a laptop, the text says so.

This book changes together with the pipeline. Appendix A is generated from the rule files, and the check `book/tools/check_sync.py` fails when a rule, tool or configuration file is not covered in the text. If you find a place where the book and the pipeline disagree, trust the pipeline and please open an issue.

How this book was written

This book was drafted with AI assistance (Claude Code) and checked against the pipeline's code and real runs. Where the pipeline comes from is described in "Provenance and acknowledgements" in the repository's `README.md`.

Part I.

Part I: Foundations

1. Linux on a cluster

This pipeline runs on a high-performance computing cluster (HPC), and clusters run Linux. Before we can talk about workflows, we need the small part of Linux that you will use every day. This chapter assumes nothing. If you already work on a cluster each day, skim the section on pipes and exit codes, because Snakemake depends on those details, and then move on.

Every command in this chapter was run on a small test cluster, and the output you see is real. You can repeat them yourself with the files in `book/examples/`, as Section 4.5 explains.

1.1. What a cluster is

A laptop has one computer in it. A cluster has hundreds or thousands of them, called **nodes**, connected by a fast network and sharing the same disks. You don't sit in front of any of them. You connect over the network, and you ask a piece of software called the **scheduler** to run your work on nodes that are free.

Three ideas explain most of how a cluster behaves.

1.1.1. Login nodes and compute nodes

When you connect, you arrive on a **login node**. Everybody on the system shares it, and it is meant for small tasks: editing files, moving data, starting workflows. Real work happens on **compute nodes**, which you can't normally log in to directly. You ask for them through the scheduler, and Chapter 2 explains how.

Running a heavy program on a login node slows the system down for everyone, and administrators may stop it. A rough rule is that anything needing more than a few minutes or more than a gigabyte of memory doesn't belong there.

Snakemake itself is the exception that surprises people. It does little work: it mostly waits and submits jobs, so it is normal to leave it running on a login node while it sends the real work to compute nodes. This pipeline works that way.

1.1.2. Shared disks, with different purposes

All nodes see the same files, because the disks are shared over the network. A typical cluster has several of these areas, and using the right one matters. The names differ between systems, and here are the ones used at CSC's Puhti system, where this pipeline was developed:

Area	Example path	Meant for	Typical limits
Home	`/users/fischerd`	Settings, small scripts	Small and backed up
Project application folder	`/projappl/project_2009831`	Installed software, containers	Moderate
Scratch	`/scratch/project_2009831`	Data and results of running projects	Large, old files may be removed
Local disk on a compute node	`$LOCAL_SCRATCH`	Temporary files during a job	Fast, gone after the job
Memory disk	`/dev/shm`	Very fast temporary files, such as databases	Counts as memory, gone after the job

For this pipeline, the most common mistake is to leave results or caches in the home folder, which fills up quickly. Your project folder belongs on scratch. You will meet the local disk and the memory disk again in Chapter 62, because copying a large database there can speed up a tool a lot.

A shared network disk also has a behaviour worth remembering: a file written on one node can take a few seconds to appear on another. That is why the pipeline's run script sets `--latency-wait 60`, which tells Snakemake to wait up to a minute for a file before calling it missing.

1.1.3. You ask for resources, and you get exactly those

On your laptop, a program can use whatever is free. On a cluster, you say up front how many CPU cores, how much memory and how much time a job needs. The scheduler finds a place for it and then usually holds it to that request: a job that exceeds its memory or time gets killed. Asking for too much isn't free either, because large requests wait longer in the queue and use up your project's allocation.

Much of the pipeline's design deals with this fact. Chapter 17 explains how it starts each rule on a small resource set and moves up to a bigger one when a job fails.

1.2. Connecting

You connect with `ssh`, which gives you a terminal on the login node:

```
ssh yourname@puhti.csc.fi
```

Everything you type then happens on the cluster. Most clusters today require an SSH key instead of a password, and your computing centre's documentation describes how to set one up. This command wasn't run for this book, because the examples run on a small test cluster.

1.2.1. Keeping things running after you disconnect

If your connection drops, programs started in that terminal normally stop too. Since a workflow can run for days, you need a way around this. The usual tool is `tmux`, which keeps a terminal alive on the login node:

```
tmux new -s metag        # start a session named "metag"
# ... start your workflow, then press Ctrl-b and then d to detach
 ↪  ...
tmux attach -t metag     # come back later, even from another
 ↪  computer
```

Some systems have several login nodes, so remember which one you were on (the command `hostname` tells you). Your tmux session exists only on that node.

1.3. Moving around

The shell is the program that reads your commands. It always has a **current folder**, and `pwd` prints it:

```
$ pwd
/scratch/project_demo/examples/00_linux
```

`ls` lists what is in a folder:

```
$ ls data
sampleA_R1.fastq.gz
sampleA_R2.fastq.gz
sampleB_R1.fastq.gz
sampleB_R2.fastq.gz
sampleC_R1.fastq.gz
sampleC_R2.fastq.gz
```

With `-l` you get details, and `-h` makes sizes readable:

```
$ ls -lh data | head -n 4
total 24K
-rw-r--r-- 1 root root 1.3K Nov  3 17:12 sampleA_R1.fastq.gz
-rw-r--r-- 1 root root 1.2K Nov  3 17:12 sampleA_R2.fastq.gz
-rw-r--r-- 1 root root 1.9K Nov  3 17:12 sampleB_R1.fastq.gz
```

The columns are the permissions, the number of links, the owner, the group, the size, the date of the last change and the name. Snakemake uses that date to decide what is out of date, so it will matter later.

A few commands cover almost all of your file handling:

Command	What it does	Example
`cd`	Change the current folder	`cd /scratch/project_2009831`
`mkdir -p`	Create a folder, including parents	`mkdir -p project/reads`
`cp`	Copy a file	`cp a.txt b.txt`
`mv`	Move or rename	`mv a.txt b.txt`
`rm`	Delete a file, with no recycle bin	`rm a.txt`
`rm -r`	Delete a folder and everything in it	`rm -r old_results`
`ln -s`	Create a symbolic link	see below
`du -sh`	Size of a folder	`du -sh results`
`df -h`	Free space on a disk	`df -h /scratch`

There is no undo for `rm`. Check twice, especially with `rm -r`.

1.3.1. Paths

A path that starts with `/` is **absolute**, and means the same from everywhere. Any other path is **relative** to the current folder. In `project/reads/a.txt`, the shell looks for

`project` inside the current folder. Snakemake rules almost always use relative paths, and Snakemake runs them from the project folder. That lets you move a whole project to another place and keep it working.

Two short forms are worth knowing: `..` is the folder above the current one, and `~` is your home folder.

1.3.2. Symbolic links

A **symbolic link** is a small file that points to another file. Programs reading the link get the content of the file it points to. This is how you use a large file in several places without copying it:

```
$ ln -s "$PWD/data/sampleB_R1.fastq.gz"
 ↪  project/reads/sampleB.fastq.gz
$ ls -l project/reads | cut -c1-10,42-
total 8
-rw-r--r--sampleA_R1.fastq.gz
lrwxrwxrwxsampleB.fastq.gz ->
 ↪  /scratch/project_demo/examples/00_linux/data/sampleB_R1.fastq.gz
```

The `l` at the start and the arrow show that it is a link. The first rule of this pipeline does exactly this: `reads__link__run` links your raw FASTQ files into `results/reads/` under uniform names, without copying them (Chapter 60). Use an absolute path as the target, because a relative target is resolved from the place of the link, which rarely is what you meant.

1.4. Looking at files

The data files in this field are too large to open in an editor, so you look at parts of them.

A **FASTQ** file stores sequencing reads, with four lines per read: a name starting with `@`, the sequence, a `+`, and a quality string with one character per base. These files are almost always compressed with gzip, and end in `.gz`. `zcat` decompresses to the screen, and `head` keeps the first lines:

```
$ zcat data/sampleA_R1.fastq.gz | head -n 4
@sampleA.0/1
AGCCCAATAAACCACTCTGACTGGCCGAATAGGGATATAGGCAA
+
```

```
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
```

The `|` is a **pipe**, and the next section is about it. To count the reads, count the lines and divide by four:

```
$ zcat data/sampleA_R1.fastq.gz | wc -l
160
```

So this file has 40 reads. These small tools can answer many questions when you combine them:

```
$ zcat data/sampleA_R1.fastq.gz | awk 'NR % 4 == 2' | head -n 2 |
 ↪  cut -c1-40
AGCCCAATAAACCACTCTGACTGGCCGAATAGGGATATAG
GACATGTGCGGCGACCCTTGCGACAGTGACGCTTTCGCCG

$ zcat data/sampleA_R1.fastq.gz | awk 'NR % 4 == 2 { print
 ↪  length($0) }' | sort -n | tail -n 1
100

$ zcat data/sampleA_R1.fastq.gz | grep -c '^@sampleA'
40
```

The first command prints the sequences only: `awk` runs a small program on each line, `NR` is the line number, and `NR % 4 == 2` is true on every second line of four. The second prints the longest read. The third counts lines starting with the read name. The `^` means "at the start of the line".

Here are the tools you will see most in this book's rules:

Tool	What it does
`less file`	Page through a file. Press `q` to leave, `/` to search
`head`, `tail`	First or last lines
`wc -l`	Count lines
`cut -f 1,5`	Keep some columns of a table
`sort`, `uniq`	Sort lines, and remove repeated ones
`grep`	Keep lines matching a pattern
`sed`	Replace text
`awk`	A small language for working on columns
`paste`	Put files or lines side by side
`zcat`, `gzip -dc`	Decompress to the screen

> ⚠ `grep -c` on FASTQ is a trap
>
> The example above works because our read names are predictable. In real FASTQ data, the quality line can also start with `@`, so counting lines that start with `@` may overcount. Always count reads as lines divided by four.

1.5. Compressed files and archives

Bioinformatics tools produce a lot of data, so files are nearly always compressed. You will see four names:

- **gzip** is the standard one, with files ending in `.gz`. `gzip file` replaces the file with a compressed one.
- **pigz** does the same as gzip but uses several CPU cores, and its files are normal gzip files.
- **bgzip** writes gzip files in small blocks, so that tools can jump into the middle. Genome FASTA files must be compressed this way before `samtools faidx` accepts them, and this is why the module `reference` recompresses the host genomes (Section 60.3).
- **tar** packs a whole folder into one file, and is usually combined with compression:

```
$ tar -czf archive.tar.gz archive
$ tar -tzf archive.tar.gz
archive/
archive/inner/
archive/inner/file.txt
```

In `-czf`, `c` means create, `z` compress with gzip and `f` names the archive. In `-tzf`, `t` lists the content, and `-xzf` unpacks. A cluster's shared disks handle millions of small files badly, so the pipeline packs the working folders of the assemblers and of GTDB-Tk into archives when they are done.

1.6. Pipes and redirection

Every program has three standard channels:

- **stdin** (channel 0) is where it reads its input.
- **stdout** (channel 1) is where it writes its results.
- **stderr** (channel 2) is where it writes messages and errors.

Normally stdout and stderr go to your screen. You can redirect them:

Syntax	Meaning
`cmd > file`	Write stdout to a file, replacing it
`cmd >> file`	Append stdout to a file
`cmd 2> file`	Write stderr to a file
`cmd > file 2>&1`	Write both to the same file
`cmd1 | cmd2`	Send stdout of `cmd1` to stdin of `cmd2`

```
$ zcat data/sampleA_R1.fastq.gz | wc -l > lines.txt
$ cat lines.txt
160

$ ls data/does_not_exist.gz > listing.txt 2> errors.txt; echo "exit
 ↪  code of ls: $?"
exit code of ls: 2
$ cat errors.txt
ls: cannot access 'data/does_not_exist.gz': No such file or
 ↪  directory
```

The separation is why tools can write results to stdout and still show progress messages: a pipe only carries stdout. That is also the reason for a very common line in this pipeline's rules:

```
exec > {log} 2>&1
```

`exec` with only redirections changes the channels of the running script itself. Everything the rule's commands print from then on goes to the log file, without repeating `> {log} 2>&1` on every line.

The order in `> file 2>&1` matters: first stdout moves to the file, then stderr goes to where stdout now points. If you write `2>&1 > file`, stderr still goes to the screen.

1.6.1. Process substitution

Some tools want a file name and can't read from a pipe. `<(command)` makes the output of a command look like a file:

```
$ paste <(zcat data/sampleA_R1.fastq.gz | head -n 1) <(zcat
 ↪  data/sampleB_R1.fastq.gz | head -n 1)
@sampleA.0/1    @sampleB.0/1
```

The pipeline uses this to hand compressed files to tools that only read plain FASTA. For example, the CONCOCT rule runs `cut_up_fasta.py <(zcat {input.contigs})` (Chapter 38). Not every tool can read from such a pseudo-file, because some need to jump around in the file. Nonpareil is one of these, so its rule writes a real temporary file.

1.7. Exit codes: how programs report failure

When a program ends, it leaves a number called the **exit code**. By convention, `0` means success, and anything else means failure. The shell keeps the last one in `$?`:

```
$ true; echo "true gives $?"; false; echo "false gives $?"
true gives 0
false gives 1
```

Exit codes are how Snakemake knows that a rule failed. There are two traps, and Snakemake deals with both.

The first trap is that a script normally continues after a failing command, and reports only the exit code of its last command. The setting `set -e` makes the script stop at the first failure.

The second trap is that a pipe reports only the exit code of its last command:

```
$ false | true; echo "without pipefail: $?"
without pipefail: 0

$ set -o pipefail; false | true; echo "with pipefail: $?"
with pipefail: 1
```

So `zcat broken.gz | wc -l > count.txt` looks like a success without `pipefail`, even if the file was damaged. You would get a wrong number and no error at all.

Snakemake runs every shell command in **strict mode**, which is `set -euo pipefail`: stop at any failure (`-e`), treat unset variables as errors (`-u`), and let a pipe fail if any part of it fails (`-o pipefail`). This protects you, as long as nothing switches it off, and Section 13.6 shows how easily that can happen. Strict mode also has one surprising effect. A command like `zcat file | head -n 1` can stop `zcat` with an error when `head` has what it needs and closes the pipe early. This depends on timing and on the amount of data, and shows up as the exit code 141. Keep it in mind if a rule with `head` fails only on large files.

1.8. Variables and quoting

A shell variable holds a text. You set it without spaces around the `=`, and read it with `$`:

```
$ SAMPLE=sampleA
$ echo "The sample is $SAMPLE and its file is
 ↪  data/${SAMPLE}_R1.fastq.gz"
The sample is sampleA and its file is data/sampleA_R1.fastq.gz
```

The braces in `${SAMPLE}` mark where the name ends. Without them, the shell would look for a variable called `SAMPLE_R1`.

Quotes control what the shell does with your text. Inside double quotes, variables are replaced and spaces stay part of the text. Inside single quotes, nothing is replaced. As a habit, write `"$file"` with double quotes, so that a file name with a space doesn't turn into two arguments.

A variable normally exists only in your current shell. `export` passes it on to programs you start:

```
$ export MY_SETTING=42
$ bash -c 'echo "A new program sees MY_SETTING=$MY_SETTING and
 ↪  YOUR_SETTING=$YOUR_SETTING"'
A new program sees MY_SETTING=42 and YOUR_SETTING=
```

This matters on a cluster, since jobs usually inherit the environment of the shell that submitted them. It is how the setting `PIPELINE_DEBUG=1` reaches the rules, and also the cause of a class of odd errors in this pipeline: the scheduler sets `$TMPDIR` to a folder on the node, and a program inside a container may not be able to see this folder. Several rules deal with this, and Chapter 3 explains why.

Some variables you will meet often:

Variable	Meaning
`$HOME`	Your home folder
`$PATH`	The folders searched for programs
`$TMPDIR`	Where programs should put temporary files
`$SLURM_JOB_ID`	The job's number, inside a SLURM job
`$LOCAL_SCRATCH`	The local disk of the node, on CSC systems

1.9. Loops and arrays

A `for` loop repeats commands for each item. Here it runs over a pattern that the shell expands to matching files:

```
$ for file in data/*_R1.fastq.gz; do
      echo "$(basename "$file" _R1.fastq.gz): $(zcat "$file" | wc
 ↪  -l) lines"
  done
sampleA: 160 lines
sampleB: 240 lines
sampleC: 100 lines
```

`$(command)` puts the output of a command into the text, and `basename` removes the folder and the given ending.

An **array** holds several values. You will see them in the binning rules, which collect the names of temporary files:

```
$ files=(data/sampleA_R1.fastq.gz data/sampleB_R1.fastq.gz)
$ echo "${#files[@]} files, the first is ${files[0]}"
2 files, the first is data/sampleA_R1.fastq.gz
```

`"${files[@]}"` expands to all values, each as its own argument, and `${#files[@]}` gives their number.

Inside a Snakemake rule, you will see these with doubled braces, such as `"${{files[@]}}"`. Chapter 5 explains why.

1.10. Scripts

A script is a text file with commands. You run it with `bash script.sh`. The first line, starting with `#!`, says which program runs the file, and a good script starts like this:

```
#!/usr/bin/env bash
set -euo pipefail

echo "Working in $(pwd)"
```

1.11. Permissions

The first column of `ls -l` shows who may do what, for example `-rw-r--r--`: `r` is read, `w` is write and `x` is execute, in three groups for the owner, the group and everyone else. On a cluster, the group is usually your project, so that colleagues can read your results.

Command	What it does
`chmod +x script.sh`	Make a script executable
`chmod g+rw file`	Let the group read and write
`chgrp -R project_2009831 folder`	Change the group of a folder and its content

1.12. Software modules

A cluster has many versions of many programs, so they aren't all available at once. A system called **environment modules** makes them available on request:

```
module avail snakemake         # which versions exist?
module load snakemake/9.11.6   # make this one available
module list                    # what is loaded now?
```

The run script of the pipeline loads Snakemake this way. These commands weren't run for this book, because the test cluster has no module system. All other tools of the pipeline come in containers instead, which Chapter 3 covers.

1.13. File formats you will meet

This table is only a first map. Part II explains each format where its tool uses it.

Format	Ending	Content
FASTQ	`.fastq.gz`, `.fq.gz`	Reads with qualities
FASTA	`.fa`, `.fasta`, `.fa.gz`	Sequences without qualities: contigs, genomes, proteins
SAM, BAM, CRAM	`.sam`, `.bam`, `.cram`	Reads aligned to a reference. BAM is the binary form, and CRAM is smaller, but needs the reference
GFF, GTF	`.gff`, `.gtf`	Positions of genes on sequences

Format	Ending	Content
TSV	`.tsv`, `.txt`	Tables with tabs between columns
YAML	`.yaml`, `.yml`	Configuration files, as used in `config/`

1.14. What's next

You can now find your way around a cluster's files and read a pipe. The next chapter is about the part that makes a cluster a cluster: asking the scheduler for resources.

2. Asking for resources with SLURM

On a cluster, you don't start heavy programs yourself. You describe a **job**, and the scheduler starts it on a suitable node when one is free. The scheduler used on most academic clusters, including CSC's, is called **SLURM**. Snakemake will talk to SLURM for you later, but you can only understand what it does, and find out why something went wrong, if you know what SLURM does. So we start by using it by hand.

All output in this chapter is from the mini cluster described in Section 4.5, which has real SLURM with one node. The example is in `book/examples/snakemake/11_slurm/`.

2.1. What the cluster looks like

`sinfo` shows the partitions and the state of their nodes:

```
$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
small*       up 3-00:00:00      1   idle localhost
large        up 3-00:00:00      1   idle localhost
```

A **partition** is a group of nodes with its own rules, such as a maximum run time. The `*` marks the default, and `idle` means that the node has nothing to do. On a real cluster, this list is much longer, with states such as `mix` (partly used), `alloc` (fully used) and `drain` (being taken out of service).

Each cluster has its own partitions. On Puhti, there are for example `small`, `large`, `longrun`, `hugemem` and `gpu`. The values in this pipeline's `config.yaml` refer to those names, so if you run on another cluster, those are what you need to change (Chapter 59).

2.2. Your first job script

A **job script** is a shell script with instructions for SLURM at the top:

```
#!/bin/bash
#SBATCH --job-name=first_job
#SBATCH --account=project_demo
#SBATCH --partition=small
#SBATCH --time=00:05:00
#SBATCH --cpus-per-task=2
#SBATCH --mem=500M
#SBATCH --output=first_job_%j.out

echo "Running on $(hostname)"
echo "Job $SLURM_JOB_ID has $SLURM_CPUS_PER_TASK CPUs"
zcat data/sampleA_R1.fastq.gz | wc -l
```

To the shell, lines starting with `#` are comments. SLURM reads the `#SBATCH` lines at the top as options:

Option	Meaning
`--job-name`	A name, so you can recognise the job
`--account`	The project that pays for the computing time
`--partition`	Which group of nodes
`--time`	The maximum run time, as `hours:minutes:seconds` or `days-hours:minutes:seconds`
`--cpus-per-task`	How many CPU cores
`--mem`	How much memory in total, like `500M` or `64G`
`--output`	Where stdout and stderr go. `%j` becomes the job number
`--gres`	Other resources. On Puhti: `--gres=nvme:100` asks for 100 GB of fast local disk

You submit it with `sbatch`, which answers immediately with the job's number. Your terminal is free again while the job waits and runs:

```
$ sbatch first_job.sh
Submitted batch job 1
```

The result ends up in the output file:

```
$ cat first_job_*.out
Running on localhost
Job 1 has 2 CPUs
160
```

The job starts in the folder where you ran `sbatch`, and inherits your environment variables. Since all nodes share the disks, it sees the same files that you do.

2.3. Watching your jobs

`squeue` lists jobs that are waiting or running. The output below comes from a job submitted just before:

```
$ squeue --format="%.8i %.10P %.10j %.2t %.8M %.6D %R"
   JOBID  PARTITION       NAME ST     TIME  NODES NODELIST(REASON)
       5      small    waiting PD     0:00      1 (None)
```

Add `-u $USER` on a real cluster to see only your own jobs, or `--me` for short. The column `ST` has the state: `PD` is pending, and `R` is running. The last column shows the node when it runs, and otherwise a reason for waiting. Here are the reasons that you will see most:

Reason	Meaning
`Priority`	Other jobs are ahead of yours
`Resources`	Your job is next, but no node has room yet
`QOSMaxJobsPerUserLimit`	You reached a limit on running jobs
`AssocGrpBillingMinutes`	Your project ran out of computing time

`scontrol show job` gives every detail of a job:

```
$ scontrol show job $JOB | grep -E
 ↪  'JobName|JobState|Partition=|NumCPUs|TimeLimit' | cut -c1-70
JobId=5 JobName=waiting
   JobState=PENDING Reason=None Dependency=(null)
   RunTime=00:00:00 TimeLimit=00:05:00 TimeMin=N/A
   Partition=small AllocNode:Sid=localhost:298
   NumNodes=1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
```

`scancel` stops a job. Use `scancel -u $USER` with care, as it stops all of your jobs:

```
$ scancel $JOB
$ sacct -j $JOB -X --format=JobID,JobName,State,ExitCode
JobID           JobName      State ExitCode
------------ ---------- ---------- --------
5               waiting CANCELLED+      0:0
```

2.4. After the job: `sacct` tells you what happened

`squeue` forgets a job soon after it ended, and `sacct` keeps the records. It is the most useful command for understanding failures:

```
$ sacct -X
 ↪  --format=JobID,JobName,Partition,State,ExitCode,Elapsed,AllocCPUS,ReqMem
 ↪  | tail -n 1
1              first_job      small  COMPLETED      0:0   00:00:00
 ↪  2       500M
```

`-X` leaves out the internal steps of a job. By default `sacct` only shows jobs from today, and `-S 2026-11-01` asks for an earlier start date. On a real cluster, also ask for the memory that the job really used:

```
sacct -j 123456 --format=
 ↪  JobID,JobName,State,ExitCode,Elapsed,Timelimit,ReqMem,MaxRSS
```

The state column tells you what to do next:

State	Meaning	What to do
`COMPLETED`	Finished with exit code 0	Nothing
`FAILED`	The script ended with an error	Read the output file
`OUT_OF_MEMORY`	It exceeded its memory	Ask for more `--mem`
`TIMEOUT`	It exceeded its time limit	Ask for more `--time`, or check whether it hung
`CANCELLED`	Someone stopped it	Check who, and whether it was you
`NODE_FAIL`	The node died	Submit it again
`PENDING`, `RUNNING`	Not done yet	Wait

2.4.1. What a time limit looks like

Let's provoke one. This job asks for one second and wants to run for five minutes:

```
#!/bin/bash
#SBATCH --job-name=too_long
#SBATCH --account=project_demo
#SBATCH --time=00:00:01
#SBATCH --output=too_long_%j.out

echo "This job asked for too little time"
sleep 300
```

```
$ sbatch too_little_time.sh
Submitted batch job 6

$ sacct -X --name=too_long
 ↪  --format=JobID,JobName,State,ExitCode,Elapsed,Timelimit
JobID           JobName      State ExitCode    Elapsed  Timelimit
------------ ---------- ---------- -------- ---------- ----------
6              too_long    TIMEOUT      0:0   00:01:04   00:01:00
```

SLURM rounded the limit up to one minute, and killed the job shortly after. See the exit code: `0:0`. **The exit code alone doesn't tell you that a job failed.** The state does.

2.4.2. What running out of memory looks like

This mini cluster doesn't limit memory, because it runs without Linux's cgroups, so we can't show it here. On Puhti and most production clusters, a job that exceeds its `--mem` gets a program killed by the system. You then usually find `OUT_OF_MEMORY` in `sacct`, and a line in the job's output with the words `oom-kill` or `Killed`.

Now an important detail for later. In this case, the system normally kills the biggest process in the job. That's usually your tool, and the rest of the job script keeps running. This detail will matter in Chapter 15.

2.5. Many similar jobs: arrays

For the same work on many samples, an **array job** avoids writing many scripts. SLURM runs one copy per index, and gives it the variable `$SLURM_ARRAY_TASK_ID`:

```
#!/bin/bash
#SBATCH --job-name=array_job
#SBATCH --account=project_demo
#SBATCH --time=00:05:00
#SBATCH --mem=100M
#SBATCH --array=0-2
#SBATCH --output=array_%A_%a.out

samples=(sampleA sampleB sampleC)
sample=${samples[$SLURM_ARRAY_TASK_ID]}
echo "Task $SLURM_ARRAY_TASK_ID works on $sample: $(zcat
 ↪  data/${sample}_R1.fastq.gz | wc -l) lines"
```

```
$ sbatch array_job.sh
Submitted batch job 2

$ cat array_*.out
Task 0 works on sampleA: 160 lines
Task 1 works on sampleB: 240 lines
Task 2 works on sampleC: 100 lines

$ sacct -X --format=JobID%14,JobName,State,ExitCode | tail -n 3
           2_0  array_job  COMPLETED      0:0
           2_1  array_job  COMPLETED      0:0
           2_2  array_job  COMPLETED      0:0
```

Arrays work well for one step on many samples. But when one step has to wait for another across hundreds of samples, you would need job dependencies, and that's where managing this by hand stops being fun. It is why we use Snakemake.

2.6. Working interactively

For debugging, you sometimes want a terminal on a compute node. Ask for one like this:

```
srun --account=project_2009831 --partition=small --time=01:00:00
 ↪  --mem=8G --cpus-per-task=2 --pty bash
```

Once SLURM finds room, your prompt moves to a compute node, and you can run commands by hand. Type `exit` to give it back. CSC also has a shortcut called `sinteractive`.

This wasn't run for this book, since a terminal session can't be recorded by our examples.

Use this when a rule fails and you want to rerun its command yourself. It is much better than doing that on the login node.

2.7. How to choose your resources

Nobody knows the right values in advance, and it depends on your data. Some advice:

- **Start from something similar.** A benchmark file from an earlier project tells you more than any rule of thumb.
- **Look at what you really used.** `sacct` shows `MaxRSS` for memory and `Elapsed` for time. Compare them with your request.
- **Don't ask for much more than you need.** Bigger requests wait longer, and on most systems you pay for what you reserve, whether you use it or not.
- **Time limits decide the partition.** On Puhti, jobs longer than three days need `longrun`.
- **More cores don't always help.** Many tools stop getting faster at some point, while the queue time keeps growing.

This pipeline's answer to the problem is to start with a modest request, and go up when a job fails. Chapter 17 describes that.

2.8. Summary

Command	Purpose
`sinfo`	Partitions and nodes
`sbatch script.sh`	Submit a job
`squeue --me`	Your waiting and running jobs
`scontrol show job ID`	All details of a job
`scancel ID`	Stop a job
`sacct -j ID`	What happened to a finished job
`srun --pty bash`	A terminal on a compute node

3. Containers

This pipeline runs about 40 tools. Each has its own dependencies, some of these conflict with each other, and results can change between versions. If everyone had to install all of this, few people could run the pipeline, and no two installations would be the same. **Containers** solve this problem.

3.1. What a container is

A **container image** is a file that holds a program together with everything it needs: libraries, other tools, even a small operating system. When you run a program from an image, it sees the files of the image instead of those of the host system, while it still uses the host's Linux kernel, CPU and memory. It isn't a virtual machine, so it starts right away and has no noticeable slowdown.

For our purposes, this brings three benefits:

- **Nothing to install**, apart from the program that runs the containers.
- **The same software everywhere.** A given image contains the same version of each tool on any computer.
- **Reproducibility.** When a paper names an image, others can use exactly the same software years later.

3.2. Docker and Apptainer

Two programs come up constantly, and they work with the same images:

Program	Used on	How it runs
Docker	Laptops and servers	With a background service and root rights, which clusters don't allow
Apptainer (formerly Singularity)	Clusters	As your own user, without a service

Images are typically built and published with Docker, and on a cluster Apptainer converts them into a single file ending in `.sif`. That is why the configuration of this pipeline holds addresses starting with `docker://`, though it runs with Apptainer on Puhti. Snakemake's option is still called `--use-singularity`, for historical reasons.

An image's address has a name and a **tag**, which usually is a version:

```
docker://fischuu/hrp_bowtie2:0.1
         ------- ----------- ---
         owner   name        tag
```

Always use a specific tag. An address without one means `latest`, which changes over time, and your results may change with it.

3.3. Running a tool from a container

This section uses a real container of the pipeline. We ran it with Docker, because Apptainer can't start containers on the Mac where the examples of this book were run: it runs in an emulator there, and can only download images.

So the output comes from this Docker command:

```
$ docker run --rm --platform linux/amd64 --entrypoint bash
 ↪  fischuu/hrp_bowtie2:0.1 -c '
      bowtie2 --version 2>&1 | head -1 | cut -c1-90
      samtools --version | head -1
      head -1 /etc/os-release
      which bowtie2 samtools
      ls /scratch'
/opt/conda/envs/hrp_bowtie2/bin/bowtie2-align-s version 2.5.1
samtools 1.18
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
/opt/conda/envs/hrp_bowtie2/bin/bowtie2
/opt/conda/envs/hrp_bowtie2/bin/samtools
ls: cannot access '/scratch': No such file or directory
```

This shows the main properties of a container: it has its own tools, it has its own operating system, and **it doesn't see the host's folders unless you allow it**. The last line is exactly the point. Inside this container, `/scratch` doesn't exist.

On the cluster, the same thing looks like this. These commands weren't run for this book, so they have no output here:

```
# Download the image and convert it into one file
apptainer pull bowtie2.sif docker://fischuu/hrp_bowtie2:0.1

# Run one command inside
apptainer exec bowtie2.sif bowtie2 --version

# Run a command that needs your data: make the folder visible
apptainer exec --bind /scratch bowtie2.sif ls /scratch

# Open a shell inside, to look around
apptainer shell --bind /scratch bowtie2.sif
```

3.4. Bind mounts: what a container can see

Making a folder visible in a container is called a **bind mount**, and `--bind` (`-B`) does this. Apptainer binds a few things on its own, such as your home folder and the current folder, but other disks need to be named.

This is the source of the most common container error:

```
No such file or directory
```

The file exists, and you can see it from outside. But the program in the container can't, because its folder wasn't bound. So the pipeline's run script binds all important places on Puhti, and if you use another cluster, this line is what you have to adjust:

```
--singularity-args "-B
 ↪  /scratch,/projappl,/users,/dev/shm:/dev/shm,/run,
 ↪  $projectFolder/tmp:/tmp"
```

`source:target` binds a folder under another name. Here, `$projectFolder/tmp:/tmp` means that `/tmp` inside the container is the folder `tmp/` of your project.

3.4.1. The `TMPDIR` trap

That last bind has a side effect, which caused real bugs in this pipeline. SLURM sets `$TMPDIR` to a folder such as `/tmp/job_12345` on the node's real `/tmp`. Inside the container, the environment variable is still set, but `/tmp` is now a different folder, and `job_12345` doesn't exist there. Tools that write temporary files to `$TMPDIR` then fail, often with a confusing message that says nothing about temporary files.

So you will see this in several rules, like those for prodigal in MAGScoT, DRAM and Bakta. The rule creates its own folder, and sets the variable itself:

```
TMPDIR=
 ↪  "/tmp/magscot_prodigal.{wildcards.assembly_id}.{resources.attempt}.
 ↪  $$"
export TMPDIR
mkdir -p "$TMPDIR"
```

`$$` is the shell's process number. It makes the name unique, since this `/tmp` is one folder shared by all jobs in the project.

3.5. Containers in Snakemake

A rule names its container with `container:`:

```
rule quantify__bowtie2__build:
    input:
        MAG_CATALOGUE,
    output:
        index=touch(MAG_INDEX / "dereplicated_genomes"),
    container:
        docker["bowtie2"]
    shell:
        "bowtie2-build --threads {threads} {input} {output.index}"
```

With `--use-singularity`, Snakemake downloads each image once, and then runs the whole `shell:` block inside it. Without the option, it ignores `container:`, and runs the command on the host.

In this pipeline, `docker` is a dictionary loaded from `config/.docker.yml`, which holds the images:

```
assemble: "docker://fischuu/hrp_assemble:0.2"
bowtie2: "docker://fischuu/hrp_bowtie2:0.1"
concoct: "docker://fischuu/hrp_concoct:0.1"
```

Most images hold the tools of one module or a tool with special needs, and some come from elsewhere, like those for 4CAC, REMAG and Tiara. Section 19.6 lists all of them, with the versions they contain.

Keep the following in mind:

- **Downloading takes time and space.** Images are often several gigabytes. The option `--singularity-prefix` keeps them in the project folder, and the variables `APPTAINER_TMPDIR` and `APPTAINER_CACHEDIR` in the run script send the temporary files of the conversion to scratch. Without these, they would go to your home folder, and fill it up.
- **A shell script in `workflow/scripts/` has to be visible in the container too.** So the pipeline folder has to be under a bound path, and in the run script, `/users` and `/projappl` are bound for this reason.
- **All tools in a rule must be in that rule's container.** A rule that calls `samtools` and `bowtie2` needs an image that has both. This is why the images group tools by module.
- **Debugging in a container** works as you would expect: open a shell in the rule's image, go to the project folder, and run the rendered command by hand.

```
apptainer shell --bind /scratch,/projappl,/users docker_images/<the
↪  image>.simg
```

To find which image file belongs to a rule, look at a dry run with `-p`, or at the rule's SLURM log, which names the image in use.

3.6. What about conda?

Snakemake can also create **conda environments** for rules, using the directive `conda:`. Many workflows do that. This pipeline uses containers only: the images save the time of solving and installing environments on every system, and they keep working when packages disappear from a channel. The images themselves were built with conda inside, which is the `/opt/conda/envs/hrp_bowtie2` you can see in the output above.

3.7. Summary

- A container bundles a tool and its dependencies, and the host's folders stay invisible unless you bind them.
- Docker builds and shares the images, and Apptainer runs them on clusters.
- `container:` names the image of a rule, and `--use-singularity` makes Snakemake use it.
- "No such file or directory" in a container usually means a missing bind.
- `$TMPDIR` set from outside may not exist inside.

4. Why a workflow system?

A metagenomics analysis isn't one program. In this pipeline, a read file passes through trimming, host removal, assembly, three binners, bin refinement, dereplication, a dozen annotation tools and quantification. That is about 170 different steps, and most run once per sample or once per assembly, so one project easily means thousands of jobs.

This chapter explains the problem a workflow system solves, and how Snakemake thinks about it. The next chapters then build up the language rule by rule.

4.1. The script that grows

Most people start with a shell script, and for two samples it works well:

```
fastp -i sampleA_R1.fq.gz -I sampleA_R2.fq.gz -o
 ↪  trimmed/sampleA_R1.fq.gz -O trimmed/sampleA_R2.fq.gz
bowtie2 -x host -1 trimmed/sampleA_R1.fq.gz -2
 ↪  trimmed/sampleA_R2.fq.gz > mapped/sampleA.sam
fastp -i sampleB_R1.fq.gz -I sampleB_R2.fq.gz -o
 ↪  trimmed/sampleB_R1.fq.gz -O trimmed/sampleB_R2.fq.gz
bowtie2 -x host -1 trimmed/sampleB_R1.fq.gz -2
 ↪  trimmed/sampleB_R2.fq.gz > mapped/sampleB.sam
```

Then the project grows, and each of these questions costs you time:

- **A job failed halfway. What still has to run?** A plain script starts over from the first line, or you comment out the lines that worked and hope you got it right.
- **You got three new samples. What now?** Only the new work should run, and every summary table has to be rebuilt.
- **You changed a setting in the trimming step. What is out of date?** Everything that depends on the trimmed reads, and nothing else. You have to remember what that is.
- **How do you run 500 jobs at once?** You write submission scripts, and handle dependencies between jobs yourself.
- **Six months later, how exactly was this result made?** The script may have changed since then, and you may have run parts of it by hand.

A workflow system answers all of these, and it does so by changing how you describe the work.

4.2. Describe the result, not the steps

A script lists actions in order: do this, then that. Snakemake turns this around. You write **rules**, and each rule says how to make one kind of file from other files. Then you ask for a result, and Snakemake works backwards.

Imagine you ask for `results/summary.tsv`. Snakemake reasons like this:

1. Which rule can produce `results/summary.tsv`? The rule `summary`, and it needs `results/counts/sampleA.txt`.
2. Does that file exist and is it current? No. Which rule produces it? The rule `count_reads`, and it needs `data/sampleA_R1.fastq.gz`.
3. That file exists. So the plan is: run `count_reads`, then `summary`.

This plan is called the **DAG**, a directed acyclic graph: boxes for the jobs and arrows for "this needs that". "Acyclic" means that no job can depend on its own result. From the DAG, the questions from above answer themselves:

- **What still has to run?** Whatever is missing or out of date. Everything else is left alone.
- **New samples?** Add them to the list, and only their jobs are new.
- **A changed setting?** Snakemake notices the change and reruns that job and everything downstream.
- **500 jobs at once?** Jobs without an arrow between them can run in parallel. Snakemake submits them for you, and respects the order.

You will come back to this mental model often: **Snakemake is driven by the output files you ask for, not by the rules in your file.** A rule that nothing asks for never runs.

4.3. A few words of vocabulary

Word	Meaning
Snakefile	The file with the rules. Snakemake looks for `Snakefile` or `workflow/Snakefile`
Rule	A recipe: from this input, make this output, with this command
Job	One concrete run of a rule, for example `count_reads` for sampleA

Word	Meaning
Target	A file or rule you ask for
Wildcard	A placeholder in a file name, such as `{sample}`, that lets one rule serve many jobs
DAG	The plan: all jobs needed and their order

4.4. Which Snakemake, and how to get it

This book uses **Snakemake 9.11.6**, the version that the pipeline's run script loads on Puhti. Newer versions work similarly, but command-line options do change between major versions, and a lot of what you find online was written for Snakemake 7 or 8. If a command from a web page fails, check the version it was written for.

On a cluster, you usually load Snakemake as a module:

```
module load snakemake/9.11.6
snakemake --version
```

Elsewhere, you can install it with conda. The second package is the plugin for submitting jobs to SLURM:

```
conda create -n snakemake -c conda-forge -c bioconda
 ↪  snakemake=9.11.6 snakemake-executor-plugin-slurm
conda activate snakemake
```

Neither was run for this book, because the examples use a test cluster with its own installation. Section 4.5 describes it.

4.5. Running the examples of this book

The chapters about Snakemake use small example workflows, which are all in `book/examples/snakemake/`. Each folder has the Snakefile and a `commands.sh` with the commands that the chapter shows. They work on six tiny FASTQ files for three samples, so every example finishes in seconds, and they need no bioinformatics tools. Only standard Linux commands are used.

To make sure the output in this book is real and stays real, the examples run on a **mini cluster**: a container with Debian Linux, a real SLURM installation with accounting, and Snakemake 9.11.6. With Docker installed, this command builds it and runs all examples:

```
bash book/examples/run_all.sh
```

This stores what each command printed in `book/examples/transcripts/`, and that is the output quoted in the chapters. It also works as a test: a command that behaves differently than expected makes the run report a problem. The output in the book is sometimes shortened, which is marked with `[...]`.

You can also use any other Linux system with Snakemake. Copy the folder of an example and the `data` folder together, and run the commands from `commands.sh`. The examples in the SLURM chapters need `sbatch` and an account, so you would have to adjust `slurm_account` in their profile.

4.6. What's next

The next chapter writes the first rule, and then each chapter adds one idea. By the end of this part, you will have seen every Snakemake feature this pipeline uses, and Chapter 17 shows how they come together in it.

5. Your first rule

We start with the smallest useful workflow: counting the reads in one file. The example is in `book/examples/snakemake/01_first_rule/`.

5.1. The Snakefile

A rule goes in a file called `Snakefile`:

```
rule count_reads:
    input:
        "data/sampleA_R1.fastq.gz",
    output:
        "results/sampleA_R1.count.txt",
    shell:
        "zcat {input} | wc -l | awk '{{print $1 / 4}}' > {output}"
```

Line by line:

- `rule count_reads:` starts a rule and gives it a name. Names can use letters, numbers and underscores.
- `input:` lists the files the rule needs. Snakemake won't start it until they exist.
- `output:` lists the files the rule promises to create. Snakemake checks this afterwards.
- `shell:` is the command to run.

Indentation matters. Everything that belongs to the rule is indented by four spaces, and the entries under `input:` by eight. The comma after a file name is optional for one entry, but it is good practice, because you will need it as soon as there are two.

5.2. How `{input}` and `{output}` work

Before the command runs, Snakemake replaces `{input}` with the input files and `{output}` with the output files. So the shell runs:

```
zcat data/sampleA_R1.fastq.gz | wc -l | awk '{print $1 / 4}' >
 ↪  results/sampleA_R1.count.txt
```

This explains the doubled braces in `{print $1 / 4}`. Single braces mean "replace this" to Snakemake, but `awk` needs real braces. A doubled brace tells Snakemake that you mean a literal brace, so `{{` becomes `{`. You will write this for every `awk` program and every shell array such as `${{files[@]}}`. Chapter 13 shows what happens if you forget.

Also notice what you don't write: the rule doesn't create the folder `results/`. Snakemake creates the folders of output files before it starts a job.

5.3. The dry run: always look first

`--dry-run`, or `-n` for short, shows the plan and runs nothing:

```
$ snakemake --dry-run
host: localhost
Building DAG of jobs...
Job stats:
job            count
-----------  -------
count_reads        1
total              1


[date]
rule count_reads:
    input: data/sampleA_R1.fastq.gz
    output: results/sampleA_R1.count.txt
    jobid: 0
    reason: Missing output files: results/sampleA_R1.count.txt
    resources: tmpdir=<TBD>
[...]
This was a dry-run (flag -n). The order of jobs does not reflect
 ↪  the order of execution.
```

Read this from the top. "Job stats" shows how many jobs of each rule would run. Then each job appears with its files. The line to look at is `reason`, which says why Snakemake wants to run it: here, the output doesn't exist.

Make the dry run a habit. On a cluster, a mistake can start thousands of jobs or rerun weeks of finished work, and a dry run costs a few seconds. This pipeline's documentation

asks you to do one before every real run, and this book does too.

5.4. Running it

To run it for real, say how many CPU cores Snakemake may use with `--cores`:

```
$ snakemake --cores 1
Assuming unrestricted shared filesystem usage.
host: localhost
Building DAG of jobs...
Using shell: /usr/bin/bash
Provided cores: 1 (use --cores to define parallelism)
Rules claiming more threads will be scaled down.
Job stats:
job            count
-----------  -------
count_reads        1
total              1

Select jobs to execute...
Execute 1 jobs...

[date]
localrule count_reads:
    input: data/sampleA_R1.fastq.gz
    output: results/sampleA_R1.count.txt
    jobid: 0
    reason: Missing output files: results/sampleA_R1.count.txt
    resources: tmpdir=/tmp
[date]
Finished jobid: 0 (Rule: count_reads)
1 of 1 steps (100%) done
Complete log(s):
 ↪  /scratch/project_demo/examples/01_first_rule/.snakemake/log/<date>.snakemake.log

$ cat results/sampleA_R1.count.txt
40
```

There is the answer: 40 reads. The last line says where Snakemake keeps a copy of everything it printed. These log files collect in the folder `.snakemake/log/`, and are the first place to look when you want to know what happened last night.

5.5. Running it again

Run the same command again:

```
$ snakemake --cores 1
Assuming unrestricted shared filesystem usage.
host: localhost
Building DAG of jobs...
Nothing to be done (all requested files are present and up to
 ↪  date).
```

This is the central feature. The output exists and is current, so there is nothing to do. On a cluster this means that you can simply start a workflow again after anything goes wrong, and it continues where it stopped.

5.6. When does a rule run again?

To decide whether a file is current, Snakemake looks at several things, called **rerun triggers**. The most obvious one is time: a job reruns if an input is newer than its output. This goes further than you might expect, so let us test it. If we only update the file's date with `touch`, without changing its content:

```
$ touch data/sampleA_R1.fastq.gz
$ snakemake --cores 1 --dry-run --quiet
host: localhost
Building DAG of jobs...
Nothing to be done (all requested files are present and up to
 ↪  date).
```

Nothing happens. That file is small, so Snakemake also stored a checksum of it, and saw that the content is the same. Now we really change the content:

```
$ cp data/sampleB_R1.fastq.gz data/sampleA_R1.fastq.gz
$ snakemake --cores 1 --dry-run | grep reason
    reason: Updated input files: data/sampleA_R1.fastq.gz

$ snakemake --cores 1 --quiet
[...]
$ cat results/sampleA_R1.count.txt
60
```

So Snakemake recognised the change and updated the result. Be careful with what you conclude from this example: the checksum comparison only works for small files, and for large files like real FASTQ data, a newer date alone can trigger a rerun.

Section 13.8 covers the other triggers, including changed code and settings, which often surprise people.

You can also force a rerun yourself:

Option	Effect
`--force`, `-f`	Rerun the job that produces the target
`--forceall`, `-F`	Rerun everything the target depends on
`--forcerun rule`, `-R rule`	Rerun all jobs of a rule, and everything downstream

```
$ snakemake --cores 1 --forceall --dry-run | grep reason
    reason: Forced execution
```

5.7. Useful options to know now

Option	Meaning
`--dry-run`, `-n`	Show the plan, run nothing
`--printshellcmds`, `-p`	Also print each command
`--cores N`, `-c N`	Use up to N CPU cores
`--snakefile FILE`, `-s FILE`	Use another file than `Snakefile`
`--quiet rules`	Print less: only the overview of rules and the progress

`-np` together is the combination you will type most often. It shows the plan with the exact commands, which is the fastest way to check that a rule does what you intended.

5.8. What Snakemake wrote to your folder

Besides your results, there is now a hidden folder:

```
.snakemake/
    log/        a log file per Snakemake run
    metadata/   what it knows about each output file: code, parameters, input checksums
    locks/      marks that a Snakemake is running in this folder
```

Snakemake keeps here what it needs to decide about reruns. Don't edit it. On the other hand, it is safe to delete the logs, and this folder doesn't belong in version control.

5.9. Summary

- A rule says how to make output files from input files with a command.
- `{input}` and `{output}` get replaced, and literal braces must be doubled.
- `snakemake -n` shows the plan and `snakemake --cores N` runs it.
- A rerun only does what is missing or out of date.

This rule has one big weakness: it only works for one file. The next chapter fixes that.

6. Wildcards: one rule for many files

The previous rule could count the reads of a single file. For 100 samples you won't write 100 rules, and this is what wildcards are for. The example is in `book/examples/snakemake/02_wildcards/`.

6.1. Replacing names with placeholders

Here is the same rule with two placeholders:

```
rule count_reads:
    input:
        "data/{sample}_R{mate}.fastq.gz",
    output:
        "results/{sample}_R{mate}.count.txt",
    shell:
        "zcat {input} | wc -l | awk '{{print $1 / 4}}' > {output}"
```

`{sample}` and `{mate}` are **wildcards**. The rule now says: to make any file named like `results/<something>_R<something>.count.txt`, read the file `data/<same something>_R<same something>.fastq.gz`.

6.2. Snakemake can't guess what you want

If you run this as before, it fails:

```
$ snakemake --cores 1
Assuming unrestricted shared filesystem usage.
host: localhost
Building DAG of jobs...
WorkflowError:
Target rules may not contain wildcards. Please specify concrete
 ↪  files or a rule without wildcards at the command line,
```

```
or have a rule without wildcards at the very top of your workflow
 ↪  (e.g. the typical "rule all" which just collects
all results you want to generate in the end).
[exit code 1]
```

Without a target, Snakemake uses the first rule in the file, and it can't fill in the wildcards on its own. This message even tells you the solution, and we will use both options. The first option is to name the file you want:

```
$ snakemake --cores 1 results/sampleB_R2.count.txt
[...]
localrule count_reads:
    input: data/sampleB_R2.fastq.gz
    output: results/sampleB_R2.count.txt
    jobid: 0
    reason: Missing output files: results/sampleB_R2.count.txt
    wildcards: sample=sampleB, mate=2
    resources: tmpdir=/tmp
[...]
```

Look at the line `wildcards: sample=sampleB, mate=2`. This is the core of how it works: Snakemake compared the name you asked for with the rule's output pattern, and worked out the values. It then put them into the input pattern. **Wildcards always flow from output to input**, which is the opposite of how you may think about the data.

You can ask for several files at once:

```
$ snakemake results/sampleC_R1.count.txt
 ↪  results/sampleC_R2.count.txt --cores 1 --quiet
[...]
$ ls results
sampleB_R2.count.txt
sampleC_R1.count.txt
sampleC_R2.count.txt
```

Notice that the targets now come first. Options like `--quiet` can take values of their own, and this is what happens if a target follows one:

```
$ snakemake --cores 1 --quiet results/sampleC_R1.count.txt 2>&1 |
 ↪  tail -n 1
```

```
snakemake: error: argument --quiet/-q: invalid choice:
 ↪  'results/sampleC_R1.count.txt' (choose from all, host,
 ↪  progress, reason, rules)
```

So put targets directly after `snakemake`. The options `--dag`, `--rulegraph` and `--set-threads` behave in the same way.

6.3. Two errors worth recognising

The output pattern may match, but the input may be missing:

```
$ snakemake --cores 1 results/sampleZ_R1.count.txt
Assuming unrestricted shared filesystem usage.
host: localhost
Building DAG of jobs...
MissingInputException in rule count_reads in file
 ↪  "/scratch/project_demo/examples/02_wildcards/Snakefile", line
 ↪  1:
Missing input files for rule count_reads:
    output: results/sampleZ_R1.count.txt
    wildcards: sample=sampleZ, mate=1
    affected files:
        data/sampleZ_R1.fastq.gz
[exit code 1]
```

This says what it could work out and which file it needs. Most of the time, it means a typo in a sample name, a wrong path, or an earlier rule that should produce the file but is missing.

Or the name you ask for may fit no rule at all:

```
$ snakemake --cores 1 results/sampleA.count.txt
Assuming unrestricted shared filesystem usage.
host: localhost
Building DAG of jobs...
MissingRuleException:
No rule to produce results/sampleA.count.txt (if you use input
 ↪  functions make sure that they don't raise unexpected
 ↪  exceptions).
[exit code 1]
```

Here `sampleA.count.txt` lacks the `_R<mate>` part, so no output pattern matches. When you see this, compare your file name character by character with the rule's output.

6.4. How matching really works

By default, a wildcard matches any text of at least one character, including slashes. This has consequences:

- Every wildcard in `input:` must also be in `output:`, since that is the only place its value can come from.
- The same wildcard can appear several times in a pattern, and has to have the same value everywhere.
- A pattern can match in more ways than you meant. If a sample is called `sample_R1`, then `sample_R1_R1.count.txt` can be split in more than one way. Snakemake's matching prefers the longest possible match.

The last point can cause confusing errors in bigger workflows. When two wildcards are next to each other or a name can contain separators, restrict what a wildcard may match with a regular expression:

```
rule count_reads:
    input:
        "data/{sample}_R{mate}.fastq.gz",
    output:
        "results/{sample}_R{mate}.count.txt",
    wildcard_constraints:
        mate="[12]",
```

The rule then only matches when `mate` is 1 or 2. This pipeline uses the same idea in its hmmsearch rule, where `marker_set="pfam|tigr"` limits a wildcard to two values (Chapter 63).

6.5. Using wildcard values in the command

Inside `shell:`, you can use the value with `{wildcards.sample}`:

```
    shell:
        "echo 'Counting {wildcards.sample}, mate {wildcards.mate}'
        ↪ && zcat {input} | wc -l > {output}"
```

6.6. Naming conventions in this pipeline

The pipeline uses three kinds of wildcards everywhere. Knowing them makes its rules much easier to read:

Wildcard	Meaning	Example
`{sample_id}`	A biological sample, from `samples.tsv`	`cow01`
`{library_id}`	One sequencing library of that sample	`L001`
`{assembly_id}`	One assembly, which may combine several libraries	`coasm`

So a file like `results/preprocess/fastp/cow01.L001_1.fq.gz` is written in a rule as `FASTP / "{sample_id}.{library_id}_1.fq.gz"`. The `FASTP` part is a path constant, which Chapter 9 explains.

6.7. Summary

- A wildcard lets one rule make many files.
- Values come from matching the requested file name against the output pattern.
- A rule with wildcards can't be a target by itself, so you have to name a concrete file.
- Put targets directly after `snakemake`.

Naming every file on the command line doesn't scale to 100 samples. Before we solve that, the next chapter connects several rules with each other.

7. Chaining rules

So far we have had one rule. A workflow gets its power from connecting rules, and in Snakemake you don't connect them explicitly: **a rule depends on another rule when its input matches the other rule's output.** That's all. The example is in `book/examples/snakemake/03_chaining/`.

7.1. Four rules

This workflow filters out short reads, then computes two statistics, and finally writes them on one line:

```
rule filter_reads:
    """Keep the reads of at least 50 bases"""
    input:
        "data/{sample}_R1.fastq.gz",
    output:
        "results/filtered/{sample}.fastq.gz",
    shell:
        """
        zcat {input} \
            | paste - - - - \
            | awk -F '\\t' 'length($2) >= 50' \
            | tr '\\t' '\\n' \
            | gzip > {output}
        """


rule count_reads:
    input:
        "results/filtered/{sample}.fastq.gz",
    output:
        "results/counts/{sample}.txt",
    shell:
        "zcat {input} | wc -l | awk '{{print $1 / 4}}' > {output}"
```

```
rule mean_length:
    input:
        "results/filtered/{sample}.fastq.gz",
    output:
        "results/lengths/{sample}.txt",
    shell:
        """
        zcat {input} \
            | awk 'NR % 4 == 2 {{ total += length($0); n++ }} END
↪   {{ print total / n }}' \
            > {output}
        """


rule summary:
    input:
        reads="results/counts/{sample}.txt",
        length="results/lengths/{sample}.txt",
    output:
        "results/summary/{sample}.tsv",
    shell:
        """
        printf '%s\\t%s\\t%s\\n' {wildcards.sample} "$(cat
↪   {input.reads})" "$(cat {input.length})" \
            > {output}
        """
```

There are several new things here, so let us go through them.

7.1.1. A docstring

The text in triple quotes right after the rule name is a short description. It shows up in `snakemake --list-rules`, and all rules of this pipeline have one.

7.1.2. Commands over several lines

Triple quotes also let a command run over several lines. Each line is a command of its own, and a backslash at the end of a line continues the command on the next one. In a normal Python string, a backslash is a special character, which is why tabs and newlines are written as `\\t` and `\\n`: the doubled backslash reaches the shell as a single one.

The filter uses a well-known trick: `paste - - - -` joins each four lines of FASTQ into one line with tabs in between, `awk` can then test the length of column 2, and `tr` turns the tabs back into newlines.

7.1.3. Named inputs

In `summary`, the inputs have names, and the command uses `{input.reads}` and `{input.length}`. Named inputs make a command easier to read, and let you use the files in any order. The same works for `output:`, and most rules in this pipeline use names when there are several files.

Choose names with care. Our first version used `count=` here, and Snakemake refused it:

```
$ snakemake -s Snakefile.reserved --dry-run
 ↪  results/summary/sampleA.tsv
AttributeError in file
 ↪  "/scratch/project_demo/examples/03_chaining/Snakefile.reserved",
 ↪  line 1:
invalid name for input, output, wildcard, params or log: count is
 ↪  reserved for internal use
[exit code 1]
```

Inputs behave like Python lists, and a Python list has a method called `count`, as well as `index` and others. That name is already taken.

7.2. What Snakemake plans

Ask for one summary and look at the plan:

```
$ snakemake --cores 1 --dry-run results/summary/sampleA.tsv
host: localhost
Building DAG of jobs...
Job stats:
job             count
------------  -------
count_reads         1
filter_reads        1
mean_length         1
summary             1
total               4
```

```
[date]
rule filter_reads:
    input: data/sampleA_R1.fastq.gz
    output: results/filtered/sampleA.fastq.gz
    jobid: 2
    reason: Missing output files: results/filtered/sampleA.fastq.gz
    wildcards: sample=sampleA
    resources: tmpdir=<TBD>
[...]
```

We named only the last file, and Snakemake found the other three steps on its own. Its reasoning went backwards from the target, and the wildcard value `sampleA` passed along the chain.

Now run it with two cores:

```
$ snakemake --cores 2 results/summary/sampleA.tsv
[...]
Select jobs to execute...
Execute 1 jobs...
[...]
localrule filter_reads:
[...]
1 of 4 steps (25%) done
Select jobs to execute...
Execute 2 jobs...
[...]
localrule count_reads:
[...]
localrule mean_length:
[...]
Finished jobid: 1 (Rule: count_reads)
2 of 4 steps (50%) done
Finished jobid: 3 (Rule: mean_length)
3 of 4 steps (75%) done
Select jobs to execute...
Execute 1 jobs...
[...]
localrule summary:
[...]
4 of 4 steps (100%) done
```

```
$ cat results/summary/sampleA.tsv
sampleA 28  74.2857
```

See what happened in the second round: "Execute 2 jobs". The two statistics don't depend on each other, so Snakemake ran them at the same time. You didn't have to say this anywhere. It follows from the DAG.

7.3. Seeing the DAG

`--dag` prints the graph in the language of a drawing program called Graphviz. You then pipe it to `dot` to make an image:

```
$ snakemake results/summary/sampleB.tsv --dag | dot -Tpng > dag.png
```

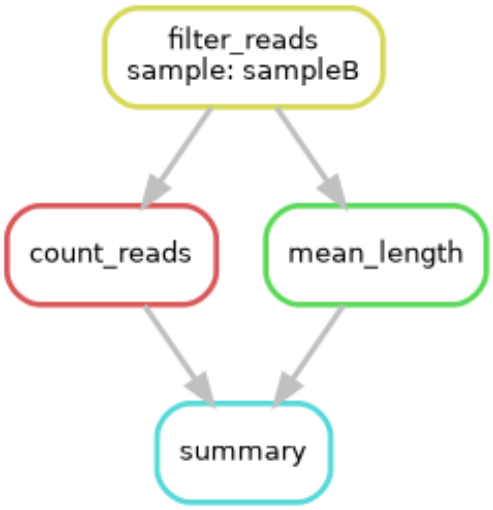


Figure 7.1.: The jobs needed for one sample. Each box is a job, and an arrow means "needs the result of".

`--rulegraph` draws one box per rule rather than per job. On a real project, the DAG can have thousands of boxes, and the rule graph stays readable:

```
$ snakemake results/summary/sampleB.tsv --rulegraph | dot -Tpng >
↪  rulegraph.png
```

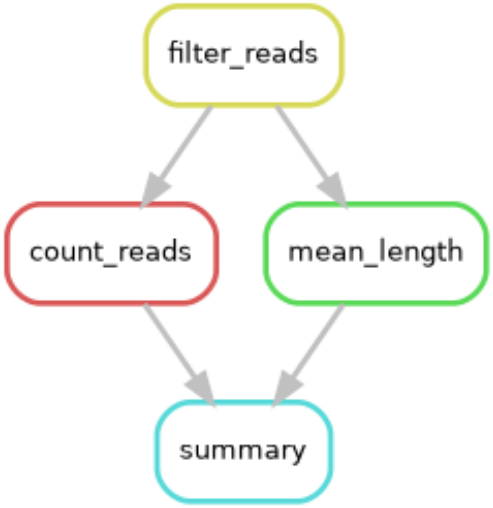


Figure 7.2.: The same workflow as a rule graph. With one sample, it looks the same as the DAG.

In a dry run, finished jobs appear with dashed lines in `--dag`, which is a nice way to see how far a project has come. The pipeline's flowchart in the README is a drawing made by hand from this kind of information.

7.4. Checking on what happened

`--summary` lists the output files with the rule that made them and their status:

```
$ snakemake --summary results/summary/sampleA.tsv | cut -f 1,3,5
output_file rule    status
results/summary/sampleA.tsv summary ok
results/counts/sampleA.txt  count_reads ok
results/filtered/sampleA.fastq.gz   filter_reads    ok
results/lengths/sampleA.txt mean_length ok
```

Status messages such as "rule implementation changed" appear here too, so this is a quick way to see what Snakemake considers out of date.

7.5. Why output files matter so much

Since Snakemake only sees files, some consequences follow:

- **The output of one rule must exactly match the input of the next**, character for character. `results/counts/sampleA.txt` and `results//counts/sampleA.txt` are different names to Snakemake. This is one reason why this pipeline defines paths once, in `folders.smk` (Section 14.5).
- **Each output file should come from one rule.** If two rules can make the same file, Snakemake stops with an `AmbiguousRuleException`, which Chapter 13 shows.
- **A rule has to create all its declared outputs.** If one is missing after the command finishes, the job counts as failed.
- **Each job should stick to its own output files.** Two jobs of the same rule run at the same time, so they must not write to the same temporary file.

7.6. Summary

- Rules connect through file names: an output of one matches an input of another.
- Snakemake works out the order and runs independent jobs in parallel.
- `--dag` and `--rulegraph` draw the plan, and `--summary` shows the state of the results.

We still ask for files one at a time. The next chapter introduces the rule at the top of every real Snakefile.

8. Targets: `rule all` and `expand()`

A real workflow has to produce results for many samples, and nobody wants to type all those file names. This chapter shows how to ask for everything at once. The example is in `book/examples/snakemake/04_targets/`.

8.1. A rule that only asks for files

If you don't name a target, Snakemake uses the **first rule** in the file. By convention that is a rule called `all` that has inputs but no output and no command:

```
SAMPLES = ["sampleA", "sampleB", "sampleC"]


rule all:
    """What a run without any target name produces"""
    input:
        "results/summary.tsv",


rule count_reads:
    input:
        "data/{sample}_R{mate}.fastq.gz",
    output:
        "results/counts/{sample}_R{mate}.txt",
    shell:
        "zcat {input} | wc -l | awk '{{print $1 / 4}}' > {output}"


rule summary:
    """One table with a line per read file"""
    input:
        expand("results/counts/{sample}_R{mate}.txt",
    ↪   sample=SAMPLES, mate=[1, 2]),
    output:
        "results/summary.tsv",
```

```
    shell:
        """
        for count in {input}; do
            printf '%s\\t%s\\n' "$(basename "$count" .txt)" "$(cat
↪   "$count")"
        done > {output}
        """
```

A rule without output and command runs nothing. It only says "I need these files", and this makes the jobs that produce them necessary. `rule all` has no wildcards, so Snakemake can work out everything from it.

8.2. `expand()` writes the list for you

`expand()` fills a pattern with all combinations of the values you give it. So this line:

```
expand("results/counts/{sample}_R{mate}.txt", sample=SAMPLES,
↪   mate=[1, 2])
```

produces this list of six files:

```
results/counts/sampleA_R1.txt
results/counts/sampleA_R2.txt
results/counts/sampleB_R1.txt
results/counts/sampleB_R2.txt
results/counts/sampleC_R1.txt
results/counts/sampleC_R2.txt
```

`SAMPLES` is a normal Python variable, and you can write Python in a Snakefile almost anywhere. Chapter 9 introduces the parts you need.

Note that `summary` itself has no wildcard: its output is one fixed file. This is the typical shape of an **aggregation step**, which collects many results into one. In the command, `{input}` becomes all six file names with spaces between them, which is what the `for` loop goes through.

Let's run it:

```
$ snakemake --cores 2 --quiet rules
[...]
Job stats:
```

```
job            count
-----------  -------
all                1
count_reads        6
summary            1
total              8
[...]
8 of 8 steps (100%) done

$ cat results/summary.tsv
sampleA_R1  40
sampleA_R2  40
sampleB_R1  60
sampleB_R2  60
sampleC_R1  25
sampleC_R2  25
```

From one target came eight jobs. If you add a sample to the list, only its two count jobs and the summary run again.

8.3. Target rules for parts of a workflow

Nothing forces you to have only one rule like this, and it doesn't have to be first. Here is another one in the same file:

```
rule counts:
    """A target rule: it runs nothing and only asks for files"""
    input:
        expand("results/counts/{sample}_R1.txt", sample=SAMPLES),
```

Now you can ask for a part of the workflow by the rule's name:

```
$ snakemake counts --cores 1 --dry-run --forceall --quiet rules
host: localhost
Building DAG of jobs...
Job stats:
job            count
-----------  -------
count_reads        3
counts             1
```

```
total              4
[...]
```

This is how the whole pipeline is organised. When you run `bash run_Snakebite-Holoruminant-MetaG.sh assemble__megahit`, then `assemble__megahit` is exactly such a rule. It lists the assemblies, and Snakemake does what is necessary to produce them. Each tool has one, each module has one, and `rule all` combines the modules.

Two commands list what is available:

```
$ snakemake --list-rules
all (What a run without any target name produces)
count_reads
counts (A target rule: it runs nothing and only asks for files)
summary (One table with a line per read file)

$ snakemake --list-target-rules
all (What a run without any target name produces)
counts (A target rule: it runs nothing and only asks for files)
summary (One table with a line per read file)
```

The second list has the rules you can use as a target by name, which are all those without wildcards.

8.4. Using another rule's inputs

To avoid writing the same list twice, a rule can refer to another one with `rules.<name>.input`:

```
rule qc:
    input:
        rules.qc__filter.input,
        rules.qc__report.input,
```

The pipeline does this constantly: `rule all` in `workflow/Snakefile` is just a list of `rules.preprocess.input`, `rules.assemble.input` and so on. One thing to watch for is that the referenced rule must already be defined at that point in the file, which is why the Snakefile includes the modules before `rule all`, and why this one rule isn't at the top.

You may wonder how `rule all` can still be the default target there, since many rules come before it. The answer is that rules from included files don't count for this: the default is the first rule in the main Snakefile itself, and in the pipeline that is `rule all`. So running the pipeline without any target means running all of it. A rule can also claim this role with the directive `default_target: True`, wherever it stands.

8.5. Finding samples from file names

Instead of listing samples, you can get them from the files that exist. `glob_wildcards()` does the reverse of `expand()`: it takes a pattern, and finds the values:

```
# Find the samples from the file names instead of listing them
SAMPLES = glob_wildcards("data/{sample}_R1.fastq.gz").sample
print("Found samples:", SAMPLES)
```

```
$ snakemake --snakefile Snakefile.glob --cores 1 --dry-run --quiet
 ↪  rules
Found samples: ['sampleB', 'sampleA', 'sampleC']
[...]
```

Note the order of the samples, which comes from the file system. This pipeline doesn't work this way for samples, and there is a good reason: a sample sheet states what you intended to analyse, including settings per sample, and a forgotten or extra file becomes an error rather than a silent change to your results. The next chapter does it that way.

8.6. Summary

- The first rule is the default target, and `rule all` by convention lists what you want in the end.
- `expand()` builds lists of file names.
- A rule without output is a target rule, and you can ask for it by name.
- This pipeline has such a rule per tool and per module.

9. Just enough Python

A Snakefile is a Python file with extra keywords. You can read and write simple workflows without knowing Python, but this pipeline uses it in many places: to read the sample sheet, to build file lists and to choose inputs per sample. This chapter teaches exactly that part of Python and nothing more. If you know Python, skim the last two sections, which are about how Snakemake uses it. The example is in `book/examples/snakemake/05_python/`.

9.1. Python in one page

Here is the whole file `python_basics.py`. You can run it with `python python_basics.py`:

```
from pathlib import Path

# A list, and a loop over it
samples = ["sampleA", "sampleB"]
for sample in samples:
    print("Sample:", sample)

# An f-string puts values into a text
sample = "sampleA"
print(f"results/{sample}.txt")

# A list comprehension builds a list from another list
print([f"results/{sample}.txt" for sample in samples])

# A dictionary maps names to values
hosts = {"cow": "cow.fa.gz", "sheep": "sheep.fa.gz"}
print(hosts["cow"], list(hosts))

# A function takes values and returns a result
def result_file(sample, mate):
    return f"results/{sample}_R{mate}.txt"
print(result_file("sampleA", 1))
```

```
# A lambda is a function without a name, written in one line
double = lambda x: x * 2
print(double(21))

# Path objects join folders with "/"
RESULTS = Path("results")
print(RESULTS / "fastp" / "sampleA.fq.gz")
```

This prints:

```
Sample: sampleA
Sample: sampleB
results/sampleA.txt
['results/sampleA.txt', 'results/sampleB.txt']
cow.fa.gz ['cow', 'sheep']
results/sampleA_R1.txt
42
results/fastp/sampleA.fq.gz
```

Now let us look at each idea and where you will meet it.

9.1.1. Lists and loops

A list is written in square brackets. In Python, indentation shows what belongs to a loop or a function, which is why the line after `for` is indented. A `#` starts a comment.

9.1.2. f-strings

An f-string has an `f` in front of the quotes, and replaces each `{name}` with the value of the variable. Be careful about the similarity to Snakemake's patterns, because they do different things:

```
"results/{sample}.txt"    # A pattern: {sample} is a wildcard,
 ↪  filled in later by Snakemake
f"results/{sample}.txt"   # An f-string: {sample} is a Python
 ↪  variable, filled in right now
```

If you need both in one text, double the braces of the wildcard: `f"{RESULTS}/{{sample}}.txt"`. You will see that in this pipeline's `folders.smk`.

9.1.3. List comprehensions

A list comprehension is a loop squeezed into one expression: "make this, for each of these". The pipeline uses them instead of `expand()` when the combinations aren't all possible. In this rule, not every library belongs to every assembly, so a plain `expand()` would ask for files that shouldn't exist:

```
[
    ASSEMBLY_MAPPING /
 ↪  f"{assembly_id}.{sample_id}.{library_id}.cram"
    for assembly_id, sample_id, library_id in ASSEMBLY_LIBRARIES
]
```

`ASSEMBLY_LIBRARIES` is a list of triples, and each round of the loop unpacks one into three variables. This is the most common kind of Python in this pipeline.

9.1.4. Dictionaries

A dictionary maps a name to a value, and you look values up in square brackets. Configuration files turn into dictionaries in Python, which is why you will see lookups like `config["assembler"]` or `params["assemble"]["drep"]["P_ani"]`.

9.1.5. Functions and lambdas

`def` defines a function, and `return` hands back its result. A `lambda` is a small function without a name, and it fits on one line. These two are the same:

```
def double(x):
    return x * 2

double = lambda x: x * 2
```

9.1.6. Paths

`Path` from Python's standard library represents a file path, and `/` joins its parts. The pipeline defines one constant per results folder in `workflow/rules/folders.smk`, and uses them everywhere:

```
ASSEMBLE = Path("results/assemble")
MEGAHIT = ASSEMBLE / "megahit"
```

Then a rule can write its output as `MEGAHIT / "{assembly_id}.fa.gz"`. If the folder ever moves, one line changes.

9.2. Reading a sample sheet with pandas

The sample sheet of this example has one line per library:

```
sample_id   library_id  forward reverse assembly_ids
sampleA L001    data/sampleA_R1.fastq.gz    data/sampleA_R2.fastq.gz    asm1, combined
sampleB L001    data/sampleB_R1.fastq.gz    data/sampleB_R2.fastq.gz    asm2, combined
sampleC L001    data/sampleC_R1.fastq.gz    data/sampleC_R2.fastq.gz    asm3
```

The last column says which assemblies a library goes into, and a library can be in several. This is a small model of the pipeline's `samples.tsv`, where several libraries that share an `assembly_id` make up a co-assembly.

pandas is a Python library for tables. Let's follow what the workflow does with this sheet, step by step:

```
import pandas as pd

samples = pd.read_table("samples.tsv")
print(samples[["sample_id", "assembly_ids"]])
```

```
  sample_id    assembly_ids
0   sampleA  asm1, combined
1   sampleB  asm2, combined
2   sampleC            asm3
```

`read_table` loads the file as a table called a **data frame**, and `samples[[...]]` selects columns. Now we split the text in the last column into a real list, using a lambda on each cell:

```
samples["assembly_id"] = samples["assembly_ids"].map(lambda cell:
 ↪  cell.replace(" ", "").split(","))
print(samples[["sample_id", "assembly_id"]])
```

```
  sample_id       assembly_id
0   sampleA  [asm1, combined]
1   sampleB  [asm2, combined]
```

```
2   sampleC            [asm3]
```

`explode` then gives each list element a row of its own:

```
samples = samples.explode("assembly_id")
print(samples[["sample_id", "assembly_id"]])
```

```
  sample_id assembly_id
0   sampleA        asm1
0   sampleA    combined
1   sampleB        asm2
1   sampleB    combined
2   sampleC        asm3
```

Now it is easy to ask which libraries belong to an assembly. A condition in square brackets keeps the matching rows:

```
members = samples[samples["assembly_id"] == "combined"]
print(list(zip(members["sample_id"], members["library_id"])))
```

```
[('sampleA', 'L001'), ('sampleB', 'L001')]
```

`zip` pairs up two columns. The pipeline's `workflow/Snakefile` and its function `libraries_of_assembly` work just like this.

9.3. Input functions: an input that depends on the wildcards

Now everything comes together. Pooling reads for an assembly has a problem: how many input files there are depends on the assembly, so no fixed pattern can describe them. For this you can write a function as the input:

```
import pandas as pd
from pathlib import Path

RESULTS = Path("results")

samples = pd.read_table("samples.tsv")
samples["assembly_id"] = samples["assembly_ids"].map(lambda cell:
 ↪  cell.replace(" ", "").split(","))
```

```
samples = samples.explode("assembly_id")
ASSEMBLIES = samples["assembly_id"].unique().tolist()


def forward_reads(wildcards):
    """The forward read files of all samples in one assembly"""
    return samples.loc[samples["assembly_id"] ==
     ↪  wildcards.assembly_id, "forward"].tolist()


rule all:
    input:
        [RESULTS / f"{assembly_id}.fastq.gz" for assembly_id in
 ↪  ASSEMBLIES],


rule pool_reads:
    """Pool the reads of all samples that belong to one assembly"""
    input:
        forward_reads,
    output:
        RESULTS / "{assembly_id}.fastq.gz",
    params:
        n_files=lambda wildcards, input: len(input),
    shell:
        """
        echo "{wildcards.assembly_id}: pooling {params.n_files}
 ↪  file(s)"
        cat {input} > {output}
        """
```

The line `forward_reads,` has no parentheses. This hands Snakemake the function itself, instead of calling it. Once Snakemake knows the wildcards of a job, it calls the function with them, and the function returns the list of input files. This is called an **input function**. The job for the assembly `combined` gets two files, and the others one each:

```
$ snakemake --cores 1
combined: pooling 2 file(s)
asm1: pooling 1 file(s)
asm2: pooling 1 file(s)
asm3: pooling 1 file(s)
[...]
```

```
localrule pool_reads:
    input: data/sampleA_R1.fastq.gz, data/sampleB_R1.fastq.gz
    output: results/combined.fastq.gz
    jobid: 2
    reason: Missing output files: results/combined.fastq.gz
    wildcards: assembly_id=combined
    resources: tmpdir=/tmp
[...]

$ for file in results/*.fastq.gz; do echo "$file: $(zcat "$file" |
 ↪  wc -l | awk '{print $1 / 4}') reads"; done
results/asm1.fastq.gz: 40 reads
results/asm2.fastq.gz: 60 reads
results/asm3.fastq.gz: 25 reads
results/combined.fastq.gz: 100 reads
```

Pooled gzip files stay valid, which is a useful property of the gzip format: `cat a.gz b.gz > c.gz` gives a file that decompresses to both contents. The metaSPAdes rule pools reads this way.

The example has another new thing: `params:` lists values for the command that aren't files. A lambda here can ask for `wildcards`, `input`, `output`, `threads` or `resources`, by naming them as its arguments. `params` plays a big role in the next chapter.

Two details make input functions easier to debug:

- A function can be tested on its own in Python, and it isn't Snakemake that has to fail first.
- If a function raises a Python error, Snakemake can report the confusing message "No rule to produce" that you saw in the previous chapter. That's why that message mentions input functions.

9.3.1. Functions that return a function

The pipeline uses one more step here. Many rules need the same function, once for mate 1 and once for mate 2. So `assembly_reads` in `workflow/rules/assemble/__functions__.smk` takes the mate as a value, and *returns a new function*:

```
def assembly_reads(end, separator=None):
    read_folder = CLEAN_READS if HOST_NAMES else FASTP

    def reads(wildcards):
```

```
        paths = [
            str(read_folder /
             ↪ f"{sample_id}.{library_id}_{end}.fq.gz")
            for sample_id, library_id in
             ↪ libraries_of_assembly(wildcards.assembly_id)
        ]
        return separator.join(paths) if separator else paths

    return reads
```

In a rule, this reads almost like a sentence, and note the parentheses this time, since the outer function does get called:

```
    input:
        forwards=assembly_reads(1),
        reverses=assembly_reads(2),
```

9.4. Python that runs at parse time

One thing confuses people about Python in a Snakefile: **when** it runs. Snakemake reads the Snakefile and runs all top-level Python code first, to learn what the rules are. It does this before it looks at any job:

- Code at the top level, such as reading the sample sheet, runs each time Snakemake starts. It runs many times on a cluster: once in the main process, and again in each SLURM job. So keep it fast, and don't make it print much or change files.
- Input functions run later, once per job, when Snakemake builds the DAG.
- The `shell:` block runs when the job runs.

The fact that a Snakefile is read again inside each SLURM job has a consequence that Chapter 15 describes, and which affects this pipeline.

9.5. Summary

- A Snakefile is Python, and the pipeline uses lists, f-strings, comprehensions, dictionaries, functions and `Path`.
- pandas reads the sample sheet, and `explode` handles libraries in several assemblies.
- An input function chooses a job's inputs from its wildcards.
- Top-level code runs every time Snakemake starts, also inside each cluster job.

10. Configuration

Rules describe how the work is done, and settings describe what this particular project needs. Keeping these apart is what makes a workflow reusable: a colleague can run your rules on other data by changing only the configuration. The example is in `book/examples/snakemake/06_config/`.

10.1. A config file

Settings go in a **YAML** file, a simple text format where indentation shows the structure:

```
# Settings a user changes without touching the rules
samples:
  - sampleA
  - sampleB
filter:
  min_length: 50
```

A line with a colon gives a name a value, and a `-` starts a list item. YAML cares about spaces and doesn't accept tabs, so if a config file fails to load, look there first.

The Snakefile loads it with `configfile:`, and the content is then available in a dictionary called `config`:

```
configfile: "config/config.yaml"


rule all:
    input:
        expand("results/{sample}.fastq.gz",
 ↪  sample=config["samples"]),


rule filter_reads:
    input:
```

```
        "data/{sample}_R1.fastq.gz",
    output:
        "results/{sample}.fastq.gz",
    params:
        min_length=config["filter"]["min_length"],
    shell:
        """
        zcat {input} \
            | paste - - - - \
            | awk -F '\\t' -v min={params.min_length} 'length($2)
↪   >= min' \
            | tr '\\t' '\\n' \
            | gzip > {output}
        """
```

The nesting in the file becomes nested lookups: `config["filter"]["min_length"]` is 50.

10.2. Why `params:` and not just `{config[...]}`?

The setting passes through `params:` and not straight into the command. This has a practical advantage, and we can see it here.

First we run the workflow, and count the lines of the result:

```
$ snakemake --cores 1 --quiet rules
[...]
$ zcat results/sampleA.fastq.gz | wc -l
112
```

Now we change the setting from the command line. `--config` overrides values for one run:

```
$ snakemake --cores 1 --dry-run --quiet rules --config
↪  filter='{min_length: 80}'
Config file config/config.yaml is extended by additional config
↪  specified via the command line.
host: localhost
Building DAG of jobs...
Job stats:
job             count
```

```
------------  -------
all                 1
filter_reads        2
total               3
[...]
Reasons:
    (check individual jobs above for details)
    input files updated by another job:
        all
    params have changed since last execution:
        filter_reads
[...]
```

Snakemake noticed the changed parameter, and wants to rerun exactly the jobs it affects. That only works because the value was in `params:`, which Snakemake records for each output. After running it, the file has fewer reads, as it should:

```
$ snakemake --cores 1 --quiet rules --config filter='{min_length:
↪  80}'
[...]
$ zcat results/sampleA.fastq.gz | wc -l
48
```

Treat `--config` as a tool for quick experiments, since a setting used only on a command line leaves no trace in your project. Results you want to keep should come from settings in a file.

10.3. How this pipeline organises its configuration

A project of this pipeline has its own `config/` folder, which is a copy of the one in the repository. So each project keeps its settings next to its results, and updating the pipeline doesn't change them. The settings are split by what they are about:

File	Content	How the Snakefile reads it
`config/config.yaml`	Paths to the other files, pipeline folder, temporary storage, assembler, resource sets	`--configfile` on the command line, as `config`

File	Content	How the Snakefile reads it
`config/samples.tsv`	The samples, libraries, reads, adapters and assemblies	pandas, as `samples`
`config/features.yaml`	Host genomes and database paths	yaml, as `features`
`config/params.yaml`	Every parameters of the tools	yaml, as `params`
`config/.docker.yaml`	The container of each module	yaml, as `docker`
`config/escalation.yaml`	Which resource sets each rule uses	yaml, as `ESCALATION_CONFIG`
`config/profiles/<cluster>/`	How jobs are submitted on this cluster	`--profile`

The main file holds the paths to the others, and the Snakefile loads them itself:

```
params = yaml.load(open(config["params-file"], "r"),
 ↪  Loader=yaml.SafeLoader)
docker = yaml.load(open(config["docker-file"], "r"),
 ↪  Loader=yaml.SafeLoader)
features = yaml.load(open(config["features-file"], "r"),
 ↪  Loader=yaml.SafeLoader)
samples = pd.read_table(config["sample-file"], comment="#",
 ↪  dtype="str")
```

So there is a global variable called `params`, in addition to the `params:` directive of each rule. These are different things, though they often appear on the same line:

```
    params:
        settings=params["assemble"]["drep"],
```

The left `params:` is the rule's directive. The right `params[...]` is the dictionary loaded from `params.yaml`. Chapter 59 describes each configuration file in detail, with all its options.

10.3.1. The default is a copy, and copies can grow old

Since each project has a copy of the configuration, a pipeline update that adds a new setting doesn't reach your existing project. The symptom is a Python `KeyError` when Snakemake starts:

```
KeyError in file "<pipeline>/workflow/rules/eukaryote/4cac.smk",
 ↪  line 90:
'eukaryote'
```

The fix is to copy the missing section from the pipeline's `config/` folder into your project's. The `RELEASE_NOTES` file in the repository lists such changes.

10.4. Summary

- Settings belong in configuration files, and code belongs in rules.
- Pass settings to commands through `params:`, so Snakemake can notice changes.
- This pipeline splits its settings over several files, and each project has its own copy.

11. Logs, benchmarks, threads and resources

So far our rules have printed their messages to the screen. On a cluster there is no screen, and many jobs run at once. This chapter adds what a production rule needs: a place for its messages, a record of what it used, and a statement of what it needs. The example is in `book/examples/snakemake/07_logs_threads/`.

11.1. A complete rule

```
rule sort_reads:
    """Sort the reads by sequence, in parallel"""
    input:
        "data/{sample}_R1.fastq.gz",
    output:
        "results/{sample}.txt",
    log:
        "logs/{sample}.log",
    benchmark:
        "benchmarks/{sample}.tsv",
    threads: 2
    resources:
        mem_mb=500,
        runtime=10,
    shell:
        """
        exec > {log} 2>&1
        echo "Sorting {wildcards.sample} with {threads} threads and
↪   {resources.mem_mb} MB"
        zcat {input} | paste - - - - | sort --parallel={threads} -S
↪   {resources.mem_mb}M -k 2 > {output}
        sleep 2
        echo "Done"
        """
```

Most rules in this pipeline have exactly this shape.

11.2. `log:` is where messages go

The `log:` directive names a file for the job's messages. It has to contain the same wildcards as the output, because each job needs its own log. Snakemake doesn't redirect anything on its own: the rule itself has to write to the file. Here, the line `exec > {log} 2>&1` from Chapter 1 does that for all commands that follow.

Log files differ from outputs in one important way: **a log file survives when the job fails**, whereas Snakemake deletes the output files of a failed job, since they may be broken. So your log is what you have to work with after a failure:

```
$ cat logs/sampleA.log
Sorting sampleA with 2 threads and 500 MB
Done
```

You will find three ways of writing to logs in this pipeline, and it is useful to know why:

```
# 1. Everything from here on goes to the log
exec > {log} 2>&1

# 2. Only messages of the tool go to the log, because its stdout is
 ↪  the result
exec 2> {log}
zcat {input} | bgzip > {output}

# 3. A log per attempt, kept in place when the attempt fails
try_log={log}.try{resources.attempt}
exec > "$try_log" 2>&1
...
mv "$try_log" {log}
```

The third form keeps the log of each failed attempt under its own name, such as `drep.log.try1`, and only a successful attempt becomes the plain `drep.log`. Chapter 15 explains attempts.

11.3. `benchmark:` measures what a job used

The `benchmark:` directive names a file where Snakemake stores what the job used. It repeats the measurement every few seconds and keeps the maximum:

```
$ cut -f 1,2,3,10 benchmarks/sampleA.tsv
s   h:m:s   max_rss cpu_time
2.0163  0:00:02 4.63    0.00
```

The file has more columns, and the end of `docs/06-Details.md` lists them all. The two that you will use most:

- `s` is the wall-clock time in seconds.
- `max_rss` is the peak memory in MB that the process really held in RAM.

Very short jobs may show `0` or `NA` here, because the measurement didn't catch them. On a real project these files tell you how to set the resources: if a rule that requests 100 GB peaks at 12 GB, you are wasting allocation and queue time. The pipeline's reports gather these files into tables for that purpose (Chapter 68).

11.4. `threads:` says how many cores a job can use

`threads:` tells Snakemake how many CPU cores a job needs. In the command, `{threads}` gives the number that it really got. Always pass it to the tool, because Snakemake doesn't limit what a program does: if the tool starts 40 threads on its own while the job asks for 2, the job will still use 40, slow down its neighbours, and might get killed.

Snakemake uses this number for its planning. If you allow `--cores 4`, two jobs with 2 threads each run at once. If you allow fewer cores than a rule claims, it scales the rule down:

```
$ snakemake --cores 1 --forceall --quiet rules
[...]
Provided cores: 1 (use --cores to define parallelism)
Rules claiming more threads will be scaled down.
[...]
```

You can change the number for a rule from the command line, which is handy when you try things out on a cluster. Remember to put the target first, as discussed in Chapter 6:

```
$ snakemake results/sampleA.txt --cores 4 --forceall --dry-run
 ↪  --set-threads sort_reads=4 | grep -E 'threads|resources'
    threads: 4
    resources: tmpdir=<TBD>, mem_mb=500, mem_mib=477, runtime=10
```

11.5. `resources:` says what else it needs

`resources:` holds anything else a job requires. Two names have a defined meaning on clusters:

Name	Meaning
`mem_mb`	Memory in megabytes
`runtime`	Maximum run time in minutes

There are others for specific systems, such as `slurm_partition`, `slurm_account` and `gres`, and Chapter 15 covers them.

Run locally, these are only used for planning: `--resources mem_mb=1000` lets Snakemake start only as many jobs as fit within 1000 MB in total. You can also make up your own. The pipeline uses this to limit how many metaSPAdes jobs run at the same time, because these put a heavy load on a shared disk. The rule says:

```
    resources:
        metaspades_slots=1,
```

and the run script says how many such slots exist in total:

```
snakemake ... --resources metaspades_slots=$metaspades_slots
```

Snakemake then never starts more metaSPAdes jobs at once than that number.

11.5.1. Resources that depend on the attempt

A resource can be a function, and it is called with the wildcards and the number of the attempt. This makes it possible to ask for more the second time:

```
    resources:
        mem_mb=lambda wildcards, attempt: 8000 * attempt,
```

This idea is the basis of the pipeline's resource escalation, and Chapter 17 describes how it works there.

11.6. The anatomy of a rule in this pipeline

With that, you can read a real rule. Here is one from the pipeline, with a comment on each directive:

```
rule assemble__drep__catalogue:                    #
 ↪  module__tool__step
    """Concatenate the dereplicated genomes into the MAG
     ↪  catalogue"""
    input:
        MAG_GENOMES,                               # a path constant
 ↪  from folders.smk
    output:
        MAG_CATALOGUE,
    log:
        DREP / "dereplicated_genomes.log",
    benchmark:
        DREP / "benchmark/join_genomes.tsv"
    container:
        docker["assemble"]                         # the software, see
 ↪  the chapter on containers
    threads: esc("cpus", "assemble__drep__catalogue")    # from the
 ↪  resource sets
    resources:
        runtime=esc("runtime", "assemble__drep__catalogue"),
        mem_mb=esc("mem_mb", "assemble__drep__catalogue"),
        cpus_per_task=esc("cpus", "assemble__drep__catalogue"),
        slurm_partition=esc("partition",
 ↪  "assemble__drep__catalogue"),
        gres=lambda wc, attempt: f"{get_resources(wc, attempt,
 ↪  'assemble__drep__catalogue')['nvme']}",
        attempt=esc("attempt", "assemble__drep__catalogue"),
    shell:
        """
        exec 2> {log}
        debug "genomes={input} threads={threads}
 ↪  catalogue={output}"
        zcat {input}/*.fa.gz | bgzip -l 9 -@ {threads} > {output}
        """
```

Don't worry about `esc` yet. It looks up a value for this rule in the configured resource sets. The command itself is the last line, and everything else is there to make it work on a cluster.

11.7. Summary

- `log:` gives each job a file for messages, which survives a failure.
- `benchmark:` records time and memory.
- `threads:` and `resources:` state what a job needs, and you pass `{threads}` to the tool.
- Resources can be functions of the attempt.

12. Special kinds of output files

By default, Snakemake treats an output as a normal file that you want to keep. A real pipeline also has temporary files, outputs that are whole folders and steps that produce no file at all. **Flags** tell Snakemake about these. The example is in `book/examples/snakemake/08_file_flags/`.

12.1. The example

```
rule all:
    input:
        "results/final.txt",
        "results/bins",
        "results/checkpoint.done",


rule unpack_reads:
    """An intermediate file: Snakemake deletes it when nothing
    ↪ needs it any more"""
    input:
        "data/sampleA_R1.fastq.gz",
    output:
        temp("results/sampleA.fastq"),
    shell:
        "zcat {input} > {output}"


rule final_result:
    """A protected file: it becomes read-only, so it cannot be
    ↪ deleted by accident"""
    input:
        "results/sampleA.fastq",
    output:
        protected("results/final.txt"),
    shell:
```

```
        "wc -l {input} > {output}"


rule make_bins:
    """A tool that writes an unknown number of files into one
    ↪ folder"""
    input:
        "data/sampleB_R1.fastq.gz",
    output:
        directory("results/bins"),
    shell:
        """
        mkdir -p {output}
        zcat {input} | split -l 100 - {output}/bin_
        """


rule mark_done:
    """A tool without a useful output file of its own: touch()
    ↪ creates an empty one on success"""
    input:
        "results/bins",
    output:
        touch("results/checkpoint.done"),
    shell:
        "ls {input} | wc -l"
```

Running it shows what Snakemake does with these flags (shortened):

```
$ snakemake --cores 1
[...]
localrule mark_done:
    input: results/bins
    output: results/checkpoint.done
    jobid: 4
    reason: Missing output files: results/checkpoint.done; Input
↪   files updated by another job: results/bins
    resources: tmpdir=/tmp
Touching output file results/checkpoint.done.
[...]
localrule final_result:
    input: results/sampleA.fastq
```

```
    output: results/final.txt
    jobid: 1
    reason: Missing output files: results/final.txt; Input files
 ↪  updated by another job: results/sampleA.fastq
    resources: tmpdir=/tmp
Write-protecting output file results/final.txt.
[...]
4 of 5 steps (80%) done
Removing temporary output results/sampleA.fastq.
[...]

$ ls -l results | cut -c1-11,25-
total 8
drwxr-xr-x 096 Nov  3 16:56 bins
-rw-r--r--   0 Nov  3 16:56 checkpoint.done
-r--r--r--  26 Nov  3 16:56 final.txt

$ ls results/bins
bin_aa
bin_ab
bin_ac
```

Let us go through them.

12.2. `temp()`: delete when no longer needed

A `temp()` file is removed as soon as all jobs that need it have finished. In the output above it says "Removing temporary output results/sampleA.fastq". This keeps intermediate files from filling your disk, and in metagenomics they can reach terabytes.

The pipeline uses it for intermediate alignments and reads: the CRAM files of each host mapping and the reads left after each host are temporary, and only the final decontaminated reads stay.

Here is a behaviour you need to know about: if a temporary file is gone, Snakemake doesn't rerun its job for that reason alone, as long as the results that depend on it are current. Only when a later step needs it again does Snakemake make the temporary file again. So this can happen: you add a new rule that reads a temporary file, or you force a rerun of one step. Snakemake then quietly reruns a long chain of earlier jobs to recreate it, which is one more reason to do a dry run first.

For debugging, `--notemp` keeps all temporary files.

12.3. `directory()`: the output is a folder

Some tools write an unknown number of files into a folder. You can't list those in advance, so the folder is the output, marked with `directory()`. The binners in this pipeline work this way, as does dRep with its `dereplicated_genomes` folder:

```
    output:
        directory(CONCOCT / "{assembly_id}"),
```

There are some rules for this:

- Your rule has to create the folder itself, unlike the parent folders of normal outputs.
- Other rules can use the folder as an input, as `mark_done` does here. They can't ask for a single file inside it by name, because no rule is known to produce that file.
- When a job fails or needs to rerun, Snakemake deletes the whole folder first.
- A folder's date changes when files are added or removed inside it. This can cause unexpected reruns if a tool writes there later.

If the number of files decides which jobs come next, a folder isn't enough, and you need a checkpoint (Chapter 16).

12.4. `touch()`: a marker file for a step with no file output

Some steps have no output file you could name, or their outputs are too many and too unpredictable. `touch()` makes Snakemake create an empty file when the command succeeds, and later rules can depend on it.

The pipeline uses this for bowtie2 indexes. `bowtie2-build` writes several files with fixed endings next to a prefix. Instead of listing them all, the rule's output is a marker file with the name of the prefix:

```
    output:
        index=touch(MAG_INDEX / "dereplicated_genomes"),
    shell:
        "bowtie2-build --threads {threads} {input} {output.index}"
```

Nonpareil uses it for another reason: on samples with too little data it doesn't produce its files, and an empty file then means "no result" without making the whole workflow fail.

12.5. `protected()`: guard against accidents

A `protected()` file becomes read-only after the job. In the listing above, `final.txt` has the permissions `-r--r--r--`. Snakemake refuses to overwrite it, so a mistaken rerun can't destroy a result that took three weeks to compute. To rerun it you first have to delete it by hand. This pipeline doesn't currently use this flag, but it is a good idea for your own expensive results.

12.6. `ancient()`: don't rerun because of the date

`ancient()` goes on an input, and tells Snakemake to ignore that file's date. This is for reference files or databases, where touching the file shouldn't invalidate all your results:

```
    input:
        reads="results/{sample}.fq.gz",
        database=ancient("resources/databases/kraken2"),
```

12.7. `pipe()`: a stream between two jobs

`pipe()` connects two jobs through a stream instead of a file. Both have to run at the same time on the same machine, so it is rarely useful on a cluster. This pipeline doesn't use it, and it pipes inside one rule's command instead.

12.8. `multiext()`: one prefix with several endings

Tools often produce several files with a common prefix. `multiext()` is a short form for listing them:

```
    output:
        multiext("results/index", ".1.bt2", ".2.bt2", ".3.bt2",
↪   ".4.bt2"),
```

12.9. Summary

Flag	Goes on	Effect
`temp()`	Output	Deleted when nothing needs it any more
`directory()`	Output	The output is a folder
`touch()`	Output	An empty file is created on success
`protected()`	Output	Becomes read-only
`ancient()`	Input	Its date doesn't trigger reruns
`pipe()`	Output	A stream between two jobs
`multiext()`	Input, output	Several files with one prefix

13. When things go wrong

You will spend more time with failed runs than with successful ones, and that's normal. Most errors fall into a few families, and once you recognise them they take seconds to solve. This chapter provokes each of them on purpose. The example is in `book/examples/snakemake/09_errors/`, with one small Snakefile per mistake.

13.1. A way of reading errors

Snakemake's messages get long, and the useful line can be anywhere in them. Three habits help:

1. **Read the first error, not the last line.** The final lines usually only say that something failed.
2. **Look for the name of the exception**, the word ending in `Error` or `Exception`. It tells you the family.
3. **Find the job's own log file.** The message from Snakemake says that a command failed, and the log says why.

13.2. Errors in the Snakefile

These appear immediately, before any job runs.

13.2.1. A typo in a keyword

```
$ snakemake -s Snakefile.typo --cores 1
SyntaxError in file
 ↪  "/scratch/project_demo/examples/09_errors/Snakefile.typo", line
 ↪  3:
Unexpected keyword outptu in rule definition:
None
[exit code 1]
```

The file and line number are right there. Syntax errors usually are this helpful, although the line number can point to the line after the true mistake, for example after a missing comma or quote. Bad indentation falls into this family too.

13.2.2. Single braces in a command

This rule uses `awk` with single braces:

```
    shell:
        "zcat {input} | awk '{print $1}' | wc -l > {output}"
```

```
$ snakemake -s Snakefile.braces --cores 1
[...]
RuleException in rule count in file
 ↪  "/scratch/project_demo/examples/09_errors/Snakefile.braces",
 ↪  line 1:
NameError: The name 'print $1' is unknown in this context. Please
 ↪  make sure that you defined that variable. Also note that braces
 ↪  not used for variable access have to be escaped by repeating
 ↪  them, i.e. {{print $1}}
zcat {input} | awk '{print $1}' | wc -l > {output}
[exit code 1]
```

Snakemake took `{print $1}` for a variable, and tells you exactly what to do. You will make this mistake, probably more than once. The error only appears when the job is about to run, so it may come hours into a workflow. A dry run with `-p` catches it early, because it has to build the command to print it.

13.3. Errors in the plan

These appear while Snakemake builds the DAG, so a dry run shows them too.

13.3.1. `MissingInputException`

Snakemake found a rule for your file, but the input doesn't exist and no rule can make it. See Chapter 6. Check the path and sample names.

13.3.2. `MissingRuleException`

No rule can make a file with this name. Also see Chapter 6. Compare the name character by character with the rule's output pattern.

13.3.3. `AmbiguousRuleException`

Two rules can make the same file, and Snakemake won't guess:

```
$ snakemake -s Snakefile.ambiguous --cores 1
[...]
AmbiguousRuleException:
Rules from_forward and from_reverse are ambiguous for the file
 ↪  results/sampleA.txt.
Consider starting rule output with a unique prefix, constrain your
 ↪  wildcards, or use the ruleorder directive.
Wildcards:
    from_forward: sample=sampleA
    from_reverse: sample=sampleA
Expected input files:
    from_forward: data/sampleA_R1.fastq.gz
    from_reverse: data/sampleA_R2.fastq.gz
Expected output files:
    from_forward: results/sampleA.txt
    from_reverse: results/sampleA.txt
[exit code 1]
```

The usual solution is to give each rule its own output folder. This pipeline does that consistently, with one folder per tool. It uses `wildcard_constraints` where patterns could still overlap.

13.4. Errors while a job runs

13.4.1. The command failed

Here is a rule whose command fails on purpose:

```
rule count:
    input:
        "data/sampleA_R1.fastq.gz",
```

```
    output:
        "results/count.txt",
    log:
        "logs/count.log",
    shell:
        """
        exec > {log} 2>&1
        echo "Working on {input}"
        zcat {input} > {output}
        echo "Something goes wrong now"
        false
        """
```

```
$ snakemake -s Snakefile.fails --cores 1
[...]
Error in rule count:
    message: None
    jobid: 0
    input: data/sampleA_R1.fastq.gz
    output: results/count.txt
    log: logs/count.log (check log file(s) for error details)
    shell:

        exec > logs/count.log 2>&1
        echo "Working on data/sampleA_R1.fastq.gz"
        zcat data/sampleA_R1.fastq.gz > results/count.txt
        echo "Something goes wrong now"
        false

        (command exited with non-zero exit code)
Removing output files of failed job count since they might be
 ↪  corrupted:
results/count.txt
[...]
WorkflowError:
At least one job did not complete successfully.
[exit code 1]
```

The message gives you the rendered command and the log's path. Two things to take from this:

Snakemake removes the output of a failed job. The file `results/count.txt` was written, and it is deleted because it may be incomplete. If you want to look at it for

debugging, run with `--keep-incomplete`.

The log file stays, and it has the answer:

```
$ cat logs/count.log
Working on data/sampleA_R1.fastq.gz
Something goes wrong now
```

A useful debugging step comes out of this as well: you can copy the command from the message and run it yourself, in the same folder. On a cluster, ask for an interactive session first. If the rule uses a container, run it in that too, as Chapter 3 shows.

13.4.2. The command worked, but the output is missing

This rule writes to a different file than it promised:

```
    output:
        "results/count.txt",
    shell:
        "zcat {input} | wc -l > results/typo.txt"
```

```
$ snakemake -s Snakefile.nooutput --cores 1 --latency-wait 1
[...]
Waiting at most 1 seconds for missing files:
results/count.txt (missing locally)
MissingOutputException in rule count in file
 ↪  "/scratch/project_demo/examples/09_errors/Snakefile.nooutput",
 ↪  line 1:
Job 0 completed successfully, but some output files are missing.
 ↪  Missing files after 1 seconds. This might be due to filesystem
 ↪  latency. If that is the case, consider to increase the wait
 ↪  time with --latency-wait:
results/count.txt (missing locally, parent dir contents: typo.txt)
[...]
```

This is helpful: it lists what *is* in the folder. On a cluster it can have two causes. Either the tool named the file differently than you expected, for example by adding its own ending. Or the shared disk is slow, and the file hasn't appeared yet, which is why the pipeline uses `--latency-wait 60`. If a file shows up when you look a minute later, it was the latter.

13.5. After a crash: incomplete files and locks

What happens when Snakemake itself dies, because of a lost connection, a killed login session or the end of a time limit? We can simulate this by killing it on purpose. This rule takes eight seconds:

```
rule slow:
    output:
        "results/slow.txt",
    shell:
        """
        echo "started" > {output}
        sleep 8
        echo "finished" >> {output}
        """
```

We start it, and kill Snakemake after four seconds with `kill -9`, which gives it no chance to clean up:

```
$ snakemake -s Snakefile.slow --cores 1 > /dev/null 2>&1 &
  sleep 4
  kill -9 $!
  sleep 1
  cat results/slow.txt
started
```

An output exists, and it is incomplete. A normal failure would have removed it, but this time nothing could. So what does the next run do?

```
$ snakemake -s Snakefile.slow --cores 1
Assuming unrestricted shared filesystem usage.
host: localhost
Building DAG of jobs...
IncompleteFilesException:
The files below seem to be incomplete. If you are sure that certain
 ↪  files are not incomplete, mark them as complete with

    snakemake --cleanup-metadata <filenames>

To re-generate the files rerun your command with the
 ↪  --rerun-incomplete flag.
Incomplete files:
```

```
results/slow.txt
[exit code 1]
```

Snakemake marks an output as "in progress" when a job starts, and removes the mark when the job finishes. So it knows that this file is suspicious, and won't build on it. We follow its advice:

```
$ snakemake -s Snakefile.slow --cores 1 --rerun-incomplete
Building DAG of jobs...
LockException:
Error: Directory cannot be locked. Please make sure that no other
 ↪ Snakemake process is trying to create the same files in the
 ↪ following directory:
/scratch/project_demo/examples/09_errors
If you are sure that no other instances of snakemake are running on
 ↪ this directory, the remaining lock was likely caused by a kill
 ↪ signal or a power loss. It can be removed with the --unlock
 ↪ argument.
[exit code 1]
```

This is the second consequence. A running Snakemake locks its folder, so that two runs can't write the same files. The killed one couldn't remove its lock. **Before you unlock, check that no Snakemake is really still running**, for example in a tmux session you forgot about. Once you are sure:

```
$ snakemake -s Snakefile.slow --unlock
Building DAG of jobs...
Unlocked working directory.

$ snakemake -s Snakefile.slow --cores 1 --rerun-incomplete --quiet
[...]

$ cat results/slow.txt
started
finished
```

There is a further lesson here. Between the kill and the unlock, the example waits for six seconds, and when we look at the file during that wait, it says "started, finished". The killed Snakemake's command had kept running on its own, and finished writing the file. **Killing Snakemake doesn't kill the jobs it started.** On a cluster, its SLURM jobs also keep running. So look with `squeue` and cancel them with `scancel` before starting again. If you don't, two jobs may write the same file.

To stop a run cleanly, press Ctrl-C once and wait. Snakemake then cancels its jobs and cleans up.

13.6. When nothing goes wrong, but something did: strict mode

The worst kind of failure gives no error message at all. Chapter 1 explained that Snakemake runs commands in strict mode, with `set -euo pipefail`, so that a failing command stops the rule. It does this by putting those words in front of each command, and that makes it easy to lose by accident.

The directive `shell.prefix` puts something of your own in front of each command. It looks harmless, and it is a convenient way to give all rules a helper function. But it **replaces** Snakemake's default prefix instead of adding to it:

```
# This replaces Snakemake's default prefix, and with it the strict
↪ mode
shell.prefix('debug() {{ echo "[DEBUG] $*" >&2; }}; ')


rule silent_failure:
    output:
        "results/silent.txt",
    shell:
        """
        false | cat > {output}
        echo "The first line failed, and this still runs" >>
↪ {output}
        """
```

The first command fails, the rule continues, and Snakemake reports a success:

```
$ snakemake -s Snakefile.prefix --cores 1 --quiet
[...]
$ cat results/silent.txt
The first line failed, and this still runs
```

Imagine that this first command is an assembler that ran out of memory, and the second line moves a log file. The job counts as `COMPLETED`, the next rules start on an incomplete result, and nothing tells you. The only trace is in the rule's log.

The fix is to repeat the strict mode in your prefix:

```
shell.prefix('set -euo pipefail; debug() {{ echo "[DEBUG] $*" >&2;
 ↪  }}; ')
```

Now the same rule fails, as it should, and the second line never runs:

```
$ snakemake -s Snakefile.strict --cores 1 --quiet 2>&1 | grep -E
 ↪  'Error in rule|exit status' | sort -u
        echo "This never runs" >> results/strict.txt' returned
 ↪  non-zero exit status 1.

$ ls results/strict.txt || true
ls: cannot access 'results/strict.txt': No such file or directory
```

This isn't a made-up example. This pipeline had that bug from September to November 2026, and the `RELEASE_NOTES` describe how to check results from such a version. A dry run can't find this kind of bug, and neither can a stand-in tool that never fails. It takes a test in which something fails on purpose (Chapter 18).

13.7. What strict mode doesn't see: `>(...)`

There's one more kind of silent failure, and it's a nasty one. Chapter 1 introduced process substitution. It's fine for reading, as in `<(zcat file)`. For writing, it hides a trap:

```
samtools fastq -1 >(pigz > reads_1.fq.gz) -2 >(pigz >
 ↪  reads_2.fq.gz) input.bam
```

Bash starts the two `pigz` processes on the side, and **it doesn't wait for them**. When `samtools` finishes, the command counts as finished, while `pigz` may still be writing. Strict mode doesn't help, since Bash doesn't look at their exit status either. So the job ends, Snakemake finds the output files, and everything looks fine. But if the job's processes are then stopped, the file is cut off.

This pipeline had exactly this bug until version 0.4.69. On a busy cluster, it would only have struck now and then, and it was only found when a real run in a container failed every time: a container stops all its processes the moment the main one ends.

When you write a rule, write files in one of these ways:

```
# The tool compresses by itself
samtools fastq -c 9 -1 reads_1.fq.gz -2 reads_2.fq.gz input.bam
```

```
# Or a plain pipe, which Bash waits for, and which strict mode
 ↪  checks
kraken2 --db db reads.fq | pigz > output.gz
```

`gzip -t file.gz` tells you whether a compressed file is complete, and it's a good check if you ever doubt a result.

13.8. Why did this job run again?

The most common surprise isn't an error: a dry run wants to redo work that you consider finished. The `reason` line tells you why. Since Snakemake 7.8, a job can rerun for any of these **rerun triggers**:

Trigger	When it fires
`mtime`	An input is newer than the output, or an output is missing
`params`	A value in `params:` changed
`input`	The list of input files changed
`software-env`	The container or conda environment changed
`code`	The rule's command or code changed

We change one word in a rule's `params:`:

```
$ sed -i 's/label="reads"/label="sequences"/' Snakefile.good
$ snakemake -s Snakefile.good --cores 1 --dry-run
[...]
rule count:
    input: data/sampleA_R1.fastq.gz
    output: results/count.txt
    jobid: 1
    reason: Params have changed since last execution: Union of
 ↪  exclusive params before and now across all output: before:
 ↪  'reads' now: 'sequences'
[...]
Some jobs were triggered by provenance information, see 'reason'
 ↪  section in the rule displays above.
If you prefer that only modification time is used to determine
 ↪  whether a job shall be executed, use the command line option
 ↪  '--rerun-triggers mtime' (also see --help).
```

```
If you are sure that a change for a certain output file (say,
 ↪  <outfile>) won't change the result (e.g. because you just
 ↪  changed the formatting of a script or environment definition),
 ↪  you can also wipe its metadata to skip such a trigger via
 ↪  'snakemake --cleanup-metadata <outfile>'.
Rules with provenance triggered jobs: count
[...]
```

This protects you from outdated results, but you sometimes know better. After a pipeline update that only reformatted some code, you don't want to redo weeks of assembly. There are two ways out.

Use only the file dates to decide, for this run:

```
$ snakemake -s Snakefile.good --cores 1 --dry-run --quiet
 ↪  --rerun-triggers mtime
host: localhost
Building DAG of jobs...
Nothing to be done (all requested files are present and up to
 ↪  date).
```

Or tell Snakemake that the existing files are fine, with `--touch`. This updates dates and records, and runs nothing:

```
$ snakemake -s Snakefile.good --cores 1 --touch --quiet
[...]
$ snakemake -s Snakefile.good --cores 1 --dry-run --quiet
host: localhost
Building DAG of jobs...
Nothing to be done (all requested files are present and up to
 ↪  date).
```

Only do this if you are sure that the existing results are still what you want. `--touch` will mark even wrong results as current, and nothing will warn you later.

13.9. A checklist for a failed run

1. Find the first `Error in rule` in Snakemake's output, or at the end of the newest file in `.snakemake/log/`.
2. Read the job's log file, which the message names.
3. On a cluster, read the SLURM log as well. Its path is in the line "has been submitted with SLURM jobid".

4. Ask `sacct` how the job ended. `OUT_OF_MEMORY` and `TIMEOUT` mean that it needs more resources (Chapter 2).
5. Copy the rendered command, and run it by hand in an interactive session.
6. After fixing it, run a dry run, and then start again. Add `--rerun-incomplete` if Snakemake asks for it.

Chapter 69 collects the problems seen with this pipeline's tools.

13.10. Summary

Message	Meaning	What to do
`SyntaxError`	A typo in the Snakefile	Look at the given line and the one before
`NameError: ... is unknown`	Single braces in a command	Double them
`MissingInputException`	Input is missing, and no rule makes it	Check paths and sample names
`MissingRuleException`	No rule makes this file	Compare with the output patterns
`AmbiguousRuleException`	Two rules make the same file	Give each its own folder or constrain wildcards
`MissingOutputException`	The command worked, but no output	Check the name, raise `--latency-wait`
`IncompleteFilesException`	A file from an interrupted run	`--rerun-incomplete`
`LockException`	Another or a dead Snakemake	Check, then `--unlock`
No message, but a wrong result	Strict mode lost through `shell.prefix`	Repeat `set -euo pipefail` in the prefix

14. Structuring a large workflow

A Snakefile with 170 rules would be unreadable. So you split a workflow into files, with a folder structure that shows how its parts relate. This chapter uses the same structure as the real pipeline, in small. The example is in `book/examples/snakemake/10_structure/`.

14.1. The layout

```
$ tree workflow config
workflow
   Snakefile
   rules
       folders.smk
       qc
           __main__.smk
           filter.smk
       stats
           __main__.smk
           count.smk
config
   config.yaml

5 directories, 7 files
```

Snakemake looks for `Snakefile` and for `workflow/Snakefile`, so you don't have to name it on the command line. The ending `.smk` is a convention for files with rules. The real pipeline has the same structure, with a folder per module under `workflow/rules/`:

```
workflow/rules/
    folders.smk           the paths of all results
    helpers/              rules that several modules use
    reads/                one folder per module
    reference/
    preprocess/
```

```
read_annotate/
assemble/
quantify/
mag_annotate/
contig_annotate/
eukaryote/
report/
```

14.2. `include:` puts files together

`include:` reads another file as if its text stood at that place. So the main Snakefile stays short:

```
configfile: "config/config.yaml"

SAMPLES = config["samples"]


include: "rules/folders.smk"
include: "rules/qc/__main__.smk"
include: "rules/stats/__main__.smk"


rule all:
    input:
        rules.qc.input,
        rules.stats.input,
```

Three things to note here:

- A path in `include:` is relative to the file with the `include:` line. That's different from input and output files, whose paths are relative to the folder where you start Snakemake.
- The order matters for plain Python: `SAMPLES` has to be defined before an included file uses it, and `rules.qc` must be defined before `rule all` refers to it. The order of rules doesn't matter otherwise, since Snakemake connects them through file names.
- All included files share one namespace. A variable or function defined in one is visible in all the others, and two functions with the same name silently overwrite each other. That has happened in this pipeline, which is why helper names are specific.

14.3. A module is a folder with a `__main__.smk`

In this pipeline, each module's folder has a file `__main__.smk`, which includes the tool files and defines a target rule named after the module:

```
include: "filter.smk"


rule qc:
    """Run the whole qc module"""
    input:
        expand(QC / "{sample}.fastq.gz", sample=SAMPLES),
```

and each tool has a file with its rules:

```
rule qc__filter:
    input:
        "data/{sample}_R1.fastq.gz",
    output:
        QC / "{sample}.fastq.gz",
    shell:
        """
        zcat {input} | paste - - - - | awk -F '\\t' 'length($2) >=
↪   50' | tr '\\t' '\\n' | gzip > {output}
        """
```

The word "module" here means a folder of rules by this pipeline's own convention. Snakemake also has a feature called `module`, which imports a whole foreign workflow, and this pipeline doesn't use it.

14.4. Rule names say where a rule lives

The rule names follow a scheme: **module, two underscores, tool, two underscores, step**. In the real pipeline, `assemble__magscot__hmmsearch` is the step `hmmsearch` of the tool `magscot` in the module `assemble`.

This pays off when you run things:

```
$ snakemake --list-rules
all
qc (Run the whole qc module)
```

```
qc__filter
stats (Run the whole stats module)
stats__count

$ snakemake qc --cores 2 --quiet rules
[...]
Job stats:
job           count
----------  -------
qc                1
qc__filter        2
total             3
[...]

$ snakemake stats --cores 2 --quiet rules
[...]
Job stats:
job             count
------------  -------
stats               1
stats__count        2
total               3
[...]

$ snakemake --cores 2 --quiet rules
[...]
Nothing to be done (all requested files are present and up to
 ↪  date).
```

The same target-rule idea exists on three levels in the real pipeline:

Level	Example target	Runs
Whole pipeline	`all`	Everything except opt-in modules
Module	`assemble`	All tools of the module
Tool	`assemble__megahit`	One tool and everything it needs

14.5. Paths in one place

The file `folders.smk` defines all results folders as constants:

```
QC = Path("results/qc")
STATS = Path("results/stats")
```

Rules then write `QC / "{sample}.fastq.gz"`. In the pipeline, this file also defines the paths that several rules share, such as the following. Each is then written exactly once, so rules can't drift apart:

```
# Contigs of one assembly: a pipeline assembler writes them here,
 ↪  self-provided ones are expected here
CONTIGS = ASSEMBLE / config["assembler"] / "{assembly_id}.fa.gz"

MAG_GENOMES = DREP / "dereplicated_genomes"
MAG_CATALOGUE = DREP / "dereplicated_genomes.fa.gz"
```

Whichever assembler you choose, downstream rules simply use `CONTIGS`. The constant `LIBRARY = "{sample_id}.{library_id}"` makes the common part of per-library paths shorter:

```
MAG_CRAM = MAG_MAPPING / f"{LIBRARY}.cram"
```

That is an f-string whose value contains wildcards, which Snakemake fills in later.

14.6. Where to run from: the project folder

One decision shapes how you use this pipeline: **the code and the data live in separate folders**. The pipeline's code sits once in a folder such as `~/git/Snakebite-Holoruminant-MetaG`, and each project has its own folder with its config, reads and results. You start Snakemake in the project folder, and point it to the code:

```
cd /scratch/project_2009831/my_project
snakemake -s
 ↪  /users/fischerd/git/Snakebite-Holoruminant-MetaG/workflow/Snakefile
 ↪  --configfile config/config.yaml ...
```

Relative paths in rules like `results/...` and `config/...` therefore refer to the project. The run script hides this detail (Chapter 17). Rule files are found through the Snakefile's location, while scripts are found through the setting `pipeline_folder` in `config.yaml`, which is what `SCRIPT_FOLDER` is built from.

14.7. Summary

- `include:` splits a workflow into files, which all share one namespace.
- In this pipeline, each module is a folder with a `__main__.smk` and one file per tool.
- Rule names follow `module__tool__step`, and target rules exist per tool and per module.
- `folders.smk` keeps each path in one place.
- Code and project are separate folders.

15. Running on the cluster

Everything so far ran on one machine. Now Snakemake gets to do what it was made for: sending each job to SLURM. You will see that your rules don't change at all. The example is in `book/examples/snakemake/12_cluster/`, and everything in this chapter ran on the mini cluster with real SLURM.

15.1. How it works

Snakemake itself stays on the login node, and acts as a dispatcher. For each job it is ready to start, it runs `sbatch`. It then checks the status of the jobs every so often, and when a job has finished, it starts those that were waiting for it.

Inside each SLURM job runs **another Snakemake**. It reads your whole Snakefile again, and runs only the one job it was sent for. So there are two kinds of Snakemake processes, and they need the same files and settings. This will become important at the end of this chapter.

15.2. Profiles: settings for the cluster

Cluster options would make for long command lines, so they go in a **profile**: a folder with a `config.yaml`, where each option of the command line can be written down:

```
executor: slurm
jobs: 10
latency-wait: 5
default-resources:
    slurm_account: "project_demo"
    slurm_partition: "small"
    runtime: 10
    mem_mb: 200
    cpus_per_task: 1
```

Setting	Meaning
`executor: slurm`	Submit jobs through SLURM. This needs the plugin `snakemake-executor-plugin-slurm`
`jobs: 10`	At most 10 jobs at once, waiting ones included
`latency-wait: 5`	Wait this many seconds for files to appear on the shared disk
`default-resources`	Used by every rule that doesn't say otherwise

`--profile profile` loads it. With a cluster, `--jobs` replaces `--cores`.

15.3. The rules stay the same

```
SAMPLES = ["sampleA", "sampleB", "sampleC"]


rule all:
    input:
        "results/summary.txt",


rule count_reads:
    input:
        "data/{sample}_R1.fastq.gz",
    output:
        "results/{sample}.txt",
    threads: 2
    resources:
        mem_mb=300,
        cpus_per_task=2,
        runtime=5,
    shell:
        "zcat {input} | wc -l > {output}"


rule summary:
    input:
        expand("results/{sample}.txt", sample=SAMPLES),
```

```
    output:
        "results/summary.txt",
    shell:
        "paste {input} > {output}"
```

Only `resources:` has grown. Here is how those values reach SLURM:

In the rule	Becomes in `sbatch`
`resources: mem_mb=300`	`--mem 300`
`resources: runtime=5`	`-t 5` (minutes)
`resources: cpus_per_task=2`	`--cpus-per-task=2`
`resources: slurm_partition="small"`	`-p small`
`resources: slurm_account="x"`	`-A x`
`resources: gres="nvme:10"`	`--gres=nvme:10`

Set both `threads:` and `cpus_per_task`, since `threads` only tells Snakemake and your command how many to use, whereas `cpus_per_task` is what SLURM reserves.

Let's run it:

```
$ snakemake --profile profile --quiet rules
 ↪  --slurm-keep-successful-logs
Using profile profile for setting default command line arguments.
host: localhost
Building DAG of jobs...
SLURM run ID: f41cde61-c9ba-4ed2-ab8a-110900e1c2df
Using shell: /usr/bin/bash
Provided remote nodes: 10
Job stats:
job            count
-----------  -------
all                1
count_reads        3
summary            1
total              5

Select jobs to execute...
Execute 3 jobs...
Job 4 has been submitted with SLURM jobid 17 (log:
 ↪  .snakemake/slurm_logs/rule_count_reads/sampleC/17.log).
Job 3 has been submitted with SLURM jobid 18 (log:
 ↪  .snakemake/slurm_logs/rule_count_reads/sampleB/18.log).
```

```
Job 2 has been submitted with SLURM jobid 19 (log:
 ↪  .snakemake/slurm_logs/rule_count_reads/sampleA/19.log).
[...]
Job 1 has been submitted with SLURM jobid 20 (log:
 ↪  .snakemake/slurm_logs/rule_summary/20.log).
[...]
5 of 5 steps (100%) done

$ sacct -X
 ↪  --format=JobID,Comment%24,Partition,AllocCPUS,ReqMem,State,ExitCode
 ↪  | tail -n 4
17           rule_count_reads_wildca+      small          2
 ↪  300M  COMPLETED      0:0
18           rule_count_reads_wildca+      small          2
 ↪  300M  COMPLETED      0:0
19           rule_count_reads_wildca+      small          2
 ↪  300M  COMPLETED      0:0
20                       rule_summary      small          1
 ↪  200M  COMPLETED      0:0
```

Each `count_reads` job got the 2 CPUs and 300 MB it asked for, and `summary` got the default values from the profile. The name of the rule is in SLURM's `Comment` field, since the job name is a random run ID. To see it in `sacct`, ask for `Comment`, and widen the column as needed.

Notice that `rule all` wasn't submitted. It has no command, so it runs locally. You can also mark quick rules with `localrules: myrule` to keep them off the queue.

15.4. Where to look: three kinds of logs

There are now three places where messages end up, and each has something the others don't:

File	What it holds
`.snakemake/log/<date>.snakemake.log`	Snakemake's own log: what was submitted and when, and what failed
`.snakemake/slurm_logs/<rule_name>/<jobid>.log`	One SLURM job, including `sbatch` for Snakemake and any crash of it
The rule's own `log:`	What the tool said

Normally you start with the last one. If it is empty or stops suddenly, read the SLURM

log, which shows whether the job was killed. The executor plugin removes the SLURM logs of successful jobs by default, and `--slurm-keep-successful-logs` keeps them.

15.5. When a job fails: retries and attempts

A job can fail because it needs more resources. Snakemake can then try again, asking for more each time. Here is a rule that simulates this. It fails whenever it gets less than 400 MB:

```
def more_memory(wildcards, attempt):
    return 200 * attempt


rule needs_more_memory:
    output:
        "results/retried.txt",
    resources:
        mem_mb=more_memory,
        attempt=lambda wildcards, attempt: attempt,
    shell:
        """
        echo "Attempt {resources.attempt} with {resources.mem_mb}
 ↪  MB"
        if [ {resources.mem_mb} -lt 400 ]; then
            echo "Not enough memory" >&2
            exit 1
        fi
        echo "Succeeded with {resources.mem_mb} MB" > {output}
        """
```

With the command-line option `--retries 2`, this is what happens:

```
$ snakemake --snakefile Snakefile.retries --profile profile
 ↪  --retries 2 --quiet rules
[...]
Job 1 has been submitted with SLURM jobid 21 (log:
 ↪  .snakemake/slurm_logs/rule_needs_more_memory/21.log).
Trying to restart job 1.
Job 1 has been submitted with SLURM jobid 22 (log:
 ↪  .snakemake/slurm_logs/rule_needs_more_memory/22.log).
[...]
```

```
$ cat results/retried.txt
Succeeded with 400 MB

$ sacct -X --format=JobID,Comment%26,ReqMem,State,ExitCode | tail
 ↪  -n 2
21               rule_needs_more_memory       200M     FAILED
 ↪  1:0
22               rule_needs_more_memory       400M  COMPLETED
 ↪  0:0
```

This is exactly what you want: the first SLURM job failed with 200 MB, and Snakemake submitted a second one with 400 MB.

15.6. A trap: `retries:` in a rule behaves differently

Snakemake also has a directive to set retries per rule, and this seems like the same thing. We add `retries: 2` to the rule and run it without the command-line option:

```
$ sed -i 's/^    resources:$/    retries: 2\n    resources:/'
 ↪  Snakefile.retries
$ snakemake --snakefile Snakefile.retries --profile profile --quiet
 ↪  rules --slurm-keep-successful-logs
[...]
Job 1 has been submitted with SLURM jobid 23 (log:
 ↪  .snakemake/slurm_logs/rule_needs_more_memory/23.log).
[...]

$ cat results/retried.txt
Succeeded with 400 MB

$ sacct -X --format=JobID,Comment%26,ReqMem,State,ExitCode | tail
 ↪  -n 1
23               rule_needs_more_memory       200M  COMPLETED
 ↪  0:0

$ grep -c 'Trying to restart'
 ↪  .snakemake/slurm_logs/rule_needs_more_memory/*.log
.snakemake/slurm_logs/rule_needs_more_memory/21.log:0
.snakemake/slurm_logs/rule_needs_more_memory/23.log:1
```

Look closely. The result says "Succeeded with 400 MB", but SLURM's record says that job 23 had **200 MB** and that it was one single job. The message "Trying to restart" is inside its SLURM log.

What happened? Remember that each SLURM job runs its own Snakemake, which reads the Snakefile too, and therefore also sees `retries: 2`. When the command failed, this inner Snakemake restarted it inside the same SLURM job, with `attempt` set to 2. So the command was told that it has 400 MB, while SLURM still only gave it 200 MB.

This has real consequences on a production cluster:

- The retry doesn't get more memory or CPUs, even though `{resources.mem_mb}` and `{threads}` now claim that it does. A tool that gets `-t {threads}` starts more threads than it has CPUs for.
- If the tool ran out of memory, the system usually kills only the tool, and the inner Snakemake survives. It then retries in the same too-small job, and probably fails the same way.
- Only when all inner attempts have failed does the SLURM job end as `FAILED`. Only then does the outer Snakemake submit a new, bigger job, which can repeat this. A rule with three retries can run up to many times more often than intended, and use a lot of allocation.
- If SLURM kills the whole job, for example at the time limit, there is no inner Snakemake left. Then the outer one resubmits right away with more resources, as you would expect.

The cure follows from the explanation. The directive should only count in the outer Snakemake. In the Snakefile, `workflow.remote_exec` tells you that you are inside a job, and the attribute of a rule that holds its retries is called `restart_times`:

```
# Inside a running SLURM job, Snakemake must not retry: the job has
 ↪  its resources already
if workflow.remote_exec:
    for rule_object in workflow.rules:
        rule_object.restart_times = 0
```

With this, the rule behaves as you would expect:

```
$ snakemake results/retried.txt --snakefile Snakefile.fixed
 ↪  --profile profile --quiet rules
[...]
Job 0 has been submitted with SLURM jobid 24 (log:
 ↪  .snakemake/slurm_logs/rule_needs_more_memory/24.log).
Trying to restart job 0.
```

```
Job 0 has been submitted with SLURM jobid 25 (log:
 ↪  .snakemake/slurm_logs/rule_needs_more_memory/25.log).
[...]

$ sacct -X --format=JobID,Comment%26,ReqMem,State,ExitCode | tail
 ↪  -n 2
24               rule_needs_more_memory       200M     FAILED
 ↪  1:0
25               rule_needs_more_memory       400M  COMPLETED
 ↪  0:0
```

These results come from Snakemake 9.11.6 with version 2.8.0 of the SLURM plugin. Other versions may differ. To find out what your own setup does, search your SLURM logs:

```
grep -l 'Trying to restart' .snakemake/slurm_logs/*/*.log
 ↪  .snakemake/slurm_logs/*/*/*.log
```

Every file listed is a retry that stayed inside one SLURM job. This pipeline had exactly this problem until version 0.4.68, and Chapter 17 shows how it sets retries now.

15.7. A few more options for the cluster

Option	Purpose
`--jobs N`	How many jobs at once
`--latency-wait N`	Seconds to wait for output files
`--keep-going`	Continue with other jobs after a failure, instead of stopping
`--rerun-incomplete`	Redo the files of interrupted jobs
`--slurm-keep-successful-logs`	Keep SLURM logs of finished jobs
`--scheduler greedy`	A faster way of choosing the next jobs, which helps with large DAGs
`--max-jobs-per-second N`	Limit how quickly jobs are submitted, to go easy on the scheduler

`--keep-going` deserves a mention. By default, Snakemake stops submitting new jobs after a failure, and waits for the running ones. On a large run, you often prefer that all the independent work continues overnight, so you can look at the failures in the morning.

15.8. Where Snakemake itself should run

Snakemake must keep running as long as the workflow does, for days if necessary. Options are:

- **A tmux session on the login node.** It uses very little CPU, and this is how the pipeline is usually run. Some clusters limit how long you can run processes on a login node, so check your centre's rules.
- **A long SLURM job of its own**, with few resources. The dispatcher then runs on a compute node, and submits the other jobs from there.

15.9. Summary

- Snakemake stays on the login node, and submits each job with `sbatch`. Inside each job, another Snakemake runs the one job.
- A profile holds the cluster options, and `resources:` becomes SLURM options.
- `sacct` with `Comment` shows the rule behind each job.
- `--retries` submits a bigger job, whereas the directive `retries:` also retries inside the running job, unless you switch that off.

16. Checkpoints: when the plan depends on results

Snakemake normally works out all jobs before it runs anything. Sometimes that's impossible, because the number of later jobs depends on what an earlier job finds. A binner decides how many bins it produces, and you may want to annotate each of them. For this Snakemake has **checkpoints**. The example is in `book/examples/snakemake/13_checkpoints/`.

16.1. The example

```
rule all:
    input:
        "results/report.txt",


checkpoint make_bins:
    """How many bins this produces is only known after it has
    ↪ run"""
    input:
        "data/sampleA_R1.fastq.gz",
    output:
        directory("results/bins"),
    shell:
        """
        mkdir -p {output}
        zcat {input} | split -l 40 -a 2 -d
    ↪ --additional-suffix=.fastq - {output}/bin_
        """


rule annotate_bin:
    input:
        "results/bins/{bin}.fastq",
```

```
    output:
        "results/annotated/{bin}.txt",
    shell:
        "wc -l < {input} > {output}"


def all_annotated_bins(wildcards):
    """Runs again after the checkpoint is done, when the bins can
    ↪  be listed"""
    folder = checkpoints.make_bins.get().output[0]
    bins = glob_wildcards(f"{folder}/{{bin}}.fastq").bin
    return expand("results/annotated/{bin}.txt", bin=sorted(bins))


rule report:
    input:
        all_annotated_bins,
    output:
        "results/report.txt",
    shell:
        "cat {input} | wc -l > {output}"
```

Three parts work together:

- `checkpoint make_bins:` looks like a rule, with the word `checkpoint` instead.
- `rule report` gets its input from a function. The function asks for the checkpoint's result with `checkpoints.make_bins.get()`.
- If the checkpoint hasn't run yet, that call stops the function, and Snakemake notes that this part of the plan can't be known yet. When the checkpoint has finished, Snakemake calls the function again, and it can now list the files.

16.2. What it looks like

The dry run can only see the first step, and says so:

```
$ snakemake --cores 1 --dry-run --quiet rules
host: localhost
Building DAG of jobs...
Job stats:
job          count
---------  -------
```

```
all              1
make_bins        1
report           1
total            3

DAG of jobs will be updated after completion.
[...]
The run involves checkpoint jobs, which will result in alteration
 ↪  of the DAG of jobs (e.g. adding more jobs) after their
 ↪  completion.
```

There are three jobs in the plan, and `annotate_bin` isn't among them. In the real run, the plan grows:

```
$ snakemake --cores 2 --quiet rules
[...]
Select jobs to execute...
Execute 1 jobs...
DAG of jobs will be updated after completion.
Finished jobid: 2 (Rule: make_bins)
1 of 3 steps (33%) done
Updating checkpoint dependencies.
Select jobs to execute...
Execute 3 jobs...
Finished jobid: 5 (Rule: annotate_bin)
2 of 7 steps (29%) done
[...]
Finished jobid: 0 (Rule: all)
7 of 7 steps (100%) done

$ ls results/annotated
bin_00.txt
bin_01.txt
bin_02.txt
bin_03.txt

$ cat results/report.txt
4
```

Watch the step count go from "1 of 3" to "2 of 7". This was the moment when Snakemake learned there are four bins.

16.3. Costs and alternatives

A checkpoint has some real drawbacks:

- **A dry run can't show the whole plan**, which you can see above. Before the checkpoint has run, you don't see what comes after it.
- **The input function runs many times**, so it should be fast.
- **It adds complexity**, and errors in such functions can be hard to read.

So use one only when you truly need to. Often a simpler way exists. Most tools in this pipeline's binning part get a folder as their input, and handle all the files in it together: dRep takes all bins and returns one folder of genomes, and CheckM2 and GTDB-Tk then process that folder as a whole. No checkpoint is necessary, since **no later job depends on the number of files**. That is why the module `assemble` doesn't need one.

This pipeline uses one checkpoint, in the contig annotation. `contig_annotate__cut_prodigal` splits the predicted proteins of an assembly into chunks of a fixed size, and how many chunks result depends on the assembly. Each chunk then gets its own eggNOG job, which makes the slow annotation run in parallel. Chapter 66 explains this in detail.

16.4. Summary

- A checkpoint is a rule after which Snakemake reconsiders the plan.
- An input function asks for its result with `checkpoints.name.get()`, and lists the files with `glob_wildcards`.
- Use a folder as output when later jobs don't depend on the number of files, and a checkpoint only when they do.

17. How this pipeline uses Snakemake

You now know all the features the pipeline uses. This chapter shows the conventions that hold it together, so that you can read any of its rules. To see them without the size of the real thing, the example in `book/examples/snakemake/14_patterns/` is a tiny pipeline that is built in the same way. It ran on the mini cluster with real SLURM.

```
$ tree -a config workflow
config
   config.yaml
   escalation.yaml
   features.yaml
   params.yaml
   profiles
       minicluster
           config.yaml
   samples.tsv
workflow
   Snakefile
   rules
       qc
           __main__.smk
           filter.smk
           fragile.smk

6 directories, 10 files
```

17.1. The run script

You don't call `snakemake` directly. A script in the project folder holds all the options of a run, so each run is the same, and you only say what to run:

```
bash run_Snakebite-Holoruminant-MetaG.sh assemble__megahit
bash run_Snakebite-Holoruminant-MetaG.sh assemble__megahit -np
```

Anything after the target goes to Snakemake, so `-np` makes it a dry run. The important part of the real script looks like this:

```
snakemake -s $pipelineFolder/workflow/Snakefile \
          --jobs 150 \
          --use-singularity \
          --configfile $projectFolder/config/config.yaml \
          --profile $Profile \
          --singularity-args "-B
           ↪  /scratch,/projappl,/users,/dev/shm:/dev/shm,/run,
           ↪  $projectFolder/tmp:/tmp"
           ↪  \
          --singularity-prefix $projectFolder/docker_images/ \
          --latency-wait 60 \
          --scheduler greedy \
          --resources metaspades_slots=$metaspades_slots \
          --restart-times 0 \
          --retries 0  \
          $@
```

Option	Why
`--jobs 150`	Up to 150 jobs at once
`--use-singularity`	Run each rule in its container (Chapter 3)
`--configfile`	The main configuration of this project
`--profile`	The SLURM settings of this cluster
`--singularity-args`	Which folders the containers can see
`--singularity-prefix`	Keep the containers in the project, not in your home folder
`--latency-wait 60`	Wait a minute for files on the shared disk
`--scheduler greedy`	Quicker planning with many jobs
`--resources metaspades_slots=...`	At most this many metaSPAdes jobs at once
`--restart-times 0 --retries 0`	No global retries, as the Snakefile sets them per rule
`$@`	Everything you typed after the script's name

The first lines of the script set the project folder, the profile and the temporary folders of Apptainer. You have to adjust them for each project (Chapter 59).

The script also reads two values from `config.yaml` with a small helper function. Since it uses `grep` and `sed`, it only works for simple one-line settings:

```
pipelineFolder="$(read_yaml pipeline_folder "$configFile")"
metaspades_slots="$(read_yaml metaspades_slots "$configFile")"
```

17.2. What happens when Snakemake reads the workflow

`workflow/Snakefile` does these things in order, each time any Snakemake reads it:

1. It sets up `debug()`, which prints only with `PIPELINE_DEBUG=1`.
2. It works out the pipeline's version from the first numbered line of the `CHANGELOG`.
3. It loads `params.yaml`, `.docker.yml`, `features.yaml` and `samples.tsv`.
4. It checks the sample sheet with `validate_samples()`. Duplicate libraries or files stop everything right away, because they would otherwise lead to silently lost data.
5. It expands `assembly_ids`, and builds the lists `SAMPLE_LIBRARY`, `ASSEMBLY_LIBRARIES` and `ASSEMBLIES`.
6. It picks the temporary storage for each database-heavy tool with `choose_temp()`.
7. It loads `escalation.yaml`.
8. It defines the `debug` shell function for all rules with `shell.prefix`.
9. It includes `folders.smk` and all modules.
10. It defines `rule all`, and sets the retries of each rule from its escalation order.

17.3. Resource sets and escalation

This is the idea you need most for reading rules. In a usual Snakefile, each rule has its own numbers for memory and time. If you then move to another cluster or other data, you have to change 170 rules. This pipeline separates the two.

`config.yaml` defines named **resource sets**, each with a size:

```
resource_sets:
  small:
    runtime: 300
    mem_mb: 16000
    cpus: 1
    partition: "small"
    nvme: "nvme:0"
```

```
  medium:
    runtime: 4000
    mem_mb: 160000
    cpus: 10
    partition: "small"
    nvme: "nvme:0"
```

`escalation.yaml` says which sets each rule uses, in order:

```
assemble__megahit__run: large
eukaryote__remag__filter: medium, large, longrun
```

The first attempt of `eukaryote__remag__filter` uses `medium`, the second `large` and the third `longrun`. A rule that isn't listed uses `small, medium, large`. So tuning for a new cluster or dataset means editing these two files, without touching any rule.

Three small functions in `workflow/rules/helpers/__functions__.smk` connect it:

```
def get_escalation_order(rule_name):
    return ESCALATION_CONFIG.get(rule_name, ["small", "medium",
     ↪  "large"])

def get_resources(wildcards, attempt, rule_name):
    escalation_order = get_escalation_order(rule_name)
    profile_idx = min(attempt - 1, len(escalation_order) - 1)
    profile_name = escalation_order[profile_idx]
    resources = config["resource_sets"][profile_name].copy()
    ...
    resources["attempt"] = attempt
    return resources

def esc(key, rule_name):
    """Return a lambda for Snakemake resources/threads, bound to a
     ↪  specific rule name."""
    return lambda wc, attempt: get_resources(wc, attempt,
     ↪  rule_name)[key]
```

`esc("mem_mb", "my_rule")` returns a function, which Snakemake calls with the attempt number. It finds the right set, and returns the value. That explains the block that you will see in every rule:

```
    threads: esc("cpus", "assemble__drep__run")
    resources:
        runtime=esc("runtime", "assemble__drep__run"),
        mem_mb=esc("mem_mb", "assemble__drep__run"),
        cpus_per_task=esc("cpus", "assemble__drep__run"),
        slurm_partition=esc("partition", "assemble__drep__run"),
        gres=lambda wc, attempt: f"{get_resources(wc, attempt,
↪   'assemble__drep__run')['nvme']}",
        attempt=esc("attempt", "assemble__drep__run"),
```

It looks long, but it is the same in all rules, and you can skip over it while reading. The name in it is what connects a rule to `escalation.yaml`, so it has to be spelled like the rule.

17.3.1. Retries are set in one place

No rule says how often it can be retried. The loop at the end of `workflow/Snakefile` decides this for all of them:

```
for r in workflow.rules:
    # One attempt per resource set
    retries = 0 if workflow.remote_exec else
↪   len(get_escalation_order(r.name)) - 1
    r.restart_times = retries
    debug(f"Rule '{r.name}': retries set to {retries}")
```

So a rule gets one attempt per resource set, and a rule with a single resource set isn't retried. When you add a rule, you only have to add its line to `escalation.yaml`.

The condition `workflow.remote_exec` is there because of the trap from Section 15.6. Each SLURM job runs its own Snakemake on this same file, and without the condition it would retry a failed command inside the job, with resources the job doesn't have. With it, only the Snakemake on the login node retries, and a retry is always a new SLURM job.

17.3.2. Escalation in action

In the example, the rule `qc__fragile__run` fails on purpose as long as it has less than 500 MB. Its configuration lets it escalate:

```
qc__fragile__run: small, medium, large
```

```
$ bash run_pipeline.sh qc --quiet rules
[...]
Job 3 has been submitted with SLURM jobid 3 (log:
 ↪  .snakemake/slurm_logs/rule_qc__fragile__run/3.log).
[...]
Trying to restart job 3.
Job 3 has been submitted with SLURM jobid 4 (log:
 ↪  .snakemake/slurm_logs/rule_qc__fragile__run/4.log).
[...]
4 of 4 steps (100%) done

$ ls results/qc
fragile.log
fragile.log.try1
fragile.txt
sampleA.fastq.gz
sampleA.log
sampleB.fastq.gz
sampleB.log

$ cat results/qc/fragile.log.try1
attempt 1: 200 MB, 1 CPUs, partition small
Pretending to run out of memory

$ cat results/qc/fragile.log
attempt 2: 500 MB, 2 CPUs, partition small
```

The log of the failed attempt stays as `fragile.log.try1`, and the successful one becomes `fragile.log`. SLURM's records show that the second attempt really did get more:

```
$ sacct -X
 ↪  --format=JobID,Comment%24,Partition,AllocCPUS,ReqMem,State |
 ↪  tail -n 4
1            rule_qc__filter__run_wi+      small          1
 ↪  200M  COMPLETED
2            rule_qc__filter__run_wi+      small          1
 ↪  200M  COMPLETED
3               rule_qc__fragile__run      small          1
 ↪  200M     FAILED
4               rule_qc__fragile__run      small          2
 ↪  500M  COMPLETED
```

Job 3 failed with 200 MB and 1 CPU, and job 4 then succeeded with 500 MB and 2

CPUs.

The test `tests/escalation/run_test.sh` checks the same behaviour with the real rules of the pipeline, also on real SLURM (Chapter 71).

> **Most rules don't escalate by default**
>
> In the current `config/escalation.yaml`, 93 of the 98 listed rules have a single resource set, so these rules aren't retried at all. If you want a rule to escalate on your data, give it several sets, for example `small, medium, large`.

17.4. Debug messages

Setting `PIPELINE_DEBUG=1` turns on extra messages, both when the Snakefile is read and in the rules:

```
$ PIPELINE_DEBUG=1 bash run_pipeline.sh qc -n --quiet rules
[DEBUG] Samples: ['sampleA', 'sampleB']
[...]
```

In the Snakefile, it is a normal Python function. For the rules, `shell.prefix` puts a bash function in front of every rule's command:

```
shell.prefix(
    'set -euo pipefail; '
    'debug() {{ '
    'if [ "${{PIPELINE_DEBUG:-0}}" = "1" ]; then echo "[DEBUG] $*"
    ↪  >&2; fi; '
    '}}; '
)
```

The first of those lines is essential. `shell.prefix` replaces Snakemake's default prefix, which is what normally turns on strict mode, so it has to be repeated here (Section 13.6). This pipeline learned that the hard way.

So each rule can write `debug "sample={wildcards.sample} threads={threads}"`. The message goes to stderr, and so into the log file when the rule has redirected it. With the variable unset, nothing is printed. It reaches the rules because SLURM jobs and containers inherit the environment here.

```
PIPELINE_DEBUG=1 bash run_Snakebite-Holoruminant-MetaG.sh
 ↪  read_annotate__kraken2
```

17.5. Conventions at a glance

Convention	Example	Where to read more
Rule names	`assemble__magscot_hamsearch`	Chapter 14
One target rule per tool and module	`assemble__magscot`, `assemble`	Chapter 8
Path constants	`MAGSCOT / "{assembly_id}.fa.gz"`	Section 14.5
Shared paths defined once	`CONTIGS`, `MAG_CATALOGUE`, `CLEAN_FORWARD`	Section 14.5
A log and a benchmark per rule	`log:`, `benchmark:`	Chapter 11
A container per module	`docker["assemble"]`	Chapter 3
Resources from resource sets	`esc("mem_mb", ...)`	This chapter
A log per attempt	`try_log={log}.try{resources.attempt}`	Chapter 11
Debug messages	`debug "..."`	This chapter
Settings from `params.yaml`	`params["assemble"]["tool"]`	Chapter 10
Scripts in `workflow/scripts/`	`{params.script_folder}/contig_ids.py`	Part VI

17.6. How to read a rule of this pipeline

When you open a rule file for the first time, read in this order:

1. **The name and the docstring** tell you what it does.
2. **`output:`** tells you what it promises, and where the result ends up.
3. **`input:`** tells you what it needs, and which rule therefore comes before it.
4. **`shell:`** is the actual work. Skip `debug` lines, and look for the tool's command.
5. **`container:`** tells you which software it runs in.
6. **`params:`** connects it to settings.

You can skip `threads:` and `resources:`, because they are the same everywhere.

Part III of this book goes through all rules in this way, and Appendix Appendix A lists them all, generated from the code.

17.7. Summary

- The run script holds all options of a run.
- Resource sets and `escalation.yaml` keep the numbers out of the rules.
- `esc()` connects a rule to them, using its own name.
- `PIPELINE_DEBUG=1` turns on extra messages.

18. Testing a workflow

A pipeline that is changed for years needs tests, or small changes break things unnoticed. Workflows have a problem here: a real run takes days and needs terabytes of databases. So you test on several levels, from checks that take seconds to complete runs. The example is in `book/examples/snakemake/15_tests/`.

18.1. Level 1: the dry run

The cheapest test is `snakemake -n`. It parses all rule files, builds the whole plan, and finds most kinds of mistakes in the plan without running anything. It needs the input files to exist, but they may be empty, which makes it easy to test a pipeline without any data:

```
mkdir -p reads resources/reference
touch reads/sample1_R1.fastq.gz reads/sample1_R2.fastq.gz
 ↪  resources/reference/host.fa.gz
snakemake -n --cores 1 all
```

Add `-p`, and each command gets built as well, which also finds problems like single braces in a command.

18.2. Level 2: the linter

`--lint` looks for known bad habits in a workflow:

```
$ snakemake --lint text | head -n 12
Lints for rule count_reads (line 10,
 ↪  /scratch/project_demo/examples/15_tests/Snakefile):
    * No log directive defined:
      Without a log directive, all output will be printed to the
 ↪  terminal. In
      distributed environments, this means that errors are harder
 ↪  to discover.
```

```
    In local environments, output of concurrent jobs will be
↪ mixed and become
    unreadable.
    Also see:

↪ https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html#log-files
  * Specify a conda environment or container for each rule.:
    This way, the used software for each specific step is
↪ documented, and the
    workflow can be executed on any machine without
↪ prerequisites.
    Also see:
```

A separate tool called `snakefmt` formats Snakefiles consistently, and the pipeline's GitHub workflow uses a linter on each pull request.

18.3. Level 3: unit tests for rules

Snakemake can write unit tests for you. After a successful run, `--generate-unit-tests` copies the input and output of one job per rule, and writes a test that runs the rule again and compares the result:

```
$ snakemake --cores 2 --quiet rules
[...]
8 of 8 steps (100%) done

$ snakemake --generate-unit-tests --cores 1 2>&1 | tail -n 1
Generating unit test for rule count_reads:
↪  .tests/unit/test_count_reads.py.

$ find .tests -type f | sort | head -n 12
.tests/unit/common.py
.tests/unit/conftest.py
.tests/unit/count_reads/config/.gitempty
.tests/unit/count_reads/data/data/sampleA_R1.fastq.gz
.tests/unit/count_reads/expected/results/counts/sampleA_R1.txt
.tests/unit/summary/config/.gitempty
.tests/unit/summary/data/results/counts/sampleA_R1.txt
.tests/unit/summary/data/results/counts/sampleA_R2.txt
.tests/unit/summary/data/results/counts/sampleB_R1.txt
.tests/unit/summary/data/results/counts/sampleB_R2.txt
```

```
.tests/unit/summary/data/results/counts/sampleC_R1.txt
.tests/unit/summary/data/results/counts/sampleC_R2.txt
```

Each rule gets a folder with `data` (its inputs) and `expected` (its outputs). A test framework called pytest runs them:

```
$ pytest -q .tests/unit 2>&1 | tail -n 3
..
 ↪  [100%]
2 passed in 2.76s
```

This is a neat idea, and has limits for a pipeline like this one:

- It stores real inputs and outputs. With tools that take gigabytes of input, this doesn't fit in a repository, so it needs a small dataset.
- It compares bytes by default. Many bioinformatics tools produce different files on each run, with dates in them, or a different order when they use several threads. You then need your own comparison.
- It needs the rule's software available when the tests run.

18.4. Level 4: running with stand-in tools

This level answers a question the others don't: does the shell code of the rules work, with its loops, renames and clean-up steps? You put small programs on the `PATH` named like the real tools, which only create the expected files. Then you run the real Snakefile for real, without containers.

It takes seconds, needs no databases, and checks everything except the tools themselves. If you run the old and the new version of the pipeline this way, and compare all results, you learn whether a rewrite changed anything. This is how the rule rewrite described in the README was verified.

A stand-in can also fail on purpose, which tests how the pipeline handles failures. `tests/escalation/run_test.sh` does this on a real SLURM cluster in a container (Chapter 71). A stand-in `bgzip` fails when a job has too little memory, and the test checks that the rule fails and that a bigger SLURM job follows. This test found two serious bugs that no dry run could have found: rules that ignored failing commands (Section 13.6), and retries that stayed inside the same job (Section 15.6).

Its limit is clear: a stand-in accepts any option you give it. It can't tell you that a real tool doesn't understand a flag.

18.5. Level 5: real runs with real data

Only this level proves that the pipeline works, and it's the most expensive. A small but real dataset keeps it possible, and the real tools then run in their containers. Part IV of this book does this, and its results are stored as tests for later changes.

18.6. What you should run when

Change	Check
Any change to a rule	Dry run in several setups
Changing shell code	Stand-in run
Changing a tool's options or container	A real run of that rule
Before a release	A complete real run on the small dataset
Any change at all	The book's sync check (`book/tools/check_sync.py`)

18.7. Summary

Test on several levels, because each finds different mistakes:

- A dry run finds problems in the plan.
- A linter finds bad habits.
- Unit tests keep single rules from changing their results.
- Stand-in runs check the shell code around the tools.
- Real runs check the tools themselves.

Part II.

Part II: The tools

19. The tools at a glance

The pipeline itself computes very little. Nearly all the work is done by about forty programs, each written by other people, and each with its own ideas about input, output and memory. To understand your results, and to know what to do when something fails, you have to know these programs. This part has a chapter for each.

19.1. How to read these chapters

Every chapter has the same structure:

- **What it does**, and the idea behind it, as far as you need that to use it well
- **Input** and **output**, in what form they usually come
- **When to use it**, and when something else would be the better choice
- **In this pipeline**: which rule runs it, how, and why it was chosen
- **Pitfalls**, the things that commonly go wrong or get misread
- **Reference**, the paper to cite

You don't have to read this part from beginning to end. When a module in Part III mentions a tool, come here for the background. The real output of the tools on real data is in Part IV, in Chapter 70, because it makes more sense in the order in which it's produced. The chapters here describe the file formats in general.

19.2. Please cite the tools

Each tool is someone's scientific work, and a citation is how that work gets recognized. If you publish results from this pipeline, cite the tools you used, with the version. The reference at the end of each chapter was checked against the publisher's record when this book was written. The file `citations.tct` in the repository has more of them.

19.3. Some terms you will meet

Read. A short piece of sequenced DNA, here about 100 to 150 bases long. Paired-end sequencing reads a fragment from both ends, which gives two reads, called mates, a few hundred bases apart.

k-mer. A piece of exactly k bases. A read of 100 bases contains $100 - k + 1$ overlapping k-mers. Many tools work with k-mers, as it's much faster to look up short words than to align whole sequences.

Contig. A longer sequence that an assembler built from overlapping reads.

Bin. A group of contigs that likely come from the same organism.

MAG. A metagenome-assembled genome, a bin of good quality.

Coverage or depth. How many reads cover a base on average.

Completeness and contamination. How much of a genome is there, and how much of it comes from other organisms. Usually estimated with marker genes.

ANI. Average nucleotide identity. It tells you how similar two genomes are, and 95% is the usual limit for one species.

HMM. A hidden Markov model. Here it's a statistical profile of a family of proteins, which can recognize distant members better than a comparison with a single sequence can.

19.4. The file formats

Most of the formats are shared between many tools, so it's enough to explain them once.

FASTQ holds reads with their quality, with four lines per read: a name that starts with `@`, the bases, a `+`, and one character per base for its quality. The character stands for a Phred score Q, where the chance that the base is wrong is $10^{(-Q/10)}$. So Q20 means 1 wrong base in 100, and Q30 means 1 in 1000.

FASTA holds sequences, with a name line that starts with `>`, and then the sequence, on one or several lines.

SAM, BAM and CRAM hold alignments of reads against a reference. SAM is text, BAM is its compressed binary form, and CRAM stores only the differences to the reference, so it's smaller and it needs the reference to be read. See Chapter 23.

GFF and GTF describe the positions of features such as genes on a sequence, with one feature per line.

TSV is a plain table, with tabs between the columns.

19.5. The containers

`config/.docker.yml` names one image per key, and a rule uses it as `docker["key"]`:

Key	Image	Used for
`reads`	`fischuu/hrp_reads:0.2`	Linking the reads
`reference`	`fischuu/hrp_reference:0.1`	Recompressing the host genomes
`helpers`	`fischuu/hrp_helpers:0.1`	FastQC and SAMtools in the generic rules
`preprocess`	`fischuu/hrp_preprocess:0.3`	fastp, Kraken2, MetaPhlAn, Nonpareil, SingleM, SAMtools
`bowtie2`	`fischuu/hrp_bowtie2:0.1`	Host removal and mapping to the catalogue
`assemble`	`fischuu/hrp_assemble:0.2`	Assemblers, MaxBin2, MetaBAT2, MAGScoT, dRep, Prodigal, HMMER
`concoct`	`fischuu/hrp_concoct:0.1`	CONCOCT
`quantify`	`fischuu/hrp_quantify:0.2`	CoverM
`subread`	`fischuu/subread:2.0.1-0.1`	featureCounts
`mag_annotate`	`fischuu/hrp_annotate:0.4`	QUAST, Bakta, PhyloPhlAn, Proteinortho, DIAMOND, nCycDB
`checkm2`	`fischuu/hrp_checkm2:0.1`	CheckM2
`gtdbtk`	`fischuu/hrp_gtdbtk:0.3`	GTDB-Tk
`dram`	`fischuu/hrp_dram:0.4`	DRAM
`camper`	`fischuu/hrp_camper:0.1`	CAMPER
`eggnog`	`fischuu/hrp_eggnog:0.1`	eggNOG-mapper on the catalogue, eggNOG 7
`humann`	`fischuu/hrp_humann:0.1`	HUMAnN
`krona`	`fischuu/hrp_krona:0.1`	Krona
`phyloflash`	`fischuu/hrp_phyloflash:0.2`	phyloFlash
`sylph`	`fischuu/sylph:0.8.1_0.1`	sylph
`r_report`	`fischuu/r-gbs:4.2.1-0.7`	The reports and the summaries of DIAMOND
`tiara`	`quay.io/biocontainers/tiara:1.0.3`	Tiara
`4cac`	`metagenlab/4cac:1.0`	4CAC
`remag`	`quay.io/biocontainers/remag:1.0.1--pyhdfd78af_0`	REMAG

19.6. Versions of the tools in the containers

Each tool runs in a container, so its version is fixed by the image in `config/.docker.yml`. This table comes from the images themselves. `book/tools/container_versions.sh` reads the installed packages of each image into `book/examples/container_versions.tsv`, and `book/tools/versions_table.py` writes the table here. If you change an image, run both of them again.

Tool	Version	Image	Used in
FastQC	0.12.1	`fischuu/hrp_helpers:0.1`	All FastQC reports
fastp	0.23.4	`fischuu/hrp_preprocess:0.1`	`preprocess`
Bowtie2	2.5.1	`fischuu/hrp_bowtie2:0.1`	`bowtie2_process`, `quantify`
Bowtie2	2.5.1	`fischuu/hrp_assemble:0.2`	`assemble`
SAMtools	1.18	`fischuu/hrp_bowtie2:0.1`	`bowtie2_process`, `quantify`
SAMtools	1.18	`fischuu/hrp_assemble:0.2`	`assemble`, `contig_annotate`
SAMtools	1.18	`fischuu/hrp_helpers:0.1`	The generic rules
SAMtools	1.20	`fischuu/hrp_concoct:0.1`	`assemble__concoct__run`
Kraken2	2.1.3	`fischuu/hrp_preprocess:0.1`	`reads_annotate`
Krona	2.8.1	`fischuu/hrp_krona:0.1`	`read_annotate`
SingleM	0.18.0	`fischuu/hrp_preprocess:0.1`	`reads_annotate`
sylph	0.8.1	`fischuu/sylph:0.8.1`	`read_annotate`
MetaPhlAn	4.1.1	`fischuu/hrp_preprocess:0.1`	`reads_annotate`
HUMAnN	3.9	`fischuu/hrp_humann:0.1`	`read_annotate`
phyloFlash	3.4.2	`fischuu/hrp_phyloflash:0.2`	`reads_annotate`
DIAMOND	2.1.8	`fischuu/hrp_annotate:0.4`	`read_annotate`, `contig_annotate`
Nonpareil	3.4.1	`fischuu/hrp_preprocess:0.1`	`reads_annotate`
MEGAHIT	1.2.9	`fischuu/hrp_assemble:0.2`	`assemble`
SPAdes (metaSPAdes)	3.15.5	`fischuu/hrp_assemble:0.2`	`assemble`
CONCOCT	1.1.0	`fischuu/hrp_concoct:0.1`	`assemble`
MaxBin2	2.2.7	`fischuu/hrp_assemble:0.2`	`assemble`
MetaBAT2	2.15	`fischuu/hrp_assemble:0.2`	`assemble`
Prodigal	2.6.3	`fischuu/hrp_assemble:0.2`	`assemble`, `contig_annotate`
HMMER	3.3.2	`fischuu/hrp_assemble:0.2`	`assemble`, `contig_annotate`
dRep	3.4.3	`fischuu/hrp_assemble:0.2`	`assemble`
CheckM	1.2.2	`fischuu/hrp_assemble:0.2`	Used by dRep
CoverM	0.6.1	`fischuu/hrp_quantify:0.1`	`quantify`
featureCounts (Subread)	2.0.1	`fischuu/subread:2.0.1`	`contig_annotate`

Tool	Version	Image	Used in
QUAST	5.2.0	fischuu/hrp_annotate:0.4	mag_annotate
CheckM2	1.0.2	fischuu/hrp_checkm2:0.1	mag_annotate
GTDB-Tk	2.4.0	fischuu/hrp_gtdbtk:0.3	mag_annotate
PhyloPhlAn	3.1.1	fischuu/hrp_annotate:0.4	mag_annotate
DRAM	1.5.0	fischuu/hrp_dram:0.4	mag_annotate
CAMPER	1.0.13	fischuu/hrp_camper:0.1	mag_annotate, contig_annotate
Bakta	1.9.3	fischuu/hrp_annotate:0.4	mag_annotate
eggNOG-mapper	2.1.12	fischuu/hrp_eggnog:0.1	mag_annotate__eggnog
eggNOG-mapper	2.1.12	fischuu/hrp_annotate:0.4	contig_annotate
Proteinortho	6.3.1	fischuu/hrp_annotate:0.4	mag_annotate
Tiara	1.0.3	quay.io/biocontainers/tiara:1.0.3	eukaryotic
REMAG	0.5.1	quay.io/biocontainers/remag:0.5.1--pyhdfd78af_0	eukaryotic

Some tools exist in more than one image with different versions, which is why the table names the image. For example, the Bowtie2 that removes host reads is another installation than the one that maps reads to the assemblies, even though they have the same version here. `hrp_preprocess:0.8` holds Bowtie2 2.5.4, DIAMOND 2.1.10 and HMMER 3.2.1, but the rules that use those tools run in other images.

The MAGScoT script and `NCycProfiler.PL` aren't in an image. They come with the pipeline, in `workflow/scripts/`. 4CAC has no package in its image, and its version is the tag of the image, 1.0.

20. FastQC

20.1. What it does

FastQC reads a FASTQ file and writes a report about its quality. It doesn't change your data. It's a quick look at whether a sequencing run looks healthy, and whether the following steps have something to fix.

The report consists of a number of checks, each with a mark of pass, warning or fail.

Check	What it shows
Per base sequence quality	The quality along the read. It usually falls toward the end
Per tile sequence quality	Quality by position on the flow cell, which shows local problems in a run
Per sequence quality scores	Whether a group of reads is poor overall
Per base sequence content	The share of A, C, G and T along the read
Per sequence GC content	The GC distribution over all reads, compared to a normal distribution
Per base N content	How often a base couldn't be called
Sequence length distribution	Whether all reads have the same length, for example before and after trimming
Sequence duplication levels	How often the same sequence occurs
Overrepresented sequences	Sequences that occur far too often, such as adapters
Adapter content	How much adapter remains, by position

20.2. Input

One or several sequence files. FASTQ with or without gzip is the usual case, and SAM and BAM also work. Each file gets its own report. Paired reads aren't connected in any way, so you get two reports per library.

20.3. Output

File	Content
`<name>_fastqc.html`	The report, a single file that you open in a browser
`<name>_fastqc.zip`	The same as data, including a text file with all numbers, which is useful for scripts and for MultiQC

20.4. When to use it

Always, at the start, and again after trimming. It takes a few minutes and it can save weeks. Compare the two reports: the adapter curve should be gone, and the quality at the ends of the reads should be better.

With many samples, reading single reports gets tedious. MultiQC, a separate tool, combines many reports into one, and it reads the zip files. It's not part of this pipeline, but you can run it on the results folder.

20.5. Reading it for a metagenome

The marks were designed with a single genome in mind, so several fail for good reasons in a metagenome. Don't be alarmed, and learn what is normal for your samples:

- **Per sequence GC content** often fails, because a community is a mix of organisms with different GC content. Several peaks are normal. You can even learn something from them. A sharp peak that doesn't belong may be a contaminant, or a very dominant organism.
- **Sequence duplication levels** can fail because abundant organisms really are sequenced many times over. In a metagenome, duplicates don't have to be an artifact.
- **Per base sequence content** often shows wobbles in the first ten or so bases of a read. This comes from the library preparation and it's common.

What does deserve attention:

- Quality that falls early in the read, and in the forward read as well
- Adapter content that rises toward the end of the read. Normally, this is what you trim
- A strange overrepresented sequence. Search for it online or in a database, as it could be a primer or a contaminant
- Reports that look very different from the other samples of the same run

20.6. In this pipeline

The generic rule `helpers__fastqc` makes a report for any file ending in `.fq.gz`, as Section 68.2 explains. Three targets ask for it:

- `reads__fastqc` for the raw reads
- `preprocess__fastqc__fastp` for the reads after trimming
- `preprocess__fastqc__nonhost` for the reads after each host

The rule simply calls `fastqc {input}`, which runs with one thread per file.

20.7. Pitfalls

- **Judging a metagenome by the marks alone.** See above.
- **Memory.** FastQC is a Java program that reserves a fixed amount of memory per file. This is no issue for short reads. For very long reads it can be too little.
- **The folder must exist.** On its own, FastQC refuses to write to a folder that doesn't exist. By default, it writes the report next to the input, as here.

20.8. Reference

Andrews S. FastQC: a quality control tool for high throughput sequence data. https://www.bioinformatics.babraham.ac.uk/projects/fastqc/. There's no paper to cite, so cite the website with the version.

21. fastp

21.1. What it does

fastp cleans FASTQ files. In a single run, it removes adapters, filters out poor reads, trims unwanted tails, and writes a report on what it did. Earlier workflows needed several programs for this, such as Trimmomatic or Cutadapt plus a quality tool. fastp does it in one step, and it's fast because it's written in C++ and uses many threads.

21.2. How it works

fastp goes through the reads and applies these steps to each read or pair.

Adapter trimming. The sequencing machine reads a fragment from its end. If the fragment is shorter than the read length, the machine reads on into the adapter on the other side. Then the end of the read isn't biological.

For pairs, fastp can find this without knowing the adapter: if the two mates overlap, then the position where they extend past each other shows where the adapter begins. You can also give the adapter sequences directly, and this pipeline does that.

Quality filtering. By default, a read is thrown away if more than 40% of its bases have a quality below Q15, or if it has more than 5 unknown bases (`N`). If one mate of a pair fails, the other can be kept as an unpaired read. Note that by default, fastp doesn't trim low-quality ends, it only judges whole reads. Trimming with a sliding window has to be switched on with options such as `--cut_tail`.

Length filtering. A read that's too short after trimming is thrown away, by default under 15 bases.

Tail trimming. Some sequencers (with two-color chemistry, such as NextSeq and NovaSeq) report `G` when they see no signal, so poor reads can end with a long run of `G`. `--trim_poly_g` removes this, and fastp turns it on by itself for those machines. `--trim_poly_x` removes tails of any other single base, such as the `A` tail of RNA.

21.3. Input

One FASTQ file for single reads, or two for pairs, with or without gzip. Optionally the adapter sequences.

21.4. Output

Option	File	Content
`--out1`, `--out2`	FASTQ	The cleaned pairs. Still in the same order, so that mates stay matched
`--unpaired1`, `--unpaired2`	FASTQ	Reads whose mate was thrown away
`--html`	HTML	The report for a browser
`--json`	JSON	The same report as data

The JSON file is what you want for summaries over many samples. Its most useful fields are:

Field	Content
`summary.before_filtering`, `summary.after_filtering`	Reads, bases, Q20 and Q30 rates, GC content
`filtering_result`	Reads that passed, and why the others failed: `low_quality_reads`, `too_many_N_reads`, `too_short_reads`
`adapter_cutting.adapter_trimmed_reads`	How many reads had adapter
`duplication.rate`	The estimated share of duplicates

21.5. When to use it

Use it as the first step on nearly any short-read data from Illumina. It's a sensible default, because it's fast and its choices are moderate.

Don't use it for long reads from PacBio or Nanopore, which have other properties and other tools. And if your data are amplicons with primers, you need to remove the primers, which is a separate matter from adapters. Cutadapt handles that better.

21.6. In this pipeline

The rule `preprocess__fastp__run` runs it once per library, as Chapter 61 shows in detail. The choices made there are:

- **Adapters** come from the sample table, so you can have different adapters per library. Using known adapters is safer than auto-detection, which only works for reads that overlap.
- `--length_required 25`, as Nonpareil needs reads of at least that length.
- `--trim_poly_g --trim_poly_x`, to remove tails of a single base.
- `--verbose`, so that the log shows the progress.
- **No sliding-window trimming**, which keeps this step close to fastp's defaults. If your quality falls sharply at the ends of the reads, you could add `--cut_tail` under `extra` in `params.yaml`.

The rule lets fastp write plain files and then compresses them with `bgzip`, as this is both faster and safe. It tests the result with `gzip --test`.

The unpaired files are kept, but no later rule reads them.

21.7. Pitfalls

- **Wrong adapters.** If the sequences in your sample table are wrong, nothing is trimmed, and there's no error. Check the adapter curve in the FastQC report after fastp. It's flat if the adapters are gone.
- **Too many reads lost.** Losing a few percent is normal. If you lose a large share, look at `filtering_result`. Low-quality reads point to the sequencing run, and too-short reads point to very short fragments in the library.
- **`--unpaired1` and `--unpaired2` pointing to the same file.** This would be valid for fastp, but then you could no longer tell the two sets apart. This pipeline keeps them separate.
- **Threads.** More than a certain number of threads doesn't speed it up, because reading and compressing limit it. That's another reason why this pipeline compresses separately.

21.8. Reference

Chen S, Zhou Y, Chen Y, Gu J (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/bty560

22. Bowtie2

22.1. What it does

Bowtie2 aligns short reads against a reference sequence: for each read, it finds where in the reference it fits best. This is called read mapping. The pipeline uses it three times, with different questions: which reads come from the host, how well does each contig get covered, and how many reads fall on each MAG.

22.2. How it works

A reference genome can have billions of bases, and there are hundreds of millions of reads. Comparing them directly would take forever. So Bowtie2 first builds an **index** of the reference, based on the FM index, a compressed structure that lets you look up short sequences very quickly while taking little memory.

To map a read, Bowtie2 takes short pieces, the seeds, of the read, and finds where they occur in the index. Then it extends the promising places with an alignment that allows mismatches and gaps. Each alignment gets a score, and the best one is reported.

If a read fits well in several places, Bowtie2 reports one of them, chosen at random among the best. It says how sure it is with the **mapping quality**: 0 means that there are equally good places elsewhere, and high values mean that the place is unique.

There are two modes. In the default `--end-to-end` mode, the whole read must fit. In `--local` mode, the ends of a read may be left out. This pipeline uses the default.

For pairs, Bowtie2 also checks whether the two mates fit together: facing each other at a suitable distance. Such a pair is called concordant, or properly paired.

22.3. Input

Step	Input
`bowtie2-build`	FASTA with the reference
`bowtie2`	The index (`-x`) and FASTQ with the reads (`-1`, `-2`)

22.4. Output

`bowtie2-build` writes six files that end in `.bt2`, or `.bt2l` for very large references. All of them together are the index, and you name it by their common prefix.

`bowtie2` writes SAM to the standard output, usually piped straight into `samtools`, since this format takes a lot of disk space. The reads that don't map are also in the output, marked with a flag. This matters here, as these are the reads the pipeline wants to keep after mapping to a host.

At the end, Bowtie2 prints a summary to the standard error, with the share of pairs that aligned once, more than once and not at all. The pipeline's logs contain it.

22.5. When to use it

Bowtie2 is a good choice for reads of 50 bases up to a few hundred bases, against a reference that is similar to the reads. It suits genomes and metagenomes well.

Use something else in these cases:

- **RNA-seq against a genome.** Reads span exons, and you need a splice-aware tool such as HISAT2 or STAR.
- **Long reads.** minimap2 is the tool for these.
- **Reads that are far from the reference**, as in a comparison between species. Bowtie2 works on DNA, and for distant protein-level comparisons you need DIAMOND, see Chapter 31.

BWA-MEM is a common alternative with similar results for many purposes.

22.6. In this pipeline

The pipeline uses it with mostly default options, in three places:

Rule	Reference	Question
`preprocess__bowtie2__build`	Each host genome	Which reads come from the host?
`preprocess__bowtie2__map`		
`assemble__bowtie2__build`	Each assembly	How well is each contig covered?
`assemble__bowtie2__map`		
`quantify__bowtie2__build`	The MAG catalogue	How much of each genome is in each sample?
`quantify__bowtie2__map`		

All mapping rules add a read group, `--rg-id` and `--rg`, which marks the reads with their library and sample. Some tools need this, and it also lets you see later where a read came from.

The output goes straight into `samtools sort`, to save disk space, as Chapter 61 shows.

22.7. Pitfalls

- **The prefix isn't a file.** `-x` wants the common prefix of the index files, and not a file name. The pipeline uses an empty placeholder for this, and you shouldn't delete it, or Snakemake will rebuild the index.
- **Mapping rate and reference.** A low mapping rate against an assembly can mean that much of the community wasn't assembled. It doesn't mean that Bowtie2 failed.
- **Multi-mapping.** A read in a repeat or a conserved gene is reported at just one place, chosen at random. With a catalogue of similar genomes, this can blur the counts between them. This is one reason to dereplicate.
- **Threads and memory.** Each mapping scales well with threads, and it doesn't take much memory, but the following `samtools sort` does.
- **Host removal isn't perfect.** Reads that come from parts of the host missing from the reference stay in the data. Reads of microbes that are similar to a host sequence are removed by mistake.

22.8. Reference

Langmead B, Salzberg SL (2012). Fast gapped-read alignment with Bowtie 2. *Nature Methods*. https://doi.org/10.1038/nmeth.1923

23. SAMtools and friends

23.1. What it does

SAMtools is the standard toolbox for alignment files. It reads, converts, sorts, indexes, filters and summarizes them. It's a set of subcommands, and the pipeline uses many of them. HTSlib, the library behind it, also provides `bgzip`, and this chapter covers that too, along with the other small helpers, `pigz` and `parallel`.

23.2. The formats

23.2.1. SAM

SAM is a plain text format, with one line per read. These are its most important columns:

Column	Name	Content
1	`QNAME`	Name of the read
2	`FLAG`	What is known about the read, see below
3	`RNAME`	The reference sequence it mapped to
4	`POS`	The position on it
5	`MAPQ`	Mapping quality
6	`CIGAR`	How it aligns: matches, insertions, deletions
10	`SEQ`	The bases
11	`QUAL`	Their qualities

23.2.2. The flag

The flag is a number that combines yes-or-no statements about a read, each with its own bit. You will see these often:

Value	Meaning
1	The read is part of a pair
2	Both mates are mapped as expected, a proper pair

Value	Meaning
4	This read is unmapped
8	Its mate is unmapped
16	This read is on the reverse strand
256	Not the main alignment of this read
2048	A supplementary alignment

`samtools view` filters by them: `-f 12` keeps the reads with both 4 and 8, so reads with both mates unmapped. `-F 4` drops the reads with 4, leaving the mapped ones. The pipeline uses both of these.

23.2.3. BAM and CRAM

BAM is SAM in a compressed binary form. CRAM goes further: it stores only how each read differs from the reference, which makes it clearly smaller. There's a price: **to read a CRAM file, you need the very same reference sequence**, so if the reference is lost or changed, the file is useless. The pipeline uses CRAM to save space, and it's why `--reference` or `-T` appears in so many commands.

Both can have an index, `.bai` for BAM and `.crai` for CRAM, for jumping to a certain place. The file has to be sorted by position for this.

23.3. The subcommands used here

Command	What it does	Where
`samtools sort`	Sorts by position, or by name with `-n`	All mapping rules
`samtools view`	Filters and converts	Everywhere
`samtools index`	Makes `.bai` or `.crai`	`helpers__samtools__index_bam`, `helpers__samtools__index_cram`
`samtools faidx`	Indexes a FASTA for random access	`helpers__samtools__faidx_fa`, `helpers__samtools__faidx_fagz`
`samtools fastq`	Turns alignments back into FASTQ	`preprocess__bowtie2__extract_nor`
`samtools flagstats`	A short summary by flags	`helpers__samtools__flagstats_cra`
`samtools stats`	A long summary	`preprocess__samtools__stats_cram` `quantify__samtools__stats_cram`
`samtools idxstats`	Reads per reference sequence	`helpers__samtools__idxstats_cram` `preprocess__bowtie2__map`
`samtools coverage`	Coverage per reference sequence	`assemble__maxbin2__run`

23.4. Reading the summaries

`samtools flagstats` prints a few lines, which are quick to read. Mostly you look at the percentage of mapped reads, and of properly paired ones.

`samtools stats` writes many sections. Each line starts with a code, and `grep ^SN` gets the summary numbers: raw total sequences, reads mapped, error rate, average length and insert size. The other sections hold distributions, and `plot-bamstats` turns them into figures.

`samtools idxstats` writes one line per reference sequence: its name, its length, the mapped reads and the unmapped reads.

`samtools coverage` also has one line per sequence, with these columns:

Column	Name	Content
1	`rname`	Sequence
2, 3	`startpos`, `endpos`	The range
4	`numreads`	Number of reads
5	`covbases`	Number of covered bases
6	`coverage`	Percentage of covered bases
7	`meandepth`	Average depth
8, 9	`meanbaseq`, `meanmapq`	Average qualities

23.5. The compressors

23.5.1. bgzip

`bgzip` compresses files in small blocks. The result is a valid gzip file, so any tool reads it, but an index can jump to any block. `samtools faidx` needs that for compressed FASTA files. So if you get an error from `faidx` about a compressed file, the file probably was made with plain `gzip`. `bgzip` also works with several threads.

23.5.2. pigz

`pigz` is `gzip` with several threads. It's faster, and the result is a normal gzip file. The pipeline uses it wherever it doesn't need an index, often together with `tar`, for packing the working folders. Give it its input through a plain pipe or a file name, but don't write through `>(pigz ...)`, as Section 13.7 explains.

23.5.3. GNU parallel

`parallel` runs a command many times at once. The pipeline uses it for Prodigal, which only has one thread: with `--pipe --recstart '>'`, it cuts the stream of FASTA records into blocks, and it gives each block to one Prodigal. See `assemble__magscot__prodigal`.

23.6. Pitfalls

- **Memory of `samtools sort`.** `-m` is per thread. With `-m 8G` and 20 threads, it can take 160 GB, so check that this fits into the job. This is a common reason for jobs being killed, and `params.yaml` has this setting for each module.
- **Temporary files of `sort`.** It writes temporary files next to its output, and a killed job leaves them behind. Some rules remove them before a new start.
- **A CRAM without its reference.** Keep the reference. Otherwise, you can't read the file any more.
- **CRAM and long contig names.** Very long names in the header can lead to errors with CRAM. In this case, set `out_type: bam` under `assemble: samtools` in `params.yaml`.
- **A broken pipe.** In `samtools view | head`, `head` stops reading, and `samtools view` then fails with a broken pipe. In strict mode, that stops a rule, as Section 13.6 explains.

23.7. Reference

Danecek P, et al. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*. https://doi.org/10.1093/gigascience/giab008

24. Kraken2

24.1. What it does

Kraken2 assigns each read to a taxon, by comparing its k-mers with a database of known genomes. It answers the question of which organisms the reads come from, and it does this very fast. It takes minutes for millions of reads, because it doesn't align anything.

24.2. How it works

The database is built from many genomes. For each k-mer in those genomes, it stores one taxon: the lowest common ancestor (LCA) of all genomes that contain this k-mer. A k-mer that only occurs in *Escherichia coli* points to that species. A k-mer found in all Enterobacteriaceae points to that family.

To classify a read, Kraken2 looks up each of its k-mers. This gives a set of hits, each hitting a node of the taxonomic tree. Kraken2 then finds the path from the root to a leaf in the tree with the most hits, and the read gets the taxon at the end of that path.

To save memory, Kraken2 doesn't store all k-mers, but only minimizers: within each stretch of 35 bases, by default, only one shorter sequence of 31 bases is stored. It also stores them in a compact hash table, which can give a small rate of wrong matches, and this is what makes it possible to hold such a large database in memory.

Important: by default, a read is classified with very little evidence. The option `--confidence` sets how many of a read's k-mers must support the taxon, and it defaults to 0. With a value of 0, a single matching k-mer can classify a read. A higher value gives fewer but more reliable assignments.

24.3. Input

FASTQ or FASTA, single or paired, with or without compression, and a database, which is a folder with three files: `hash.k2d`, `opts.k2d` and `taxo.k2d`.

24.4. Output

The report (`--report`) is what you usually read. It has one line per taxon, indented by rank, and it's in the form of a tree:

Column	Content
1	Percentage of reads in this clade
2	Number of reads in this clade, including those below it
3	Number of reads assigned directly to this taxon
4	Rank code: `U` unclassified, `R` root, `D` domain, `K`, `P`, `C`, `O`, `F`, `G`, `S` for kingdom through species
5	NCBI taxon number
6	Name, indented

The per-read output goes to the file named by `--output`, or to the standard output without this option. It has one line per read:

Column	Content
1	`C` classified or `U` unclassified
2	Name of the read
3	Taxon number
4	Length
5	Which k-mers hit which taxa

This file is as long as your read file, which is why the pipeline compresses it.

24.5. When to use it

Use it for a quick overview of what's in a sample, especially if a good reference database exists for your habitat, and for finding contamination: human or other unexpected taxa stand out at once.

Keep these limits in mind:

- **It only finds what's in the database.** In rumen samples, many organisms have no close reference. A large share of unclassified reads is normal. It doesn't say that your sample is poor.

- **Percentages of reads aren't abundances of organisms.** Large genomes get more reads than small ones, and Kraken2 doesn't correct for this. Bracken, a separate tool, estimates abundance from the reports. It's not in this pipeline.
- **Species-level calls are often uncertain**, and a tool based on genomes or marker genes, such as sylph or SingleM, is better for that.

24.6. In this pipeline

The rule `read_annotate__kraken2__assign`, which Chapter 62 explains in detail, runs all samples in one job, so that the database has to be loaded once. It copies the database to `/dev/shm` or to a fast disk first, and it reads it with `--memory-mapping`. This is also why this rule uses the whole node, in the resource set `full_node`.

It uses the reads after fastp and **before** host removal. So the report also tells you how much host DNA there is.

No `--confidence` is set, so the default of 0 applies, and you should keep that in mind when reading low-abundance taxa.

Databases are listed in `features.yaml`. The `standard` database needs about 140 GB of memory, and `refseq500` about 1.4 TB.

24.7. Pitfalls

- **False positives at low abundance.** With `--confidence 0`, you'll see many taxa with a few reads each. Most of them aren't real. Look at the upper part of the report first.
- **Too little memory or disk.** The database has to fit. If it doesn't fit into `/dev/shm`, the job fails on purpose in this pipeline, and it waits for a bigger resource set.
- **Shared nodes.** Files in `/dev/shm` count as memory, so on a shared node this could disturb others. That's why it's a full node here.
- **Comparing databases.** Results from two databases aren't directly comparable, because the share of classified reads depends on what's in them.

24.8. Reference

Wood DE, Lu J, Langmead B (2019). Improved metagenomic analysis with Kraken 2. *Genome Biology*. https://doi.org/10.1186/s13059-019-1891-0

25. Krona

25.1. What it does

Krona draws a taxonomic profile as an interactive pie chart with many rings. The inner rings are the high ranks, and they get finer toward the outside. You can click on any part to zoom into it. The result is a single HTML file, which needs nothing but a browser, so it's easy to share.

It's a tool for looking at data and showing it. It doesn't compute anything new.

25.2. Input

`ktImportTaxonomy` reads a table with a taxon number per line. The options say which column holds what:

Option	Meaning	Default
`-q`	Column of the query ID, such as a read name	1
`-t`	Column of the taxon number	2
`-m`	Column of the magnitude, such as a count. Without it, each line counts once	None
`--tax`	Folder of the taxonomy database	Inside the installation

The taxonomy is a small database (`taxonomy.tab`), and it has to fit the taxon numbers in your data.

25.3. Output

One HTML file. Everything is inside it, and you can add several samples to a single chart.

25.4. When to use it

Use it for exploring one or a few samples, and for showing a colleague what's in them. For comparing many samples, a table or a bar chart works better, as it's hard to compare two pie charts by eye.

25.5. In this pipeline

The rule `read_annotate__krona__visualize` makes one chart per sample and Kraken2 database, from the report:

```
ktImportTaxonomy -t 5 -m 3 {input} -o {output} --tax
 ↪  {params.krona_db}
```

The columns are those of a Kraken2 report, which Chapter 24 lists: the taxon is in column 5, and the reads assigned directly to it are in column 3.

Krona prints `Score column already in use; not reading scores`. This is harmless: the default column for scores is 3 too, and a report has no scores.

> ⚠ Charts from versions before 0.4.69 are wrong
>
> The rule used to say `-q 2 -t 3`. These options fit the per-read output of Kraken2, in which column 2 is the read name and column 3 is the taxon. But the rule reads the report, in which these two columns are read counts, and so Krona took a count for a taxon number.
> In a test with the real tool and the pipeline's taxonomy database, on a small report written by hand, 800 reads assigned to *Escherichia coli* (taxon 562) appeared as *Ancylobacter aquaticus*. `tests/krona/run_test.sh` now checks this. The Kraken2 reports themselves were always right.

25.6. Pitfalls

- **A wrong column gives no error**, as this bug shows. Krona draws whatever it gets, so check a chart against the table it comes from.
- **Counts are added up by Krona.** Use the column of the reads assigned directly to a taxon, and not the one with the whole clade. Otherwise, reads get counted several times.
- **An old taxonomy.** Taxon numbers that aren't in the database appear as "No hits". The taxonomy should be about as recent as the database of the classifier.

25.7. Reference

Ondov BD, Bergman NH, Phillippy AM (2011). Interactive metagenomic visualization in a Web browser. *BMC Bioinformatics*. https://doi.org/10.1186/1471-2105-12-385

26. SingleM

26.1. What it does

SingleM profiles a community with marker genes that nearly all Bacteria and Archaea carry once. It tells you which organisms are present and how much of each. It can also estimate which share of a sample is microbial at all. Because it looks at conserved genes, it can see organisms that have no genome in any database, at least at the level of a higher rank.

26.2. How it works

SingleM looks for a set of single-copy marker genes in the reads. In these genes, it looks at a fixed short window of 60 bases. Reads that cover the window are compared with a reference set, and each window sequence gets a taxonomy.

The idea is that a window in a conserved gene is nearly unique to each species, and comparing all the reads with such windows is much cheaper than comparing them with whole genomes. Because the genes occur once per genome, the number of reads on them directly reflects the number of cells, and it doesn't depend on the size of the genome. This is a real advantage over counting reads on genomes.

The results of all markers are then combined into one profile, in the step called `condense`.

`singlem microbial_fraction` compares two numbers: how many bases should come from Bacteria and Archaea according to the profile and their known genome sizes, and how many bases there are in total. The ratio estimates the share of microbial DNA. In a host-associated sample, this is often the first thing you want to know.

26.3. Input

Command	Input
`singlem pipe`	FASTQ (`--forward`, `--reverse`) and a metapackage
`singlem condense`	Archive tables from `pipe`

Command	Input
`singlem microbial_fraction`	FASTQ and a profile from `pipe`

The metapackage is SingleM's database. Here it's the one for GTDB R220.

26.4. Output

File	Content
`--taxonomic-profile`	The profile: sample, coverage and taxonomy per line. This is usually what you read
`--otu-table`	Window sequences per marker, with counts and taxonomy
`--archive-otu-table`	The same in JSON, for combining later
`microbial_fraction` output	The share of microbial reads per sample, with its details

26.5. When to use it

Use it when you want a profile that doesn't depend on a reference genome for each species, and when you want to know how much of a sample is microbial. It's fast enough for many samples and its memory use is moderate.

It doesn't look at eukaryotes or viruses. It says nothing about function. And an organism without a close relative in GTDB is reported only at a higher rank, such as order or family, which is what you'd want.

26.6. In this pipeline

Rule	Runs
`read_annotate__singlem__pipe`	`singlem pipe` per library
`read_annotate__singlem__condense`	`singlem condense` over all libraries
`read_annotate__singlem__microbial_fraction`	`singlem microbial_fraction` per library
`read_annotate__singlem__aggregate_microbial_fraction`	Combine fractions over all libraries

As the comment in the rule says, SingleM's documentation recommends using raw reads, because it doesn't need trimming. The pipeline gives it the clean reads, so that all profilers see the same data. This also means that host reads are gone before the microbial fraction is computed. **So in this pipeline, this share refers to the reads after host removal, and it doesn't tell you how much of the raw sample was host.** For that, look at the mapping to the host genomes in Chapter 61.

26.7. Pitfalls

- **Temporary space.** `singlem pipe` can write a lot to `$TMPDIR`, which is why the rule writes the free space there to its log.
- **Differences in coverage.** The coverage in the profile isn't a percentage of reads. It's the estimated average coverage of each organism's genome. Convert it to relative abundance if you need that.
- **Low depth.** With little sequencing, rare organisms aren't seen. This happens with any method.

26.8. Reference

Woodcroft BJ, et al. (2025). Comprehensive taxonomic identification of microbial species in metagenomic data using SingleM and Sandpiper. *Nature Biotechnology*. https://doi.org/10.1038/s41587-025-02738-1

27. sylph

27.1. What it does

sylph looks for known genomes in a metagenome. For each genome in its database, it estimates how similar the closest organism in your sample is (its ANI), and how abundant it is. It's very fast and needs little memory, and it works even for organisms that have very low coverage.

27.2. How it works

sylph doesn't align reads. It uses a small sample of the k-mers of each genome and of the reads, a sketch. It then compares them: if the sample contains an organism identical to a genome in the database, then all k-mers of that genome should show up. If the organism differs slightly, fewer do.

The problem at low coverage is that many k-mers are missing just because they weren't sequenced, and this would look like a low similarity. sylph corrects for this: from how often the k-mers that are present are seen, it estimates the true coverage, and from that the expected share of missing k-mers. This lets it estimate a reliable ANI even at a coverage of well below 1x.

From these values it computes the abundance of each genome, in two forms: by the number of cells (the taxonomic abundance) and by the amount of sequence (the sequence abundance).

27.3. Input

A database of sketched genomes (`.syldb`) and reads, either as FASTQ or as prepared sketches. `-1` and `-2` take paired reads.

27.4. Output

`sylph profile` writes a table with one line per genome that it detects. These are the most useful columns:

Column	Content
`Sample_file`	The reads
`Genome_file`	The genome in the database
`Taxonomic_abundance`	Percentage by number of cells
`Sequence_abundance`	Percentage by amount of sequence
`Adjusted_ANI`	Estimated ANI to the closest organism in the sample
`Eff_cov`	Estimated coverage
`ANI_5-95_percentile`	How certain the ANI is
`Contig_name`	A contig of this genome, whose name often holds the taxonomy

27.5. When to use it

Use it for a fast and accurate profile on the level of species, if your organisms are represented in the database. The `Adjusted_ANI` is a helpful extra, because it tells you how close your organism is to the reference. A value around 95% or below means that it's a relative and not that species.

Like every database method, it doesn't see what isn't in there. It only looks at the genomes in the database, and it has nothing to say about novel groups. For this, SingleM is the better tool, and they complement each other.

27.6. In this pipeline

The rule `read_annotate__sylph_profile` runs one call per library, against the database in `features.yaml`, by default GTDB R220:

```
sylph profile {input.db} -1 {input.forwards} -2 {input.reverses}
 ↪  -t {threads} > {output}
```

There's no step that combines the libraries. You get one table per library, and they are simple to join yourself by `Genome_file`.

The rule still contains a commented part for testing the escalation, and you can ignore it.

27.7. Pitfalls

- **The genome names are file names.** To get a taxonomy, you need the metadata of the database. For GTDB, the accession in `Genome_file` is the link.
- **Abundance refers to what was found.** The percentages add up to 100 over the detected genomes, not over your whole sample. If much of the community isn't in the database, the percentages are too high in absolute terms.
- **A database of the wrong type.** Sketches made for short reads and for long reads are built with different settings. Use the right one.

27.8. Reference

Shaw J, Yu YW (2024). Rapid species-level metagenome profiling and containment estimation with sylph. *Nature Biotechnology*. https://doi.org/10.1038/s41587-024-02412-y

28. MetaPhlAn

28.1. What it does

MetaPhlAn estimates which organisms are in a sample and how abundant they are. It uses marker genes that are specific to a clade: genes that all members of a group have, and that no other organisms have. A read on such a gene is strong evidence for that group.

MetaPhlAn is one of the most used profilers, particularly for human samples, for which it has the best reference data.

28.2. How it works

Its database holds millions of marker genes. Version 4 includes genomes assembled from metagenomes, and it groups them into species-level genome bins (SGBs). This makes it able to report species that were never cultured.

MetaPhlAn maps the reads against all markers with Bowtie2. For each clade, it counts the coverage of its markers, and it computes the average, ignoring the markers with extreme values. It then normalizes these so that they sum up to 100%.

As the markers are specific, a read that maps to them is nearly always right, so this method has few false positives. But it only sees a small part of the reads: most of them don't fall on any marker, and that's expected.

28.3. Input

FASTQ, with the option `--input_type fastq`, and a Bowtie2 index of the markers (`--bowtie2db`). Paired files are given separated by a comma, and they are treated as single reads: MetaPhlAn doesn't use the pairing.

28.4. Output

File	Content
`-o`	The profile, as a table
`--bowtie2out`	The mapping, which allows running MetaPhlAn again with other settings, without mapping again

The profile has one line per clade, at all ranks from kingdom to SGB:

Column	Content
`clade_name`	The clade, such as `k__Bacteria|p__Bacteroidota|...`. The levels are `k__`, `p__`, `c__`, `o__`, `f__`, `g__`, `s__`, `t__`
`NCBI_tax_id`	The taxon numbers of that path
`relative_abundance`	Percentage
`additional_species`	Other species in this SGB

`merge_metaphlan_tables.py` joins the profiles of many samples.

28.5. When to use it

Use it for human samples, and in general where the organisms are well represented in the database. Since it reports relative abundances of the known species, it's a common choice for comparing many samples.

Keep the database in mind in other habitats. In rumen samples, for example, its reports can cover only part of the community. The abundances add up to 100% regardless, so they don't show you how much is missing. An option, `--unclassified_estimation`, adds an estimate of the unknown part.

28.6. In this pipeline

The rule `read_annotate__metaphlan__run` maps the clean reads of each library, and `read_annotate__metaphlan__condense` joins the tables into `metaphlan_profiled.tsv`. As HUMAnN uses the profile, this rule also runs if you only ask for HUMAnN.

The database comes from `features.yaml`, and it is the MetaPhlAn database in version 4.

28.7. Pitfalls

- **A profile that adds up to 100%** can suggest that everything is known, as said above.
- **Database and tool versions must fit.** A version of MetaPhlAn expects a certain database. If they don't match, it stops with an error, or it tries to download one, which can't work in a compute job.
- **Rerunning.** If `--bowtie2out` exists already, MetaPhlAn refuses to run again. Snakemake removes it before a rerun, but keep it in mind for runs by hand.
- **Very low coverage.** A clade needs a minimum share of its markers to be reported. So rare species don't appear, and this gets worse with low sequencing depth.

28.8. Reference

Blanco-Míguez A, et al. (2023). Extending and improving metagenomic taxonomic profiling with uncharacterized species using MetaPhlAn 4. *Nature Biotechnology*. https://doi.org/10.1038/s41587-023-01688-w

29. HUMAnN

29.1. What it does

HUMAnN finds out what a community can do, by profiling gene families and metabolic pathways from the reads. It answers questions like "how much of a certain enzyme family is there, and which species contributes it?"

29.2. How it works

The search works in tiers, starting with what's cheap and precise. HUMAnN uses these steps:

1. **Which species are there?** It takes the profile from MetaPhlAn.
2. **Nucleotide search.** It builds a custom database from the pangenomes of just these species, from the ChocoPhlAn database, and it maps the reads against that with Bowtie2. This is quick, and the result is linked to a species.
3. **Translated search.** The reads that didn't map are translated, and they are compared with a large protein database, UniRef, using DIAMOND. This finds genes of organisms without a pangenome. It's slower, and the results aren't linked to a species, so they appear as "unclassified".
4. **Pathways.** From the gene families it infers pathways, using the MetaCyc database.

The abundances are given in reads per kilobase (RPK), so they are corrected for the length of a gene but not yet for sequencing depth.

29.3. Input

One file of reads, such as FASTQ, plus the databases. Pairing isn't used, so paired reads are simply joined. The MetaPhlAn profile is optional, and HUMAnN would otherwise run MetaPhlAn itself.

29.4. Output

File	Content
`<name>_genefamilies.tsv`	Abundance of each gene family, in total and per species
`<name>_pathabundance.tsv`	Abundance of each pathway
`<name>_pathcoverage.tsv`	How complete each pathway is

In all of these, a line without `|` is the total for that family. A line with `|` is the share of one species. The special lines `UNMAPPED` and `UNINTEGRATED` hold what couldn't be assigned, and they are a useful measure of how much of your sample HUMAnN could explain.

The tool comes with helpers: `humann_renorm_table` normalizes for depth, `humann_join_tables` joins samples, and `humann_regroup_table` translates gene families into other systems, such as KEGG orthologs.

29.5. When to use it

Use it for comparing functions between samples, particularly for human-associated communities where the reference data are best. It's a read-based alternative to annotating an assembly, and it doesn't need any assembly, so it also sees rare organisms that can't be assembled.

The limits are in the databases: much of what's in a rumen sample isn't in ChocoPhlAn, and so it depends on the slower translated search. It's also heavy, with long run times and a need for lots of temporary disk space. For new or poorly known communities, the annotation of contigs in Chapter 66 is often more informative.

29.6. In this pipeline

The rule `read_annotate__humann__run` joins the clean reads of a library and runs:

```
humann --input {output.cat} --output {params.out_folder} \
       --threads {threads} \
       --protein-database {input.prot_dbs} \
       --nucleotide-database {input.nt_dbs} \
       --output-basename {params.out_name} \
       --taxonomic-profile {input.mp_out} \
```

```
        {params.additional_options}
```

The option `--remove-temp-output` deletes the large intermediate files at the end.

`read_annotate__humann__condense` then writes `humann_genefamilies.tsv`. Note that this is a plain concatenation of the tables, so the result has one block per library, each with its own header. For a real table with one column per sample, use `humann_join_tables` on the single files.

Only the gene families are declared as output, but the pathway tables are also in the same folder.

29.7. Pitfalls

- **Temporary space.** The rule warns about this: `TMPDIR` has to be large enough for big data sets.
- **Normalizing.** The values are in RPK. To compare samples, you must normalize first, usually to counts per million (CPM) with `humann_renorm_table`.
- **Versions.** The versions of HUMAnN, MetaPhlAn and the ChocoPhlAn database have to fit together. If the profile comes from a version with other species names, HUMAnN finds no matching pangenomes.
- **Comparing `UNMAPPED` between samples.** It depends strongly on the database, and it's not a biological result.

29.8. Reference

Beghini F, et al. (2021). Integrating taxonomic, functional, and strain-level profiling of diverse microbial communities with bioBakery 3. *eLife*. https://doi.org/10.7554/eLife.65088

30. phyloFlash

30.1. What it does

phyloFlash looks at the small-subunit ribosomal RNA genes (SSU rRNA, 16S in Bacteria and Archaea, 18S in eukaryotes) in a metagenome. These genes are the classic marker of microbial taxonomy, and their databases are huge. So this tool links your metagenome to decades of work with amplicons. Unlike most of the other profilers here, it sees all three domains of life, and so eukaryotes as well.

30.2. How it works

1. It maps the reads against the SILVA database of SSU rRNA with BBmap, and it extracts those that hit.
2. It gets a taxonomy for these reads from their best hit, which gives a quick profile.
3. It assembles the extracted reads with SPAdes to get full-length SSU sequences, and it compares them with the database.
4. It estimates the abundance of each assembled sequence, and how much of the SSU reads the assembled sequences explain.

30.3. Input

Reads as FASTQ (`-read1`, `-read2`), and a database folder (`-dbhome`) that was prepared with a script for it, from SILVA. Here this is version 138.2. It also needs a library name (`-lib`).

30.4. Output

File	Content
`<lib>.phyloFlash.html`	The report for a browser, the best place to start
`<lib>.phyloFlash.log`	The log, with a summary
`<lib>.phyloFlash.extractedSSUclassifications.tsv`	Taxonomic assignments of the mapped reads, with counts

File	Content
`<lib>.spades_rRNAs.final.fasta`	Assembled SSU sequences
`<lib>.phyloFlash.tar.gz`	The other results, as an archive

30.5. When to use it

Use it when you want to compare with amplicon studies, when you have many eukaryotes or want to see them at all, or when you want full-length rRNA sequences from a metagenome. It's a good second opinion on the composition of a sample.

Be careful with abundances. The number of rRNA gene copies varies between organisms, from one to more than ten, so counting these genes doesn't count cells. And since the gene is highly conserved, the assembly can merge similar species, and it can create chimeras.

30.6. In this pipeline

The rule `read_annotate__phyloflash__run` runs it on the clean reads of each library:

```
phyloFlash.pl -dbhome {input.phyloflash_dbs} \
              -lib {params.lib} -CPUs {threads} \
              -read1 {input.forward_} -read2 {input.reverse_} \
              -almosteverything
```

`-almosteverything` switches on the optional analyses, except EMIRGE, a slower alternative for reconstructing sequences.

phyloFlash writes its results to the current folder, here the project folder, so the rule moves them afterwards. If a job is killed, files named after the library can stay behind in the project folder.

The library name has an underscore instead of the usual dot, as in `ERR2019410_lib1`.

`read_annotate__phyloflash__condense` concatenates the logs into `phyloflash_all.log`.

30.7. Pitfalls

- **Host rRNA.** Eukaryotic SSU reads of the host would dominate, if host reads were still there. This pipeline removes them first, so this isn't an issue here, as long as the host genome is in `features.yaml` and it contains those genes.
- **Read length.** Very short reads don't work well. The tool has a lower limit on read length, and it warns about it.
- **The share of SSU reads is small.** Usually it's well below 1% of reads. So with little sequencing, there may be too few reads for an assembly, and the report will be empty.

30.8. Reference

Gruber-Vodicka HR, Seah BKB, Pruesse E (2020). phyloFlash: Rapid Small-Subunit rRNA Profiling and Targeted Assembly from Metagenomes. *mSystems*. https://doi.org/10.1128/mSystems.00920-20

31. DIAMOND

31.1. What it does

DIAMOND compares DNA or protein sequences with a protein database, as BLAST does, but thousands of times faster. It's the tool to use when you want to know which known proteins your reads or genes resemble. It's often a hidden component of other tools too, like eggNOG-mapper, HUMAnN, CAMPER and DRAM.

31.2. Why compare at the protein level

DNA changes faster than proteins, because different codons can code for the same amino acid, so two distantly related genes can look unrelated as DNA, but alike as proteins. So to find the function of a gene, you often want to compare proteins.

31.3. How it works

Like BLAST, DIAMOND looks for short exact matches, called seeds, and extends them into alignments. To make that fast, it uses a few tricks: it reduces the alphabet of amino acids, it uses seeds with a pattern of positions that have to match, and it sorts both the query seeds and the database seeds and walks through both lists together.

The sensitivity is a choice. The default is quick, and it finds strong similarities. Options such as `--sensitive` and `--ultra-sensitive` find more distant matches, and they take longer.

31.4. Input

Command	Input
`diamond makedb`	Protein FASTA, which it turns into a `.dmnd` database

Command	Input
`diamond blastx`	DNA reads or genes (`-q`), translated in all six reading frames, and a database (`-d`)
`diamond blastp`	Proteins (`-q`), and a database (`-d`)

31.5. Output

By default, DIAMOND writes the tabular format of BLAST (format 6), with these columns:

Column	Content
1, 2	Query and database sequence
3	Percentage of identical positions
4	Length of the alignment
5, 6	Mismatches and gap openings
7, 8	Start and end on the query
9, 10	Start and end on the database sequence
11	E-value: how many such hits you'd expect by chance
12	Bit score

A query with no hit doesn't appear. So to get the share of reads with a hit, you have to divide by the number of reads yourself.

31.6. When to use it

Use it to search reads or proteins against a specific protein database, when you're interested in a certain function: CAZymes, hydrogenases, antibiotic resistance. Read-based use has one great advantage, in that it needs no assembly. It also has weaknesses: a read of 100 bases gives only about 33 amino acids, so distant matches are uncertain.

Comparing reads with a whole general database like UniRef takes much longer, and it can be done, but specialized tools like HUMAnN do it in a smarter way.

31.7. In this pipeline

Rule	Mode	Query	Database
`read_annotate__diamond__assign`	blastx	Each mate of the clean reads	`diamond` in `features.yaml`, by default CAZy
`contig_annotate_diamond__assign`	blastp	Proteins from Prodigal	`diamond_contig_protein`, by default HydDB

The rule for reads uses `--max-target-seqs 1`, so it reports just one hit per read, and this makes counting simple. Both rules first copy the database to fast storage, as Chapter 62 explains.

Sensitivity stays at the default, so distant homologs may be missed.

An R script turns the hits of the reads into feature tables, in `read_annotate__diamond__summarise`. It's chosen by the name of the database, so for a new database you have to provide such a script.

31.8. Pitfalls

- **`--max-target-seqs 1` doesn't mean the best hit.** DIAMOND reports the best hit it finds with the current settings, and with the fast defaults this is usually, but not always, the best one in the database.
- **Memory.** It's controlled by the block size `-b`, and the number of chunks `-c`. By default a job of this kind needs some tens of GB. If it's killed for memory, make the blocks smaller.
- **The two mates count separately.** In the rule for reads, each mate gets its own hit. Don't count them as two independent observations of a fragment.
- **Hits aren't functions.** A hit to an enzyme with low identity doesn't prove the same activity. Filter by identity and by alignment length.

31.9. Reference

Buchfink B, Reuter K, Drost HG (2021). Sensitive protein alignments at tree-of-life scale using DIAMOND. *Nature Methods*. https://doi.org/10.1038/s41592-021-01101-x

32. NCycDB

32.1. What it does

NCycDB is a curated database of the genes of the nitrogen cycle, along with a script that profiles samples against it. Nitrogen matters in a rumen, in soil, and in many other places: fixation, nitrification, denitrification, and the conversions between them. General databases can blur these gene families. For example, some of them are close relatives that do different things. NCycDB was built to keep these apart.

32.2. How it works

The database holds the protein sequences of 68 gene families of the nitrogen cycle, collected and curated by hand. It also contains homologs from related families. They don't count, but they catch reads that would otherwise be assigned wrongly to a family of interest.

The Perl script `NCycProfiler.PL` compares each sample with the database, using a search tool such as DIAMOND, keeps the best hits, and adds up the counts per gene family. It normalizes them by the sequencing depth of each sample.

32.3. Input

Option	Content
`-d`	Folder with the sequence files
`-f`	Their ending, such as `fq.gz` or `fa`
`-s`	The type of the sequences, `nucl` or `prot`
`-si`	A table of sample names and files
`-m`	The search tool, here `diamond`
`-db_faa`, `-db_folder`	The database and where to find its files

32.4. Output

One table, with gene families as rows and samples as columns, normalized. The result file is what `-o` names, and `-of` is the folder for its working files.

32.5. When to use it

Use it when the nitrogen cycle is part of your question. For a general view of function, other tools are a better start. This one is a specialized addition, and that's why it isn't in the main targets.

32.6. In this pipeline

It runs twice, and both are outside the default run. You call them with `read_annotate_extra` and `contig_annotate_extra`.

Rule	Input	Mode
`read_annotate__ncyc__prepare_inputs`, `read_annotate__ncyc__run`	Clean reads	`-s nucl`
`contig_annotate__ncyc__prepare_inputs`, `contig_annotate__ncyc__run`	Proteins from Prodigal	`-s prot`

`workflow/scripts/prepare_ncyc_inputs.sh` writes the sample table. The Perl script comes with the pipeline, in `workflow/scripts/NCycProfiler.PL`, so that the version is fixed. The database is the entry `ncyc` in `features.yaml`, and all its files have to be in the same folder as `NCyc_100.faa`, as the rule assumes.

32.7. Pitfalls

- **All samples at once.** Both rules run all samples in one job. It has to finish as a whole, so a failure late in the job costs the time for all samples.
- **Reads versus proteins.** The two runs measure different things: one counts reads per family, the other counts predicted genes per family. A high abundance of a gene in the reads doesn't need to mean many different genes.
- **Fixed database file name.** `features.yaml` points to one FASTA file, and the rule derives the folder from it.

32.8. Reference

Tu Q, Lin L, Cheng L, Deng Y, He Z (2019). NCycDB: a curated integrative database for fast and accurate metagenomic profiling of nitrogen cycling genes. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/bty741

33. Nonpareil

33.1. What it does

Nonpareil estimates how well your sequencing covered the community. In other words: of all the DNA in the sample, what share did you see at least once? It answers a question you want answered before spending weeks on an assembly: was this enough sequencing, or would more help?

It needs no database or reference, which makes it especially useful for poorly known communities.

33.2. How it works

The idea is simple. If you sequence a community deeply, a new read will very likely overlap with a read you already have. If you sequenced too little, most new reads are about something you haven't seen yet. So the redundancy of the reads tells you about the coverage.

Nonpareil takes a sample of the reads as queries, and for each it checks whether it matches any of the other reads. In `kmer` mode, it does this by comparing k-mers of 24 bases, and this is fast. From this it computes the redundancy at many depths, by sub-sampling, and this gives a curve of coverage against sequencing effort.

The R package `Nonpareil` fits a model to this curve, and it reports these values:

Value	Meaning
`C`	The coverage of the data set, from 0 to 1
`LR`	How much sequence you've actually got
`LRstar`	How much sequence you'd need to reach 95% coverage
`diversity`	Nonpareil diversity (Nd), a measure of the complexity of the community

Because the diversity is on a scale that doesn't depend on depth, you can use it to compare communities. Rumen samples tend to have high values.

33.3. Input

One FASTQ or FASTA file (`-s`). It uses **one mate only**, because the two mates of a pair aren't independent. It needs an uncompressed file, and it can't read from a pipe. Reads have to be at least as long as the k-mer, and this is why fastp keeps reads of 25 bases or more.

33.4. Output

All files share the prefix from `-b`:

File	Content
`.npo`	The redundancy curve, which is the main result
`.npa`	All pairs of values, in full detail
`.npc`	Raw counts per query read
`.npl`	The log

33.5. When to use it

Use it early, on all samples, in any metagenome project. The results help you to decide about further sequencing, and they can explain poor assemblies later. If the coverage is low, say below 60%, you shouldn't expect to recover genomes of more than the most abundant organisms.

33.6. In this pipeline

The rule `read_annotate__nonpareil__run` unpacks the forward reads, and it runs:

```
nonpareil -s "${{prefix}}_1.fq" -T kmer -b "$prefix" -f fastq -t
 ↪  {threads} -X {params.X}
```

`read_annotate__nonpareil__aggregate` then runs `workflow/scripts/nonpareil_summary.R`, which reads all `.npo` files with the R package, and it writes `nonpareil.tsv`, with one line per library.

`-X` is the number of query reads, and the default in `params.yaml` is 100, for quick testing. The comment there says so. Fewer query reads make the estimate less precise, so raise this value for an analysis you want to report.

33.7. Pitfalls

- **`-X` too low**, as just explained.
- **Too little data.** With low coverage, the model can't be fitted. The values `C` and `LR` are still reported, but `modelR` and `LRstar` are `NA` in the summary, and `diversity` is 0, which doesn't mean a community without diversity. The R package warns with `Median of the curve is zero at 50% of the reads`. With even less data, Nonpareil can leave empty files. Their outputs are marked `touch()` in the rule, so that Snakemake can go on, and such a library is left out of the summary.
- **Too many reads for its memory.** Nonpareil has its own memory setting, `-R`, and on large read files it may use only part of the reads.
- **Disk space.** The uncompressed file takes space until the rule deletes it, and if a job gets killed, it stays there.

33.8. References

Rodriguez-R LM, Konstantinidis KT (2014). Nonpareil: a redundancy-based approach to assess the level of coverage in metagenomic datasets. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btt584

Rodriguez-R LM, Gunturu S, Tiedje JM, Cole JR, Konstantinidis KT (2018). Nonpareil 3: Fast Estimation of Metagenomic Coverage and Sequence Diversity. *mSystems*. https://doi.org/10.1128/mSystems.00039-18

34. Assembly: the idea

Assembly is the step that turns short reads into long contigs. Both assemblers of the pipeline work on the same principle, so here's the idea, before looking at each of them.

34.1. The de Bruijn graph

If you wanted to find overlaps directly, you would compare each read with every other. With hundreds of millions of reads, that's out of the question. Short-read assemblers do something smarter.

They cut all reads into k-mers. Then they build a graph in which each k-mer is a connection between two nodes, its first k−1 bases and its last k−1 bases. Overlapping k-mers end up on a path through this graph. If a stretch of the genome is unique, there's a single path, and you can walk along it and read off a contig.

The graph has to hold only the distinct k-mers and not all reads, so its size depends on how complex the community is, rather than on the amount of sequencing. That's why deep sequencing of a simple sample needs less memory than shallow sequencing of soil.

34.2. The choice of k

The value of k is a trade-off:

- **A small k** connects reads even at low coverage. But short k-mers occur in many places by chance, so the graph gets tangled.
- **A large k** separates repeats and similar organisms well. But two reads must overlap by at least k bases, so regions of low coverage fall apart.

A community has abundant organisms and rare ones, so no single k fits. Both assemblers therefore run several times with growing k, and each round builds on the contigs of the round before. This is called iterative assembly. Small k-mers recover the rare organisms, and large ones resolve the abundant ones.

A read shorter than k can't add a k-mer of its own, so the largest useful k is usually close to the read length. Beyond that, a round can only build on the contigs of the round before.

34.3. Why metagenomes are hard

Assemblers for a single genome expect even coverage, and they treat a sudden change in coverage as an error. In a metagenome, coverage differs by orders of magnitude between organisms, so low coverage can be real.

Worse, close relatives share long stretches of DNA. In the graph, these form bubbles and tangles. Assemblers break contigs at such points to avoid mixing organisms, and that's why metagenome assemblies are so fragmented, and why strain diversity usually lowers their quality.

34.4. Which to choose

	metaSPAdes	MEGAHIT
Quality	Usually longer contigs	Usually somewhat shorter
Memory	High, often several hundred GB for large data	Low, often an order of magnitude less
Time	Long	Short
Co-assembly of many samples	Often too heavy	Designed for it
Assembly graph available	Yes, needed by 4CAC	Not kept here
Several libraries	Only one, so they are pooled	Yes

For a first look, or for very large co-assemblies, start with MEGAHIT. If metaSPAdes finishes on your data, its assemblies tend to give better bins. The setting `assembler` in `config.yaml` switches between them.

35. MEGAHIT

35.1. What it does

MEGAHIT assembles metagenomes, with very little memory for the size of data it can handle. It's the assembler of choice for very large or very complex data.

35.2. How it works

It uses a succinct de Bruijn graph, a compressed form of the graph that takes only a few bits per k-mer. It runs iteratively, from `--k-min` to `--k-max` in steps of `--k-step`. At low k, it also adds so-called mercy k-mers, to rescue k-mers of low coverage that would otherwise break a contig.

35.3. Input

FASTQ or FASTA. Paired reads go in with `-1` and `-2`, and several libraries are given as a list separated by commas.

35.4. Output

File	Content
`final.contigs.fa`	The contigs. The names look like `k141_1234`, and the header holds the coverage and length
`intermediate_contigs/`	Results of each k
`log`	All steps. It also includes a summary with the number of contigs and the N50
`options.json`, `done`	State, which allows resuming with `--continue`

35.5. When to use it

Use it for large co-assemblies, for soil and other very complex samples, and when memory is limited. Its presets are named for this: `meta-sensitive` for more sensitivity at the price of time, and `meta-large` for large and complex data, with `--k-min 27 --k-max 127 --k-step 10`.

35.6. In this pipeline

The rule `assemble__megahit__run` runs it when `assembler` is `"megahit"`, as Chapter 63 shows. All the k-mer options come from `params.yaml`.

- The default of `--k-max 141` suits reads of 150 bases. MEGAHIT doesn't lower it for shorter reads: in a real run with reads of 101 bases, it went through all k-mer sizes up to 141. For such reads, a lower value in `params.yaml` can save time.
- `--min-contig-len 1500` removes short contigs right away. MEGAHIT's own default is 200, but the binners use contigs from about 1500 bases, so shorter ones would mostly be ballast here. If you want the short contigs too, for gene-based work, then lower this value. The comment in `params.yaml` says the same.
- `--force` is needed because the folder exists already, and `--continue` lets it resume.

35.7. Pitfalls

- **`--memory`.** By default, MEGAHIT may use 90% of the machine's memory. In a SLURM job, this means the node's memory, not the job's, and so it can go beyond the allocation and get killed. If this happens, give it a limit through `additional_options` in `params.yaml`.
- **Temporary disk space.** It needs a lot of space in the output folder while running.
- **`--no-mercy`.** Recommended for samples with deep coverage (over 30x), where the mercy k-mers only add noise to the graph.

35.8. Reference

Li D, Liu CM, Luo R, Sadakane K, Lam TW (2015). MEGAHIT: an ultra-fast single-node solution for large and complex metagenomics assembly via succinct de Bruijn graph. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btv033

36. metaSPAdes

36.1. What it does

metaSPAdes is the metagenome mode of the SPAdes assembler. It usually yields longer and more accurate contigs than MEGAHIT, and it needs much more memory and time.

36.2. How it works

It starts by correcting read errors, using BayesHammer. It then builds de Bruijn graphs at several values of k, and it simplifies them with methods for uneven coverage. It uses the pairing of the reads to resolve repeats, and it tries to follow the dominant strain through the variation between close relatives, rather than breaking at each difference.

36.3. Input

Paired reads with `-1` and `-2`. In this mode, it accepts exactly **one paired-end library**. So several libraries have to be pooled first. Single-end reads alone aren't supported in the metagenome mode.

36.4. Output

File	Content
`contigs.fasta`	The contigs, named like `NODE_1_length_5000_cov_12.5`
`scaffolds.fasta`	Contigs joined with the help of read pairs
`assembly_graph_with_scaffolds.gfa`	The assembly graph
`contigs.paths`, `scaffolds.paths`	How the sequences run through the graph
`spades.log`, `params.txt`	The log and the settings

The name of a contig tells you about its length and coverage, which is handy. A folder for each k holds the intermediate results, so that a working folder can hold a large number of files.

36.5. When to use it

Use it for samples of moderate size, when you have the memory. It's a good default for single samples or small co-assemblies. For a very large co-assembly, it may not finish, so then go to MEGAHIT.

36.6. In this pipeline

The rule `assemble__metaspades__run` runs it, as Chapter 63 shows, and it uses `contigs.fasta`, not the scaffolds. The working folder gets packed into an archive at the end, and 4CAC reads its graph from there.

Three settings are important:

- `-k 21,33,55.` This is the default in the metagenome mode for reads of this length. If you have reads of 150 bases, you could try larger values, such as `21,33,55,77`.
- `--only-assembler` skips the read correction, which saves much time and memory, at some cost in quality. This was chosen for this pipeline, and you can remove it in `params.yaml` if you can afford it.
- `-m.` metaSPAdes takes this in gigabytes, and it stops when it reaches that limit. The rule gives it `{resources.mem_mb}`, which is in megabytes, so the value is about a thousand times too high. In effect it's no limit, and it's the job's memory limit that counts. The job then gets killed by SLURM, instead of metaSPAdes stopping by itself.

`metaspades_slots` in `config.yaml` limits how many run at the same time.

36.7. Pitfalls

- **Running out of memory** is the most common failure. The log's last lines tell you how far it got. To give it more, add a larger resource set to the rule in `escalation.yaml`.
- **Restarting.** After a failure of the job, the working folder stays there. To use it, you could resume with `--continue`, but the rule doesn't do that.
- **Many small files.** These can be a real problem for shared file systems, and they are also a reason for packing them into an archive.

36.8. Reference

Nurk S, Meleshko D, Korobeynikov A, Pevzner PA (2017). metaSPAdes: a new versatile metagenomic assembler. *Genome Research*. https://doi.org/10.1101/gr.213959.116

37. Binning: the idea

After assembly, you have contigs from many organisms, all mixed. Binning sorts them into groups, one per organism, without knowing in advance which organisms there are. It relies on two signals.

Composition. Each genome has a typical pattern of short words. The frequency of the four-letter words (tetranucleotides) is rather stable along a genome, and it differs between genomes. This works on contigs of a few kilobases, but it's weak on shorter ones. It also can't separate close relatives well.

Abundance. All contigs of a genome come from the same cells, so they should have about the same coverage in a sample. An organism that's abundant in one sample and rare in another shows this pattern on all of its contigs. With many samples, this signal is very strong, as two organisms rarely follow exactly the same pattern.

This is why **the number of samples mapped to an assembly matters so much**. With one sample, you have one coverage value per contig, and organisms with the same abundance can't be told apart. With a co-assembly of several samples, you have many values.

37.1. What can go wrong

- **Contamination:** a bin holds contigs of different organisms.
- **Fragmentation:** one genome ends up in several bins.
- **Missing parts:** mobile elements and islands often have an unusual composition and coverage, so they stay out of the bin they belong to. Even a very good MAG usually lacks them.
- **Unbinned contigs:** many contigs, particularly short ones, aren't in any bin.

Each binner makes different mistakes. That's the reason the pipeline runs three, and uses MAGScoT to combine the results.

38. CONCOCT

38.1. What it does

CONCOCT clusters contigs by composition and coverage, using a statistical model. It's the first of the three binners of this pipeline.

38.2. How it works

It first cuts long contigs into pieces of 10 kb. In this way, a long contig gets more weight, and pieces of one contig can even be compared with each other.

For each piece, it computes the k-mer frequencies and the coverage in each sample. It reduces this to a few dimensions with a principal component analysis. Then it fits a Gaussian mixture model, with an upper limit of clusters. By default, this limit is 400.

Finally, each contig goes to the cluster that got most of its pieces.

38.3. Input

Tool	Input
`cut_up_fasta.py`	The contigs
`concoct_coverage_table.py`	The pieces as a BED file, and sorted, indexed BAM files
`concoct`	The pieces (`--composition_file`) and the coverage table (`--coverage_file`)
`merge_cutup_clustering.py`	The clustering of the pieces
`extract_fasta_bins.py`	The contigs and the merged clustering

38.4. Output

File	Content
`clustering_gt1000.csv`	Which cluster each piece went to, for contigs of at least 1000 bases
`clustering_merged.csv`	The same for whole contigs
`<n>.fa`	One FASTA file per bin

38.5. When to use it

Use it as one voice among several. It tends to work best with many samples, and it needs them more than others do. On its own, it can split genomes into several parts, and that's another reason for following it with a refinement step.

38.6. In this pipeline

The rule `assemble__concoct__run` follows the steps of the manual, as Chapter 63 describes, and it keeps only the bins. It's the only binner with its own container, `hrp_concoct`.

The rule uses the default settings of CONCOCT, and it doesn't filter bins by size. Small bins are filtered out later on, by MAGScoT.

38.7. Pitfalls

- **Few samples.** With a single sample, the coverage signal is nearly useless.
- **Memory.** The size of the matrix grows with the number of contigs and samples.
- **Many small bins.** That's expected, and it's no reason for concern, as long as a refinement step follows.

38.8. Reference

Alneberg J, et al. (2014). Binning metagenomic contigs by coverage and composition. *Nature Methods*. https://doi.org/10.1038/nmeth.3103

39. MaxBin2

39.1. What it does

MaxBin2 assigns contigs to bins with an iterative statistical method. Unlike the others, it first estimates how many genomes there are by using marker genes.

39.2. How it works

1. It looks for 107 single-copy marker genes on the contigs, using FragGeneScan and HMMER. Their counts tell it about how many genomes to expect, and contigs with these genes become the starting points.
2. It runs an expectation-maximization algorithm: from the current guess, it computes for each contig how likely it is to belong to each bin, by tetranucleotide frequencies and coverage. Then it updates the bins, and it repeats this.

39.3. Input

Option	Content
`-contig`	The contigs
`-abund`	A table with contig name and abundance. For several samples, use `-abund2`, `-abund3`, and so on, or `-abund_list`
`-out`	The prefix of the output
`-min_contig_length`	Ignore shorter contigs. The default is 1000

39.4. Output

File	Content
`<out>.001.fasta` and so on	The bins
`<out>.summary`	Completeness and size of each bin

File	Content
`<out>.noclass`	Contigs that didn't fit any bin
`<out>.tooshort`	Contigs that were too short
`<out>.log`	The log

39.5. When to use it

Use it as one voice among several, particularly if you have few samples, since it's less dependent on many of them. Its choice of the number of bins is sound, but it finds only prokaryotes, because of the marker genes.

39.6. In this pipeline

The rule `assemble__maxbin2__run`, which Chapter 63 shows in detail, gives it a single abundance file, even with several libraries. The abundance comes from `samtools coverage`, by adding up over all libraries:

```
samtools coverage {input.alignments} | sed '/^#/d' | cut -f 1,5 >
 ↪  {output}/maxbin2.coverage
```

There are two things to note here. First, column 5 is the number of covered bases, while MaxBin2 asks for an abundance, such as the mean depth, which is column 7. So a longer contig gets a higher value. And second, as all libraries are summed up, MaxBin2 doesn't get the differences between samples, which is its strongest signal.

The rest of the rule renames the bins to `.fa` and compresses them. Other files of MaxBin2 are removed.

39.7. Pitfalls

- **Few marker genes.** With a small assembly, it may find too few marker genes to start, and it then produces no bins. On small test data, this is what to expect.
- **Long run time.** On large assemblies, MaxBin2 can be the slowest of the three.

39.8. Reference

Wu YW, Simmons BA, Singer SW (2016). MaxBin 2.0: an automated binning algorithm to recover genomes from multiple metagenomic datasets. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btv638

40. MetaBAT2

40.1. What it does

MetaBAT2 bins contigs by tetranucleotide frequencies and abundance, using a graph. It's fast, it needs few settings, and it's one of the most widely used binners.

40.2. How it works

For every pair of contigs, it computes a score of how alike they are, in composition and in abundance. It then treats the contigs as nodes of a graph, with these scores on the edges, and it partitions this graph into groups. Contigs that aren't placed at first are added later, by looking at their neighbors. It adapts the weights automatically, which is why it hardly needs any tuning.

40.3. Input

Tool	Input
`jgi_summarize_bam_contig_depths`	Sorted BAM files
`metabat2`	The contigs (`-i`) and the depth table (`-a`)

40.4. Output

File	Content
Depth table	One line per contig: name, length, total depth, and a mean and variance per library
`<o>.1.fa` and so on	The bins

40.5. When to use it

Use it nearly always. It's a good default, and on its own, it often gives the largest number of good bins. It makes use of many samples, but it also works with one.

40.6. In this pipeline

The rule `assemble__metabat2__run` makes BAM files, computes the depths and runs:

```
metabat2 -i {input.contigs} -a "$depth" -o {output}/bin -m
 ↪  {params.min_contig} -t {threads} -v
```

`-m 1500` is the smallest value MetaBAT2 accepts. Also note that MetaBAT2 drops bins smaller than 200 kb by default, and it doesn't report unbinned contigs.

40.7. Pitfalls

- **`jgi_summarize_bam_contig_depths` needs sorted BAM files.** The mapping rule sorts them.
- **No bins.** On a small or poor assembly, it can produce nothing at all without any error, and this is a valid result. Look at the log.
- **Differences between runs.** A random seed influences the result. `--seed` makes runs repeatable.

40.8. Reference

Kang DD, et al. (2019). MetaBAT 2: an adaptive binning algorithm for robust and efficient genome reconstruction from metagenome assemblies. *PeerJ*. https://doi.org/10.7717/peerj.7359

41. Prodigal

41.1. What it does

Prodigal finds protein-coding genes in the DNA of Bacteria and Archaea. It takes a sequence, and it reports where genes lie and which proteins they code for. Nearly all annotation in this pipeline starts from its output.

41.2. How it works

A gene of a prokaryote is a long open reading frame, without a stop codon, and it has a typical pattern of codons. Prodigal looks at all possible open reading frames, and it scores them by things such as the use of codons, the start codon and the site where the ribosome binds. Then it chooses the best-scoring set of genes that fit together.

Normally, it learns these patterns from the genome itself, in a training step. A contig from a metagenome is too short for that, and it comes from an unknown organism. So in **meta mode** (`-p meta`), Prodigal uses a set of precomputed models, and for each sequence it picks the model that fits best.

41.3. Input

FASTA with nucleotide sequences (`-i`, or the standard input).

41.4. Output

Option	Content
`-a`	Protein sequences
`-d`	Nucleotide sequences of the genes
`-o`	Positions, in the format from `-f`, such as `gff` or `gbk`

A protein is named after its contig plus a number, as in `contig_1_3` for the third gene on `contig_1`. Several tools rely on this. MAGScoT, for example, finds the contig by removing the last number. The header of each protein holds its start, its end, its strand and details such as:

Field	Meaning
`partial`	`00` for a complete gene. A 1 means that it runs off the start or the end of the contig
`start_type`	The start codon
`rbs_motif`	The binding site of the ribosome
`gc_cont`	GC content

41.5. When to use it

Use it for Bacteria and Archaea. It's fast, and it's accurate for them.

Don't use it for eukaryotes, as it doesn't handle introns. Be careful with organisms that use another genetic code, too. Prodigal assumes code 11 unless told otherwise, and some groups, such as Mycoplasma and CPR bacteria, use other ones. In meta mode, it has models for this.

Since eukaryotic contigs get wrong gene calls, a eukaryote-rich sample needs other tools, such as those of Chapter 67, before you interpret these results.

41.6. In this pipeline

It runs in two places, always in meta mode:

Rule	On	Why
`assemble__magscot__prodigal`	Contigs of each assembly	For finding marker genes
`contig_annotate__prodigal__run`	Contigs of each assembly	For annotation and counting

The first uses `parallel`, as Prodigal only has a single thread, and it needs just the proteins. The second writes proteins and positions, and it runs on a single thread. So the same genes are predicted twice, for these different purposes.

Bakta, DRAM and eggNOG-mapper each run their own gene prediction, which also uses Prodigal.

41.7. Pitfalls

- **Single-threaded.** A large assembly takes hours. `parallel` is a good way to speed it up, as long as you keep the order.
- **Partial genes.** On contigs, many genes are cut off at the edge. It's normal, and they count as genes.
- **A protein ending in `*`.** Prodigal marks stop codons with an asterisk at the end of the protein. Some tools don't accept it.

41.8. Reference

Hyatt D, et al. (2010). Prodigal: prokaryotic gene recognition and translation initiation site identification. *BMC Bioinformatics*. https://doi.org/10.1186/1471-2105-11-119

42. HMMER

42.1. What it does

HMMER searches protein sequences using profile hidden Markov models. A profile doesn't describe one sequence, but a whole family: which positions are conserved, which vary and where gaps are common. So it finds distant family members that a comparison with a single sequence would miss.

42.2. How it works

A profile is built from an alignment of a protein family. It holds, for each position, how likely each amino acid is, and how likely an insertion or a deletion is. HMMER then computes how well a protein fits this model, and it does it fast by using a series of filters that throw out hopeless candidates early.

The result is a score in bits, and an E-value: how many such hits you'd expect by chance in a search of this size. Many profile collections come with thresholds for each family, called gathering (GA), noise (NC) and trusted (TC) cutoffs. `--cut_nc` uses the noise cutoff, which is what MAGScoT asks for here.

42.3. Input

`hmmsearch` takes a file of profiles and a file of protein sequences. It searches each profile against all the sequences. The profiles come from a database such as Pfam or TIGRFAM, or you can build them from your own alignment with `hmmbuild`.

42.4. Output

Option	Content
`-o`	The human-readable report
`-A`	An alignment of the hits
`--tblout`	A table with one line per protein and profile

Option	Content
`--domtblout`	A table with one line per domain
`--pfamtblout`	A compact table for Pfam

The columns of `--tblout` are separated by spaces, and they start like this:

Column	Content
1, 2	Name and accession of the protein
3, 4	Name and accession of the profile
5, 6, 7	E-value, score and bias of the whole sequence

Lines that start with `#` are comments.

42.5. When to use it

Use it when you search for a certain protein family, and particularly for families with a good curated profile. It's the standard for marker genes. It's more sensitive than DIAMOND for distant members, and it's slower. So for comparing with very large collections, you'd use DIAMOND.

42.6. In this pipeline

Rule	Profiles	Question
`assemble__magscot_hmmsearch`	Bacterial marker genes of GTDB	Which bins are complete?
`contig_annotate__hmmer_search`	The hydrogenases of HydDB	Which genes are hydrogenases?

In the second rule, `--domT` sets a minimum score per domain, a separate value per database, as Chapter 66 shows. This option only filters the domains: `--tblout` still reports sequences with a lower score, down to an E-value of 10.

42.7. Pitfalls

- **Whitespace in the tables.** Columns are padded with spaces, not tabs. Read them with `awk`, as the pipeline does.

- **Which accession?** Pfam and TIGRFAM don't fill the same columns with the name of the marker. For this reason, `assemble__magscot__marker_table` uses column 3 for one and column 4 for the other.
- **E-value and database size.** The E-value grows with the number of sequences you search. Scores don't, which is why fixed score thresholds are a good choice.

42.8. Reference

Eddy SR (2011). Accelerated Profile HMM Searches. *PLoS Computational Biology*. https://doi.org/10.1371/journal.pcbi.1002195

43. MAGScoT

43.1. What it does

MAGScoT takes the bins of several binners, scores them by marker genes, and builds the best possible set of bins out of them. So it refines rather than bins. DAS Tool and MetaWRAP's refinement module are tools of the same kind.

43.2. How it works

Scoring. Each bin gets an estimate of completeness and contamination from single-copy marker genes. These are the marker sets of GTDB release 207: 120 for Bacteria and 53 for Archaea. If a bin has most of the markers, it's complete. If it has some markers twice, it's contaminated. These go into one score, with three weights you can set:

Option	Default
`-a`, `--score_a`	1
`-b`, `--score_b`	0.5
`-c`, `--score_c`	0.5

Merging. MAGScoT also makes new candidates: if bins from different binners share many markers, then it builds combined bins from them. `--n_iterations` says how many rounds of this it does, and `--skip_merge_bins` turns it off.

Selecting. It then goes down the list: it takes the best bin, removes its contigs from all other bins, computes the scores again, and takes the next. Each contig ends up in at most one bin. It stops when no bin reaches the minimum completeness (`-t`).

43.3. Input

Option	Content	Columns
`-i`	Which contig is in which bin	Bin, contig, name of the binner. No header
`--hmm`	Marker gene hits	Gene, marker, E-value

The gene name must be the contig name plus `_` and a number, as Prodigal writes it.

43.4. Output

File	Content
`<out>.refined.out`	The final bins, with their scores
`<out>.refined.contig_to_bin.out`	Which contig is in which final bin
`<out>.scores.out`	Scores of all bins of all binners
`<out>.gtdb_rel207_bac120.out`, `<out>.gtdb_rel207_ar53.out`	Marker counts of the bins

The file `scores.out` lets you see which binner did best on your data.

43.5. When to use it

Use it after running several binners, as a step before dereplication. It's light on time and memory.

It only knows about Bacteria and Archaea, since the marker genes are theirs. This also means that eukaryotic bins disappear in this step, as they don't have these markers. See Chapter 67.

43.6. In this pipeline

Six rules prepare and run it, as Chapter 63 shows. The script comes with the pipeline, in `workflow/scripts/MAGScoT/`, under its GPL-3 licence.

Two settings are worth a look:

- `threshold` is 0.5, so bins that are at least 50% complete can pass. This is the default of the tool.

- `extra` holds `--n_iter=1`, which R's option parser takes as `--n_iterations=1`. So there's one round of merging, while the default is 2. The comment in `params.yaml` says that this value might be a relic of testing.

If MAGScoT finds no viable bin, it fails, and the rule accepts this and continues with empty tables. This means that real errors are hidden too, so check the log.

43.7. Pitfalls

- **Names.** The contig names must match exactly between the files. The pipeline's `bin_NA@contig_…` format takes care of this.
- **Few bins.** On a small test data set, there may be no bins that pass. That isn't an error.
- **Missed markers.** Organisms with reduced genomes, such as CPR bacteria, lack some markers, so they look incomplete.

43.8. Reference

Rühlemann MC, Wacker EM, Ellinghaus D, Franke A (2022). MAGScoT: a fast, lightweight and accurate bin-refinement tool. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btac694

44. dRep

44.1. What it does

dRep reduces a set of genomes to one representative per species, or per another chosen level of similarity. Several assemblies usually recover the same organism again and again. If you were to map reads to all of these copies, the reads would spread randomly among them, and this would ruin your quantification. So a catalogue should hold only one of them, and this is called dereplication.

44.2. How it works

Comparing all genomes with each other exactly would take too long, so dRep does it in two steps.

1. **Filter.** Genomes that fail the length, completeness or contamination limits are removed. It uses CheckM for the estimates, unless you provide values through `--genomeInfo`.
2. **Primary clustering.** A very quick method (Mash) groups the genomes at a low threshold, by default 90% (`-pa`).
3. **Secondary clustering.** Inside each group, a more accurate method compares the genomes (by default fastANI) and groups them at the final threshold, by default 95% (`-sa`). They also need to align over a minimum share of their length, set by `-nc`.
4. **Choosing the winner.** For each final cluster, the genome with the best score is kept. The score combines completeness, contamination, N50 and size.

44.3. Input

Genomes in FASTA format, with one file per genome (`-g`).

44.4. Output

All of these are in the working directory, which is the first argument:

Path	Content
`dereplicated_genomes/`	The chosen genomes
`data_tables/Widb.csv`	The winners with their scores
`data_tables/Cdb.csv`	Which genome is in which cluster
`data_tables/Mdb.csv`, `Ndb.csv`	All pairwise comparisons of both steps
`data_tables/genomeInfo.csv`	Quality of all genomes
`figures/`	Plots of the clusters and the scores
`log/`	Logs

44.5. When to use it

Use it each time you combine genomes from several assemblies. The threshold depends on your question: 95% for one genome per species, which is the usual choice, and 98% or 99% for strains. Be aware that a too high threshold brings back the problem of reads spreading between very similar genomes.

44.6. In this pipeline

The rule `assemble__drep__run`, explained in Chapter 63, reads the single bins made by `assemble__drep__split_bins`. It uses these options:

Option in dRep	Key in `params.yaml`	Value
`-comp`	`completeness`	50
`-con`	`contamination`	50
`-pa`	`P_ani`	0.9
`-sa`	`S_ani`	0.95
`-nc`	`nc`	0.1

Note `-con 50`. dRep's own default is 25, and a genome with up to 50% contamination can get through this filter. The limit is permissive on purpose, and quality filtering is left to you, based on CheckM2, in Chapter 48.

The container `hrp_assemble` holds CheckM with its reference data, so dRep can run it without a separate database.

44.7. Pitfalls

- **A single genome.** dRep can't cluster one genome: its run ends with `ValueError: The number of observations cannot be determined on an empty distance matrix`. A real project has more bins than that, but a small test can run into it, as Chapter 70 shows.
- **CheckM needs a lot of memory.** Its step with `pplacer` needs tens of GB. If it fails, dRep prints `!!! checkM failed !!!`, but it **still exits with 0** and leaves no genomes. So the rule checks for this and stops with a clear message.
- **Hard to resume.** dRep won't start in a folder with old results, and the rule removes them first.
- **The time of CheckM.** Most of the run time may be in CheckM, and not the comparison. Giving it results through `--genomeInfo` avoids this. You could add that through `extra`.
- **The name of the winner.** The file names are kept, so you can see which assembly each winner came from.

44.8. Reference

Olm MR, Brown CT, Brooks B, Banfield JF (2017). dRep: a tool for fast and accurate genomic comparisons that enables improved genome recovery from metagenomes through de-replication. *The ISME Journal*. https://doi.org/10.1038/ismej.2017.126

45. CoverM

45.1. What it does

CoverM computes how much of each genome or contig is covered by reads, from alignments. It's the tool that turns a mapping into the abundance table. It knows about genomes that consist of many contigs, and it offers a long list of ways of measuring.

45.2. How it works

It reads an alignment, and it counts per contig. In `coverm genome`, it adds up the contigs of each genome. To know which contigs form a genome, it can take a list, a folder of FASTA files, or simply a character in the contig names, the separator `-s`.

45.3. Input

BAM files (`-b`), sorted by position, or reads with a reference, in which case it does the mapping itself. In `genome` mode, it also needs one way to tell which contig belongs to which genome.

45.4. Output

A table with one row per genome or contig, and one column per sample and method. The first column holds the name. The available methods include:

Method (`-m`)	Meaning
`count`	Number of mapped reads
`covered_bases`	Number of bases with at least one read
`length`	Length of the genome or contig
`mean`	Average depth

Method (`-m`)	Meaning
`trimmed_mean`	Average depth without the highest and lowest 5% of the positions
`relative_abundance`	Percentage of mapped reads on this genome
`rpkm`	Reads per kilobase and million reads
`tpm`	Like RPKM, normalized so that a sample sums to one million

There's more: https://github.com/wwood/CoverM#calculation-methods.

`--min-covered-fraction` matters in `genome` mode. A genome counts as present only if this percentage of it is covered, and otherwise its value is set to 0. By default this is 10. The idea is that a few reads can map to a genome by chance, through a conserved gene, for example, without that genome being in the sample.

45.5. When to use it

Use it for quantifying genomes from a catalogue. For gene-level counts, you need a tool that uses gene positions, such as featureCounts.

45.6. In this pipeline

The rules are in Chapter 64. `samtools view` first turns CRAM into BAM, keeping only mapped reads, and then CoverM reads it from a pipe:

```
samtools view -1 -F 4 -T {input.catalogue} {input.alignment} \
    | coverm genome -b /dev/stdin -m {wildcards.method} -s
     ↪ {params.settings[separator]} \
       --min-covered-fraction
        ↪ {params.settings[min_covered_fraction]} \
    > {output}
```

Each method runs as its own job, per library, and `workflow/scripts/coverm_wide_table.R` then joins them into one table. The default methods are `count`, `covered_bases`, `length` and `tpm`.

Note that `min_covered_fraction` is 0 here, so even a genome with a single read gets a value. You'll want to filter these yourself, using `covered_bases` divided by `length`.

45.7. Pitfalls

- `min_covered_fraction`, as above.
- **Conserved regions.** With 0 as the fraction, conserved genes produce low values for genomes that aren't there. A genome with a low covered fraction shouldn't be counted as present.
- **The separator.** All contig names need the `@`, and the genome is what's left of it. The pipeline's names take care of that.
- **Which measure?** Use `count` for statistical tests, which want raw counts, and `tpm` or `relative_abundance` for descriptions.

45.8. Reference

Aroney STN, et al. (2025). CoverM: read alignment statistics for metagenomics. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btaf147

46. featureCounts

46.1. What it does

featureCounts counts how many reads fall on each gene. It takes alignments and a table of gene positions, and it writes a table of counts. It's part of the Subread package, and it's a very common tool in RNA-seq, but it's equally useful here.

46.2. How it works

For each read or fragment, it finds the features it overlaps. If it overlaps exactly one, this feature gets a count. By default, a read that overlaps several features isn't counted, nor is a read that maps to several places. This is cautious, and it can be changed with `-O` and `-M`.

46.3. Input

Option	Content
`-a`	The annotation, as GTF or GFF
`-F`	The format
`-t`	Which type of feature to count, from column 3 of the file
`-g`	Which attribute names the gene, from column 9
`-p`	The data are paired. In this version, it counts fragments instead of reads
The last arguments	One or several BAM files

It reads BAM and SAM, not CRAM.

46.4. Output

File	Content
`-o`	The counts: gene, contig, start, end, strand, length, and one column per BAM file
`<o>.summary`	How many reads were assigned, and why the others weren't

The first line of the count table is a comment that holds the command. Look at the `.summary` file: `Assigned` is what counts, and a high value of `Unassigned_NoFeatures` means that many reads don't fall on any gene.

46.5. When to use it

Use it for counting reads on genes. Compared with a mapper-level tool like CoverM, it knows about the position of genes. It's very fast.

46.6. In this pipeline

The rule `contig_annotate__featurecounts__run` counts per library and assembly, as Chapter 66 shows:

```
featureCounts -p -T {threads} -F GFF3 \
              -a {input.gtf} -o {output.file} \
              -t CDS -g ID \
              {input.bam}
```

The file with the positions comes from Prodigal, which marks genes as `CDS` and gives each an `ID` like `1_5`, the fifth gene on the first contig in the file. So the names in this table differ from the names in the protein file. Each gene's ID is built from the position of the contig in the file. To join the tables, use the contig and the gene number.

The image `subread` holds version 2.0.1.

46.7. Pitfalls

- **`-p` changed its meaning.** In version 2.0.2 and later, `-p` only says that the data are paired, and `--countReadPairs` is needed to count fragments. So if you change the image, then check this, or your counts could double.
- **Multi-mapped reads** aren't counted. In a metagenome with very similar genes, this can undercount them.
- **Annotation and BAM must agree on the contig names.** Otherwise, everything is unassigned.

46.8. Reference

Liao Y, Smyth GK, Shi W (2014). featureCounts: an efficient general purpose program for assigning sequence reads to genomic features. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btt656

47. QUAST

47.1. What it does

QUAST evaluates assemblies. Without a reference, it reports length statistics, and with a reference, it can also find misassemblies. It's a quick way of getting the numbers everyone asks about.

47.2. Understanding the numbers

Value	Meaning
`# contigs`	How many sequences, usually counted from a length of 500 bases
`Largest contig`	The longest
`Total length`	The sum of all lengths
`N50`	Half of all bases are in contigs of at least this length. Larger is better
`L50`	How many contigs make up that half. Smaller is better
`GC (%)`	The GC content

N50 is a measure of how fragmented the assembly is, and it's the most quoted number. Be aware that it says nothing about whether the contigs are correct, and that it can be raised by simply dropping short contigs.

47.3. Input

FASTA files with the assemblies. A reference is optional (`-r`). `metaquast.py` is a version for metagenomes that can work with several references.

47.4. Output

File in the output folder	Content
`report.txt`, `report.tsv`	The table of numbers
`transposed_report.tsv`	The same, with one line per assembly, for many assemblies
`report.html`	The report for a browser
`icarus.html`	An interactive view of contigs
`quast.log`	The log

47.5. When to use it

Use it for comparing assemblies or assemblers on the same data. Don't compare the N50 between very different samples, as it depends on the complexity of the community and on the depth.

47.6. In this pipeline

The rule `mag_annotate__quast` runs it on the catalogue, as one file:

```
quast -o {output} -t {threads} {input}
```

So the numbers describe the set of contigs in the catalogue as a whole, and not each genome. The value of `# contigs` isn't per genome, and N50 is over all genomes. It doesn't evaluate the individual assemblies. To do that, run it by hand on `results/assemble/<assembler>/*.fa.gz`. For the quality of single genomes, CheckM2 is the right tool, and its table also gives an N50 per genome.

47.7. Reference

Gurevich A, Saveliev V, Vyahhi N, Tesler G (2013). QUAST: quality assessment tool for genome assemblies. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btt086

48. CheckM2

48.1. What it does

CheckM2 estimates the completeness and the contamination of a genome. This is the most important check of a MAG. Everything else you say about a genome rests on it.

48.2. How it works

The original CheckM counted marker genes specific to a lineage. That doesn't work well for groups without good reference genomes, or for those with reduced genomes.

CheckM2 uses machine learning. It predicts the genes of a genome, and it compares them with a database with DIAMOND. The presence and absence of many gene families and the amino acid composition go into two models, trained on genomes that were artificially thinned out and mixed: a general gradient-boosting model and a neural network. It picks which to trust depending on how close the genome is to its training data. This works without first placing the genome in a tree.

48.3. Input

A folder or list of genomes (`-i`) in FASTA, with their file ending (`-x`), and the database (`--database_path`).

48.4. Output

`quality_report.tsv`, with one line per genome:

Column	Content
`Name`	The genome
`Completeness`	Estimated percentage
`Contamination`	Estimated percentage
`Completeness_Model_Used`	Which model was used

Column	Content
`Translation_Table_Used`	The genetic code
`Coding_Density`	Share that codes for proteins
`Contig_N50`, `Average_Gene_Length`, `Genome_Size`, `GC_Content`, `Total_Coded_Sequences`	Statistics
`Additional_Notes`	Warnings, if any

48.5. What's a good genome?

A widely used standard is MIMAG (Bowers et al. 2017):

Quality	Completeness	Contamination	Also needs
High	Over 90%	Below 5%	rRNA genes (23S, 16S, 5S) and at least 18 tRNAs
Medium	At least 50%	Below 10%	
Low	Below 50%	Below 10%	

Note the additional demand for rRNA genes. Short-read assemblies seldom get these, so many MAGs that reach the numbers aren't "high quality" by the strict definition. DRAM reports rRNA and tRNA genes per genome, so you can check.

48.6. When to use it

Use it on every MAG you want to use. It doesn't work for eukaryotes or viruses, for which you need EukCC or BUSCO, and CheckV.

48.7. In this pipeline

The rule `mag_annotate__checkm2__predict` runs it on the dereplicated genomes. The result is at `results/mag_annotate/checkm2/quality_report.tsv`.

dRep uses the older CheckM for its filter, as Chapter 44 explains, so you can see two sets of values for a genome. Use CheckM2's for your report.

48.8. Pitfalls

- **Numbers are estimates.** They aren't exact, and a MAG with very low estimated contamination can still have a wrong contig.
- **Completeness isn't correctness.** Chimeric bins with parts of relatives can look fine.
- **File ending.** `-x` must match exactly, here `.fa.gz`. If not, CheckM2 finds no genomes.

48.9. References

Chklovski A, Parks DH, Woodcroft BJ, Tyson GW (2023). CheckM2: a rapid, scalable and accurate tool for assessing microbial genome quality using machine learning. *Nature Methods*. https://doi.org/10.1038/s41592-023-01940-w

Bowers RM, et al. (2017). Minimum information about a single amplified genome (MISAG) and a metagenome-assembled genome (MIMAG) of bacteria and archaea. *Nature Biotechnology*. https://doi.org/10.1038/nbt.3893

49. GTDB-Tk

49.1. What it does

GTDB-Tk assigns a taxonomy to genomes of Bacteria and Archaea, based on the Genome Taxonomy Database (GTDB). It tells you where a MAG belongs in the tree of life, and also whether it's a known species.

49.2. What's GTDB?

GTDB is a taxonomy built directly from genome phylogeny, with ranks defined by a consistent measure. So it differs from the NCBI taxonomy in many places, and names may change. For example, Firmicutes is now split up and called Bacillota in several parts. Remember this when you compare with older literature.

49.3. How it works

The workflow `classify_wf` runs three steps.

1. **identify.** It finds 120 bacterial and 53 archaeal marker genes in each genome, with Prodigal and HMMER.
2. **align.** It aligns these markers, and it concatenates them into one long sequence per genome.
3. **classify.** It places each genome in a reference tree, with pplacer. It uses the position in the tree and the relative evolutionary divergence (RED) to decide the rank. Then it compares it with close reference genomes, by ANI, to decide on the species.

49.4. Input

A folder of genomes (`--genome_dir`), with their ending (`-x`), and the database, through the environment variable `GTDBTK_DATA_PATH`. The version of the tool and the release of the database must fit each other.

49.5. Output

File	Content
`gtdbtk.bac120.summary.tsv`	Results for Bacteria
`gtdbtk.ar53.summary.tsv`	Results for Archaea, if there are any
`classify/`, `align/`, `identify/`	Intermediate results and trees
`gtdbtk.warnings.log`	Genomes with problems

The most useful columns are:

Column	Content
`user_genome`	Your genome
`classification`	The taxonomy, as `d__Bacteria;p__...;c__...;o__...;f__...;g__...;s__...`
`classification_method`	How it was decided
`note`	Remarks, for example on ANI
`msa_percent`	Share of the alignment that's filled
`warnings`	Warnings

An empty name at a rank, such as a line ending in `s__`, says that no known species fits at that rank. **So your MAG may represent a new species or genus**, and that's one of the exciting results in a less-known habitat like the rumen.

49.6. When to use it

Use it for any MAG of Bacteria or Archaea. It needs a large database and quite some memory, but only once per genome set.

Low-quality MAGs have too few markers. So they can get a poor placement, and you see this in `msa_percent`.

49.7. In this pipeline

The rule `mag_annotate__gtdbtk__classify` runs it on the dereplicated genomes, with the database from `features.yaml`, by default release 220:

```
GTDBTK_DATA_PATH={input.reference_data} gtdbtk classify_wf \
    --genome_dir {input.genomes} -x fa.gz --out_dir {GTDBTK}
    ↪  --cpus {threads} --skip_ani_screen
```

`--skip_ani_screen` skips the first quick comparison, which would need one more database. It joins the two tables into `gtdbtk.summary.tsv`, and packs the folders with intermediate results into archives. DRAM uses the table.

49.8. Pitfalls

- **Memory.** It can use a lot, and the amount depends on the database release. This rule runs in the resource set `large`.
- **The version and the release.** A tool of one version doesn't work with a database of another.
- **`--skip_ani_screen`.** Genomes are placed through the tree, not by a first ANI screen. This costs time, but it doesn't change much of the result.
- **Taxonomy names in the literature.** Compare carefully, as the names differ from NCBI's.

49.9. References

Chaumeil PA, Mussig AJ, Hugenholtz P, Parks DH (2022). GTDB-Tk v2: memory friendly classification with the genome taxonomy database. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btac672

Parks DH, et al. (2022). GTDB: an ongoing census of bacterial and archaeal diversity through a phylogenetically consistent, rank normalized and complete genome-based taxonomy. *Nucleic Acids Research*. https://doi.org/10.1093/nar/gkab776

50. PhyloPhlAn

50.1. What it does

PhyloPhlAn is a set of tools for phylogenetics of microbial genomes. The pipeline uses one of them, `phylophlan_assign_sgbs`. It assigns each MAG to a species-level genome bin (SGB) from a large collection, which holds both known species and species that are known only from metagenomes.

So you learn whether a MAG belongs to a group that has been seen before in other studies, even if nobody has given it a name.

50.2. How it works

It sketches each genome with Mash, and it compares this with the sketches of all SGBs. Each SGB stands for a species-level cluster, with genomes that have at least 95% ANI to each other. It reports the closest SGBs, with a distance. A label shows if it's a known species (kSGB) or one without a named species (uSGB).

50.3. Input

A folder of genomes (`-i`), their ending (`-e`), a database name (`-d`) and folder (`--database_folder`).

50.4. Output

A table (`-o`, written as `<o>.tsv`), with one line per genome, and with the closest SGBs and their distances.

50.5. When to use it

Use it as a companion to GTDB-Tk. The names don't follow GTDB, and the SGB collection ties in with the MetaPhlAn world, which makes it easy to compare with studies that use it.

For building a real phylogenetic tree of your genomes, the main `phylophlan` program is what you want. This pipeline doesn't run that.

50.6. In this pipeline

The rule `mag_annotate__phylophlan` runs it on the dereplicated genomes:

```
phylophlan_assign_sgbs -i {input} \
                       -d SGB.Jun23 \
                       --database_folder
                        ↪  resources/databases/phylophlan/ \
                       -e fa.gz -o {params.result} \
                       --nproc {threads} --verbose --overwrite
```

As the comment in the rule says, the database name and folder are written into the rule. The list under `phylophlan` in `features.yaml` has no effect on it, nor do `phylophlan-config` in `config.yaml` and `diversity` in `params.yaml`.

50.7. Reference

Asnicar F, et al. (2020). Precise phylogenetic analysis of microbial isolates and genomes from metagenomes using PhyloPhlAn 3.0. *Nature Communications*. https://doi.org/10.1038/s41467-020-16366-7

51. DRAM

51.1. What it does

DRAM (Distilled and Refined Annotation of Metabolism) annotates genomes with a focus on metabolism. It finds out what each gene is and then it "distills" this into an overview of what a genome can do: for example, which carbohydrates it can use, and whether it can make methane.

51.2. How it works

Annotate. DRAM takes the contigs, predicts genes with Prodigal, and compares them with many databases: Pfam, dbCAN for carbohydrate-active enzymes, MEROPS for peptidases, VOGDB for viruses, UniRef90, and optionally KEGG, which needs a licence. KOfam is used in its place in this pipeline. It also finds tRNA genes with tRNAscan-SE, and rRNA genes with barrnap.

Distill. It then uses hand-curated tables to turn the annotation into a list of metabolic functions, and it summarizes them per genome.

51.3. Input

Step	Input
`DRAM.py annotate`	FASTA with contigs. Each file counts as one genome. Optionally a taxonomy from GTDB-Tk
`DRAM.py distill`	The tables from the annotation step
Both	A config file that tells where the databases are

51.4. Output

File	Step	Content
`annotations.tsv`	annotate	One line per gene, with all database hits
`genes.faa`, `genes.fna`, `genes.gff`	annotate	Genes as proteins, as DNA and as positions
`trnas.tsv`, `rrnas.tsv`	annotate	tRNA and rRNA genes
`genome_stats.tsv`	distill	Quality per genome, including the checks for rRNA and tRNA
`metabolism_summary.xlsx`	distill	A workbook with one sheet per group of functions
`product.html`, `product.tsv`	distill	A heatmap of functions per genome, and its data

`product.html` is what to look at first. `metabolism_summary.xlsx` is the place for details.

51.5. When to use it

Use it to get an overview of the metabolism of a set of MAGs, in particular for carbon and energy metabolism. It's a heavy tool: the databases take hundreds of GB and it needs much memory. If you only need a general annotation, Bakta or eggNOG-mapper is lighter.

51.6. In this pipeline

Each of the two runs has three steps. `mag_annotate__dram__annotate`, `mag_annotate__fix_dram_annotations_scaffold` and `mag_annotate__dram__distill` run on the catalogue. `mag_annotate__dram_mag__annotate`, `mag_annotate__fix_dram_mag_annotations_scaffold` and `mag_annotate__dram_mag__distill` run on the binned contigs of each assembly.

Note what happens to genome names. DRAM treats one FASTA file as one genome, but the catalogue is a single file with all MAGs. So DRAM would see one huge genome. The repair step, `workflow/scripts/fix_annotations.py`, writes the part of the contig name before the `@` into the column for the scaffold. So the distillation can tell the MAGs apart, and that's why the contig names matter here as well.

`config/dram_config.txt` holds the paths to the databases, all relative to the project folder. `kegg` is `null`, so KEGG isn't used.

51.7. Pitfalls

- **Temporary files.** DRAM starts tRNAscan-SE, which reads `$TMPDIR`, and on the cluster the inherited value points to a place that the container can't see. Without the rule's fix, DRAM would crash in this step. The comment in the rule explains this.
- **Empty tables.** If there's no tRNA, DRAM writes no table. The rule then leaves an empty file, as the next step needs one.
- **Paths in the config.** All database paths have to exist from the project folder. If you keep the databases elsewhere, you have to change this file.

51.8. Reference

Shaffer M, et al. (2020). DRAM for distilling microbial metabolism to automate the curation of microbiome function. *Nucleic Acids Research*. https://doi.org/10.1093/nar/gkaa621

52. CAMPER

52.1. What it does

CAMPER (Curated Annotations for Microbial Polyphenol Enzymes and Reactions) looks for genes that break down or modify polyphenols. Polyphenols are plant compounds such as tannins. For a long time, it was assumed that they weren't metabolized in low-oxygen environments, but studies have since shown that microbes do act on them. In a rumen, feed tannins are an important topic in their own right.

General annotation tools hardly cover these functions. That's why this special database exists.

52.2. How it works

It has two sets of references: protein sequences with score cutoffs for a DIAMOND search, and protein profiles for an HMMER search, each with cutoffs by hand. It searches with both, and then it writes a summary, called distillate.

52.3. Input

Protein sequences (`-i`), and the paths to the files of the database.

52.4. Output

File	Content
`annotations.tsv`	One line per gene with hits
`distillate.tsv`	The summary

52.5. When to use it

Use it when polyphenols are part of your question, as a supplement to a general annotation.

52.6. In this pipeline

It runs twice, and both runs use the same commands:

Rule	Proteins from
`mag_annotate__camper__annotate`	DRAM on the catalogue
`contig_annotate__camper__annotate`	Prodigal on all contigs of each assembly

The rules copy the database first, to `nvme_storage`, and they run in a temporary folder that has to not exist yet. This is the strange step `mkdir -p`, then `rm -rf`, which a comment in the rule calls "a strange camper requirement".

52.7. Pitfalls

- **Without a local disk.** The rule falls back to a folder in the results if `nvme_storage` is empty.
- **A shared folder for the database copy.** In `mag_annotate__camper__annotate`, the database goes straight into `nvme_storage`, without a subfolder for the job. If a local disk is shared with other jobs, they could interfere.

52.8. Reference

McGivern BB, et al. (2024). Microbial polyphenol metabolism is part of the thawing permafrost carbon cycle. *Nature Microbiology*. https://doi.org/10.1038/s41564-024-01691-0

53. Bakta

53.1. What it does

Bakta annotates bacterial genomes in a standardized way, and it writes the files needed for submission to the public databases. It finds genes of all kinds, and it gives them consistent names, with links to other databases.

53.2. How it works

1. It finds non-coding features: tRNA, tmRNA, rRNA, ncRNA and others. It also finds CRISPR arrays if you want them.
2. It predicts coding genes with Pyrodigal, a Python version of Prodigal.
3. To name them, it computes a hash of each protein, and looks it up in a large table of known proteins. An exact match is found immediately, without any alignment. Only the rest go to DIAMOND. This makes it fast.

53.3. Input

FASTA with contigs, and a database (`--db`), which comes in a full and a smaller form.

53.4. Output

All files share the prefix from `--prefix`:

File	Content
`.tsv`	The annotation as a table, one line per feature
`.gff3`, `.gbff`, `.embl`	The annotation in the standard formats
`.faa`, `.ffn`, `.fna`	Proteins, genes and contigs
`.hypotheticals.tsv`	Details on proteins without a known function
`.txt`, `.json`	Summary and full data

53.5. When to use it

Use it for single, reasonably complete genomes, when you want a standard annotation. It's also useful if you want to submit genomes. It's meant for Bacteria, and for genomes rather than whole metagenomes.

53.6. In this pipeline

Rule	Input
`mag_annotate__bakta`	The whole catalogue, as one input
`mag_annotate__bakta_mags__run`	The binned contigs of each assembly

So it doesn't run per genome. The catalogue looks like one large genome to Bakta. Its results are still annotated by contig, so you can split them up by the contig names. `--skip-crispr --skip-plot` comes from `params.yaml`. Note that Bakta also has an option `--meta`, which lets the gene prediction expect a mixture. This pipeline doesn't use it by default, and you could add it to `additional_options`.

The proteins from `mag_annotate__bakta_mags__run` are the input for Proteinortho.

As in DRAM, the rule sets its own `TMPDIR`, because of tRNAscan-SE. The comment explains why.

53.7. Pitfalls

- **Many contigs.** Bakta is meant for genomes, and on a large assembly, it takes a long time.
- **Database version.** The database has to fit the version of the tool.
- `--force` overwrites the output folder, which is what the rules want here. Keep this in mind if you run it by hand.

53.8. Reference

Schwengers O, et al. (2021). Bakta: rapid and standardized annotation of bacterial genomes via alignment-free sequence identification. *Microbial Genomics*. https://doi.org/10.1099/mgen.0.000685

54. eggNOG-mapper

54.1. What it does

eggNOG-mapper gives genes a functional annotation, based on groups of orthologs: genes in different organisms that come from one gene in a common ancestor. Orthologs tend to keep their function, so that the function of one known member can be transferred to the others. This tends to be more precise than transferring it from any similar sequence.

54.2. How it works

It has two steps, and you can run them separately:

1. **Search.** It compares your proteins with the eggNOG database, using DIAMOND or another search tool. For each protein, it finds the best match, the seed ortholog.
2. **Annotate.** It looks up the seed in the eggNOG database, finds its orthologs, and it transfers their annotation.

The first step is CPU-heavy, and it benefits from many threads. The second one reads a large SQLite database a lot, so it benefits from a fast disk or from memory (`--dbmem`).

54.3. Input

Option	Content
`-i`	Sequences. `--itype` says which: `proteins`, `CDS`, `genome`, `metagenome`
`--genepred`	How to find genes in the last two: `search` or `prodigal`
`--annotate_hits_table`	For the second step, the results of the first
`--data_dir`	The database folder
`-m`	The search tool, here `diamond`

54.4. Output

All files share the prefix from `-o`:

File	Content
`.emapper.seed_orthologs`	The results of the search
`.emapper.hits`	All hits
`.emapper.annotations`	The annotation

The columns of the annotation include:

Column	Content
`query`	Your gene
`seed_ortholog`, `evalue`, `score`	The best match
`eggNOG_OGs`	The groups of orthologs
`COG_category`	The broad functional class
`Description`, `Preferred_name`	What it is
`GOs`, `EC`, `KEGG_ko`, `KEGG_Pathway`, `KEGG_Module`, `CAZy`, `PFAMs`	Links to other systems

The KEGG columns are often the most useful, as they let you map genes onto pathways.

54.5. When to use it

Use it to annotate many genes at once, in a form you can summarize by categories. It's a good general choice for all contigs of an assembly.

54.6. In this pipeline

It's used in two ways, and these differ.

On all contigs, as Chapter 66 shows, it does both steps, on chunks of proteins from Prodigal: `contig_annotate__eggnog_find_homology` searches, and `contig_annotate__eggnog_orthology_chunk` annotates. `contig_annotate__eggnog_merge_annotations` joins the results.

On the catalogue, the rule `mag_annotate__eggnog` runs one call with `--itype metagenome --genepred prodigal`, and with `--no_annot`. So this call only performs

the search, and the result is in `.emapper.seed_orthologs`. No annotation is written, and you would have to run the second step by hand with `--annotate_hits_table`.

54.6.1. eggNOG version 7

The rule `contig_annotate__eggnog7__run` uses a newer annotator for version 7 of the database, through `workflow/scripts/eggnog7_annotator.sh`. This script is simpler: it searches a DIAMOND database (`_proteins.dmnd`) with an E-value of `1e-5`, and it looks up the hits in a table (`_master_search_table.tsv.gz`). The result is `{assembly_id}.eggnog.tsv.gz`. Note that it's a separate, newer data set, so the results don't exactly match the classic one.

54.7. Pitfalls

- **Comment lines.** `--no_file_comments` removes the headers that start with `##`. That's useful when you concatenate chunks.
- **Mixing versions.** The results from a certain database version shouldn't be mixed with those from another.
- **Only the best match.** A distant seed gets an annotation just like a close one. Filter by `evalue` and `score` if needed.

54.8. Reference

Cantalapiedra CP, Hernández-Plaza A, Letunic I, Bork P, Huerta-Cepas J (2021). eggNOG-mapper v2: Functional Annotation, Orthology Assignments, and Domain Prediction at the Metagenomic Scale. *Molecular Biology and Evolution*. https://doi.org/10.1093/molbev/msab293

55. Proteinortho

55.1. What it does

Proteinortho finds orthologs between several sets of proteins. So it tells you which genes different genomes or assemblies have in common, and which are unique to some of them.

55.2. How it works

It compares all proteins with each other, set against set, with DIAMOND by default. It keeps reciprocal best hits: pairs of proteins that are each other's best match. These form a graph, and a clustering step splits it into groups of orthologs.

55.3. Input

Two or more protein FASTA files, one per species or genome.

55.4. Output

All files share the prefix from `-project`:

File	Content
`.proteinortho.tsv`	The main table
`.proteinortho.html`	The same for a browser
`.blast-graph`, `.proteinortho-graph`	The graphs

The main table has one line per group of orthologs, with these columns:

Column	Content
`# Species`	How many sets are in this group

Column	Content
`Genes`	How many genes are in it
`Alg.-Conn.`	How well the group is connected, 1 being best
One per input	The genes of this set, or `*` if there's none

55.5. When to use it

Use it for comparing the gene content of several genomes, to find a core and an accessory part. It works for hundreds of genomes.

55.6. In this pipeline

The rule `mag_annotate__proteinortho` compares the proteins from Bakta's run on each assembly:

```
proteinortho {input.faa} -cpus={threads} -project=proteinortho
mv proteinortho.* {params.outdir}
```

Note what the sets are: **each set is one assembly, not one genome**. So the table shows what's shared between assemblies. To compare genomes, you'd have to split the proteins by genome first.

The rule needs at least two assemblies, so with a single assembly, it fails.

55.7. Reference

Lechner M, et al. (2011). Proteinortho: Detection of (Co-)orthologs in large-scale analysis. *BMC Bioinformatics*. https://doi.org/10.1186/1471-2105-12-124

56. Tiara

56.1. What it does

Tiara sorts sequences of a metagenome into classes, in particular eukaryotic or prokaryotic, with a neural network that uses k-mer frequencies. It also picks out sequences from organelles. So it's the quick first step when you want to find the eukaryotes in an assembly.

56.2. How it works

It classifies each sequence in two stages:

1. The first stage decides between `archaea`, `bacteria`, `prokarya`, `eukarya`, `organelle` and `unknown`.
2. The second looks at the organelles, and it decides between `mitochondrion`, `plastid` and `unknown`.

A class needs a probability above a cutoff for each stage (`--prob_cutoff`). Otherwise it's `unknown`.

56.3. Input

A FASTA file with the contigs (`--input`), which can be compressed. By default, it ignores those shorter than 3000 bases (`--min_len`), since classification on short sequences is less reliable.

56.4. Output

File	Content
`--output`	The table

`log_<name>`	The number of sequences per class. This is the quickest summary

The table holds these columns:

Column	Content
`sequence_id`	The contig
`class_fst_stage`	Class from the first stage
`class_snd_stage`	Class from the second stage, or `n/a`
Further columns	With `--probabilities`, the probabilities per class

56.5. When to use it

Use it as a fast filter for eukaryotic contigs. It has only modest resource needs.

56.6. In this pipeline

The rule `eukaryote__tiara__run` runs it per assembly, as Chapter 67 shows. `min_len` is 1000 here, and the comment in `params.yaml` explains that this is the tool's documented lower limit, chosen so that assemblies with short contigs, such as those from soil, don't get thinned out.

56.7. Pitfalls

- **Short contigs.** With `min_len` 1000, the shortest contigs get the least reliable labels.
- **`prokarya` versus `bacteria`.** A contig in `prokarya` is one that's clearly prokaryotic, but not clearly one of the two domains.
- **Eukaryotic genes in bacteria**, and the other way around. Horizontal gene transfer can mislead classification by composition.

56.8. Reference

Karlicki M, Antonowicz S, Karnkowska A (2022). Tiara: deep learning-based classification system for eukaryotic sequences. *Bioinformatics*. https://doi.org/10.1093/bioinformatics/btab672

57. 4CAC

57.1. What it does

4CAC classifies the contigs of an assembly into four groups: phages, plasmids, prokaryotic chromosomes and eukaryotes. Other tools usually only do one of those distinctions. Its special contribution is that it uses the assembly graph, in addition to the sequences.

57.2. How it works

1. A classifier (XGBoost) gives each contig a score for each class, based on its k-mers.
2. For contigs whose class is clear, the label is kept. Then 4CAC looks at the assembly graph: neighboring contigs often come from one element. So a contig without a clear label can take the class from its neighbors.

The graph is the reason why it has to be an assembly with a graph it can read.

57.3. Input

Option	Content
`classify_xgb.py --fasta`	The contigs
`classify_4CAC.py --asmdir`	The assembler's folder, with the contigs, the paths and the graph
`--assembler`	Which assembler made these, here `metaSPAdes`
`--phage`, `--plasmid`	Minimum scores

57.4. Output

`4CAC_classification.fasta`, whose sequence names hold the classes. A script of this pipeline turns that into a table, see below.

57.5. When to use it

Use it when plasmids and phages matter too, or to back up other classifiers. It needs an assembler's graph, and in this pipeline it only works for metaSPAdes.

57.6. In this pipeline

Two rules run it: `eukaryote__4cac__extract_graph` gets the files from the archive of the working folder of metaSPAdes, and `eukaryote__4cac__run` runs the two steps, plus the translation of names. The result has one line per contig:

Column	Content
`contig_id`	The name in this pipeline
`node_id`	The name from metaSPAdes
`class`	`phage`, `plasmid`, `prokarya`, `eukarya` or `uncertain`
Four more	The scores of the first step

This comes from a third-party image, `metagenlab/4cac:1.0`, with the scripts in `/pkg/4CAC`. `workflow/scripts/fourcac_to_contig_ids.py` translates the names, by repeating the numbering of `contig_ids.py`. So if you ever change one of the two, change the other too.

57.7. Pitfalls

- **MEGAHIT or your own assemblies.** The target does nothing then, as Chapter 67 explains.
- **Memory.** The resource sets of this rule escalate up to `highmem`, as large graphs can take a lot of it.

57.8. Reference

Pu L, Shamir R (2024). 4CAC: 4-class classifier of metagenome contigs using machine learning and assembly graphs. *Nucleic Acids Research*. https://doi.org/10.1093/nar/gkae799

58. REMAG

58.1. What it does

REMAG recovers genomes of eukaryotes from a metagenome. It does two things that are usually separate: it filters the contigs for eukaryotes, and then it bins them. Most binners and their refinement steps are built for Bacteria and Archaea, so a tool like this fills a real gap.

58.2. How it works

1. **Filter.** A classifier based on HyenaDNA, a deep language model for DNA, gives each contig a probability of being eukaryotic.
2. **Bin.** It learns a representation of the contigs using self-supervised learning, based on their sequences and coverage, and it clusters them into bins.

58.3. Input

Option	Content
`--fasta`	The contigs, which can be compressed
`--coverage`	A BAM or CRAM file, one option per sample
`--output`	The folder
`--filter-only`	Stop after the filter
`--min-contig-length`, `--min-bin-size`	Limits

58.4. Output

File	Content
`<name>.fa_hyenadna_classification.tsv`	The classification
`<name>.fa_eukaryotic_filtered.fasta`	The contigs called eukaryotic
`bins/`	The bins

File	Content
`bins.csv`	Which contig is in which bin
`remag*.log`	Logs. A log file gets replaced by a new one at 10 MB

The classification table has these columns: `contig_id`, `length`, `prediction`, `eukaryote_prob`, `confidence`, `num_windows`, `resampled`.

58.5. When to use it

Use it for samples with a real share of eukaryotes, such as soil, sediments, or a rumen with fungi and protists. It's a young tool, so expect it to change, and check its bins with care.

It benefits from a GPU, but it also works without one, only more slowly.

58.6. In this pipeline

Two rules run it, as Chapter 67 shows in detail. `eukaryote__remag__filter` only classifies, and `eukaryote__remag__bin` continues in the same folder.

Two precautions in these rules are worth noting:

- REMAG doesn't stop with an error when its classifier can't be loaded or a batch fails. It keeps the contigs as eukaryotic, and it only writes this to the log. So the rule looks for these messages and fails by itself.
- If the contigs called eukaryotic add up to less than `min_bin_size`, then the rule skips the binning, as REMAG would stop with an error.

58.7. Pitfalls

- **`min_contig_length` must stay the same** between the two steps, or REMAG refuses to resume.
- **Bins aren't in the catalogue.** They don't go through CheckM2, MAGScoT or dRep. Check them with a tool for eukaryotes.
- **Time.** The rule can escalate to `longrun` and `highmem_longrun`, as training takes long without a GPU.

58.8. Reference

Gómez-Pérez D, et al. (2026). REMAG: recovery of eukaryotic genomes from metagenomic data using non-contrastive self-supervised learning. Preprint. https://doi.org/10.64898/2026.03.05.709928

Part III.

Part III: The modules

59. Setting up a project

Part I showed how a Snakemake pipeline is built. From this chapter on we look at this one. Before any module can run, a project needs its reads, its databases and its configuration. This chapter goes through all of that, file by file.

59.1. Two folders

The pipeline separates two things, and it helps to keep them apart in your head.

The **pipeline folder** is a clone of the repository. It holds `workflow/`, the rules and scripts. You don't change anything here during an analysis, and several projects can use the same clone. `pipeline_folder` in the configuration points to it.

The **project folder** is where one analysis lives, usually on scratch space. You start Snakemake here, and all paths in the configuration are relative to it. After a run it looks like this:

```
Snakebite_project/
   config/           your own copy of the configuration
   reads/            the raw FASTQ files, or a link to them
   resources/        databases and reference genomes, or a link to them
   docker_images/    the containers, which Snakemake downloads on the first run
   results/          everything the pipeline writes
   tmp/              temporary files, bound to /tmp in the containers
   .snakemake/       Snakemake's bookkeeping and job logs
   run_Snakebite-Holoruminant-MetaG.sh
```

Because the configuration is copied into the project, you can read it years later and see how that analysis was run. Copy it again for each new project, with `cp -r $PIPELINEFOLDER/config $PROJECTFOLDER`. This copies the hidden `.docker.yml` too.

59.2. Getting the databases

The databases and reference genomes are prepared on a public server, and `docs/02-Installation.md` lists the commands. Plan for about 3 TB and up to three days. Unpacking is much quicker on a fast local disk, if you have one.

They are the same for every project, so download them once and link them:

```
cd $PROJECTFOLDER
ln -s /scratch/project_2009831/RESOURCES/resources .
```

The link must point to a place that the containers can see, which Chapter 3 explains. If the target isn't bound into the container, the tools report that a database file doesn't exist, although you can see it from the login node. This is one of the most common first errors.

59.3. The run script

`run_Snakebite-Holoruminant-MetaG.sh` is copied to the project too. Chapter 17 explains why it exists. These are the lines you edit:

Line	What to set
`projectFolder=`	The project folder
`Profile=`	The profile of your cluster, see below
`module load snakemake/9.11.6`	How Snakemake is made available on your system
`APPTAINER_TMPDIR`, `APPTAINER_CACHEDIR`	Where Apptainer unpacks images. The default in the home folder is usually too small
`XDG_CACHE_HOME`	Where Snakemake keeps its cache, for the same reason. Commented out by default
`--jobs 150`	How many jobs may be in the queue at once
`--singularity-args "-B ..."`	The folders the containers can see. Everything the pipeline reads must be under one of them

A few options deserve a comment.

`--singularity-prefix $projectFolder/docker_images/` is where the images go. They take several gigabytes each, and with this setting each project downloads its own copy. To share them between projects, point it to a common folder.

`--resources metaspades_slots=$metaspades_slots` limits how many metaSPAdes jobs run at once in the whole project. The value comes from `config.yaml`, and Chapter 63 returns to it.

`--restart-times 0 --retries 0` look as if they switched off the resource escalation, but they don't. They only set the default, and the Snakefile sets the number of attempts for every rule itself. Chapter 17 explains this.

`--scheduler greedy` makes Snakemake use a quick way of choosing the next jobs. The default solver can take a long time with thousands of jobs.

`--latency-wait 60` gives a shared file system a minute to show a job's files on the login node.

Anything you write after the target is passed to Snakemake:

```
bash run_Snakebite-Holoruminant-MetaG.sh assemble__megahit -np
 ↪  # a dry run with the commands
PIPELINE_DEBUG=1 bash run_Snakebite-Holoruminant-MetaG.sh reads
 ↪  # with the debug messages of the rules
```

59.4. `config/config.yaml`

This is the file that Snakemake receives with `--configfile`. It names the other files, and it holds the settings of the system.

59.4.1. Paths to the other files

Key	Default	Content
`pipeline_folder`	a path	The clone of the repository. A trailing `/` is added if it's missing
`features-file`	`config/features.yaml`	Reference genomes and databases
`params-file`	`config/params.yaml`	Options of the tools
`docker-file`	`config/.docker.yml`	Which container each module uses
`sample-file`	`config/samples.tsv`	The samples
`dram-config`	`config/dram_config.txt`	Paths for DRAM, see Chapter 51
`phylophlan-config`	`config/phylophlan/supertree_aa.cfg`	No rule reads this key at the moment

Key	Default	Content
`proteinortho-config`	`config/proteinortho_config.txt`	No rule reads this key at the moment, and the file doesn't exist in `config/`

`config/escalation.yaml` is missing from this list because the Snakefile always reads it from that path.

59.4.2. Fast storage

Some tools work on databases that are too big to read quickly over a network file system. The pipeline can copy these to a fast disk on the compute node first. Three settings say where temporary files go:

Key	Default	Meaning
`tmp_storage`	`$TMP`	Normal temporary files
`nvme_storage`	`$LOCAL_SCRATCH`	A fast local disk
`shm_storage`	`/dev/shm`	Memory used as a disk, which is the fastest

These values are written into the shell commands, so a variable like `$LOCAL_SCRATCH` is only read on the compute node when the job runs. Check that the variables exist on your cluster. The names here are the ones used on CSC's Puhti.

The switches say whether the fast storage is used at all:

Key	Default	Meaning
`copy_dbs`	`"True"`	Copy databases to the fast storage where a rule supports it
`kraken2_shm`	`"True"`	Kraken2 uses `/dev/shm`. With `"False"` it uses `nvme_storage`
`diamond_shm`	`"True"`	The same for DIAMOND
`eggnog_shm`	`"True"`	The same for eggNOG-mapper

If you use `/dev/shm`, the database counts as memory of the job. On a shared node this can get in the way of other jobs, and the comment in the file warns about this. That is why the resource set `full_node` exists. Don't rely on `/dev/shm` being large unless you have a whole node for yourself.

If a rule uses these settings, its chapter in this part says so.

59.4.3. The assembler

```
assembler: "metaspades"
```

`metaspades` and `megahit` run these assemblers. Any other name means that you provide the assemblies yourself. Put them into `results/assemble/<name>/<assembly_id>.fa.gz`. Chapter 63 describes this, including how the contigs have to be named.

Note that 4CAC, in the module `eukaryote`, needs the assembly graph of metaSPAdes and doesn't run with the other choices.

59.4.4. Limits and resource sets

`metaspades_slots: 10` is the limit mentioned above.

`resource_sets` is the list of job sizes. These are the defaults, written for CSC's Puhti:

Set	Time (min)	Memory (MB)	CPUs	Partition	Local disk (GB)
`small`	300	16000	1	`small`	0
`medium`	4000	160000	10	`small`	0
`medium_nvme`	4000	160000	10	`small`	10
`large`	4000	320000	20	`small`	0
`large_nvme`	4000	320000	20	`small`	50
`highmem`	4000	1400000	20	`hugemem`	250
`longrun`	20000	320000	20	`longrun`	250
`highmem_longrun`	20000	1400000	20	`hugemem_longrun`	250
`full_node`	4000	360000	40	`small`	1000

On another cluster, change the partitions first, because SLURM refuses a job for a partition that doesn't exist. The values are also generous on purpose and not calibrated. A real run will show you how much your rules really need, because `sacct` reports it, and then you can make them smaller. Queues get much shorter when a job asks for less.

59.5. `config/escalation.yaml`

Each line gives a rule its resource sets, in the order they are tried:

```
assemble__concoct__run: large
eukaryote__tiara__run: medium, large
```

Chapter 17 explains the mechanism. In short: each set gets one attempt, each attempt is a new SLURM job, and a rule that isn't listed gets `small, medium, large`.

Most rules have only one set, 93 of the 98 lines at the moment. A rule like this isn't retried: if it runs out of memory, the pipeline stops and you have to act. To let a rule escalate, add a second set to its line:

```
assemble__concoct__run: large, highmem
```

There are two things to look out for. A failure that has nothing to do with resources, such as a wrong path, fails on every set, and you wait for every attempt. And while you are testing, a single small set saves a lot of waiting.

59.6. `config/features.yaml`

This file says which reference data to use. All paths are relative to the project folder.

59.6.1. `hosts`

```
hosts:
  bovine: resources/reference/Bos_taurus.ARS-UCD1.3.dna.toplevel.fa.gz
  human: resources/reference/Homo_sapiens.GRCh38.dna.toplevel.fa.gz
  ...
```

The reads are decontaminated against these genomes, and **the order matters**. Each genome only gets the reads the previous one left over, as Chapter 61 shows. So if you want to know how many reads came from the human genome, a read that maps to both cow and human is counted as cow if cow is listed first. Put the genome you care most about first.

The names you choose become folder names. Leave the section empty for samples without a host, and then the reads skip this step.

59.6.2. `magscot`

Two sets of marker gene profiles for scoring bins, see Chapter 43.

59.6.3. `databases`

One path per tool. Four tools take a list, and they then run once per entry. Each name you give becomes a folder name in `results/`:

Key	Entries
`kraken2`	`standard`, needing about 140 GB of memory. `refseq500`, commented out by default, needs about 1.4 TB
`diamond`	Protein databases for the reads, by default `cazy`
`diamond_contig_protein`	Protein databases for the contigs, by default `hyddb`
`hmmer`	Profile files for the contigs, by default the three hydrogenase classes `fe`, `fefe` and `nife`
`phylophlan`	`amphora2` and `phylophlan`

59.7. `config/params.yaml`

The options of the tools, sorted by module. The chapter of each module in this part lists them with their meaning. Not every option of every tool is here: an option that isn't listed is either written directly into the rule or left at the default of the tool.

59.8. `config/.docker.yml`

This file says which container each part of the pipeline uses, like `assemble: "docker://fischuu/hrp_assemble:0.2"`. Rules refer to it as `docker["assemble"]`. Section 19.6 lists what's inside. You only change this when you want to try another version of a tool. If you do, write down why.

59.9. `config/samples.tsv`

This is the table of your samples, with one line per library and tabs between the columns:

Column	Meaning
`sample_id`	The sample
`library_id`	The library. Use `L001`, `L002` when one sample was sequenced in several lanes

Column	Meaning
`forward_filename`	Path to the forward reads, relative to the project folder
`reverse_filename`	Path to the reverse reads
`forward_adapter`	Adapter sequence of the forward reads
`reverse_adapter`	Adapter sequence of the reverse reads
`assembly_ids`	Which assemblies these reads go into

The last column decides how samples are assembled:

- Give each sample its own name, and you get one assembly per sample.
- Give several samples the same name, and you get one co-assembly of them.
- Give several names with commas, like `S1, coasm`, and the library goes into both.
- Leave it empty, and the library is processed but not assembled.

The pipeline checks this table when it starts. It refuses a sheet in which a pair of `sample_id` and `library_id` appears twice, or in which a file is listed twice. Without this check, the reads of the second row would be ignored without a warning, because all files are named after this pair. Don't use the Illumina S-number as `library_id`, because all lanes of a sample share it.

`workflow/scripts/createSampleSheet.sh` writes a table from the files in `reads/`. It expects names like `<name>_R1_001.fastq.gz`, so you may have to change two lines for other names, and `docs/03-Setup.md` shows how. `workflow/scripts/concatenateFiles.sh` joins several lanes into one file before that.

59.10. `config/profiles/`

A profile holds the settings of a cluster, as Chapter 15 explains. The repository has `Puhti` and `Migale`. For another system, copy one to a new folder and adjust it. The file must be called `config.yaml`:

```
executor: slurm
jobs: 5
default-resources:
    runtime: 30
    mem_mb: 16000
    cpus_per_task: 1
    slurm_partition: "small"
    slurm_account: "project_2010176"
    gres: "'nvme:0'"
```

`slurm_account` is the project that pays for the jobs, so change it to yours. `gres` is specific to clusters with local disks that have to be requested. On a system without them, remove it here, and set `nvme` in the resource sets accordingly.

59.11. Checking the setup

Before you start real jobs, let Snakemake tell you what it would do:

```
bash run_Snakebite-Holoruminant-MetaG.sh -np
```

This reads all configuration files and checks that the sample table is valid, that all rules fit together and that the input files exist. The table at the end shows how many jobs each rule would get. If the numbers look wrong, such as one assembly when you expected three, look at `assembly_ids`.

`tests/real_run/` contains scripts that do all of this for the example data, and Chapter 70 uses them.

59.12. Summary

- The pipeline folder holds the code, and the project folder holds one analysis and its own copy of the configuration.
- All paths are relative to the project folder, and everything has to be visible inside the containers.
- `config.yaml` describes the system, `features.yaml` the reference data, `params.yaml` the tool options, `escalation.yaml` the job sizes and `samples.tsv` your data.
- The order of the host genomes matters.
- The resource sets are starting values. Change the partitions for your cluster, and calibrate the sizes after a first run.

60. Modules `reads` and `reference`

These two modules do the preparation. They are short, and they are a good first look at how the rules of the pipeline work.

60.1. What they are for

Every project names its files differently. The module `reads` makes one consistent set of names, so that all later rules can rely on them, and it checks the quality of the raw reads.

The module `reference` prepares the host genomes in the compressed format that the mapping tools need.

60.2. The module `reads`

```
bash run_Snakebite-Holoruminant-MetaG.sh reads
```

This runs two steps per library.

60.2.1. `reads__link__run`: one consistent name

This rule looks up the two read files of a library in `samples.tsv`, and links them to a fixed name:

```
results/reads/{sample}.{library}_1.fq.gz
results/reads/{sample}.{library}_2.fq.gz
```

The inputs come from the functions `get_forward` and `get_reverse`. They take the wildcards and look up the row of the sample table, as Chapter 9 explains. Both functions use `_reads_sample_row`, which raises a clear error if it doesn't find exactly one row.

The command is only this:

```
ln --symbolic "$(readlink --canonicalize {input.forward_})"
 ↪  {output.forward_}
ln --symbolic "$(readlink --canonicalize {input.reverse_})"
 ↪  {output.reverse_}
```

`readlink --canonicalize` turns the path into an absolute one first. Otherwise the link would point to a relative path, and it would be wrong as seen from `results/reads/`. No data is copied, so this step takes no time and no space, and all later rules read the original files. This also means that the target of the link must remain in place and that it must be visible in the containers.

`reads__link` is the target of this step, and it asks for the links of all libraries.

60.2.2. `reads__fastqc`: first look at the quality

`reads__fastqc` asks for a report on each read file:

```
results/reads/{sample}.{library}_{1,2}_fastqc.zip
```

It has no command of its own. The rule that produces these files is the generic `helpers__fastqc` from Section 68.2, which works for any `.fq.gz` file. Chapter 20 explains how to read the report.

Look at these reports before you go on. They tell you the read length, whether adapters are present and how the quality falls along the reads. This is what you need to judge the next step.

60.2.3. What you get

File	Content
`{sample}.{library}_1.fq.gz`, `_2.fq.gz`	Links to your reads
`{sample}.{library}_{1,2}_fastqc.html`	The report to open in a browser
`{sample}.{library}_{1,2}_fastqc.zip`	The same as data
`{sample}.{library}.log`, `benchmark/`	Log and resource use of the link step

60.3. The module reference

```
bash run_Snakebite-Holoruminant-MetaG.sh reference
```

60.3.1. reference__hosts__recompress

The host genomes are listed in features.yaml, and they usually come compressed with gzip. Some steps have to jump to a certain position in the genome, and this is impossible in a normal gzip file: you would have to read everything from the start. BGZF, which bgzip writes, solves this by compressing in small blocks. A BGZF file is still a valid gzip file, so all tools can read it, and with an index they can jump to any place.

So this rule uncompresses each genome and compresses it again:

```
zcat {input} | bgzip -@ {threads} > {output}
```

The result is results/reference/hosts/{genome}.fa.gz, where {genome} is the name you chose in features.yaml. The input path is looked up with a function again:

```
input:
    lambda wildcards: features["hosts"][wildcards.genome],
```

reference__hosts asks for one such file per host, and reference asks for all of them. Since you rarely change the genomes, and this is the same for all projects, this module usually runs only once per project.

The index files .fai and .gzi aren't made here. The generic rule helpers__samtools__faidx_fagz makes them when a later rule asks for them.

60.4. Summary

- reads links your files to the names {sample}.{library}_{1,2}.fq.gz and runs FastQC on them.
- reference recompresses the host genomes with bgzip.
- Neither module needs a database, so they are a cheap first test of a new setup.

61. Module preprocess

Raw reads aren't ready for analysis. They contain adapter sequence, bases of poor quality, and in samples from an animal many reads that come from the animal itself. This module removes all of that. Its result, the clean reads, is the input of nearly everything that follows.

```
bash run_Snakebite-Holoruminant-MetaG.sh preprocess
```

61.1. The flow

```
results/reads/S.L_{1,2}.fq.gz
          preprocess__fastp__run

results/preprocess/fastp/S.L_{1,2}.fq.gz
          preprocess__bowtie2__map                  (first host)

results/preprocess/bowtie2/bovine/S.L.cram
          preprocess__bowtie2__extract_nonhost__run

results/preprocess/bowtie2/nonbovine/S.L_{1,2}.fq.gz
          preprocess__bowtie2__map                  (next host)

       ...
          preprocess__store_final_fastq

results/preprocess/bowtie2/decontaminated_reads/S.L_{1,2}.fq.gz  (the clean reads)
```

With no hosts in `features.yaml`, the arrow goes directly from fastp to the clean reads. The name of the last folder comes from the constant `CLEAN_READS` in `folders.smk`, as all paths do in this pipeline. `S.L` stands for `{sample_id}.{library_id}`, which the constant `LIBRARY` holds.

61.2. Trimming: preprocess__fastp__run

This rule runs fastp on one library, and Chapter 21 explains the tool. This is the core of the rule:

```
fastp \
    --in1 {input.forward_} --in2 {input.reverse_} \
    --out1 {params.work_prefix}_1.fq --out2
     ↪  {params.work_prefix}_2.fq \
    --unpaired1 {params.work_prefix}_u1.fq --unpaired2
     ↪  {params.work_prefix}_u2.fq \
    --html {output.html} --json {output.json} \
    --verbose \
    --adapter_sequence {params.adapter_forward} \
    --adapter_sequence_r2 {params.adapter_reverse} \
    --length_required {params.length_required} \
    --thread {threads} \
    {params.extra}
```

The adapters come from `samples.tsv`, through the functions `get_forward_adapter` and `get_reverse_adapter`. The other options come from `params.yaml`:

Option	Default	Meaning
`preprocess: fastp: length_required`	25	Throw away reads that are shorter than this after trimming. Nonpareil needs at least 25 bases
`preprocess: fastp: extra`	`--trim_poly_g` `--trim_poly_x`	Further options, here for removing tails of a single base, which two-color sequencers tend to produce

Note that fastp first writes uncompressed files. After that, the rule compresses them with `bgzip`, one by one, and tests each file:

```
bgzip --compress-level 9 --threads {threads}
 ↪  {params.work_prefix}_${{suffix}}.fq
mv {params.work_prefix}_${{suffix}}.fq.gz "${{dest[$suffix]}}"
gzip --test "${{dest[$suffix]}}"
```

fastp can only use a limited number of threads for its own compression, and `bgzip` is faster here. Only a file that passed the test gets its final name, so if anything fails along the way, the rule fails and you don't get a broken file.

The doubled braces are needed because Snakemake reads single braces as its own placeholders, as Chapter 5 explains. They turn into normal Bash variables when the command runs.

The log uses the pattern of Chapter 17: it's first written to `{log}.try{resources.attempt}`, and only a successful run moves it to `{log}`.

`preprocess__fastp` is the target, and it asks for the outcome of all libraries.

61.2.1. What you get

File	Content
`S.L_1.fq.gz`, `S.L_2.fq.gz`	Read pairs that both passed
`S.L_u1.fq.gz`, `S.L_u2.fq.gz`	Reads whose mate was thrown away. They aren't used later
`S.L_fastp.html`	Report for a browser
`S.L_fastp.json`	The same as data

61.3. Removing host reads

The pipeline maps the reads against each host genome in turn, and it keeps only the reads that don't map. Chapter 22 explains the mapping.

61.3.1. `preprocess__bowtie2__build`

Bowtie2 needs an index of each genome, and this rule builds it once per host:

```
bowtie2-build --threads {threads} {input.genome} {output.index}
```

The index consists of several files that share the prefix `{output.index}`. Snakemake can't easily follow this, so the rule says `index=touch(HOST_INDEX / "{genome}")`. This makes an empty file with the name of the prefix, and all other rules depend on that file. Chapter 12 explains `touch`.

61.3.2. preprocess__bowtie2__map

This rule maps one library to one host:

```
bowtie2 --threads {threads} --rg-id '{params.rg_id}' --rg
 ↪  '{params.rg_extra}' \
        -x {input.index} -1 {input.forward_} -2 {input.reverse_} \
    | samtools sort -@ {threads} -m {params.sort_memory} -l 9 -M
     ↪  --reference {input.genome} \
        -o {output.cram} -
samtools idxstats {output.cram} | awk '{{print $1, $3}}' >
 ↪  {output.counts}
```

Which reads come in depends on the position of the host. The function `reads_for_host_mapping` looks at the order in `HOST_NAMES`: the first host gets fastp's reads, and all others get what the host before left. This is what the output of Snakemake's dry run shows as a chain of jobs, and Chapter 7 explains why it just works: each rule asks for its input, and Snakemake finds the rule that makes it.

The read group options `--rg-id` and `--rg` mark the reads with their library and sample, built by `compose_rg_id` and `compose_rg_extra`. `samtools sort` gets `-M`, which places unmapped reads next to each other according to similarity, and this compresses them better. The output is a CRAM file because of its extension, and CRAM needs the reference sequence, hence `--reference`.

The result in `.chr_alignment_counts.tsv` is a simple count of the reads per chromosome.

The CRAM is marked `temp()` because nothing later needs it, and Snakemake removes it as soon as the rules that read it are finished.

One option in `params.yaml` applies here:

Option	Default	Meaning
`preprocess: bowtie2: samtools: mem_per_thread`	`1G`	Memory for `samtools sort`, **per thread**

Make sure that this memory, multiplied by the threads, fits into the job and leaves room for Bowtie2. It can be a reason for a job to be killed.

The line with `find ... -delete` before the mapping removes temporary files of an earlier attempt, as they would disturb `samtools sort`.

61.3.3. preprocess__bowtie2__extract_nonhost__run

This rule writes the reads that didn't map to FASTQ:

```
samtools view -u -f 12 -T {input.genome} {input.cram} \
    | samtools sort -n -l 0 -m 500M -@ {threads} -T
     ↪  {params.sort_tmpdir} \
    | samtools fastq -N -0 /dev/null -c 9 -@ {threads} \
        -1 {output.forward_} \
        -2 {output.reverse_}
```

Reading it from left to right:

- `-f 12` keeps reads with both flags 4 and 8, meaning that the read and its mate are both unmapped. A pair in which only one mate maps is treated as host and removed. This is the careful choice, and it loses a few real microbial reads.
- `samtools sort -n` sorts by name, because `samtools fastq` needs the two mates next to each other.
- `samtools fastq` writes the two files. `-N` adds `/1` and `/2` to the names, and `-0 /dev/null` throws away reads that lack this information. It compresses them by itself, because the names end in `.gz`, and `-c 9` sets the level.

Until version 0.4.69, this rule wrote through `>(pigz ...)`, and this could cut off the files, as Section 13.7 explains. If you have clean reads from an earlier version, check them as `RELEASE_NOTES` describes.

These FASTQ files are temporary as well, as long as they aren't the last of the chain.

61.3.4. preprocess__store_final_fastq

This rule copies the reads at the end of the chain to the place of the clean reads. `reads_after_last_host` finds them. With no hosts, these are fastp's reads.

`preprocess__bowtie2__extract_nonhost` asks for these files for all libraries, and `preprocess__bowtie2` is the target of the whole chain.

Since each host removes reads before the next one sees them, you can read how many reads were lost at each step from the sizes of the FASTQ reports, or from `samtools stats` below.

61.4. Checking the result

61.4.1. preprocess__samtools__stats_cram

This rule writes `samtools stats` for each mapping, and `preprocess__samtools` asks for these files and for the `flagstats` of the generic rule `helpers__samtools__flagstats_cram`. Chapter 23 explains them. `flagstats` tells you quickly how many reads mapped to a host. In rumen samples, you would expect a share of cow reads, and hardly any human ones. A high human share points to contamination in the lab. The share of each host is a result in its own right.

61.4.2. preprocess__fastqc__fastp and preprocess__fastqc__nonhost

These two rules ask for FastQC reports of the reads after fastp and after each host. `preprocess__fastqc` is the target for both. `helpers__fastqc` does the real work. Compare the report before and after trimming, and you see if the adapters are gone.

61.5. Things to look at after a run

1. How many reads did fastp keep? The JSON or the HTML report has it. Losing a few percent is normal. Losing half is not.
2. Are the adapters gone? Look at the FastQC report after fastp.
3. How many reads mapped to each host? Look at `flagstats`.
4. How many clean reads does each sample have? A sample with far fewer than the others will produce less of everything later on.

61.6. Summary

- fastp trims the reads with the adapters from the sample table.
- Bowtie2 maps them against each host in turn, and only pairs with both mates unmapped go on.
- The order of the hosts in `features.yaml` decides the order of the chain.
- The clean reads at the end are what all later modules read.
- Temporary CRAM and FASTQ files of the chain are removed automatically.

62. Module `read_annotate`

This module asks what is in a sample before anything is assembled: which organisms, how many of each, which functions, and how well the sequencing covered the community. All tools work on the reads directly, and each answers a slightly different question. This is the module that needs the largest databases.

```
bash run_Snakebite-Holoruminant-MetaG.sh read_annotate

# Or a single tool:
bash run_Snakebite-Holoruminant-MetaG.sh read_annotate__kraken2
```

62.1. The tools at a glance

Target	Tool	Answers	Reads it uses
`read_annotate__kraken2`	Chapter 24	Which taxon does each read belong to?	After fastp
`read_annotate__krona`	Chapter 25	Kraken2's answer as an interactive chart	
`read_annotate__singlem`	Chapter 26	Which organisms, from marker genes, and how much of the sample is microbial?	Clean
`read_annotate__sylph`	Chapter 27	Which known genomes are in the sample?	Clean
`read_annotate__metaphlan`	Chapter 28	Which organisms, from clade-specific genes?	Clean
`read_annotate__humann`	Chapter 29	Which gene families and functions?	Clean

Target	Tool	Answers	Reads it uses
read_annotate__phyloflash	Chapter 30	Which small-subunit rRNA genes?	Clean
read_annotate__diamond	Chapter 31	Which proteins of a chosen database, such as CAZymes?	Clean
read_annotate__nonpareil	Chapter 33	How well did the sequencing cover the community?	Clean
read_annotate__nifh	Chapter 32	Which genes of the nitrogen cycle? Only in read_annotate_extra	Clean

Why so many? No single profiler is right for all questions, and they fail in different ways. If three of them agree that a sample is dominated by Prevotella, you can believe it. If they disagree, you've learned something about the sample, or about the databases. For a large project, you would pick the ones you need and not run all of them. Each can be called on its own.

Note the different starting points: Kraken2 reads what fastp left, **before** host removal. So its report also shows you the host, which the others can't. The other tools read the clean reads.

62.2. Fast storage for big databases

Before the rules, one mechanism needs explaining, since the rules of Kraken2 and DIAMOND are long because of it.

Kraken2 with the standard database has to read 140 GB from disk, and on a network file system this takes very long. So the rule first copies the database to a fast local place. It has three choices:

1. `/dev/shm`, memory used as a disk, if the database fits and `kraken2_shm` is `"True"`
2. The local NVMe disk, if the database fits there
3. The original place, without copying, but only as a last resort

The rule measures what is free with `df` and compares this with the size of the database:

```
if [ "$db_size" -lt "$shm_avail" ]; then
    db_dst="$db_shm"
```

```
elif [ "$db_size" -lt "$nvme_avail" ]; then
    db_dst="$db_nvme"
elif [ {resources.attempt} -eq {params.max_attempt} ]; then
    db_dst="$db_src"
else
    echo "database too large for available fast storage; aborting
    ↪  attempt {resources.attempt} to trigger escalation"
    exit 1
fi
```

The third case explains the escalation here. If the database fits nowhere, the attempt fails on purpose, and the next resource set may offer a bigger disk. Only the last attempt runs from the slow original place.

At the end the rule removes the copy in `/dev/shm`. If the job gets killed before that, the copy stays until the node cleans up. This can be a reason why a later job on the same node sees less space.

With `copy_dbs: "False"` nothing is copied, and the database is read in place.

62.3. Kraken2

62.3.1. `read_annotate__kraken2__assign`

This is the only rule of the module that handles all samples in one job. Kraken2 loads its database for each run, and with a database in memory it's much quicker to reuse that one copy for all samples than to copy it for each. This has a consequence: the job gets all samples, so it's long, and if it dies, all samples of this database run again.

After staging, it loops over the samples:

```
kraken2 \
    --db "$db_dst" \
    --threads {threads} \
    --gzip-compressed \
    --paired \
    --report "$report" \
    --memory-mapping \
    "$forward" "$reverse" \
    2> "$sample_log" \
    | pigz --processes {threads} > "$out_gz"
```

`--memory-mapping` makes Kraken2 read the database files directly instead of loading them into its own memory. This is why they should be on a fast place, and why this saves memory when the files are in `/dev/shm`.

Without the option `--output`, Kraken2 writes its per-read output to the standard output, which goes through a pipe into `pigz`. Its messages go to the standard error, and so to the log of the sample.

The wildcard is `{kraken_db}`, so this rule runs once per database in `features.yaml`.

File in `results/read_annotate/kraken2/{sample}/{library}/{kraken_db}/`	
`{sample}.{library}.report`	The summary per taxon, which is what you normally read
`{sample}.{library}.out.gz`	One line per read
`{sample}.{library}.log`	What Kraken2 printed, with the share of classified reads

`read_annotate__kraken2` asks for the reports of all samples and databases.

62.3.2. `read_annotate__krona__visualize`

This rule turns each report into an interactive chart:

```
ktImportTaxonomy -t 5 -m 3 {input} -o {output} --tax
 ↪  {params.krona_db}
```

The options say which columns of the report to use: the taxon from column 5, and its reads from column 3. Chapter 25 explains why this matters, and that charts from versions before 0.4.69 are wrong.

`read_annotate__krona` asks for one HTML file per sample and database.

62.4. SingleM

62.4.1. `read_annotate__singlem__pipe`

This rule runs `singlem pipe` on one library and writes three tables: the marker gene sequences (`otu_table`), the same in a form that can be combined later (`archive.json`), and the species profile (`condense.tsv`).

The rule writes the free space of `$TMPDIR` into the log, as SingleM can need a lot of it, and an error of a full disk is easier to see that way.

A comment in the rule points out that SingleM's documentation recommends raw reads. The pipeline uses clean reads here so that all tools see the same data.

62.4.2. read_annotate__singlem__condense

This rule combines the `archive.json` files of all samples into one profile, `singlem.tsv`.

62.4.3. read_annotate__singlem__microbial_fraction

This rule estimates which share of a library comes from Bacteria and Archaea, by comparing the profile with the amount of sequence. This is often the first number you want for a host-associated sample.

62.4.4. read_annotate__singlem__aggregate_microbial_fraction

This rule stacks these results into one table with `csvstack`. `read_annotate__singlem` is the target.

62.5. sylph

62.5.1. read_annotate__sylph_profile

This is one simple call per library. Unlike the others, it reports genomes rather than taxa, and it needs no mapping or marker gene step:

```
sylph profile {input.db} -1 {input.forwards} -2 {input.reverses}
 ↪  -t {threads} > {output}
```

`read_annotate__sylph` asks for one table per library. There's no step that combines them. Note that the name of the first rule breaks the usual naming pattern, without the double underscore before `profile`.

62.6. MetaPhlAn

62.6.1. read_annotate__metaphlan__run

```
metaphlan {input.forward_},{input.reverse_} \
          --bowtie2out {output.bt2_out} \
          --nproc {threads} \
          --input_type fastq \
          -o {output.mp_out} \
          --bowtie2db {input.data}
```

`--bowtie2out` keeps the intermediate mapping, so that MetaPhlAn could be rerun with other settings without mapping again.

62.6.2. read_annotate__metaphlan__condense

This rule merges the profiles into one table with `merge_metaphlan_tables.py`. `read_annotate__metaphlan` is the target.

62.7. HUMAnN

62.7.1. read_annotate__humann__run

HUMAnN doesn't use the pairing of the reads, so the rule simply puts both files together first, and this is valid for gzip files. The profile of MetaPhlAn is an input, which tells HUMAnN which species to look at first, so this rule runs after MetaPhlAn:

```
cat {input.forward_} {input.reverse_} > {output.cat}
humann --input {output.cat} --output {params.out_folder} \
       --threads {threads} \
       --protein-database {input.prot_dbs} \
       --nucleotide-database {input.nt_dbs} \
       --output-basename {params.out_name} \
       --taxonomic-profile {input.mp_out} \
       {params.additional_options}
```

Option	Default	Meaning
read_annotate: humann: additional_options	--remove-temp-output	Removes HUMAnN's large intermediate files

The rule prints a warning about TMPDIR: on large data sets this folder needs a lot of space.

62.7.2. read_annotate__humann__condense

This rule concatenates the tables of the gene families into humann_genefamilies.tsv. Note that it's a plain cat, so the result holds one block per sample, each with its own header, and not a table with one column per sample. Use humann_join_tables if you need that. read_annotate__humann is the target.

62.8. phyloFlash

62.8.1. read_annotate__phyloflash__run

```
phyloFlash.pl -dbhome {input.phyloflash_dbs} \
              -lib {params.lib} -CPUs {threads} \
              -read1 {input.forward_} -read2 {input.reverse_} \
              -almosteverything
mkdir -p {params.outdir}
mv {params.lib}.* {params.outdir}
```

phyloFlash writes to the current folder, which here is the project folder, and so the rule moves the files afterwards. The library name uses an underscore instead of a dot, as in ERR2019410_lib1. -almosteverything switches on the optional analyses, except the one with EMIRGE.

62.8.2. read_annotate__phyloflash__condense

This rule concatenates the logs of all libraries. read_annotate__phyloflash is the target.

62.9. DIAMOND

62.9.1. read_annotate__diamond__assign

This rule compares the reads with a protein database. It does its staging in nearly the same way as Kraken2, and then runs twice, once per read file:

```
diamond blastx -d "$DB_LOC" -q {input.forwards} -o {output.out_R1}
 ↪  -p {threads} --max-target-seqs {params.max_target_seqs}
diamond blastx -d "$DB_LOC" -q {input.reverses} -o {output.out_R2}
 ↪  -p {threads} --max-target-seqs {params.max_target_seqs}
```

The two files aren't linked: each read gets its own best hit, also when the two mates of a pair disagree.

Option	Default	Meaning
read_annotate: diamond: max-target-seqs	1	How many hits to keep per read

The wildcard {diamond_db} is a name in features.yaml.

62.9.2. read_annotate__diamond__summarise

This rule runs an R script that turns the hits into feature tables, summary_a.tsv and summary_b.tsv. The script is found by the name of the database, as workflow/scripts/downstream_scripts/create_featuretable_diamond_{diamond_db}.R. **So if you add your own database under another name, you also have to provide this script**, otherwise this rule fails. read_annotate__diamond is the target.

62.10. Nonpareil

62.10.1. read_annotate__nonpareil__run

Nonpareil only uses one read of each pair, and it needs an uncompressed file:

```
zcat {input.forward_} > "${{prefix}}_1.fq"
nonpareil -s "${{prefix}}_1.fq" -T kmer -b "$prefix" -f fastq -t
 ↪  {threads} -X {params.X}
rm -f "${{prefix}}_1.fq"
```

Option	Default	Meaning
`read_annotate:` `nonpareil: X`	100	How many reads to compare against the rest. Smaller is quicker, and the comment in `params.yaml` says that this value is for testing, with 1000 as the default. Raise it for a real analysis

All four outputs are marked `touch()`. The reason is in the comment of the rule: on samples with little coverage Nonpareil may not converge, and it can then leave empty files. `touch()` makes sure that Snakemake still sees the files that it expects, as Chapter 12 explains.

62.10.2. `read_annotate__nonpareil__aggregate`

This rule runs `workflow/scripts/nonpareil_summary.R`, which reads all results and writes `nonpareil.tsv`, with one line per library. `read_annotate__nonpareil` is the target.

62.11. nCycDB

These rules are only part of the target `read_annotate_extra`, not of `read_annotate` or `all`.

62.11.1. `read_annotate__ncyc__prepare_input`

This rule writes the sample sheet that the Perl script expects, using `workflow/scripts/prepare_ncyc_inputs.sh`.

62.11.2. read_annotate__ncyc__run

This rule runs workflow/scripts/NCycProfiler.PL, which comes with the pipeline, on all samples at once. It uses DIAMOND for the search, and writes one table, ncyc_results.tsv. read_annotate__ncyc is the target.

62.12. The options in one place

All options of this module in params.yaml are listed above. Everything else, such as the choice of databases, is in features.yaml, and the fast storage is set in config.yaml.

62.13. What to look at after a run

1. **Kraken2:** what share of the reads was classified at all? In rumen samples, a low value is normal, because many rumen microbes aren't in the standard database. It says more about the database than about your sample.
2. **SingleM microbial fraction:** does it fit with what you know about the sample?
3. **Nonpareil:** is the coverage high enough? If it's low, a deeper sequencing would find many more organisms, and few MAGs are to be expected.
4. **Compare the profilers:** do they agree on the dominant groups?

62.14. Summary

- The tools work on the reads, and each answers another question. You can run each on its own.
- Kraken2 uses the reads after fastp. The others use the clean reads.
- Kraken2 and DIAMOND copy their database to fast storage, and fail on purpose if it doesn't fit, so that a bigger resource set can take over.
- A new DIAMOND database needs a matching R script.
- nCycDB is only part of read_annotate_extra.

63. Module `assemble`

This is the heart of the pipeline. It turns millions of short reads into a catalogue of genomes, known as metagenome-assembled genomes or MAGs. Everything in the modules `quantify` and `mag_annotate` builds on this catalogue.

```
bash run_Snakebite-Holoruminant-MetaG.sh assemble
```

63.1. The idea

The reads of a metagenome come from thousands of organisms, all mixed. To get genomes out of this, four steps are needed.

1. **Assemble.** Overlapping reads are joined into longer pieces, the contigs. At this stage, you don't know which contig belongs to which organism.
2. **Bin.** Contigs of one organism look similar: they have a similar base composition, and a similar coverage in each sample, because the amount of an organism in a sample affects all its contigs alike. A binner uses this to sort contigs into groups, the bins.
3. **Refine.** Binners make different mistakes. The pipeline runs three of them, scores all their bins by marker genes, and builds the best set from all of them.
4. **Dereplicate.** The same species can come out of several assemblies. dRep keeps one genome per species, which gives the final catalogue.

63.2. The flow

```
clean reads of the libraries of one assembly
          assemble__megahit__run or assemble__metaspades__run

results/assemble/{assembler}/{assembly_id}.fa.gz                    the contigs
          assemble__bowtie2__build__run, assemble__bowtie2__map

results/assemble/bowtie2/{assembly_id}.{sample_id}.{library_id}.cram
          assemble__concoct__run, assemble__maxbin2__run, assemble__metabat2__run
```

```
results/assemble/{concoct,maxbin2,metabat2}/{assembly_id}/*.fa.gz   the bins of each binn
          the six MAGScoT rules, see below

results/assemble/magscot/{assembly_id}.fa.gz                        the contigs in refine
          assemble__drep__split_bins, assemble__drep__run, assemble__drep__catalogue

results/assemble/drep/dereplicated_genomes.fa.gz                    the MAG catalogue
```

Note that the last steps collect all assemblies: the wildcard `{assembly_id}` disappears at dRep, and there is one catalogue in the end, however many assemblies you have.

The final target only asks for the catalogue:

```
rule assemble:
    input:
        MAG_CATALOGUE,
```

Snakemake works out the rest backwards, as Chapter 7 explains.

63.3. Which reads go into which assembly

The column `assembly_ids` of `samples.tsv` decides this, as Chapter 59 explains. Three helper functions in `workflow/rules/assemble/__functions__.smk` use it.

`libraries_of_assembly` lists the libraries of one assembly. `assembly_reads(end)` turns this into paths of one mate. These are the clean reads, or fastp's reads if no host is configured. `assembly_alignments` lists the mapping of each library against an assembly. This is how the binners find their input. Its companion `assembly_alignment_indexes` adds the index files.

Contigs and mappings are linked through the constant `CONTIGS`. It holds `results/assemble/<assembler>/{assembly_id}.fa.gz`, where `<assembler>` comes from the configuration. All later rules use this constant, and so they don't care which assembler made the contigs.

63.4. Assembling

Only one of the two assemblers runs in a project, namely the one in `config.yaml`. Chapter 36 and Chapter 35 compare them. In short, metaSPAdes tends to give better assemblies, and it needs much more memory. MEGAHIT is the economical choice, and it's a fall-back if metaSPAdes doesn't finish.

63.4.1. assemble__metaspades__run

```
cat {input.forwards} > {output.pooled_forward}
cat {input.reverses} > {output.pooled_reverse}
metaspades.py -1 {output.pooled_forward} -2 {output.pooled_reverse}
 ↪  -o {params.workdir} \
    -t {threads} -m {resources.mem_mb} -k
     ↪  {params.settings[kmer_size]} \
    {params.settings[additional_options]}
python {params.script_folder}/contig_ids.py {wildcards.assembly_id}
 ↪  {params.workdir}/contigs.fasta \
    | bgzip -l 9 -@ {threads} > {output.contigs}
tar -cv -I "pigz -9 -p {threads}" --remove-files -f
 ↪  {output.workdir_archive} {params.workdir}
```

metaSPAdes takes only one library in this mode, so the rule pools the reads first, and it deletes the pooled files later, because they are `temp()`.

The working folder holds hundreds of thousands of small files, which is bad for a shared file system. So the rule packs it into one archive at the end, and it removes the original. The module `eukaryote` later reads the assembly graph from this archive.

Option	Default	Meaning
`assemble: metaspades: kmer_size`	`"21,33,55"`	The k-mer sizes to use. See Chapter 36
`assemble: metaspades: additional_options`	`"--only-assembler"`	Skips the read error correction, which saves a lot of time and memory

Be aware that `-m` gets the memory in megabytes, while metaSPAdes reads this number as gigabytes. So it doesn't act as a limit here, and the memory limit of the SLURM job is what counts.

The target `assemble__metaspades` asks for all assemblies. `metaspades_slots` in `config.yaml` limits how many of these jobs run at once.

63.4.2. assemble__megahit__run

```
megahit -1 {params.forwards} -2 {params.reverses} -o
 ↪  {params.workdir} -t {threads} \
    --min-count {params.settings[mincount]} \
    --k-min {params.settings[kmin]} --k-max {params.settings[kmax]}
     ↪  --k-step {params.settings[kstep]} \
    --min-contig-len {params.settings[min_contig_len]} \
    --force --continue --verbose
     ↪  {params.settings[additional_options]}
```

MEGAHIT takes several libraries directly, separated by commas, which `assembly_reads(end, separator=",")` builds. `--force` is needed because Snakemake has already created the output folder, and MEGAHIT would otherwise refuse to write there.

Option	Default	Meaning
`assemble: megahit: mincount`	2	How often a k-mer must occur to count
`assemble: megahit: kmin`	21	Smallest k-mer. Use 27 for complex communities
`assemble: megahit: kmax`	141	Largest k-mer. MEGAHIT goes up to this value also with shorter reads. Use 127 for complex communities
`assemble: megahit: kstep`	10	Step between k-mer sizes
`assemble: megahit: min_contig_len`	1500	Shorter contigs aren't reported
`assemble: megahit: additional_options`	`""`	For example `--no-mercy` for samples with deep coverage

The target `assemble__megahit` asks for all assemblies.

63.4.3. The contig names

Both assemblers end with `workflow/scripts/contig_ids.py`, which renames the contigs like this:

```
ERR2019410:bin_NA@contig_00000001
```

This name carries information that later steps read: the assembly, the bin (NA until there is a bin), and a number. The @ separates the genome from the contig, and CoverM uses that later on to add up the contigs of a genome. **Don't change this format**, as several scripts depend on it.

63.4.4. Your own assemblies

With another value of assembler, such as long_reads, no assembler runs. Put your files into results/assemble/long_reads/{assembly_id}.fa.gz. Rename the contigs first with workflow/scripts/rename_provided_assemblies.py, because the format above is required.

63.5. Mapping the reads back

Binners need to know the coverage of each contig in each library.

63.5.1. assemble__bowtie2__build__run

This rule indexes the contigs of one assembly. As in the module preprocess, an empty file made by touch() stands in for the index. The target assemble__bowtie2__build asks for the index of all assemblies.

63.5.2. assemble__bowtie2__map

```
bowtie2 --threads {threads} --rg-id '{params.rg_id}' --rg
 ↪  '{params.rg_extra}' \
        -x {input.index} -1 {input.forward_} -2 {input.reverse_} \
    | samtools view -b -o {output.unsorted} -
samtools sort -@ {threads} -m {params.sort_memory} -l 9 -O
 ↪  {SAMTOOLS_OUTTYPE} \
    -o {output.alignment} {output.unsorted}
```

Each library is mapped only to the assemblies it belongs to. So in a co-assembly, every sample of the group is mapped. If each sample has its own assembly, a sample is only mapped to its own assembly, and a binner then has one coverage value per contig. This is one reason why co-assemblies often give better bins.

Option	Default	Meaning
assemble: samtools: mem	8G	Memory for samtools sort, per thread. Make sure this multiplied by the threads fits into the job
assemble: samtools: out_type	cram	Change this to bam if the mapping rule fails with an error about the CRAM header. Very long contig names can cause this

The target assemble__bowtie2 asks for all mappings of all assemblies.

63.6. Binning

All three binners get the contigs and the mappings, and all three produce a folder with one .fa.gz file per bin. Chapter 38, Chapter 39 and Chapter 40 explain how they differ.

These rules declare a whole folder as output with directory(), because nobody can know beforehand how many bins there will be.

63.6.1. assemble__concoct__run

The rule follows the steps of CONCOCT's manual:

1. cut_up_fasta.py cuts the contigs into pieces of 10 kb, so that long contigs get more weight.
2. samtools view makes a BAM file of the mapped reads, for each library, as the next step needs BAM.
3. concoct_coverage_table.py computes the coverage of the pieces.
4. concoct clusters the pieces.
5. merge_cutup_clustering.py goes back from pieces to contigs.
6. extract_fasta_bins.py writes the bins.

At the end only the bins remain. Note the expression <(zcat {input.contigs}), which hands the uncompressed contigs to a tool that wants a file name, without writing them to the disk.

The target is assemble__concoct. This rule is the only one of the three with its own container, docker["concoct"].

63.6.2. assemble__maxbin2__run

MaxBin2 only takes one abundance value per contig:

```
samtools coverage {input.alignments} | sed '/^#/d' | cut -f 1,5 >
 ↪  {output}/maxbin2.coverage
run_MaxBin.pl -contig {input.contigs} -abund
 ↪  {output}/maxbin2.coverage -out {output}/maxbin2 \
    -min_contig_length {params.min_contig} -thread {threads}
```

Note what the comment of the rule points out: column 5 of `samtools coverage` is the number of covered bases, and not the mean depth, which is column 7. So it's the amount of covered sequence that is used as the abundance. Also, with several libraries, `samtools coverage` adds them up into one value.

The rule then renames `.fasta` to `.fa`, compresses the bins, and removes the other files of MaxBin2. The target is `assemble__maxbin2`.

Option	Default	Meaning
`assemble: maxbin: min_contig_len`	1500	Ignore shorter contigs

63.6.3. assemble__metabat2__run

```
jgi_summarize_bam_contig_depths --outputDepth "$depth"
 ↪  --pairedContigs "$pairs" "${{bams[@]}}"
metabat2 -i {input.contigs} -a "$depth" -o {output}/bin \
    -m {params.min_contig} -t {threads} -v
```

MetaBAT2 uses one depth per contig and library, so it gains the most from having several samples in an assembly. The target is `assemble__metabat2`.

Option	Default	Meaning
`assemble: metabat: min_contig_len`	1500	Ignore shorter contigs. MetaBAT2 doesn't accept less than 1500

63.7. Refining: MAGScoT

MAGScoT takes the bins of all binners, scores them by marker genes, and combines them. Chapter 43 explains the method. Six rules prepare and run it. Everything of one

assembly stays in results/assemble/magscot/{assembly_id}/.

63.7.1. assemble__magscot__prodigal

Marker genes are proteins, so we need the proteins first:

```
zcat {input} \
    | parallel -j {threads} --block 1M --recstart '>' --pipe
    ↪  --keep-order \
       prodigal -p meta -a /dev/stdout -d /dev/null -o /dev/null \
    > {output}
```

Prodigal itself uses one thread. parallel splits the FASTA stream into blocks of 1 MB at the start of a record, and it hands each to its own Prodigal. --keep-order keeps the result in the original order. -p meta is Prodigal's mode for metagenomes.

The comment in the rule explains why it sets its own TMPDIR: GNU parallel keeps control files there, and the usual value points to a place that the container can't see.

63.7.2. assemble__magscot__hmmsearch

```
hmmsearch -o /dev/null --tblout "${{hits%.gz}}" --noali --notextw
 ↪  --cut_nc --cpu {threads} \
    {input.hmm} {input.proteins}
```

This rule runs once per assembly and marker set. Its wildcard {marker_set} can only be pfam or tigr, as wildcard_constraints says, and these are the two profiles in features.yaml. --cut_nc uses the score threshold that comes with each profile.

63.7.3. assemble__magscot__marker_table

This rule brings both results into the format MAGScoT wants: protein, marker and e-value. Note that TIGRFAM names the marker in column 3, and Pfam in column 4.

63.7.4. assemble__magscot__merge_contig_to_bin

This rule lists which contig is in which bin of which binner. It reads all bins with zcat, and writes one line per contig: bin, contig, binner.

63.7.5. assemble__magscot__run

```
Rscript --vanilla {params.script_folder}/MAGScoT/MAGScoT.R -i
 ↪  {input.contigs_to_bin} \
    --hmm {input.markers} -o {params.prefix} -t
     ↪  {params.settings[threshold]} \
    {params.settings[extra]} \
    || echo "MAGScoT failed, most likely no viable bins: continuing
     ↪  without bins"
touch {output}
```

Look at the `||`. A poor assembly can leave MAGScoT without a viable bin, and it reports this by failing. The rule treats this as a result and not as a problem, and it continues with empty tables, so that one bad assembly doesn't stop a whole project. **This also hides real failures of MAGScoT**, so if an assembly has no bins, read this log.

Option	Default	Meaning
`assemble: magscot: threshold`	`"0.5"`	Minimum completeness of a bin, as a fraction
`assemble: magscot: extra`	`"--n_iter=1"`	Further options. The comment in `params.yaml` says that this value could be a relict from testing, so check it for a real analysis

MAGScoT's script comes with the pipeline, in `workflow/scripts/MAGScoT/`.

63.7.6. assemble__magscot__label_contigs

This rule writes the refined bin into each contig name, and it keeps only the contigs in a bin:

```
ERR2019410:bin_NA@contig_00000001      before
ERR2019410:bin_00012@contig_00000001   after
```

Contigs without a bin are dropped here, and they don't go on to the catalogue. The target of the whole section is `assemble__magscot`.

63.8. Dereplicating: dRep

63.8.1. assemble__drep__split_bins

dRep wants one file per genome. This rule splits the contigs of all assemblies by their bin, with workflow/scripts/split_bins.sh. Since the assembly is part of the name, bins from different assemblies don't get mixed up.

63.8.2. assemble__drep__run

```
rm -rf {DREP}/{{data,data_tables,dereplicated_genomes,figures,log}}
dRep dereplicate {DREP} -g {input}/*.fa -p {threads} \
    -comp {params.settings[completeness]} -con
     ↪  {params.settings[contamination]} \
    -pa {params.settings[P_ani]} -sa {params.settings[S_ani]} -nc
     ↪  {params.settings[nc]} \
    {params.settings[extra]}
```

The rm removes what a failed attempt may have left, because dRep won't start in such a folder. Chapter 44 explains the method.

After dRep, the rule checks that there are genomes, and it stops with a message if there aren't. This is needed because dRep can report that CheckM failed, and still exit with 0.

Option	Default	Meaning
assemble: drep: completeness	50	Minimum completeness of a genome, in percent
assemble: drep: contamination	50	Maximum contamination, in percent
assemble: drep: P_ani	0.9	Primary clustering threshold, using a fast estimate
assemble: drep: S_ani	0.95	Secondary clustering threshold. 95% ANI is the usual definition of a species
assemble: drep: nc	0.1	Minimum share of the genome that two genomes must have in common
assemble: drep: extra	" "	Further options

With the default of S_ani, the catalogue holds about one genome per species. For strain-level questions, you would raise it to 0.99.

Note also the permissive limits for completeness and contamination: a genome with 50% contamination is a poor one. Look at the results of CheckM2 in Chapter 65 before using a genome.

63.8.3. assemble__drep__catalogue

This rule concatenates all genomes into one file, dereplicated_genomes.fa.gz, the MAG catalogue. The target is assemble__drep.

63.9. What you get

Path in results/assemble/	Content
{assembler}/{assembly_id}.fa.gz	The contigs
{assembler}/{assembly_id}.tar.gz	The working folder of the assembler
bowtie2/{assembly_id}.{sample}.{library}.cram	The mapping
concoct/, maxbin2/, metabat2/	The bins of each binner
magscot/{assembly_id}/magscot.refined.out	The refined bins with their scores
magscot/{assembly_id}.fa.gz	The contigs in refined bins
drep/dereplicated_genomes/	One FASTA per genome
drep/dereplicated_genomes.fa.gz	The MAG catalogue

63.10. What to look at after a run

1. **Assembly:** how many contigs over 1500 bases, and how long are they? Chapter 47 explains the numbers.
2. **Mapping rate:** how many clean reads map back to their assembly? If only a small share does, much of the community isn't assembled.
3. **MAGScoT logs:** did every assembly give bins?
4. **dRep:** how many genomes came in, and how many are left?

63.11. Summary

- Contigs are made by one assembler, and reads are mapped back to them.
- Three binners sort the contigs into bins, and MAGScoT makes the best set out of them.

- dRep reduces the bins of all assemblies to one catalogue, with about one genome per species.
- The contig names carry information, and they must keep their format.
- A failure of MAGScoT is treated as “no bins”, so check its log if bins are missing.

64. Module `quantify`

Now that there is a catalogue of genomes, we can ask how much of each genome is in each sample. The module maps the clean reads of every library against the catalogue, and it counts. The result is the table you will most likely use for your statistics: one row per MAG and one column per sample.

```
bash run_Snakebite-Holoruminant-MetaG.sh quantify
```

64.1. Why map to a catalogue

You could count from the assembly-specific mappings of the module `assemble`, but these are not comparable. In this module, all libraries of all samples are mapped to the same reference, the catalogue. So a value for a genome means the same thing in every sample, including samples that weren't part of the assembly of that genome.

There's also a limit to keep in mind: only what is in the catalogue can be counted. Reads of organisms without a MAG don't appear here. The share of reads that do map tells you how much of the community your catalogue represents. In a complex rumen sample, it's common that a good part of the reads doesn't map.

64.2. The flow

```
results/assemble/drep/dereplicated_genomes.fa.gz
          quantify__bowtie2__build

results/quantify/index/dereplicated_genomes
          quantify__bowtie2__map, with the clean reads of each library

results/quantify/bowtie2/{sample_id}.{library_id}.cram
          quantify__coverm__genome__run, quantify__coverm__contig_one

results/quantify/coverm/{genome,contig}/{method}/{sample_id}.{library_id}.tsv
          quantify__coverm__genome_aggregate, quantify__coverm__contig_aggregate
```

`results/quantify/coverm/{genome,contig}.{method}.tsv`

64.3. Mapping

64.3.1. quantify__bowtie2__build

This rule indexes the catalogue once.

64.3.2. quantify__bowtie2__map

```
bowtie2 --threads {threads} --rg-id '{params.rg_id}' --rg
 ↪  '{params.rg_extra}' \
       -x {input.index} -1 {input.forward_} -2 {input.reverse_} \
    | samtools sort -@ {threads} -m {params.sort_memory} -l 9 -M
     ↪  --reference {input.catalogue} \
       -o {output} -
```

This is the same pattern as in the module `preprocess`, with the same explanation of the options. The target `quantify__bowtie2` asks for one CRAM per library. Since this file is the basis of everything else in this module, it isn't temporary.

Option	Default	Meaning
`quantify: samtools: mem`	`1G`	Memory for `samtools sort`, per thread

`quantify__samtools__stats_cram` writes `samtools stats` of each mapping, and `quantify__samtools` asks for these files and for `flagstats`. **Look at the mapping rate first**, as this is the share of your reads that the catalogue can explain.

64.4. Counting

Chapter 45 explains the tool and its measures. The rules run once per library and per method.

64.4.1. quantify__coverm__genome__run

```
samtools view -1 -F 4 -T {input.catalogue} {input.alignment} \
    | coverm genome -b /dev/stdin -m {wildcards.method} -s
    ↪ {params.settings[separator]} \
       --min-covered-fraction
        ↪ {params.settings[min_covered_fraction]} \
    > {output}
```

`samtools view -F 4` passes on the mapped reads as BAM, since CoverM can't read CRAM. `-s @` is why contig names have their special format: CoverM takes everything before the `@` as the name of the genome, so `ERR2019410:bin_00012@contig_00000001` counts for the genome `ERR2019410:bin_00012`. This way the catalogue doesn't need a separate list of which contig belongs to which genome.

64.4.2. quantify__coverm__genome_aggregate

This rule joins the tables of all libraries into one, using `workflow/scripts/coverm_wide_table.R`, with the MAGs as rows and the libraries as columns. The target for all methods is `quantify__coverm__genome`.

64.4.3. quantify__coverm__contig_one

```
samtools view -1 -F 4 -T {input.catalogue} {input.alignment} \
    | coverm contig -b /dev/stdin -m {wildcards.method}
    ↪ --proper-pairs-only \
    > {output}
```

The same on the level of contigs. Note `--proper-pairs-only`, which counts only reads whose mate maps as expected. The genome-level rule doesn't use this option, so the two levels don't add up exactly.

64.4.4. quantify__coverm__contig_aggregate and quantify__coverm__contig

These do the same as on the genome level. `quantify__coverm` asks for both levels.

64.5. The options

Option	Default	Meaning
quantify: coverm: genome: methods	count, covered_bases, length, tpm	Which measures to compute, each gives its own table
quantify: coverm: genome: min_covered_fraction	0.0	Report a genome only if this share of it is covered
quantify: coverm: genome: separator	@	What separates genome and contig in a name
quantify: coverm: contig: methods	count, covered_bases, length, tpm	The same for contigs

About min_covered_fraction: with 0.0, a genome counts as present with a single read. CoverM's own default is higher, because a few reads can map to a genome that isn't really there, for example through a conserved gene. With this setting, you have to do this filtering yourself. The table covered_bases divided by length gives you the share of each genome that is covered, and this is the basis for it.

64.6. What you get

File in results/quantify/coverm/	Content	Use it for
genome.count.tsv	Reads per MAG and library	Statistics for differential abundance
genome.tpm.tsv	Reads normalized for genome length and depth	Comparing MAGs within a sample
genome.covered_bases.tsv	Covered bases	Filtering, see above
genome.length.tsv	Length of each MAG	Normalizing
contig.*.tsv	The same per contig	Details within a genome

Note that a value of TPM is relative: the sum over a sample is fixed, and it covers the mapped reads only. Comparing it between samples tells you about the shares, not the absolute amount.

64.7. Summary

- All libraries are mapped to the same catalogue, so the values are comparable between samples.
- Only organisms with a MAG are counted, so check the share of mapped reads.
- `genome.count.tsv` and `genome.tpm.tsv` are the main results.
- With the default `min_covered_fraction`, you have to filter weakly covered genomes yourself.

65. Module `mag_annotate`

The catalogue from Chapter 63 is a set of DNA sequences. This module finds out what they are: how good each genome is, which organism it comes from, and what it can do.

```
bash run_Snakebite-Holoruminant-MetaG.sh mag_annotate

# Or a single tool:
bash run_Snakebite-Holoruminant-MetaG.sh mag_annotate__checkm2
```

65.1. The tools at a glance

Target	Tool	Answers	Works on
`mag_annotate__quast`	Chapter 47	How long and how fragmented are the sequences?	Catalogue
`mag_annotate__checkm2`	Chapter 48	How complete and how contaminated is each genome?	Each genome
`mag_annotate__gtdbtk`	Chapter 49	Which organism is it?	Each genome
`mag_annotate__phylophlan`	Chapter 50	Which known species-level group does it belong to?	Each genome
`mag_annotate__dram`	Chapter 51	What can the genomes do, in metabolism?	Catalogue
`mag_annotate__dram_bins`	Chapter 51	The same per assembly	Binned contigs
`mag_annotate__camper`	Chapter 52	Which polyphenol-related genes?	DRAM's genes
`mag_annotate__bakta`	Chapter 53	Standardized gene annotation	Catalogue

Target	Tool	Answers	Works on
mag_annotate__bakta__mags	Chapter 53	The same per assembly	Binned contigs
mag_annotate__eggnog	Chapter 54	Which orthologous groups do the genes belong to?	Catalogue
mag_annotate__proteinortho	Chapter 55	Which genes are shared between assemblies?	Bakta's proteins

Notice the order they depend on each other. DRAM reads the taxonomy from GTDB-Tk, CAMPER reads DRAM's genes, and Proteinortho reads Bakta's proteins. Snakemake takes care of this if you call a single tool.

All rules of this module share the path `results/mag_annotate/<tool>/`.

Start with CheckM2. Everything else says something about the genomes, but only CheckM2 tells you which genomes to trust.

65.2. Quality

65.2.1. mag_annotate__quast

```
quast -o {output} -t {threads} {input}
```

Note that the input is the whole catalogue as one file. So this gives numbers for the catalogue as a whole, and not per genome.

65.2.2. mag_annotate__checkm2__predict

```
checkm2 predict -i {input.genomes} -x .fa.gz -o "$workdir" -t
 ↪  {threads} \
    --database_path {input.reference_data} --remove_intermediates
mv "$workdir/quality_report.tsv" {output}
```

This rule writes `quality_report.tsv`, with one line per genome. CheckM2 wants to make its own output folder, so the rule works in a temporary folder and moves the result. `mag_annotate__checkm2` is the target.

65.3. Taxonomy

65.3.1. mag_annotate__gtdbtk__classify

```
GTDBTK_DATA_PATH={input.reference_data} gtdbtk classify_wf \
    --genome_dir {input.genomes} -x fa.gz --out_dir {GTDBTK}
     ↪  --cpus {threads} --skip_ani_screen
```

GTDB-Tk needs a lot of memory, which is why this rule has the resource set `large`. Note the three points in the rule:

- `--skip_ani_screen` skips the first comparison against all reference genomes, which needs an additional database.
- GTDB-Tk treats Bacteria and Archaea separately, so it writes two tables. Only if there are archaea does the second exist. The rule joins them into `gtdbtk.summary.tsv`, or it copies the bacterial table if there's only one.
- The working folders `align`, `classify` and `identify` are packed into archives.

If a MAG can't be placed, GTDB-Tk reports it in the warnings. `mag_annotate__gtdbtk` is the target.

65.3.2. mag_annotate__phylophlan

```
phylophlan_assign_sgbs -i {input} \
                       -d SGB.Jun23 \
                       --database_folder
                        ↪  resources/databases/phylophlan/ \
                       -e fa.gz \
                       -o {params.result} \
                       --nproc {threads} \
                       --verbose \
                       --overwrite
```

This assigns each MAG to a species-level genome bin. Note what the comment in the rule says: the database name and folder are written into the rule, and don't come from `features.yaml`. The entries `amphora2` and `phylophlan` there aren't used by this rule. If you keep this database elsewhere, change the rule.

65.4. Function

65.4.1. DRAM on the catalogue

DRAM works in two steps, and between them one small repair.

`mag_annotate__dram__annotate` predicts genes and compares them with DRAM's databases:

```
DRAM.py annotate -i {input.catalogue} -o {_DRAM_TABLES} --threads
 ↪  {threads} \
    --gtdb_taxonomy {input.taxonomy} --config_loc
     ↪  {params.dram_config}
```

`--config_loc` points to `config/dram_config.txt`, which holds the paths of DRAM's databases. **Check these paths in your project**, as they are relative to the project folder.

DRAM skips a table if it finds nothing of that kind, for example no tRNAs. The rule then writes an empty file, as the next rules expect all three.

Like several tools in this module, DRAM starts other programs that use `$TMPDIR`, and on the cluster the inherited value points to a place that the container can't see. So the rule sets its own. The comments in the rules of Bakta and DRAM explain this.

`mag_annotate__fix_dram_annotations_scaffold` then repairs the annotation table with `workflow/scripts/fix_annotations.py`, so that the next step can read it.

`mag_annotate__dram__distill` summarizes the annotation, and writes `genome_stats.tsv`, `metabolism_summary.xlsx`, `product.html` and `product.tsv`. DRAM insists on making its own folder, so the rule works in a temporary one again. `mag_annotate__dram` is the target.

65.4.2. DRAM per assembly

The same three steps, but on the binned contigs of each assembly and without the taxonomy: `mag_annotate__dram_mag__annotate`, `mag_annotate__fix_dram_mag_annotations_scaffold` and `mag_annotate__dram_mag__distill`. `mag_annotate__dram_mags` is the target. The results are in `results/mag_annotate/dram_mags/{assembly_id}/`.

Why both? The catalogue holds one genome per species over all samples. The per-assembly view keeps what each assembly found, also for genomes that were dropped by dRep.

65.4.3. mag_annotate__camper__annotate

CAMPER looks for genes of the metabolism of polyphenols. It uses the genes from DRAM. The rule first copies the database to fast storage, and runs two steps:

```
camper_annotate -i {input.genes} -o $tmpdir --threads {threads} \
                --camper_fa_db_loc $tmp_db_dir/CAMPER_blast.faa \
                --camper_fa_db_cutoffs_loc
                 ↪  $tmp_db_dir/CAMPER_blast_scores.tsv \
                --camper_hmm_loc $tmp_db_dir/CAMPER.hmm \
                --camper_hmm_cutoffs_loc
                 ↪  $tmp_db_dir/CAMPER_hmm_scores.tsv
camper_distill  -a $tmpdir/annotations.tsv -o {output.distill} \
                --camper_distillate
                 ↪  $tmp_db_dir/CAMPER_distillate.tsv
```

`mag_annotate__camper` is the target.

65.4.4. Bakta

`mag_annotate__bakta` annotates the catalogue, which it sees as one genome:

```
bakta --db {params.db} --force --verbose \
      --output {params.out_dir} --prefix bakta --threads {threads}
       ↪  \
      {params.options} {input.contigs}
```

Option	Default	Meaning
`mag_annotate: bakta: additional_options`	`--skip-crispr --skip-plot`	Skip the search for CRISPR arrays and the circular plot

`mag_annotate__bakta_mags__run` does the same for the binned contigs of each assembly. Its docstring calls these dereplicated MAGs, but its input is the output of MAGScoT, so they aren't dereplicated. `mag_annotate__bakta_mags` is the target.

65.4.5. mag_annotate__eggnog

```
emapper.py -m diamond --data_dir {params.tmp_dir} \
           --itype metagenome --genepred prodigal \
           --dbmem --no_annot --no_file_comments \
           --cpu {threads} -i {input.contigs} \
           --output_dir {params.out_dir} -o {params.prefix}
```

The rule first copies the database to `tmp_storage`. `--dbmem` loads eggNOG's annotation database into memory, and `--itype metagenome --genepred prodigal` makes eggNOG-mapper predict the genes itself.

Notice `--no_annot`: this option stops after the search for orthologs, so the output doesn't hold the functional annotation yet. Chapter 54 explains the two steps, and how to run the second.

The log of this rule is marked `protected()` and lies inside the output folder. So to run it again, you have to remove that folder, and you may need to do this with `rm -rf` as the file is read-only.

65.4.6. mag_annotate__proteinortho

```
proteinortho {input.faa} -cpus={threads} -project=proteinortho
mv proteinortho.* {params.outdir}
```

This compares the proteins of all assemblies from Bakta. It needs at least two assemblies, so with only one assembly this rule can't work, and you should run single tools instead of the whole module. Note that it writes to the current folder, which is why the results are moved.

65.5. What to look at after a run

1. **CheckM2:** how many genomes reach at least 90% completeness and at most 5% contamination? These are the usual marks of a high-quality draft, and at least 50% and at most 10% those of a medium-quality one.
2. **GTDB-Tk:** which groups are there, and do they fit the habitat? Are some classified only up to the family or the genus? These may be new species.
3. **DRAM:** `product.html` gives you the overview of what your genomes can do.

65.6. Summary

- Run CheckM2 first, and use it to decide which genomes to trust.
- GTDB-Tk and PhyloPhlAn give the taxonomy, DRAM, CAMPER, Bakta and eggNOG-mapper the function.
- Many rules work in a temporary folder, because the tools insist on creating their own.
- Some rules run on the catalogue, and others per assembly.
- `mag_annotate__proteinortho` needs at least two assemblies, and PhyloPhlAn's database path is written into its rule.

66. Module contig_annotate

The module `mag_annotate` annotates genomes. But binning loses a lot: most contigs of a complex community never end up in a bin, and they would be missing from any analysis that starts at the catalogue. This module looks at **all contigs of each assembly**. It predicts their genes, annotates them, and counts how many reads hit each gene. This gives you a view of the functions of the whole community, whichever organism they belong to.

```
bash run_Snakebite-Holoruminant-MetaG.sh contig_annotate

# Or a single tool:
bash run_Snakebite-Holoruminant-MetaG.sh contig_annotate__eggnog
```

66.1. The flow

```
results/assemble/{assembler}/{assembly_id}.fa.gz
          contig_annotate__prodigal__run

results/contig_annotate/prodigal/{assembly_id}/{assembly_id}.prodigal.fa   (proteins)
results/contig_annotate/prodigal/{assembly_id}/{assembly_id}.prodigal.gtf  (positions)

          contig_annotate__featurecounts__run      counts per gene
          contig_annotate__eggnog                  orthologs and functions
          contig_annotate__eggnog7__run            eggNOG version 7
          contig_annotate__camper__annotate        polyphenol genes
          contig_annotate__diamond__assign         a chosen protein database
          contig_annotate__hmmer__assign           chosen protein profiles
          contig_annotate__ncyc__run               nitrogen cycle, only in contig_annota
```

All results are per assembly, as this module has no catalogue. The target `contig_annotate` asks for all of them, except nCycDB.

66.2. Finding the genes

66.2.1. contig_annotate__prodigal__run

```
prodigal -i <(gunzip -c {input.assembly}) \
         -o {output.gtf} \
         -a {output.fa} \
         -p meta -f gff
grep -v '^#' {output.gtf} > {output.gtfplain}
```

Chapter 41 explains the tool. The rule writes proteins to the `.fa` file and positions to the `.gtf` file. Despite the name, that file is in GFF format, as `-f gff` says. The `_plain` version lacks the comment lines. `contig_annotate__prodigal` is the target.

66.2.2. contig_annotate__cut_prodigal

An assembly can contain many millions of genes, and the search with eggNOG-mapper is slow. So this step cuts the proteins into chunks, which run as separate jobs, using `workflow/scripts/cutProdigal.sh`.

Option	Default	Meaning
`contig_annotate: prodigal: split`	1000000	Size of a chunk, in proteins

You can't know how many chunks there will be before the step has run. This is why it's a **checkpoint**, the only one in the pipeline, as Chapter 16 explains. The functions `contig_annotate_aggregate_annotations_eggnog_search` and `contig_annotate_aggregate_assembly_eggnog_search` read the list of chunks after the checkpoint has finished.

66.3. eggNOG-mapper

These rules do what Chapter 54 describes, in its two steps. Each chunk has its own job in each step.

66.3.1. contig_annotate__eggnog_find_homology

```
emapper.py -m diamond --data_dir $DATA_DIR --override --no_annot
 ↪  --no_file_comments --cpu {threads} -i {input.files} ...
```

This is the search with DIAMOND. It only needs the fast DIAMOND database, and this is copied to `nvme_storage` first, if that's configured. `--no_annot` stops the program after the search, and leaves a `.emapper.seed_orthologs` file.

66.3.2. contig_annotate__eggnog_orthology_chunk

```
emapper.py --data_dir $DATA_DIR \
           --annotate_hits_table {input.seed} \
           --no_file_comments \
           -o {params.out} --output_dir {params.outdir} \
           --override --cpu {threads}
```

This step turns the hits into annotations. It reads a big database all the time, so it should be on fast storage, and many jobs may share one copy on a node. This explains why this rule is so long: it has to make sure that exactly one job copies the database, that the others wait, and that the last job deletes it.

- A **counter** file says how many jobs are using the copy. `flock` protects it, so two jobs can't change it at the same moment.
- A **lock** folder says who does the copying. `mkdir` either creates a folder or fails, so only one job can succeed.
- A **done** file tells the waiting jobs that the copy is finished.

If a job gets killed, the counter stays too high, and the copy in `/dev/shm` remains until the node cleans it up. With `copy_dbs: "False"`, all this is skipped.

66.3.3. contig_annotate__eggnog_merge_annotations

This rule concatenates the chunks into `eggnog_output.emapper.annotations`. `contig_annotate__eggnog` is the target.

66.3.4. contig_annotate__eggnog7__run

Version 7 of eggNOG has a new tool, which is called with `workflow/scripts/eggnog7_annotator.sh`. It searches a DIAMOND database and looks up the hits in a master table. Its path comes from the entry `eggnog7` in `features.yaml`. `contig_annotate__eggnog7` is the target.

66.4. Counting reads per gene

66.4.1. contig_annotate__cram_to_bam

featureCounts can't read CRAM, so with `out_type: cram` this rule makes a temporary BAM file. With `out_type: bam` the rule doesn't exist, as it's inside a Python `if`. This is a nice example of Chapter 9: the Snakefile decides which rules exist when it's read.

66.4.2. contig_annotate__featurecounts__run

```
featureCounts -p -T {threads} -F GFF3 \
              -a {input.gtf} -o {output.file} \
              -t CDS -g ID \
              {input.bam}
```

This counts the read pairs that fall on each gene, per library and assembly. Chapter 46 explains the options. Only libraries that belong to an assembly are counted against it. `contig_annotate__featurecounts` is the target.

66.5. Other annotations

66.5.1. contig_annotate__camper__annotate

The same tool as in Chapter 65, here on the proteins from Prodigal. `contig_annotate__camper` is the target.

66.5.2. contig_annotate__diamond__assign

This rule uses `diamond blastp` to compare proteins with each database in `diamond_contig_protein` in `features.yaml`, and it stages them in the same way as the rule for the reads, which Chapter 62 describes. `contig_annotate__diamond` is the target.

The rule `contig_annotate__diamond__summarise` exists, but the target doesn't ask for its output. So it only runs if you request its output files by name. It also needs an R script for your database, and at the moment there are scripts for `cazy`, `funcgenes` and `kegg`, but none for the default `hyddb`.

66.5.3. contig_annotate__hmmer__assign

```
hmmsearch -o {output.out} -A {output.aln} \
    --tblout {output.tblout} --domtblout {output.domtblout}
     ↪  --pfamtblout {output.pfam} \
    --acc --domT {params.domt} \
    {input.database} {input.fa}
```

This searches the proteins with the profiles in the entry `hmmer` of `features.yaml`. By default, these are the three classes of hydrogenases. `--domT` is a minimum score, and it has a value for each database:

Option	Default	Meaning
`contig_annotate: hmmer: domt: fefe`	15.9	Minimum score for FeFe hydrogenases
`contig_annotate: hmmer: domt: nife`	34.5	For NiFe hydrogenases
`contig_annotate: hmmer: domt: fe`	54.4	For Fe hydrogenases
`contig_annotate: hmmer: domt: fallback`	10.0	For any other database

If you add your own profiles, set a score for them that makes sense, as 10.0 is only a rough fallback. `contig_annotate__hmmer` is the target.

`--domT` only filters the domains. The table of sequences, `{assembly_id}.tblout.tsv`, can still hold hits below the threshold, as Chapter 70 shows on real data. So use `{assembly_id}.domains-tblout.tsv` for the result, or apply the threshold yourself.

66.5.4. nCycDB

`contig_annotate__ncyc__prepare_input` links the proteins into one folder and writes a sample sheet. `contig_annotate__ncyc__run` runs the Perl script `NCycProfiler.PL`. These are the same steps as for the reads, this time with `-s prot`. `contig_annotate__ncyc` is the target, and it's only part of `contig_annotate_extra`.

66.6. What you get

Path in `results/contig_annotate/`	Content
`prodigal/{assembly_id}/{assembly_id}.prodigal.fa`	The proteins
`prodigal/{assembly_id}/{assembly_id}.prodigal.gtf`	The genes
`featurecounts/{assembly_id}/{assembly_id}.{sample}.{library}.prodigal_fc.txt`	Read counts per gene
`eggnog/{assembly_id}/eggnog_output.emapper.annotations`	The eggNOG-mapper annotations
`eggnog7/{assembly_id}/{assembly_id}.eggnog.tsv.gz`	The same from eggNOG 7
`camper/{assembly_id}/annotations.tsv`	Polyphenol genes
`diamond/{diamond_db}/{assembly_id}.tsv`	DIAMOND hits
`hmmer/{hmmer_db}/{assembly_id}.tblout.txt`	HMMER hits

To get the amount of a function in a sample, you join two tables: the annotation of a gene from eggNOG, and the count of that gene from featureCounts.

66.7. Summary

- This module works on all contigs of each assembly, whether binned or not.
- Prodigal finds the genes, and featureCounts counts the reads on them.
- The proteins are cut into chunks for eggNOG-mapper, which is why the pipeline needs a checkpoint here.
- The chunks share a copy of the database on the node, and files with counters and locks keep them from getting in each other's way.

67. Module `eukaryote`

The binning of Chapter 63 scores bins with marker genes of Bacteria and Archaea. So eukaryotic bins don't come out of it: they would look like incomplete bins and be dropped. In soil or in samples with many fungi and protists, this can be a large part of the community. This module labels the contigs of each assembly as eukaryotic or prokaryotic, and it also bins the eukaryotic ones.

It isn't part of a complete run, so you have to call it yourself. It needs no database.

```
bash run_Snakebite-Holoruminant-MetaG.sh eukaryote

# Or a single tool:
bash run_Snakebite-Holoruminant-MetaG.sh eukaryote__tiara
```

67.1. Why three tools

Telling apart the contigs of different domains is hard, especially for short ones. The three tools use different information, so where they agree you can be more confident:

Target	Tool	Uses	Also does
`eukaryote__tiara`	Chapter 56	The k-mers of each contig	Finds organelles
`eukaryote__4cac`	Chapter 57	K-mers, then the assembly graph	Finds plasmids and phages
`eukaryote__remag`	Chapter 58	A deep language model of DNA	Bins the eukaryotic contigs

67.2. Tiara

67.2.1. eukaryote__tiara__run

```
tiara \
    --input {input.assembly} --output {output.tsv} \
    --min_len {params.min_len} --prob_cutoff {params.prob_cutoff} \
    --probabilities --threads {threads}
```

This writes one table per assembly, and a short count of the classes as `log_{assembly_id}.tsv`, which is the first thing to look at. `eukaryote__tiara` is the target.

Option	Default	Meaning
`eukaryote: tiara: min_len`	1000	Ignore shorter contigs. Tiara's default is 3000, and 1000 is its documented lower limit
`eukaryote: tiara: prob_cutoff`	`[0.65, 0.65]`	The cutoff for each of the two stages. This is Tiara's default

67.3. 4CAC

4CAC refines its result with the assembly graph, and only metaSPAdes keeps this. The constant `FOURCAC_SUPPORTED` checks the configuration, and with MEGAHIT or your own assemblies, `eukaryote__4cac` asks for nothing, so it doesn't run, while the other two tools still do.

67.3.1. eukaryote__4cac__extract_graph

The module `assemble` packed the working folder of metaSPAdes into an archive. This rule gets three files out of it. It extracts the contigs under the names of the scaffolds, as 4CAC expects these names. In this way, 4CAC classifies exactly the contigs that the rest of the pipeline uses, and the files are temporary.

67.3.2. eukaryote__4cac__run

```
python {params.install_dir}/classify_xgb.py --fasta {input.contigs}
 ↪  \
    --outfile {params.workdir}/scaffolds.fasta.probs_xgb_4class.out
     ↪  --num_processes {threads}
python {params.install_dir}/classify_4CAC.py --assembler metaSPAdes
 ↪  \
    --asmdir {params.workdir}/ --outdir {params.workdir}/ \
    --phage {params.phage} --plasmid {params.plasmid}
python {params.script_folder}/fourcac_to_contig_ids.py ...
```

There are three steps: a first classification, the refinement on the graph, and finally the translation of metaSPAdes's names to the names of the pipeline. The last step needs the numbering of `contig_ids.py`, as Chapter 63 explains, so these two scripts have to change together.

The result has one line per contig, with the class `phage`, `plasmid`, `prokarya`, `eukarya` or `uncertain`, and the four scores. `eukaryote__4cac` is the target.

Option	Default	Meaning
`eukaryote: 4cac: install_dir`	`/pkg/4CAC`	Where the scripts are inside the container
`eukaryote: 4cac: phage`	0.95	Score needed for a phage
`eukaryote: 4cac: plasmid`	0.95	Score needed for a plasmid

67.4. REMAG

67.4.1. eukaryote__remag__filter

```
rm -rf {params.out_dir}
remag --fasta {input.assembly} --output {params.out_dir} \
    --filter-only --min-contig-length {params.min_contig_length} \
    --hyenadna-batch-size {params.hyenadna_batch_size} --threads
     ↪  {threads}
```

This classifies the contigs. The rule then looks into REMAG's log for two problems:

```
if grep -q -E "HyenaDNA classifier not found|Failed to initialize
 ↪  HyenaDNA classifier|Classification error for bat..." ...
```

The reason is in the comment: if the classifier can't be loaded, or a batch fails, REMAG keeps those contigs as eukaryotic and only writes this to its log. A wrong result without an error is the worst kind, so the rule turns this into a failure.

67.4.2. eukaryote__remag__bin

This rule runs in the folder of the previous one, where REMAG finds its results and doesn't need to classify again. It first adds up the length of the contigs that were called eukaryotic:

```
if awk -F '\\t' -v min_bp={params.min_bin_size} '
        NR > 1 && $3 == "eukaryote" {{ bp += $2 }}
        END {{ printf "%d bp classified eukaryotic\\n", bp; exit
 ↪  !(bp < min_bp) }}
    ' {input.tsv}; then
```

If this is less than the smallest allowed bin, no bin is possible, and REMAG would stop with an error. So the rule skips the binning, and writes an empty `bins/` folder and a `bins.csv` with only a header. Otherwise, it runs REMAG with one `--coverage` per mapping of the assembly, which the function `compose_coverage_args_for_remag` builds. So this rule needs the mappings of the module `assemble`. `eukaryote__remag` is the target.

Option	Default	Meaning
`eukaryote: remag: min_contig_length`	1000	Ignore shorter contigs. It must be the same in both steps, or REMAG refuses to continue. Don't change it without starting over
`eukaryote: remag: min_bin_size`	500000	Smallest bin, in bases
`eukaryote: remag: hyenadna_batch_size`	1024	Batch size of the classifier. Raise it if you have a GPU
`eukaryote: remag: extra`	""	Further options, for example `--epochs 50`

67.5. Reading the results together

You get three tables that use the same contig names, so you can join them. A contig is much more likely to be really eukaryotic if all three agree.

Be careful with short contigs: all three classifiers are least reliable there. And note what is **not** in this module: REMAG's bins aren't part of the catalogue of Chapter 63, so they aren't quantified or annotated by the other modules. Check them with a tool for eukaryotes, such as EukCC or BUSCO, as CheckM2 doesn't apply to them.

67.6. Summary

- This module is optional, and useful in samples with a real share of eukaryotes.
- Tiara, 4CAC and REMAG each label contigs in a different way, and REMAG also bins.
- 4CAC needs metaSPAdes, and it is skipped otherwise.
- Bins from REMAG stay outside the catalogue, and they need their own quality check.

68. Modules `report` and `helpers`

The last two groups of rules are different from the others. They don't add a scientific step: `report` summarizes, and `helpers` does small jobs for the other modules.

68.1. The module `report`

> ⚠ Under development
>
> These reports aren't finished. They may not produce anything useful and they may crash. The rule `all` doesn't include them for this reason.

Each module has one report, an R Markdown file in `workflow/scripts/R/`, which is rendered to a PDF in the folder `Rreports/`:

```
bash run_Snakebite-Holoruminant-MetaG.sh report_preprocess
```

All these rules are built in the same way. For example, `report__assemble` asks for all results of the module `assemble`, and runs:

```
R -e "project_folder <- '{params.project_folder}'; \
      pipeline_folder <- '{params.pipeline_folder}'; \
      features_file <- '{params.features}'; \
      sample_file <- '{params.sample_file}'; \
      rmarkdown::render('{params.script}',
↪   output_format='pdf_document', \

↪   output_file=file.path('{params.project_folder}',
↪   '{output.pdf}'))"
```

Careful: since a report asks for all results of its module, calling it starts that module if it's not done yet.

Rule doing the work	Short target	Report on
`report__reads`	`report_reads`	Raw reads

Rule doing the work	Short target	Report on
report__reference	report_reference	Host genomes
report__preprocess	report_preprocess	Trimming and host removal
report__read_annotate	report_read_annotate	Read-based annotation
report__assemble	report_assemble	Assembly and binning
report__quantify	report_quantify	Quantification
report__mag_annotate	report_mag_annotate	Annotation of MAGs
report__contig_annotate	report_contig_annotate	Annotation of contigs

The target report asks for all of them. The container r_report is used only here, and for the R scripts of the DIAMOND summaries.

68.2. The module helpers

These are generic rules. Their output patterns have only one wildcard, {prefix}, and so they match any file with the right ending, wherever it is. A rule anywhere in the pipeline can ask for something.cram.crai, and helpers__samtools__index_cram will make it, as Chapter 6 explains. So they are never called directly, and there's no target for this module.

Rule	Makes	From	Runs
helpers__fastqc	{prefix}_fastqc.html, {prefix}_fastqc.zip	{prefix}.fq.gz	fastqc
helpers__samtools__index_bam	{prefix}.bam.bai	{prefix}.bam	samtools index
helpers__samtools__index_cram	{prefix}.cram.crai	{prefix}.cram	samtools index
helpers__samtools__faidx	{prefix}.fa.fai	{prefix}.fa	samtools faidx
helpers__samtools__faidx_fagz	{prefix}.fa.gz.fai, {prefix}.fa.gz.gzi	{prefix}.fa.gz	samtools faidx
helpers__samtools__idxstats	{prefix}.idxstats.tsv	{prefix}.cram	samtools idxstats
helpers__samtools__flagstats	{prefix}.flagstats.txt	{prefix}.cram	samtools flagstats

Having these in one place avoids writing the same rule many times. There's one thing to note. helpers__samtools__faidx_fagz only works on a file made with bgzip, not with plain gzip, and this is why host genomes get recompressed first.

The helper functions of Part I, get_resources, esc, esc_val and get_escalation_order, are in the same folder. Chapter 17 explains them.

68.3. The rule `test_error`

`workflow/rules/devel/throw_error.smk` has one rule, `test_error`, for testing the resource escalation by hand. It waits for a while and fails on the first attempt. You don't need it for an analysis, and no target asks for it. `tests/escalation/` has replaced it for regular checks.

69. Common problems

Something will fail. With this many tools, this isn't a sign that you did anything wrong. This chapter shows how to find the cause, and it lists the problems you're most likely to meet. Chapter 13 explains the Snakemake side in general.

69.1. How to find out what happened

Work through these from the outside in:

1. **Snakemake's output.** Look for the `Error in rule` block. It tells you the rule, the wildcards, the log files and the SLURM job number.
2. **The rule's log**, the file under `log:` in the block. This is what the tool itself said, and it's usually where the answer is. If the rule uses the `.try` pattern, look at `{log}.try1` as well, as a successful run renames this file, but a failed one doesn't.
3. **The job log**, in `.snakemake/slurm_logs/rule_<name>/<wildcards>/<jobid>.log`. This holds what Snakemake printed within the job. It's useful if the rule's log is empty.
4. **SLURM's view**, which tells you whether the job was killed:

```
sacct -j <jobid>
 ↪  --format=JobID,State,ExitCode,ReqMem,MaxRSS,Elapsed,Timelimit
```

A state of `OUT_OF_MEMORY` or `TIMEOUT` tells you what to do next, and in these cases the tool's log simply stops without any error message.

69.2. Quick table

What you see	Likely cause	What to do
`Invalid account or account/partition combination`	`slurm_account` in the profile, or a partition in `resource_sets`	See Chapter 59

What you see	Likely cause	What to do
`OUT_OF_MEMORY` in `sacct`, a log that just stops	Too little memory	Add a larger set to the rule in `escalation.yaml`
`TIMEOUT`	Too little time	Use a set with more time, such as `longrun`
A file "doesn't exist", but you can see it	The folder isn't bound into the container	Add it to `--singularity-args`
`No space left on device`	Temporary folder, `/dev/shm`, or quota	See below
`LockException`	A previous Snakemake died	`--unlock`, see below
`IncompleteFilesException`	A job was killed halfway	`--rerun-incomplete`
`MissingOutputException`	The file system is slow, or the tool wrote elsewhere	Check the log, raise `--latency-wait`
`MissingInputException`	A file isn't there and no rule makes it	Check the paths in `samples.tsv` and `features.yaml`
`Invalid sample sheet`	A library or a file is listed twice	The message tells you which
An error about the CRAM header	Long contig names	`out_type: bam` in `params.yaml`
`gzip: ... unexpected end of file`, or `Error occurs when reading inputs` from MEGAHIT	A truncated file, for example clean reads from a version before 0.4.69	Delete it and what's built on it, see `RELEASE_NOTES`
An assembly without bins	Too little data, or a real error	See below
`dRep wrote no genomes`, with `!!! checkM failed !!!` above it	CheckM, inside dRep, ran out of memory	Add a larger set to `assemble__drep__run`
`ValueError: The number of observations cannot be determined on an empty distance matrix` from dRep	There's only one genome	Nothing to dereplicate. See below
Many lines `[W::cram_encode_container] Failed to build reference, switching to non-ref mode`	`assemble__bowtie2__map` writes CRAM without a reference	Harmless: the file just stores the sequences itself

69.3. Memory

The most common failure. The values in `resource_sets` are starting points, and most rules have only one set, so a job that runs out of memory stops the pipeline rather than trying again with more.

To let a rule try again with more, extend its line in `config/escalation.yaml`:

```
assemble__metaspades__run: large, highmem
```

Check the memory settings given to the tools too: `samtools sort` gets a value **per thread**, in `params.yaml`, and if this multiplied by the threads is more than the job has, the job gets killed, however large the set.

Snakemake restarts the failed job with the next set when you start the pipeline again, or within the same run, if it's still going.

69.4. Time

Long jobs can hit the limit of a partition. On Puhti, `small` allows 3 days, and `longrun` allows 14. The sets `longrun` and `highmem_longrun` are for that. Rules like metaSPAdes on a big co-assembly, MaxBin2 and REMAG are the usual candidates.

69.5. Containers can't see a file

The tools run in containers, and they see only folders that are bound. The list is in the run script, under `--singularity-args`. If you link `resources` or `reads` to another place, that place has to be bound too. From the login node, the link looks fine, so this is confusing at first.

A related trap is `$TMPDIR`. The run script binds the project's `tmp/` to `/tmp` in the container, while SLURM often sets `$TMPDIR` to a path on the real `/tmp` of the node, and this doesn't exist in the container. Programs that use it fail, often deep inside another program. Several rules set their own `TMPDIR` for this reason, namely those of MAGScoT, Bakta, CAMPER and DRAM. The comments in these rules explain it. If you see such an error in another rule, this is the first thing to check.

69.6. Disk space

`No space left on device` can have several sources, and the error rarely says which:

Where	Check	Fix
Your quota in the project	`df`, or the quota command of your cluster	Remove old results
The home folder	Apptainer's and Snakemake's caches	Set `APPTAINER_CACHEDIR` and `XDG_CACHE_HOME` in the run script
The project's `tmp/`	`du -sh tmp`	Remove what killed jobs left behind
`/dev/shm` on the node	The log of the rule	Use the set `full_node`, or set `kraken2_shm` and the like to `"False"`
The local disk	The `nvme` value in the resource set	Use a set with more disk

69.7. Starting again after a problem

`LockException`. Snakemake locks the folder while it runs. If it gets killed, the lock stays. Check that no other run is active, and then:

```
bash run_Snakebite-Holoruminant-MetaG.sh --unlock
```

`IncompleteFilesException`. Some files may be broken because a job was killed. Add `--rerun-incomplete`, and Snakemake makes them again.

Jobs still running. If you stop Snakemake, the jobs already in SLURM carry on. Look at `squeue --me`, and cancel them with `scancel` if you want to start over.

Things that are rerun without need. Snakemake reruns a rule if its code, its parameters or its input changed. After an update of the pipeline, this can mean a lot of jobs. Look at the reason in a dry run, as Section 13.8 explains. If you know that the results are still valid, `--rerun-triggers mtime` limits the reasons to changed files.

69.8. Problems of single rules

69.8.1. Kraken2 or DIAMOND: "database too large"

The rules fail on purpose if the database fits neither in `/dev/shm` nor on the local disk, as Chapter 62 explains. Give the rule a set with more local disk, or set `copy_dbs: "False"`.

For Kraken2, the database then runs from the network disk, and this can get very slow. Also, look out for leftover copies in `/dev/shm` from killed jobs.

69.8.2. eggNOG-mapper: waiting forever

`contig_annotate__eggnog_orthology_chunk` waits while another job copies the database. If this other job was killed during the copy, the folder `$DATA_DIR/.lock` stays there and other jobs on that node wait until their time limit runs out. In the log, the last line is `Another job is copying the DB, waiting...`. Cancel the jobs, and let the node be cleaned up, or use `copy_dbs: "False"`.

69.8.3. MAGScoT or dRep: no bins

With little data, this can be a valid result. The rule of MAGScoT catches a failure and continues with empty tables, which also hides real errors. Read `results/assemble/magscot/{assembly_id}/magscot.log`. dRep can't run on no genomes, and it also can't run on a single one. With one genome, you could lower `threshold` under `assemble: magscot` in `params.yaml` to let more bins through, as Chapter 70 does for its small data, but judge those bins with care.

If all assemblies are affected, check the mapping rate and the Nonpareil coverage. These usually explain it.

69.8.4. DRAM: a confusing error in `distill`

A failure in the annotation step can surface later. If `DRAM.py distill` complains about an empty file, then look at the log of the annotation first, and in particular at tRNAscan-SE. See "Containers can't see a file" above.

69.8.5. `mag_annotate__proteinortho`

It needs at least two assemblies. Run the other tools one by one for a project with a single assembly.

69.8.6. `read_annotate__diamond__summarise`

It fails if there's no R script for your database's name, as Chapter 31 explains.

69.8.7. Pandas errors at the start

Some combinations of Python and Pandas fail in strange ways. `docs/02-Installation.md` suggests a version that works.

69.9. Failures that didn't exist before

Since version 0.4.68, a failing command stops its rule. Before that, from September 2026 on, a rule could ignore a failing command, and it only failed when its last command failed. Section 13.6 explains the background. So a rule that always "worked" can now stop with an error. That's the point, and the error was there before, only hidden. Read the log, and take the first error you find seriously.

69.10. Asking for help

If you open an issue on GitHub, please give the rule, its log, the output of `sacct` for the job, the version from `CHANGELOG`, and your changes to the configuration. Don't paste data or paths that you can't share.

69.11. Summary

- Read the rule's log first, then ask `sacct` whether the job was killed.
- Most failures are about memory, time, temporary space or a folder that the containers can't see.
- A failed run can be continued: `--unlock`, `--rerun-incomplete`, and start it again.
- A strict rule that fails is telling you about a real problem.

Part IV.

Part IV: Tutorial

70. Tutorial with real data

This part runs the pipeline on real data, and it shows what comes out and how to read it. You can follow it on your own cluster, using the scripts in `tests/real_run/`.

> **i** Where the output in this part comes from
>
> All the output here is real. It comes from the pipeline's own rules, with the real tools in their containers, on **1 million read pairs per sample**. That's about a sixtieth of the data, on a computer with only Docker and 8 GB of memory. It's enough to see every file and to read every table, but the biology is thin, as you'll see.
> The tools that need large databases couldn't run there, and neither could metaSPAdes. Section 70.13 lists them, and those sections will be added from a complete run on a cluster.

70.1. The data

We use three public cow rumen metagenomes of the Ruminomics project (ENA study `PRJEB21508`):

Run	Sample name	Read pairs	Bases
`ERR2019410`	UK54	60,740,071	12.3 Gb
`ERR2019412`	UK7	69,876,338	14.1 Gb
`ERR2019415`	UK8	77,030,215	15.6 Gb

They were sequenced with an Illumina HiSeq 2000, with 2 x 101 bases. The prepared archive at https://a3s.fi/Holoruminant-data/2025.11.25.reads.tar.gz holds the full files, and also subsets with the first 10 million pairs of each file. Remember that these are the first reads of the file, not a random sample.

Rumen samples are a good example, and a hard one. The community is very diverse, and many of its members aren't in the databases. So you'll see lots of unclassified reads, and a modest assembly. That's what real data look like.

70.2. Running it yourself

70.2.1. On a cluster

These are the steps. `tests/real_run/README.md` has all the details, and Chapter 59 explains the settings.

```
git clone
 ↪  https://github.com/fischuu/Snakebite-Holoruminant-MetaG.git
cd Snakebite-Holoruminant-MetaG
PIPELINE=$(pwd)

bash tests/real_run/setup_project.sh \
    --project /scratch/project_2009831/Snakebite_real_run \
    --account project_2009831 \
    --resources /scratch/project_2009831/RESOURCES/resources

cd /scratch/project_2009831/Snakebite_real_run
for stage in reads reference preprocess read_annotate assemble
 ↪  quantify contig_annotate mag_annotate eukaryote
 ↪  read_annotate_extra contig_annotate_extra; do
    bash $PIPELINE/tests/real_run/run_stage.sh "$stage" || break
done
```

70.2.2. With only Docker

`book/examples/real_tools/` holds the scripts for the small run of this chapter. A stand-in for Apptainer starts the commands with Docker, as its `README.md` explains. Compared with a real project, this run differs in these ways:

What	Here	In a real project
Reads	1 million pairs per sample	All of them, or the 10-million-pair subsets
Host genome	PhiX, a small phage genome	The genomes of your host
Assembler	MEGAHIT, as metaSPAdes needs more memory	metaSPAdes by default

What	Here	In a real project
Jobs	One at a time, without SLURM	On SLURM, in parallel
Images	Simple steps share an image, and QUAST and nCycDB ran in another image with the same tool	As in `.docker.yml`
MAGScoT threshold, dRep filter	Changed, as the section on dRep below explains	The defaults

The tools ran under emulation, so ignore the run times, which are several times longer than they'd be on a normal x86 computer.

70.2.3. The samples

```
$ cat config/samples.tsv | cut -f 1,2,3,7
sample_id   library_id  forward_filename    assembly_ids
ERR2019410  lib1    reads/ERR2019410_1.fastq.gz ERR2019410, coasm
ERR2019412  lib1    reads/ERR2019412_1.fastq.gz coasm
ERR2019415  lib1    reads/ERR2019415_1.fastq.gz coasm
```

So there are two assemblies: `ERR2019410`, from one sample, and `coasm`, a co-assembly of all three. Comparing them shows what a co-assembly brings.

70.3. Stage `reads`

Chapter 60 explains this stage. Here's the first read:

```
$ gzip -dc reads/ERR2019410_1.fastq.gz | head -n 4
@ERR2019410.1 HWI-1KL157:190:C7RW0ACXX:3:1101:1438:1968/1
NTTTTGCTTAAATGCCTTTTTTGTATGATATGTTTTACTCATTGTTTTATTTCAGTATTGTAATAATTGGGGTTTAGGTAAT
+
```

```
]
 ↪  #0<FFFFFFFFFFIIIIIIIIIIFIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIFFIIFIIIIIIFIIIIBFFFFFBFFF
```

The name tells you the flow cell and the position on it. The first base couldn't be called, so it's `N` with the lowest quality, `#`. The `I` stands for a quality of Q40.

The stage links the reads under fixed names, one per mate:

```
$ ls results/reads | head -n 2
ERR2019410.lib1.log
ERR2019410.lib1_1.fq.gz
```

FastQC is part of this stage too. Its Java runs very slowly under emulation, so its report isn't shown here.

70.4. Stage `reference`

Section 60.3 explains this stage. It recompresses each host genome. The generic rules then make the index files `.fai` and `.gzi`:

```
$ ls results/reference/hosts
benchmark
phix.fa.gz
phix.fa.gz.fai
phix.fa.gz.gzi
phix.fa.gz.log
phix.log
```

70.5. Stage `preprocess`

Chapter 61 explains this stage.

70.5.1. What fastp did

fastp writes its report to the log, and the same numbers are in the JSON file. The first part shows the quality before:

```
$ sed -n '/^Read1 before filtering/,/^$/p'
 ↪  results/preprocess/fastp/ERR2019410.lib1.log
Read1 before filtering:
total reads: 1000000
total bases: 101000000
Q20 bases: 100008538(99.0184%)
Q30 bases: 98147857(97.1761%)
```

And after:

```
$ sed -n '/^Read1 after filtering/,/^$/p'
 ↪  results/preprocess/fastp/ERR2019410.lib1.log
Read1 after filtering:
total reads: 935567
total bases: 94418098
Q20 bases: 93982480(99.5386%)
Q30 bases: 92470684(97.9375%)
```

The share of good bases went up, and the reads with poor quality are gone. Why they went is in the next part:

```
$ sed -n '/^Filtering result/,/^Insert size/p'
 ↪  results/preprocess/fastp/ERR2019410.lib1.log
Filtering result:
reads passed filter: 1871134
reads failed due to low quality: 125202
reads failed due to too many N: 2908
reads failed due to too short: 756
reads with adapter trimmed: 8757
bases trimmed due to adapters: 204920
reads with polyX in 3' end: 17840
bases trimmed in polyX tail: 387161

Duplication rate: 0.0513%

Insert size peak (evaluated by paired-end reads): 169
```

Here's how to read this:

- **93.6% of the reads passed.** Nearly all the rest failed on quality. That's a healthy result.
- **Few adapters.** 8,757 of 2 million reads had any, which fits the insert size of 169 bases: most fragments are longer than the reads.

- **Few duplicates.** 0.05%, which is what you expect in a very diverse community.

The other two samples look alike:

```
$ grep -E '^reads passed filter|^reads failed due to low'
 ↪  results/preprocess/fastp/ERR2019412.lib1.log
reads passed filter: 1900732
reads failed due to low quality: 96792
```

```
$ grep -E '^reads passed filter|^reads failed due to low'
 ↪  results/preprocess/fastp/ERR2019415.lib1.log
reads passed filter: 1910824
reads failed due to low quality: 83680
```

70.5.2. What host removal did

Bowtie2's summary is at the start of the log of each mapping:

```
$ cat results/preprocess/bowtie2/phix/ERR2019410.lib1.log | grep -v
 ↪  bam_sort
935567 reads; of these:
  935567 (100.00%) were paired; of these:
    933569 (99.79%) aligned concordantly 0 times
    1998 (0.21%) aligned concordantly exactly 1 time
    0 (0.00%) aligned concordantly >1 times
    ----
    933569 pairs aligned concordantly 0 times; of these:
      70 (0.01%) aligned discordantly 1 time
    ----
    933499 pairs aligned 0 times concordantly or discordantly; of
 ↪  these:
      1866998 mates make up the pairs; of these:
        1866947 (100.00%) aligned 0 times
        51 (0.00%) aligned exactly 1 time
        0 (0.00%) aligned >1 times
0.22% overall alignment rate
```

Read it from the top. 1,998 pairs mapped as expected, 70 pairs mapped in an odd way, and from the rest, 51 single mates mapped. In the other two samples, the rate was 0.20% and 0.10%. PhiX is added to Illumina runs as a control, so these are likely real PhiX

reads. In a real project this is where you would see how much cow DNA there is, and that can be a large share.

`flagstats` gives the same as a short list:

```
$ cat results/preprocess/bowtie2/phix/ERR2019410.lib1.flagstats.txt
 ↪  | head -n 8
1871134 + 0 in total (QC-passed reads + QC-failed reads)
1871134 + 0 primary
0 + 0 secondary
0 + 0 supplementary
0 + 0 duplicates
0 + 0 primary duplicates
4187 + 0 mapped (0.22% : N/A)
4187 + 0 primary mapped (0.22% : N/A)
```

And the counts per sequence of the host, here with a single sequence, and with `*` for unplaced reads:

```
$ cat
 ↪  results/preprocess/quantification/phix/ERR2019410.lib1.chr_alignment_counts.tsv
ENA|J02482|J02482.1 4187
* 0
```

70.5.3. The clean reads

These are the files that everything else reads:

```
$ ls results/preprocess/bowtie2/decontaminated_reads | grep -v -E
 ↪  'log|benchmark'
ERR2019410.lib1_1.fq.gz
ERR2019410.lib1_2.fq.gz
ERR2019412.lib1_1.fq.gz
ERR2019412.lib1_2.fq.gz
ERR2019415.lib1_1.fq.gz
ERR2019415.lib1_2.fq.gz
```

Check that they're complete. This is the check that this very run made necessary, as Section 70.12 tells:

```
$ gzip -t results/preprocess/bowtie2/decontaminated_reads/*.gz &&
 ↪  echo all complete
all complete
```

The number of lines, divided by 4, is the number of pairs:

```
$ gzip -dc
 ↪  results/preprocess/bowtie2/decontaminated_reads/ERR2019410.lib1_1.fq.gz
 ↪  | wc -l
 3733792
```

That's 933,448 pairs. Before host removal there were 935,567, so 2,119 pairs were removed. That's more than the 1,998 proper pairs, because a pair with only one mapped mate is removed too.

Note that the names got shorter on the way, as `samtools fastq` keeps only the name and adds `/1`:

```
$ gzip -dc
 ↪  results/preprocess/bowtie2/decontaminated_reads/ERR2019410.lib1_1.fq.gz
 ↪  | head -n 1
@ERR2019410.1/1
```

70.6. Stage `read_annotate`

Chapter 62 explains this stage. Most of it needs large databases, so here we can only look at two tools.

70.6.1. Nonpareil

```
$ cat results/read_annotate/nonpareil/nonpareil.tsv
sample_id   kappa   C   LR  modelR  LRstar  diversity
ERR2019410.lib1 0.13006 0.15496064380054214 94204505.60799976
 ↪  0.9991269048010931  17344018571.196674  20.112522154756906
ERR2019412.lib1 0.04249 0.05573011944627605 95719612.90200004   NA
 ↪  NA  0
ERR2019415.lib1 0.07556 0.09432453034231177 96251992.69799998   NA
 ↪  NA  0
```

This is a useful table, and it says a lot:

- **`C` is the coverage.** With 1 million pairs, we've seen about 16%, 6% and 9% of the community. So most of what's there hasn't been sequenced at all, and this explains everything that follows.
- **`LR` is the amount of sequence used**, about 95 Mb.
- **`LRstar` is the sequence you'd need for 95% coverage**: 17 Gb in the first sample. The full file has 12.3 Gb, so even all of it wouldn't quite reach this. Treat this as a rough figure, as it's based on very little data and on a small `-X`.
- **`NA` and a diversity of 0** mean that the coverage of the other two samples is too low for a model. It doesn't mean that they have no diversity.

Also note that the two samples differ: the same amount of sequencing covers `ERR2019410` much better, so it's less diverse or it has more dominant organisms. We'll see this again below.

70.6.2. nCycDB

This one needs only a small database. It's in `read_annotate_extra`.

```
$ cut -f 1-4 results/read_annotate/ncyc/ncyc_results.tsv | head -n
 ↪  6
#random sampling: 933448
Gene    ERR2019410.lib1_1   ERR2019410.lib1_2   ERR2019415.lib1_1
ansB    28  29  16
asnB    297 292 267
gdh_K00260  1   2   0
gdh_K00261  25  24  20
```

There's one row per gene family and one column per read file, not per library. All files were sampled to the same depth, as the first line says, so that the columns can be compared. `asnB` is an asparagine synthetase, which turns up in many organisms, so it's no surprise that it's the most frequent here.

70.7. Stage `assemble`

Chapter 63 explains this stage, step by step.

70.7.1. Assembly

MEGAHIT goes through the k-mer sizes in its list. Note that it goes all the way up to 141 with reads of 101 bases:

```
$ grep -h 'k list' results/assemble/megahit/coasm.log | cut -d ' '
 ↪  -f 4-
k list: 21,31,41,51,61,71,81,91,101,111,121,131,141
```

The summaries of the two assemblies:

```
$ grep -h 'contigs, total' results/assemble/megahit/ERR2019410.log
 ↪  | cut -d ' ' -f 4-
549 contigs, total 1370369 bp, min 1500 bp, max 9706 bp, avg 2496
 ↪  bp, N50 2415 bp
```

```
$ grep -h 'contigs, total' results/assemble/megahit/coasm.log | cut
 ↪  -d ' ' -f 4-
900 contigs, total 2480429 bp, min 1500 bp, max 24283 bp, avg 2756
 ↪  bp, N50 2858 bp
```

These are tiny assemblies, as you'd expect from a coverage of 5 to 16%. Only organisms that are abundant enough get contigs of at least 1500 bases. The co-assembly has more data, so it has more contigs, and the longest is much longer. On the full data, you can expect orders of magnitude more.

The contigs already have their pipeline names:

```
$ gzip -dc results/assemble/megahit/coasm.fa.gz | head -n 1
>coasm:bin_NA@contig_00000001
```

70.7.2. Mapping back

How many reads of each sample are in the co-assembly?

```
$ grep 'overall alignment'
 ↪  results/assemble/bowtie2/coasm.ERR2019410.lib1.log
6.51% overall alignment rate
```

```
$ grep 'overall alignment'
 ↪  results/assemble/bowtie2/coasm.ERR2019412.lib1.log
2.31% overall alignment rate
```

```
$ grep 'overall alignment'
 ↪  results/assemble/bowtie2/coasm.ERR2019415.lib1.log
1.34% overall alignment rate
```

So more than 90% of the reads don't end up in the assembly. Again, this fits the coverage from Nonpareil. It's also the first sign that the assembled organisms are more abundant in `ERR2019410`.

These logs also hold many lines like the following. They're harmless: this rule writes CRAM without giving a reference, so `samtools` stores the sequences in the file itself.

```
$ grep -m 1 'Failed to build reference'
 ↪  results/assemble/bowtie2/coasm.ERR2019410.lib1.log
[W::cram_encode_container] Failed to build reference, switching to
 ↪  non-ref mode
```

70.7.3. Binning

MetaBAT2 reports what it did at the end of its log:

```
$ tail -n 3 results/assemble/metabat2/ERR2019410.log
[00:00:02] 61.78% (846568 bases) of large (>=1500) and 0.00% (0
 ↪  bases) of small (<1500) contigs were binned.
2 bins (846568 bases in total) formed.
[00:00:02] Finished
```

```
$ tail -n 3 results/assemble/metabat2/coasm.log
[00:00:04] 50.61% (1254393 bases) of large (>=1500) and 0.00% (0
 ↪  bases) of small (<1500) contigs were binned.
1 bins (1254393 bases in total) formed.
[00:00:04] Finished
```

For the co-assembly, the other binners give these:

```
$ ls results/assemble/maxbin2/coasm results/assemble/metabat2/coasm
results/assemble/maxbin2/coasm:
maxbin2.001.fa.gz
maxbin2.002.fa.gz

results/assemble/metabat2/coasm:
```

```
bin.1.fa.gz
```

```
$ ls results/assemble/concoct/coasm | wc -l
      21
```

Notice how differently the binners behave on the same contigs: MetaBAT2 makes one bin, MaxBin2 two, and CONCOCT twenty-one. That's why a refinement step follows.

70.7.4. MAGScoT

MAGScoT needs the proteins, the marker genes and the bins:

```
$ wc -l results/assemble/magscot/coasm/prodigal.faa
 ↪  results/assemble/magscot/coasm/hmm.tblout
 ↪  results/assemble/magscot/coasm/contigs_to_bin.tsv | head -n 3
   16099 results/assemble/magscot/coasm/prodigal.faa
      84 results/assemble/magscot/coasm/hmm.tblout
    2161 results/assemble/magscot/coasm/contigs_to_bin.tsv
```

There are 84 marker gene hits, in the format of protein, marker and E-value:

```
$ head -n 3 results/assemble/magscot/coasm/hmm.tblout
coasm:bin_NA@contig_00000809_1  TIGR00029   2.7e-11
coasm:bin_NA@contig_00000758_1  TIGR00054   1.6e-63
coasm:bin_NA@contig_00000218_3  TIGR00059   1.6e-40
```

And here's the list of bins, with bin, contig and binner:

```
$ head -n 2 results/assemble/magscot/coasm/contigs_to_bin.tsv
bin_20.fa.gz    coasm:bin_NA@contig_00000002    concoct
bin_3.fa.gz coasm:bin_NA@contig_00000003    concoct
```

The scores of all the candidate bins are the most instructive table of this step:

```
$ cut -f 1,2,3,4,7,8,10,13,14
 ↪  results/assemble/magscot/coasm/magscot.scores.out
bin set uniqueSCGs  multipleSCGs    n_contigs   Completeness
 ↪  Contamination   score   bintool
bin_11.fa.gz    gtdb_rel207_bac120  69  2   554 0.575
 ↪  0.028985507246376812    0.5605072463768116  concoct
```

```
MAGScoT_1_1 gtdb_rel207_bac120  69  2   565 0.575
 ↪  0.028985507246376812    0.5605072463768116  MAGScoT_1
MAGScoT_1_2 gtdb_rel207_bac120  69  2   612 0.575
 ↪  0.028985507246376812    0.5605072463768116  MAGScoT_1
MAGScoT_1_3 gtdb_rel207_bac120  68  2   536 0.5666666666666667
 ↪  0.029411764705882353    0.5519607843137255  MAGScoT_1
bin_bin.1.fa.gz gtdb_rel207_bac120  67  2   409 0.5583333333333333
 ↪  0.029850746268656716    0.543407960199005   metabat2
bin_maxbin2.001.fa.gz   gtdb_rel207_bac120  52  1   483
 ↪  0.43333333333333335 0.019230769230769232    0.4237179487179487
 ↪  maxbin2
bin_3.fa.gz gtdb_rel207_ar53    1   0   171 0.018867924528301886
 ↪  0   0.018867924528301886    concoct
bin_5.fa.gz gtdb_rel207_bac120  1   0   7   0.008333333333333333
 ↪  0   0.008333333333333333    concoct
bin_maxbin2.002.fa.gz   gtdb_rel207_bac120  20  6   369
 ↪  0.16666666666666666 0.3 0.008333333333333331    maxbin2
```

Here's how to read the first line. Of the 120 bacterial marker genes, 69 occur once and 2 occur more than once, so the bin is 57.5% complete. The score is the completeness minus half the contamination: $0.575 - 0.5 \times 0.029 = 0.5605$.

You can see the logic at work here. All three binners found versions of the same organism, and the candidates in the upper lines share most of their markers. The `MAGScoT_1_` lines are the merged candidates that MAGScoT made itself. In this case, merging adds contigs but no new markers, so they don't beat the CONCOCT bin, which comes first. MaxBin2's second bin shows the opposite: it has 30% contamination, and this pulls its score down to almost nothing.

So one bin stands out, and it holds 554 of the 900 contigs. With the default threshold of 0.5, this is the result:

```
$ cut -f 1,2,3,4,7,8,10,13,14
 ↪  results/assemble/magscot/coasm/magscot.refined.out
bin set uniqueSCGs  multipleSCGs    n_contigs   Completeness
 ↪  Contamination   score   bintool
bin_11.fa.gz    gtdb_rel207_bac120  69  2   554 0.575
 ↪  0.028985507246376812    0.5605072463768116  concoct
```

```
results/assemble/magscot/coasm/magscot.log:Refinement lead to a
 ↪  total of 1  bins with a score >= 0.5
```

The assembly of the single sample doesn't reach the threshold with any bin.

The contigs now carry their bin in the name:

```
$ gzip -dc results/assemble/magscot/coasm.fa.gz | head -n 1
>coasm:bin_000001@contig_00000001
```

70.7.5. dRep, and two things that went wrong

Here the small data ran into two limits, and both are instructive.

The first attempt failed with this in its log:

```
Running checkM
!!! checkM failed !!!
See
 ↪ https://drep.readthedocs.io/en/latest/advanced_use.html#troubleshooting-checkm
 ↪ for help troubleshooting
dRep wrote no genomes. Its messages are above, and
 ↪ results/assemble/drep/log/logger.log has the details.
```

CheckM, which dRep uses for its quality filter, ran its step with `pplacer`, and that step needs far more than the 7 GB available here. It's no issue on a cluster, as `assemble__drep__run` asks for 160 GB there. The last line of the message comes from the pipeline's rule. dRep itself reports success at this point, as Chapter 44 explains.

So we skipped the filter for this small run, by adding `--ignoreGenomeQuality` under `assemble: drep: extra` in `params.yaml`. Then dRep failed in another way:

```
1 genomes were input to dRep
Running primary clustering
Running pair-wise MASH clustering
ValueError: The number of observations cannot be determined on an
 ↪ empty distance matrix.
```

With a single genome, it has nothing to compare. So, also for this small run only, we lowered `threshold` under `assemble: magscot` to 0.2, which also lets the single-sample assembly keep a bin:

```
$ cut -f 1,2,3,4,7,8,10,13,14
 ↪ results/assemble/magscot/ERR2019410/magscot.refined.out
bin set uniqueSCGs  multipleSCGs    n_contigs   Completeness
 ↪ Contamination   score   bintool
```

```
bin_0.fa.gz gtdb_rel207_bac120  51  2   450 0.425
 ↪  0.0392156862745098  0.4053921568627451  concoct
```

Don't do this in a real analysis without a good reason. A genome that's 42% complete is a poor genome. Here we only want to see the following steps work.

Now dRep has two genomes, one from each assembly:

```
$ ls results/assemble/drep/separated_bins
 ↪  results/assemble/drep/dereplicated_genomes | grep -v timestamp
results/assemble/drep/dereplicated_genomes:
coasm:bin_000001.fa.gz

results/assemble/drep/separated_bins:
ERR2019410:bin_000001.fa
coasm:bin_000001.fa
```

Two genomes went in, and one came out. They are the same species, and dRep kept the better one, the one from the co-assembly. That's exactly its job. The catalogue is this one genome:

```
$ gzip -dc results/assemble/drep/dereplicated_genomes.fa.gz | grep
 ↪  -c '>'
554
```

```
$ gzip -dc results/assemble/drep/dereplicated_genomes.fa.gz | grep
 ↪  -v '>' | tr -d '\n' | wc -c
 1647187
```

So 554 contigs and 1.65 Mb, from one organism that's abundant enough to be assembled from a few million reads.

70.8. Stage `quantify`

Chapter 64 explains this stage. All reads go to the catalogue:

```
$ grep 'overall alignment'
 ↪  results/quantify/bowtie2/ERR2019410.lib1.log
4.77% overall alignment rate
```

Now the tables. They have one row per genome, and one column per library:

```
$ cat results/quantify/coverm/genome.count.tsv
sequence_id ERR2019410.lib1 ERR2019412.lib1 ERR2019415.lib1
coasm:bin_000001    89007   23170   5028
```

The genome is about 18 times as abundant in the first sample as in the third. This is what we saw in the mapping rates, and it's what the coverage from Nonpareil hinted at. It's a real difference between these cows, or at least between these parts of the data.

```
$ cat results/quantify/coverm/genome.covered_bases.tsv
sequence_id ERR2019410.lib1 ERR2019412.lib1 ERR2019415.lib1
coasm:bin_000001    1630044 1035667 412130
```

```
$ cat results/quantify/coverm/genome.length.tsv
sequence_id ERR2019410.lib1 ERR2019412.lib1 ERR2019415.lib1
coasm:bin_000001    1647187 1647187 1647187
```

Dividing the two gives the share of the genome that's covered: 99%, 63% and 25%. That's the check that Chapter 64 recommends for whether a genome is really present. In all three samples, reads are spread over a good part of it, so it's there. The same 5,000 reads on one conserved gene would look quite different.

```
$ cat results/quantify/coverm/genome.tpm.tsv
sequence_id ERR2019410.lib1 ERR2019412.lib1 ERR2019415.lib1
coasm:bin_000001    1000000 1000000 1000000
```

Look at this table carefully, as it's a good warning. TPM says 1,000,000 in all three samples, while the counts differ by a factor of 18. The reason is that TPM is relative to what's mapped: with only one genome in the catalogue, it gets the whole share every time. **TPM can't tell you how much of a sample a genome is**, and only with many genomes does it say how they compare within a sample.

There's the same on the level of contigs:

```
$ head -n 3 results/quantify/coverm/contig.count.tsv
sequence_id ERR2019410.lib1 ERR2019412.lib1 ERR2019415.lib1
coasm:bin_000001@contig_00000001    78  10  2
coasm:bin_000001@contig_00000005    184 28  10
```

70.9. Stage `contig_annotate`

Chapter 66 explains this stage. It looks at all contigs of the assemblies, not only those in bins.

70.9.1. Genes

```
$ grep -c '>'
 ↪  results/contig_annotate/prodigal/coasm/coasm.prodigal.fa
2828
```

That's 2,828 proteins on 2.5 Mb, or about one gene per 880 bases, which is what you expect from Bacteria. Here are the first two, whose headers have a lot of information:

```
$ grep '>' results/contig_annotate/prodigal/coasm/coasm.prodigal.fa
 ↪  | head -n 2
>coasm:bin_NA@contig_00000001_1 # 3 # 2132 # 1 #
 ↪  ID=1_1;partial=11;start_type=Edge;rbs_motif=None;rbs_spacer=None;gc_cont=0.596
>coasm:bin_NA@contig_00000002_1 # 3 # 212 # -1 #
 ↪  ID=2_1;partial=10;start_type=ATG;rbs_motif=AGGTG;rbs_spacer=5bp;gc_cont=0.567
```

The first gene runs from base 3 to base 2132 of the first contig, on the forward strand. `partial=11` says that it's cut off at both ends, as it runs past both ends of the contig. That's very common with such short contigs.

70.9.2. Counting reads per gene

```
$ sed -n 2,4p
 ↪  results/contig_annotate/featurecounts/coasm/coasm_ERR2019410.lib1.prodigal_fc.txt
Geneid  Chr Start   End Strand  Length
 ↪  results/assemble/bowtie2/coasm.ERR2019410.lib1.bam
1_1 coasm:bin_NA@contig_00000001    3   2132    +   2130    52
2_1 coasm:bin_NA@contig_00000002    3   212 -   210 35
```

The last column is the number of read pairs on that gene. Note that the gene names are `1_1`, and so on. They count the contigs in the file, so they are only unique within one assembly. The summary of how reads were assigned:

```
$ grep -v -E '\s0$'
 ↪  results/contig_annotate/featurecounts/coasm/coasm_ERR2019410.lib1.prodigal_fc.txt.
Status  results/assemble/bowtie2/coasm.ERR2019410.lib1.bam
Assigned    54908
Unassigned_Unmapped 866638
Unassigned_NoFeatures   3772
Unassigned_Ambiguity    8130
```

Most fragments didn't map at all, as we know. Of those that mapped, most fell on a gene (54,908), and a few fell between genes or on two at once. This is typical of Bacteria, where genes cover most of the genome.

70.9.3. Hydrogenases

HMMER looks for three classes of hydrogenases. There are no hits for two of them, and two for the third:

```
$ grep -v -c '^#'
 ↪  results/contig_annotate/hmmer/fefe/coasm.tblout.tsv
0
```

```
$ grep -v '^#' results/contig_annotate/hmmer/nife/coasm.tblout.tsv
 ↪  | awk '{print $1, $3, $5, $6}'
coasm:bin_NA@contig_00000421_2 NiFe 0.0068 12.4
coasm:bin_NA@contig_00000178_2 NiFe 0.056 9.4
```

But look at the scores in the last column: 12.4 and 9.4. The rule uses `--domT 34.5` for this profile, and it only applies to the table of domains, which has nothing:

```
$ grep -v -c '^#'
 ↪  results/contig_annotate/hmmer/nife/coasm.domains-tblout.tsv
0
```

So these two weak hits don't pass the threshold. **Read the domain table for this rule**, not the `tblout` one, or apply the threshold yourself. With so few proteins, it's no surprise that there are no real hydrogenases here. You can expect them on the full data, as hydrogen metabolism is central in the rumen.

70.9.4. CAMPER

Of all genes, a few have a hit to the polyphenol database:

```
$ cut -f 1,3 results/contig_annotate/camper/coasm/annotations.tsv |
 ↪  awk -F '\t' 'NR > 1 && $2 != ""' | wc -l
      4
```

70.9.5. nCycDB on contigs

As on the reads, but with one column per assembly:

```
$ cut -f 1-3 results/contig_annotate/ncyc/ncyc_results.tsv | head
 ↪  -n 4
#random sampling: 1578
Gene    coasm.prodigal  ERR2019410.prodigal
asnB    0   0
gdh_K00262  0   1
```

70.10. Stage `mag_annotate`

Chapter 65 explains this stage. Nearly all of it needs databases. Only QUAST could run here:

```
$ cut -f 1,2 results/mag_annotate/quast/report.tsv | head -n 20
Assembly    dereplicated_genomes
# contigs (>= 0 bp) 554
# contigs (>= 1000 bp)  554
# contigs (>= 5000 bp)  56
# contigs (>= 10000 bp) 6
# contigs (>= 25000 bp) 0
# contigs (>= 50000 bp) 0
Total length (>= 0 bp)  1647187
Total length (>= 1000 bp)   1647187
Total length (>= 5000 bp)   401728
Total length (>= 10000 bp)  79168
Total length (>= 25000 bp)  0
Total length (>= 50000 bp)  0
# contigs   554
```

```
Largest contig  24283
Total length    1647187
GC (%)  56.88
N50 3208
N90 1726
auN 4161.0
```

An N50 of 3,208 bases says that the genome is in very small pieces. A good MAG from real sequencing depth has an N50 of tens of kilobases.

70.11. Stage `eukaryote`

Chapter 67 explains this stage. Of its tools, Tiara could run here:

```
$ cat results/eukaryote/tiara/log_coasm.tsv | head -n 7
First iteration statistics:
    archaea: 110
    bacteria: 723
    eukarya: 40
    prokarya: 22
    unknown: 5
```

So 40 of 900 contigs look eukaryotic. Rumen samples do contain protists and fungi, so that's plausible, but with contigs this short, don't lean on single labels. This is exactly the case for comparing with the other two tools of this module.

The table has one line per contig:

```
$ cut -f 1,2,3 results/eukaryote/tiara/coasm.tsv | head -n 3
sequence_id class_fst_stage class_snd_stage
coasm:bin_NA@contig_00000001    bacteria    n/a
coasm:bin_NA@contig_00000002    bacteria    n/a
```

70.12. What this run found

Before these real runs, the rewritten pipeline had only been tested with stand-ins for its tools. Its first contact with the real ones found these problems, all of which are fixed now:

What happened	Why stand-ins missed it
The clean reads were cut off, and MEGAHIT and Nonpareil failed on them with `unexpected end of file`	The stand-in tools finished at once, so nothing was still being written when the rule ended
`nonpareil.tsv` had no values at all when one library had too little coverage	Only the real R package returns this special form of table
dRep reported a failure of CheckM, but exited with 0	A stand-in doesn't do this

Reading the rules while writing this book found more, like the wrong columns for Krona. A dry run of each target found that `contig_annotate__featurecounts` couldn't start.

The lesson is general: **stand-ins test your plumbing, and only real tools test your assumptions about them.** Chapter 18 returns to this. The numbers of this run are now stored as a test, as Chapter 71 describes.

70.13. What's still waiting for the complete run

These parts need a large database, or more memory than this run had, so they have no real output in this book yet. When the complete run on a cluster is done, its archive from `tests/real_run/collect.py` will fill them in. Their tool chapters in Part II describe the output formats.

Stage	Tools	Why they couldn't run here
`reads`, `preprocess`	FastQC	Far too slow under emulation
`read_annotate`	Kraken2, Krona	Database of about 140 GB of memory
`read_annotate`	SingleM, MetaPhlAn	Crash under emulation, and large databases
`read_annotate`	sylph, HUMAnN, phyloFlash, DIAMOND with CAZy	Databases
`assemble`	metaSPAdes	Memory

Stage	Tools	Why they couldn't run here
`contig_annotate`	eggNOG-mapper, eggNOG 7, DIAMOND with HydDB	Databases
`mag_annotate`	CheckM2, GTDB-Tk, PhyloPhlAn, DRAM, Bakta, eggNOG-mapper, Proteinortho, CAMPER	Databases
`eukaryote`	4CAC	It needs the graph of metaSPAdes
`eukaryote`	REMAG	Too slow without a suitable processor

Run times and memory on real hardware will come from that run too, from `sacct`.

71. The tests in the repository

`tests/` holds checks to run when you change the pipeline, and `tests/README.md` lists them. Chapter 18 explains the different kinds of tests in general.

71.1. The escalation test

`tests/escalation/run_test.sh` needs only Docker. It runs the real `workflow/Snakefile` and rules on a real SLURM cluster in a container, with a stand-in `bgzip` that fails when a job has too little memory. It checks that a failing command stops its rule, that a rule with two resource sets gets a new and bigger SLURM job, that no retry happens inside a job, that each resource set gets one attempt, and that a rule with one resource set isn't retried. It takes about ten minutes, most of which is Snakemake waiting between its status checks.

71.2. The DIAMOND test

`tests/diamond_staging/run_test.sh` needs only Docker. It runs the real rules `read_annotate__diamond__assign` and `contig_annotate__diamond__assign` with a stand-in for `diamond`. It checks where the database comes from in each situation: without any fast storage, with one and with two resource sets, with `copy_dbs: "False"`, and with a copy in `/dev/shm` or on the local disk. On the rules before version 0.4.69, 8 of its checks fail.

71.3. The Krona test

`tests/krona/run_test.sh` uses the real Krona in its container with the pipeline's taxonomy, and it reads the options from the rule. It draws a small report written by hand, and checks that each organism has its reads. On the rule before version 0.4.69, all 4 of its checks fail.

71.4. The real results

`tests/real_tools/` stores the numbers from the run in Chapter 70 as a test. It has two parts.

`check_results.py <project>` reads 41 numbers from the results of a project and compares them with `expected.tsv`. These include the reads before and after fastp, the reads on the host, the clean reads, the contigs, bases and bins of each assembly, the genomes of dRep, the genes from Prodigal and the contigs classified by Tiara. It only takes a moment. A file that's missing or cut off counts as a failure.

`run_test.sh <subsets> <new project>` makes such a project from scratch and then runs the check. This takes hours under emulation.

Each number has a tolerance in `expected.tsv`. It's 0 where a tool gives the same result each time, 5% for the assemblies, and 50% for the number of bins.

When a difference is right, because you meant to change something, run `check_results.py <project> --write`, and say why in the commit. Look at the differences first, as this is the whole point of the test. When a number changes, ask why.

It runs on the small data of Chapter 70, so it can't say anything about the tools in Section 70.13.

71.5. The complete run

`tests/real_run/` isn't a test yet. It runs the whole pipeline on a real cluster with the example data, stage by stage, and packs a record of the run into one small archive. The expected results of the tools with large databases will come from this, in the same way as above.

A. Rule reference

This appendix is generated from the rule files by `book/tools/rule_reference.py`. Do not edit it by hand. Paths are shown as they are written in the rules, so they use the path constants of `workflow/rules/folders.smk`, which Section 14.5 explains.

Rules without their own entry in `config/escalation.yaml` use the default escalation (`small, medium, large`).

A.1. Module `reads`

A.1.1. `reads/__main__.smk`

`reads`

Symlink the raw reads and run FastQC on them

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.reads__link.input`
- input: `rules.reads__fastqc.input`

A.1.2. `reads/fastqc.smk`

`reads__fastqc`

Aggregate FastQC reports for every raw forward/reverse read file

Target rule: it runs nothing itself and only asks for these files.

- input: `[READS / f"{sample_id}.{library_id}_{end}_fastqc.zip" for sample_id, library_id in SAMPLE_LIBRARY for end in (1, 2)]`

A.1.3. `reads/link.smk`

`reads__link__run`

Symlink one sample's raw reads under a normalized {sample}.{library}_{1,2}.fq.gz name

- input: `forward_=get_forward`
- input: `reverse_=get_reverse`
- output: `forward_=READS / "{sample}.{library}_1.fq.gz"`
- output: `reverse_=READS / "{sample}.{library}_2.fq.gz"`
- log: `READS / "{sample}.{library}.log"`
- benchmark: `READS / "benchmark/{sample}.{library}.tsv"`
- container: `reads` (see `config/.docker.yml`)
- escalation: default

`reads__link`

Symlink every sample/library's raw reads listed in samples.tsv

Target rule: it runs nothing itself and only asks for these files.

- input: `[READS / f"{sample}.{library}_{end}.fq.gz" for sample, library in SAMPLE_LIBRARY for end in (1, 2)]`

A.2. Module `reference`

A.2.1. `reference/__main__.smk`

`reference`

Stage and recompress the reference host genomes

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.reference__hosts.input`

A.2.2. `reference/hosts.smk`

`reference__hosts__recompress`

Recompress one host genome with bgzip, which samtools faidx needs

- input: `lambda wildcards: features["hosts"][wildcards.genome]`
- output: `HOST_FASTA`
- log: `HOST_GENOMES / "{genome}.log"`

- benchmark: `HOST_GENOMES / "benchmark/{genome}.tsv"`
- container: `reference` (see `config/.docker.yml`)
- escalation: small

`reference__hosts`

Aggregate the recompressed genome for every configured host

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(HOST_FASTA, genome=HOST_NAMES)`

A.3. Module `preprocess`

A.3.1. `preprocess/__main__.smk`

`preprocess`

Trim reads with fastp, remove host reads with bowtie2, and evaluate with fastqc/samtools

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.preprocess__bowtie2.input`
- input: `rules.preprocess__fastp.input`
- input: `rules.preprocess__samtools.input`
- input: `rules.preprocess__fastqc.input`

A.3.2. `preprocess/bowtie2.smk`

`preprocess__bowtie2__build`

Index one host genome for read mapping

- input: `genome=HOST_FASTA`
- input: `fai=HOST_FAI`
- output: `index=touch(HOST_INDEX / "{genome}")`
- log: `HOST_INDEX / "{genome}.log"`
- benchmark: `HOST_INDEX / "benchmark/{genome}.tsv"`
- container: `bowtie2` (see `config/.docker.yml`)
- escalation: medium

`preprocess__bowtie2__map`

Map one library against one host genome, as one step of the decontamination chain

- input: forward_=reads_for_host_mapping(1)
- input: reverse_=reads_for_host_mapping(2)
- input: index=HOST_INDEX / "{genome}"
- input: genome=HOST_FASTA
- input: fai=HOST_FAI
- output: cram=temp(HOST_CRAM)
- output: counts=HOST_COUNTS / "{genome}" / f"{LIBRARY}.chr_alignment_counts.tsv"
- log: HOST_MAPPING / "{genome}" / f"{LIBRARY}.log"
- benchmark: HOST_MAPPING / "benchmark/{genome}" / f"{LIBRARY}.tsv"
- params: sort_memory=params["preprocess"]["bowtie2"]["samtools"]["mem_per_thread"]
- params: rg_id=compose_rg_id
- params: rg_extra=compose_rg_extra
- container: bowtie2 (see config/.docker.yml)
- escalation: medium

preprocess__bowtie2__extract_nonhost__run

Write the read pairs with neither mate on this host genome to FASTQ, for the next step of the chain

- input: cram=HOST_CRAM
- input: genome=HOST_FASTA
- input: fai=HOST_FAI
- output: forward_=temp(HOST_MAPPING / "non{genome}" / f"{LIBRARY}_1.fq.gz")
- output: reverse_=temp(HOST_MAPPING / "non{genome}" / f"{LIBRARY}_2.fq.gz")
- log: HOST_MAPPING / "non{genome}" / f"{LIBRARY}.log"
- benchmark: HOST_MAPPING / "benchmark/non{genome}" / f"{LIBRARY}.tsv"
- params: sort_tmpdir=lambda wc: f"./samtools_sort_tmp/{wc.sample_id}.{wc.library_id}"
- container: bowtie2 (see config/.docker.yml)
- escalation: small

preprocess__store_final_fastq

Copy the fully preprocessed reads (post-decontamination, or post-fastp if no hosts) to their final path

- input: forward_=reads_after_last_host(1)
- input: reverse_=reads_after_last_host(2)
- output: forward_=CLEAN_FORWARD
- output: reverse_=CLEAN_REVERSE
- log: CLEAN_READS / "log" / f"{LIBRARY}.log"
- benchmark: CLEAN_READS / "benchmark" / f"{LIBRARY}.tsv"
- container: bowtie2 (see config/.docker.yml)
- escalation: small

preprocess__bowtie2__extract_nonhost

Store the final decontaminated FASTQs for every sample/library

Target rule: it runs nothing itself and only asks for these files.

- input: `[CLEAN_READS / f"{sample_id}.{library_id}_{end}.fq.gz" for sample_id, library_id in SAMPLE_LIBRARY for end in (1, 2)]`

preprocess__bowtie2

Run the full host-decontamination chain (index, map, extract, store) for every host

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.preprocess__bowtie2__extract_nonhost.input`

A.3.3. `preprocess/fastp.smk`

preprocess__fastp__run

Run fastp on one library

- input: `forward_=READS / "{sample_id}.{library_id}_1.fq.gz"`
- input: `reverse_=READS / "{sample_id}.{library_id}_2.fq.gz"`
- output: `forward_=FASTP / "{sample_id}.{library_id}_1.fq.gz"`
- output: `reverse_=FASTP / "{sample_id}.{library_id}_2.fq.gz"`
- output: `unpaired1=FASTP / "{sample_id}.{library_id}_u1.fq.gz"`
- output: `unpaired2=FASTP / "{sample_id}.{library_id}_u2.fq.gz"`
- output: `html=FASTP / "{sample_id}.{library_id}_fastp.html"`
- output: `json=FASTP / "{sample_id}.{library_id}_fastp.json"`
- log: `FASTP / "{sample_id}.{library_id}.log"`
- benchmark: `FASTP / "benchmark/{sample_id}.{library_id}.tsv"`
- params: `adapter_forward=get_forward_adapter`
- params: `adapter_reverse=get_reverse_adapter`
- params: `extra=params["preprocess"]["fastp"]["extra"]`
- params: `length_required=params["preprocess"]["fastp"]["length_required"]`
- params: `work_prefix=lambda w: FASTP / f"{w.sample_id}.{w.library_id}_tmp"`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: medium

preprocess__fastp

Aggregate all fastp outputs

Target rule: it runs nothing itself and only asks for these files.

- input: reads=[FASTP / f"{sample_id}.{library_id}_{end}.fq.gz" for sample_id, library_id in SAMPLE_LIBRARY for end in "1 2 u1 u2".split()]
- input: json=[FASTP / f"{sample_id}.{library_id}_fastp.json" for sample_id, library_id in SAMPLE_LIBRARY]

A.3.4. preprocess/fastqc.smk

preprocess__fastqc__fastp

FastQC reports for the fastp-trimmed reads

Target rule: it runs nothing itself and only asks for these files.

- input: [FASTP / f"{sample_id}.{library_id}_{end}_fastqc.{ext}" for sample_id, library_id in SAMPLE_LIBRARY for end in (1, 2) for ext in ("html", "zip")]

preprocess__fastqc__nonhost

FastQC reports for the reads left over after each host-decontamination step

Target rule: it runs nothing itself and only asks for these files.

- input: [HOST_MAPPING / f"non{genome}" / f"{sample_id}.{library_id}_{end}_fastqc.{ext} for genome in HOST_NAMES for sample_id, library_id in SAMPLE_LIBRARY for end in (1, 2) for ext in ("html", "zip")]

preprocess__fastqc

Aggregate every FastQC report produced during preprocessing

Target rule: it runs nothing itself and only asks for these files.

- input: rules.preprocess__fastqc__fastp.input
- input: rules.preprocess__fastqc__nonhost.input

A.3.5. preprocess/samtools.smk

preprocess__samtools__stats_cram

Mapping statistics of one library on one host genome with samtools stats

- input: cram=HOST_CRAM
- input: crai=HOST_CRAI
- input: genome=HOST_FASTA
- input: fai=HOST_FAI

- output: `HOST_MAPPING / "{genome}" / f"{LIBRARY}.stats.txt"`
- log: `HOST_MAPPING / "{genome}" / f"{LIBRARY}.stats.log"`
- benchmark: `HOST_MAPPING / "benchmark/{genome}" / f"{LIBRARY}.stats.tsv"`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: small

`preprocess__samtools`

Collect samtools stats/flagstats for every host-mapping CRAM

Target rule: it runs nothing itself and only asks for these files.

- input: `[HOST_MAPPING / genome / f"{sample_id}.{library_id}.{ext}" for genome in HOST_NAMES for sample_id, library_id in SAMPLE_LIBRARY for ext in ("stats.txt", "flagstats.txt")]`

A.4. Module `read_annotate`

A.4.1. `read_annotate/__main__.smk`

`read_annotate`

Run the read-based annotation tools

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__diamond.input`
- input: `rules.read_annotate__kraken2.input`
- input: `rules.read_annotate__krona.input`
- input: `rules.read_annotate__humann.input`
- input: `rules.read_annotate__metaphlan.input`
- input: `rules.read_annotate__nonpareil.input`
- input: `rules.read_annotate__phyloflash.input`
- input: `rules.read_annotate__singlem.input`
- input: `rules.read_annotate__sylph.input`

`read_annotate_extra`

Run the read-based annotation tools

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__ncyc.input`

A.4.2. `read_annotate/diamond.smk`

`read_annotate__diamond__assign`

Run Diamond with SHM/NVME staging if possible.

- input: `forwards=CLEAN_FORWARD`
- input: `reverses=CLEAN_REVERSE`
- input: `database=lambda w: features["databases"]["diamond"][w.diamond_db]`
- output: `out_R1=DIAMOND / "{diamond_db}" / "{sample_id}.{library_id}_R1.out"`
- output: `out_R2=DIAMOND / "{diamond_db}" / "{sample_id}.{library_id}_R2.out"`
- log: `DIAMOND / "{diamond_db}_{sample_id}_{library_id}.log"`
- benchmark: `DIAMOND / "benchmark/{diamond_db}_{sample_id}_{library_id}.tsv"`
- params: `max_target_seqs=params["read_annotate"]["diamond"]["max-target-seqs"]`
- params: `max_attempt=len(get_escalation_order("read_annotate__diamond__assign"))`
- params: `in_folder=FASTP`
- params: `out_folder=lambda w: DIAMOND / w.diamond_db`
- params: `run_in_shm=config["diamond_shm"]`
- params: `copy_dbs=config["copy_dbs"]`
- params: `diamond_shm=DIAMONDSHM`
- params: `diamond_nvme=DIAMONDNVME`
- params: `diamond_db_shm=lambda w: os.path.join(DIAMONDSHM, w.diamond_db)`
- params: `diamond_path=lambda w: os.path.basename(features["databases"]["diamond"][w.di`
- params: `diamond_db_nvme=lambda w: os.path.join(DIAMONDNVME, w.diamond_db)`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium_nvme

`read_annotate__diamond__summarise`

Run R script to summarise Diamond results

- input: `lambda w: [DIAMOND / w.diamond_db / f"{sample_id}.{library_id}_R1.out" for sample_id, library_id in SAMPLE_LIBRARY] + [DIAMOND / w.diamond_db / f"{sample_id}.{library_id}_R2.out" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `DIAMOND / "{diamond_db}" / "summary_a.tsv"`
- output: `DIAMOND / "{diamond_db}" / "summary_b.tsv"`
- log: `DIAMOND / "{diamond_db}" / "read_annotate__diamond__summarise.log"`
- benchmark: `DIAMOND / "{diamond_db}" / "benchmark_summarise.tsv"`
- params: `script=lambda w: config["pipeline_folder"] + "workflow/scripts/downstream_scripts/create_featuretable_diamond_" + w.diamond_db + ".R"`

- params: `project_folder=WD`
- params: `database=lambda w: w.diamond_db`
- params: `max_target_seqs=params["read_annotate"]["diamond"]["max-target-seqs"]`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

`read_annotate__diamond`

Run all Diamond steps

Target rule: it runs nothing itself and only asks for these files.

- input: `[DIAMOND / diamond_db / f"{sample_id}.{library_id}_R1.out" for sample_id, library_id in SAMPLE_LIBRARY for diamond_db in features["databases"]["diamond"]] + [DIAMOND / diamond_db / f"{sample_id}.{library_id}_R2.out" for sample_id, library_id in SAMPLE_LIBRARY for diamond_db in features["databases"]["diamond"]] + [DIAMOND / diamond_db / "summary_a.tsv" for diamond_db in features["databases"]["diamond"]] + [DIAMOND / diamond_db / "summary_b.tsv" for diamond_db in features["databases"]["diamond"]]`

A.4.3. `read_annotate/humann.smk`

`read_annotate__humann__run`

Run HumanN3 over one sample

- input: `forward_=CLEAN_FORWARD`
- input: `reverse_=CLEAN_REVERSE`
- input: `prot_dbs=features["databases"]["humann_prot_dbs"]`
- input: `nt_dbs=features["databases"]["humann_nt_dbs"]`
- input: `mp_out=METAPHLAN / "profiled" / "{sample_id}.{library_id}.txt"`
- output: `humann_out=HUMANN / "{sample_id}.{library_id}_genefamilies.tsv"`
- output: `cat=temp(HUMANN / "{sample_id}.{library_id}_concatenated.fastq.gz")`
- log: `HUMANN / "log" / "{sample_id}.{library_id}.log"`
- benchmark: `HUMANN / "benchmark" / "{sample_id}.{library_id}.tsv"`
- params: `out_folder=HUMANN`
- params: `out_name="{sample_id}.{library_id}"`
- params: `additional_options=params["read_annotate"]["humann"]["additional_options"]`
- container: `humann` (see `config/.docker.yml`)
- escalation: medium

`read_annotate__humann__condense`

Aggregate all the HumanN results into a single table

- input: `genefamily_data=[HUMANN / f"{sample_id}.{library_id}_genefamilies.tsv" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `HUMANN / "humann_genefamilies.tsv"`
- log: `HUMANN / "humann.log"`
- benchmark: `HUMANN / "benchmark/humann.tsv"`
- params: `input_dir=HUMANN`
- container: `humann` (see `config/.docker.yml`)
- escalation: small

`read_annotate__humann`

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__humann__condense.output`

A.4.4. `read_annotate/kraken2.smk`

`read_annotate__kraken2__assign`

Classify every sample against one kraken2 database in a single job kraken2 has no way to keep the database resident across multiple single-sample invocations, so it's staged once onto fast local storage (/dev/shm, falling back to NVMe, falling back to running in place on the final retry) and every sample is classified against that one copy. NOTE: if the job is killed, a staged copy may be left behind in /dev/shm.

- input: `forwards=[FASTP / f"{sample_id}.{library_id}_1.fq.gz" for sample_id, library_id in SAMPLE_LIBRARY]`
- input: `reverses=[FASTP / f"{sample_id}.{library_id}_2.fq.gz" for sample_id, library_id in SAMPLE_LIBRARY]`
- input: `database=lambda w: features["databases"]["kraken2"][w.kraken_db]`
- output: `out_gzs=[KRAKEN2 / "{kraken_db}" / f"{sample_id}.{library_id}.out.gz" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `reports=[KRAKEN2 / "{kraken_db}" / f"{sample_id}.{library_id}.report" for sample_id, library_id in SAMPLE_LIBRARY]`
- log: `KRAKEN2 / "{kraken_db}.log"`
- benchmark: `KRAKEN2 / "benchmark/{kraken_db}.tsv"`
- params: `max_attempt=len(get_escalation_order("read_annotate__kraken2__assign"))`
- params: `nvme=esc_val("nvme", "read_annotate__kraken2__assign")`
- params: `in_folder=FASTP`
- params: `out_folder=lambda w: KRAKEN2 / w.kraken_db`
- params: `run_in_shm=config["kraken2_shm"]`
- params: `copy_dbs=config["copy_dbs"]`
- params: `shm_dir=KRAKEN2SHM`

- params: `nvme_dir=KRAKEN2NVME`
- params: `db_shm=lambda w: os.path.join(KRAKEN2SHM, w.kraken_db)`
- params: `db_nvme=lambda w: os.path.join(KRAKEN2NVME, w.kraken_db)`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: full_node

`read_annotate__kraken2`

Aggregate kraken2 reports for every sample against every configured database

Target rule: it runs nothing itself and only asks for these files.

- input: `[KRAKEN2 / kraken_db / f"{sample_id}.{library_id}.report" for sample_id, library_id in SAMPLE_LIBRARY for kraken_db in features["databases"]["kraken2"]]`

A.4.5. `read_annotate/krona.smk`

`read_annotate__krona__visualize`

Create Krona plots for Kraken2 output.

- input: `KRAKEN2 / "{kraken_db}" / "{sample_id}.{library_id}.report"`
- output: `KRONA / "{kraken_db}" / "{sample_id}.{library_id}.html"`
- log: `KRONA / "{kraken_db}_{sample_id}.{library_id}.log"`
- benchmark: `KRONA / "benchmark/{kraken_db}_{sample_id}.{library_id}bsk.tsv"`
- params: `krona_db=features["databases"]["krona"]`
- container: `krona` (see `config/.docker.yml`)
- escalation: medium

`read_annotate__krona`

Run krona for the kraken output

Target rule: it runs nothing itself and only asks for these files.

- input: `[KRONA / kraken_db / f"{sample_id}.{library_id}.html" for sample_id, library_id in SAMPLE_LIBRARY for kraken_db in features["databases"]["kraken2"]]`

A.4.6. `read_annotate/metaphlan.smk`

`read_annotate__metaphlan__run`

Run metaphlan over one sample

- input: `forward_=CLEAN_FORWARD`
- input: `reverse_=CLEAN_REVERSE`
- input: `data=features["databases"]["metaphlan4"]`
- output: `bt2_out=METAPHLAN / "bowtie2_out" / "{sample_id}.{library_id}.bz2"`
- output: `mp_out=METAPHLAN / "profiled" / "{sample_id}.{library_id}.txt"`
- log: `METAPHLAN / "log" / "{sample_id}.{library_id}.log"`
- benchmark: `METAPHLAN / "benchmark" / "{sample_id}.{library_id}.tsv"`
- params: `tmp=config["tmp_storage"]`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: medium

`read_annotate__metaphlan__condense`

Aggregate all the metaphlan results into a single table

- input: `profiled_data=[METAPHLAN / "profiled" / f"{sample_id}.{library_id}.txt" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `METAPHLAN / "metaphlan_profiled.tsv"`
- log: `METAPHLAN / "metaphlan.log"`
- benchmark: `METAPHLAN / "benchmark/metaphlan.tsv"`
- params: `input_dir=METAPHLAN`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: small

`read_annotate__metaphlan`

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__metaphlan__condense.output`

A.4.7. `read_annotate/ncyc.smk`

`read_annotate__ncyc__prepare_input`

Create sample sheet for NCycDB Perl wrapper (runs all samples at once)

- input: `forwards=[CLEAN_READS / f"{sample_id}.{library_id}_1.fq.gz" for sample_id, library_id in SAMPLE_LIBRARY]`

- input: `reverses=[CLEAN_READS / f"{sample_id}.{library_id}_2.fq.gz" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `NCYC / "ncyc_samples.tsv"`
- log: `NCYC / "log" / "ncyc_samples.log"`
- benchmark: `NCYC / "benchmark" / "ncyc_samples.tsv"`
- params: `folder=config["pipeline_folder"]`
- params: `reads=CLEAN_READS`
- params: `outdir=NCYC / "results"`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: small

`read_annotate__ncyc__run`

Run NCycDB Perl wrapper on all samples

- input: `NCYC / "ncyc_samples.tsv"`
- output: `NCYC / "ncyc_results.tsv"`
- log: `NCYC / "log" / "ncyc_run.log"`
- benchmark: `NCYC / "benchmark" / "ncyc_run.tsv"`
- params: `folder=config["pipeline_folder"]`
- params: `outdir=NCYC`
- params: `reads_dir=CLEAN_READS`
- params: `faa=features["databases"]["ncyc"]`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium

`read_annotate__ncyc`

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__ncyc__run.output`

A.4.8. `read_annotate/nonpareil.smk`

`read_annotate__nonpareil__run`

Estimate sequence coverage/diversity for one sample with Nonpareil Nonpareil only takes one read of the pair, and needs a real (decompressed) FASTQ file on disk – it doesn't accept a process-substitution pipe. On low-coverage samples it can legitimately fail to converge, in which case it produces empty output files rather than erroring; the `touch()` wrappers below make sure Snakemake still sees the expected outputs.

- input: `forward_=CLEAN_FORWARD`
- output: `touch(NONPAREIL / f"{LIBRARY}.npa")`
- output: `touch(NONPAREIL / f"{LIBRARY}.npc")`
- output: `touch(NONPAREIL / f"{LIBRARY}.npl")`

- output: `touch(NONPAREIL / f"{LIBRARY}.npo")`
- log: `NONPAREIL / f"{LIBRARY}.log"`
- benchmark: `NONPAREIL / f"benchmark/{LIBRARY}.tsv"`
- params: `X=params["read_annotate"]["nonpareil"]["X"]`
- params: `tmp=config["tmp_storage"]`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: medium

`read_annotate__nonpareil__aggregate`

Aggregate all the nonpareil results into a single table

- input: `[f"{NONPAREIL}/{sample_id}.{library_id}.npo" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `NONPAREIL / "nonpareil.tsv"`
- log: `NONPAREIL / "nonpareil.log"`
- benchmark: `NONPAREIL / "benchmark/nonpareil.tsv"`
- params: `input_dir=NONPAREIL`
- params: `script_folder=SCRIPT_FOLDER`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: small

`read_annotate__nonpareil`

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__nonpareil__aggregate.output`

A.4.9. `read_annotate/phyloflash.smk`

`read_annotate__phyloflash__run`

Run PhyloFlash over one sample

- input: `forward_=CLEAN_FORWARD`
- input: `reverse_=CLEAN_REVERSE`
- input: `phyloflash_dbs=PHYLOFLASH_DBS`
- output: `phyloflash_out=PHYLOFLASH / "{sample_id}_{library_id}.phyloFlash.html"`
- output: `phyloflash_log=PHYLOFLASH / "{sample_id}_{library_id}.phyloFlash.log"`
- log: `PHYLOFLASH / "log" / "{sample_id}.{library_id}.log"`
- benchmark: `PHYLOFLASH / "benchmark" / "{sample_id}.{library_id}.tsv"`
- params: `lib="{sample_id}_{library_id}"`
- params: `outdir=PHYLOFLASH`
- container: `phyloflash` (see `config/.docker.yml`)
- escalation: medium_nvme

read_annotate__phyloflash__condense

Aggregate all the PhyloFlash results into a single table

- input: phyloflash_logs=[PHYLOFLASH / f"{sample_id}_{library_id}.phyloFlash.log" for sample_id, library_id in SAMPLE_LIBRARY]
- output: PHYLOFLASH / "phyloflash_all.log"
- log: PHYLOFLASH / "phyloflash.log"
- benchmark: PHYLOFLASH / "benchmark/phyloflash.tsv"
- params: input_dir=PHYLOFLASH
- container: phyloflash (see config/.docker.yml)
- escalation: small

read_annotate__phyloflash

Target rule: it runs nothing itself and only asks for these files.

- input: rules.read_annotate__phyloflash__condense.output

A.4.10. read_annotate/singlem.smk

read_annotate__singlem__pipe

Profile marker-gene taxonomy for one sample with SingleM SingleM's own docs recommend raw reads, but this pipeline feeds it the decontaminated, quality-trimmed reads instead, consistent with every other read-based annotation tool here.

- input: forward_=CLEAN_FORWARD
- input: reverse_=CLEAN_REVERSE
- input: data=features["databases"]["singlem"]
- output: archive_otu_table=SINGLEM / "pipe" / "{sample_id}.{library_id}.archive.json"
- output: otu_table=SINGLEM / "pipe" / "{sample_id}.{library_id}.otu_table.tsv"
- output: condense=SINGLEM / "pipe" / "{sample_id}.{library_id}.condense.tsv"
- log: SINGLEM / "pipe" / "{sample_id}.{library_id}.log"
- benchmark: SINGLEM / "benchmark/pipe" / "{sample_id}.{library_id}.tsv"
- params: tmp=config["tmp_storage"]
- container: preprocess (see config/.docker.yml)
- escalation: medium_nvme

read_annotate__singlem__condense

Aggregate all the singlem results into a single table

- input: archive_otu_tables=[SINGLEM / "pipe" / f"{sample_id}.{library_id}.archive.json" for sample_id, library_id in SAMPLE_LIBRARY]

- input: `database=features["databases"]["singlem"]`
- output: `condense=SINGLEM / "singlem.tsv"`
- log: `SINGLEM / "singlem.log"`
- benchmark: `SINGLEM / "benchmark/singlem.tsv"`
- params: `input_dir=SINGLEM`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: small

`read_annotate__singlem__microbial_fraction`

Run singlem microbial_fraction over one sample

- input: `forward_=CLEAN_FORWARD`
- input: `reverse_=CLEAN_REVERSE`
- input: `data=features["databases"]["singlem"]`
- input: `condense=SINGLEM / "pipe" / "{sample_id}.{library_id}.condense.tsv"`
- output: `microbial_fraction=SINGLEM / "microbial_fraction" / "{sample_id}.{library_id}.tsv"`
- log: `SINGLEM / "microbial_fraction" / "{sample_id}.{library_id}.log"`
- benchmark: `SINGLEM / "benchmark/microbial_fraction" / "{sample_id}.{library_id}.tsv"`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: small

`read_annotate__singlem__aggregate_microbial_fraction`

Aggregate all the microbial_fraction files into one tsv

- input: `tsvs=[SINGLEM / "microbial_fraction" / f"{sample_id}.{library_id}.tsv" for sample_id, library_id in SAMPLE_LIBRARY]`
- output: `tsv=SINGLEM / "microbial_fraction.tsv"`
- log: `SINGLEM / "microbial_fraction.log"`
- benchmark: `SINGLEM / "benchmark/microbial_fraction.tsv"`
- container: `preprocess` (see `config/.docker.yml`)
- escalation: small

`read_annotate__singlem`

Aggregate the SingleM taxonomic profile and microbial-fraction results

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.read_annotate__singlem__condense.output`
- input: `rules.read_annotate__singlem__aggregate_microbial_fraction.output`

A.4.11. `read_annotate/sylph.smk`

`read_annotate__sylph_profile`

Run Sylph profiler

- input: `forwards=CLEAN_FORWARD`
- input: `reverses=CLEAN_REVERSE`
- input: `db=features["databases"]["sylph"]`
- output: `PRE_SYLPH / "{sample_id}.{library_id}.profiling.tsv"`
- log: `PRE_SYLPH / "{sample_id}.{library_id}.profiling_report.log"`
- benchmark: `PRE_SYLPH / "benchmark/{sample_id}.{library_id}.tsv"`
- container: `sylph` (see `config/.docker.yml`)
- escalation: medium_nvme

`read_annotate__sylph`

Run Sylph

Target rule: it runs nothing itself and only asks for these files.

- input: `[PRE_SYLPH / f"{sample_id}.{library_id}.profiling.tsv" for sample_id, library_id in SAMPLE_LIBRARY]`

A.5. Module `assemble`

A.5.1. `assemble/__main__.smk`

`assemble`

Assemble, bin (CONCOCT/MaxBin2/MetaBAT2), combine bins with MAGScoT and dereplicate with dRep

Target rule: it runs nothing itself and only asks for these files.

- input: `MAG_CATALOGUE`

A.5.2. `assemble/binners/concoct.smk`

`assemble__concoct__run`

Bin the contigs of one assembly with CONCOCT, following the steps of its manual

- input: `contigs=CONTIGS`
- input: `alignments=assembly_alignments`
- output: `directory(CONCOCT / "{assembly_id}")`

- log: `CONCOCT / "{assembly_id}.log"`
- benchmark: `CONCOCT / "benchmark/{assembly_id}.tsv"`
- container: `concoct` (see `config/.docker.yml`)
- escalation: large

`assemble__concoct`

Run CONCOCT for every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(CONCOCT / "{assembly_id}", assembly_id=ASSEMBLIES)`

A.5.3. `assemble/binners/maxbin2.smk`

`assemble__maxbin2__run`

Bin the contigs of one assembly with MaxBin2, using one abundance value per contig

- input: `contigs=CONTIGS`
- input: `alignments=assembly_alignments`
- output: `directory(MAXBIN2 / "{assembly_id}")`
- log: `MAXBIN2 / "{assembly_id}.log"`
- benchmark: `MAXBIN2 / "benchmark/{assembly_id}.tsv"`
- params: `min_contig=params["assemble"]["maxbin"]["min_contig_len"]`
- container: `assemble` (see `config/.docker.yml`)
- escalation: large

`assemble__maxbin2`

Run MaxBin2 for every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(MAXBIN2 / "{assembly_id}", assembly_id=ASSEMBLIES)`

A.5.4. `assemble/binners/metabat2.smk`

`assemble__metabat2__run`

Bin the contigs of one assembly with MetaBAT2, using each contig's depth in each library

- input: `contigs=CONTIGS`
- input: `alignments=assembly_alignments`
- input: `indexes=assembly_alignment_indexes`
- output: `directory(METABAT2 / "{assembly_id}")`

- log: `METABAT2 / "{assembly_id}.log"`
- benchmark: `METABAT2 / "benchmark/{assembly_id}.tsv"`
- params: `min_contig=params["assemble"]["metabat"]["min_contig_len"]`
- container: `assemble` (see `config/.docker.yml`)
- escalation: large

`assemble__metabat2`

Run MetaBAT2 for every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(METABAT2 / "{assembly_id}", assembly_id=ASSEMBLIES)`

A.5.5. `assemble/bowtie2.smk`

`assemble__bowtie2__build__run`

Index the contigs of one assembly for read mapping

- input: `CONTIGS`
- output: `index=touch(ASSEMBLY_INDEX / "{assembly_id}")`
- log: `ASSEMBLY_INDEX / "{assembly_id}.log"`
- benchmark: `ASSEMBLY_INDEX / "benchmark/{assembly_id}.tsv"`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium

`assemble__bowtie2__build`

Index every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(ASSEMBLY_INDEX / "{assembly_id}", assembly_id=ASSEMBLIES)`

`assemble__bowtie2__map`

Map one library against one assembly and sort it into the configured format

- input: `index=ASSEMBLY_INDEX / "{assembly_id}"`
- input: `contigs=CONTIGS`
- input: `forward_=CLEAN_FORWARD`
- input: `reverse_=CLEAN_REVERSE`
- output: `alignment=ASSEMBLY_MAPPING / f"{_LIBRARY_ON_ASSEMBLY}.{ALIGN_EXT}"`
- output: `unsorted=temp(ASSEMBLY_MAPPING / f"{_LIBRARY_ON_ASSEMBLY}.tmp.bam")`
- log: `ASSEMBLY_MAPPING / f"{_LIBRARY_ON_ASSEMBLY}.log"`
- benchmark: `ASSEMBLY_MAPPING / f"benchmark/{_LIBRARY_ON_ASSEMBLY}.tsv"`

- params: `sort_memory=params["assemble"]["samtools"]["mem"]`
- params: `rg_id=compose_rg_id`
- params: `rg_extra=compose_rg_extra`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium

`assemble__bowtie2`

Map every library against every assembly it belongs to

Target rule: it runs nothing itself and only asks for these files.

- input: `[ASSEMBLY_MAPPING / f"{assembly_id}.{sample_id}.{library_id}.{ALIGN_EXT}" for assembly_id, sample_id, library_id in ASSEMBLY_LIBRARIES]`

A.5.6. `assemble/drep.smk`

`assemble__drep__split_bins`

Write every refined bin of all assemblies to its own FASTA, the way dRep takes its genomes

- input: `expand(MAGSCOT / "{assembly_id}.fa.gz", assembly_id=ASSEMBLIES)`
- output: `directory(_BIN_FILES)`
- log: `DREP / "separate_bins.log"`
- benchmark: `DREP / "benchmark/separate_bins.tsv"`
- params: `script_folder=SCRIPT_FOLDER`
- container: `assemble` (see `config/.docker.yml`)
- escalation: small

`assemble__drep__run`

Dereplicate the bins of all assemblies into one set of genomes with dRep

- input: `_BIN_FILES`
- output: `genomes=directory(MAG_GENOMES)`
- output: `archives=expand(DREP / "{section}.tar.gz", section=("data", "data_tables"))`
- log: `DREP / "drep.log"`
- benchmark: `DREP / "benchmark/drep_run.tsv"`
- params: `settings=params["assemble"]["drep"]`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium_nvme

assemble__drep__catalogue

Concatenate the dereplicated genomes into the MAG catalogue

- input: `MAG_GENOMES`
- output: `MAG_CATALOGUE`
- log: `DREP / "dereplicated_genomes.log"`
- benchmark: `DREP / "benchmark/join_genomes.tsv"`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium_nvme

assemble__drep

Build the MAG catalogue from the bins of all assemblies

Target rule: it runs nothing itself and only asks for these files.

- input: `MAG_CATALOGUE`

A.5.7. assemble/magscot.smk

assemble__magscot__prodigal

Predict the proteins of one assembly, with one prodigal per FASTA chunk

- input: `CONTIGS`
- output: `_MAGSCOT_RUN / "prodigal.faa"`
- log: `_MAGSCOT_RUN / "prodigal.log"`
- benchmark: `_MAGSCOT_RUN / "benchmark_prodigal.tsv"`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium_nvme

assemble__magscot__hmmsearch

Search the proteins of one assembly against one marker gene set of MAGScoT (Pfam or TIGRFAM)

- input: `proteins=_MAGSCOT_RUN / "prodigal.faa"`
- input: `hmm=lambda wildcards: features["magscot"][f"{wildcards.marker_set}_hmm"]`
- output: `hits=_MAGSCOT_RUN / "{marker_set}.tblout.gz"`
- log: `_MAGSCOT_RUN / "{marker_set}.log"`
- benchmark: `_MAGSCOT_RUN / "benchmark_hmmsearch_{marker_set}.tsv"`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium

assemble__magscot__marker_table

Write the marker gene hits in MAGScoT's format: protein, marker, e-value

- input: tigr=_MAGSCOT_RUN / "tigr.tblout.gz"
- input: pfam=_MAGSCOT_RUN / "pfam.tblout.gz"
- output: _MAGSCOT_RUN / "hmm.tblout"
- log: _MAGSCOT_RUN / "hmm.log"
- benchmark: _MAGSCOT_RUN / "benchmark_join_hmms.tsv"
- container: assemble (see config/.docker.yml)
- escalation: small

assemble__magscot__merge_contig_to_bin

List the contigs of every bin of the three binners: bin, contig, binner

- input: concoct=CONCOCT / "{assembly_id}"
- input: maxbin2=MAXBIN2 / "{assembly_id}"
- input: metabat2=METABAT2 / "{assembly_id}"
- output: _MAGSCOT_RUN / "contigs_to_bin.tsv"
- log: _MAGSCOT_RUN / "contigs_to_bin.log"
- benchmark: _MAGSCOT_RUN / "benchmark_merge_contig_to_bin.tsv"
- container: assemble (see config/.docker.yml)
- escalation: small

assemble__magscot__run

Score and refine the bins of one assembly with MAGScoT A poor assembly can leave MAGScoT without any viable bin, which it reports by failing. That is no reason to stop the pipeline, so the rule carries on with empty tables in that case.

- input: contigs_to_bin=_MAGSCOT_RUN / "contigs_to_bin.tsv"
- input: markers=_MAGSCOT_RUN / "hmm.tblout"
- output: _MAGSCOT_RUN / "magscot.gtdb_rel207_ar53.out"
- output: _MAGSCOT_RUN / "magscot.gtdb_rel207_bac120.out"
- output: _MAGSCOT_RUN / "magscot.refined.contig_to_bin.out"
- output: _MAGSCOT_RUN / "magscot.refined.out"
- output: _MAGSCOT_RUN / "magscot.scores.out"
- log: _MAGSCOT_RUN / "magscot.log"
- benchmark: _MAGSCOT_RUN / "benchmark_run.tsv"
- params: prefix=lambda w: MAGSCOT / w.assembly_id / "magscot"
- params: settings=params["assemble"]["magscot"]
- params: script_folder=SCRIPT_FOLDER
- container: assemble (see config/.docker.yml)
- escalation: small

assemble__magscot__label_contigs

Write the refined bin into the contig ids, keeping only the contigs that are in a bin

- input: contigs=CONTIGS

- input: `refined=_MAGSCOT_RUN / "magscot.refined.contig_to_bin.out"`
- output: `table=_MAGSCOT_RUN / "magscot.reformat.tsv"`
- output: `binned_contigs=MAGSCOT / "{assembly_id}.fa.gz"`
- log: `_MAGSCOT_RUN / "magscot.rename.log"`
- benchmark: `_MAGSCOT_RUN / "benchmark_rename.tsv"`
- params: `script_folder=SCRIPT_FOLDER`
- container: `assemble` (see `config/.docker.yml`)
- escalation: small

`assemble__magscot`

Run MAGScoT for every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(MAGSCOT / "{assembly_id}.fa.gz", assembly_id=ASSEMBLIES)`

A.5.8. `assemble/megahit.smk`

`assemble__megahit__run`

Assemble the libraries of one assembly_id with MEGAHIT and give the contigs pipeline ids –force because Snakemake has already created the output folder, which MEGAHIT otherwise refuses to write into.

- input: `forwards=assembly_reads(1)`
- input: `reverses=assembly_reads(2)`
- output: `contigs=MEGAHIT / "{assembly_id}.fa.gz"`
- output: `workdir_archive=MEGAHIT / "{assembly_id}.tar.gz"`
- log: `MEGAHIT / "{assembly_id}.log"`
- benchmark: `MEGAHIT / "benchmark/{assembly_id}.tsv"`
- params: `workdir=lambda w: MEGAHIT / w.assembly_id`
- params: `forwards=assembly_reads(1, separator=",")`
- params: `reverses=assembly_reads(2, separator=",")`
- params: `settings=params["assemble"]["megahit"]`
- params: `script_folder=SCRIPT_FOLDER`
- container: `assemble` (see `config/.docker.yml`)
- escalation: large

`assemble__megahit`

Run MEGAHIT for every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(MEGAHIT / "{assembly_id}.fa.gz", assembly_id=ASSEMBLIES)`

A.5.9. `assemble/metaspades.smk`

`assemble__metaspades__run`

Assemble the pooled libraries of one assembly_id with metaSPAdes and give the contigs pipeline ids metaSPAdes takes a single library per run in meta mode, hence the pooling. The eukaryote module reads the assembly graph back from the archive of the working folder.

- input: `forwards=assembly_reads(1)`
- input: `reverses=assembly_reads(2)`
- output: `contigs=METASPADES / "{assembly_id}.fa.gz"`
- output: `workdir_archive=METASPADES / "{assembly_id}.tar.gz"`
- output: `pooled_forward=temp(METASPADES / "{assembly_id}_R1_concat.fastq.gz")`
- output: `pooled_reverse=temp(METASPADES / "{assembly_id}_R2_concat.fastq.gz")`
- log: `METASPADES / "{assembly_id}.log"`
- benchmark: `METASPADES / "benchmark/{assembly_id}.tsv"`
- params: `workdir=lambda w: METASPADES / w.assembly_id`
- params: `settings=params["assemble"]["metaspades"]`
- params: `script_folder=SCRIPT_FOLDER`
- container: `assemble` (see `config/.docker.yml`)
- escalation: large

`assemble__metaspades`

Run metaSPAdes for every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(METASPADES / "{assembly_id}.fa.gz", assembly_id=ASSEMBLIES)`

A.6. Module `quantify`

A.6.1. `quantify/__main__.smk`

`quantify`

Map reads against the assemblies and quantify coverage with CoverM

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.quantify__coverm.input`
- input: `rules.quantify__samtools.input`

A.6.2. `quantify/bowtie2.smk`

`quantify__bowtie2__build`

Index the MAG catalogue for read mapping

- input: `MAG_CATALOGUE`
- output: `index=touch(MAG_INDEX / "dereplicated_genomes")`
- log: `MAG_INDEX / "dereplicated_genomes.log"`
- benchmark: `MAG_INDEX / "benchmark/dereplicated_genomes.tsv"`
- container: `bowtie2` (see `config/.docker.yml`)
- escalation: small

`quantify__bowtie2__map`

Map one library against the MAG catalogue

- input: `index=MAG_INDEX / "dereplicated_genomes"`
- input: `forward_=CLEAN_FORWARD`
- input: `reverse_=CLEAN_REVERSE`
- input: `catalogue=MAG_CATALOGUE`
- input: `fai=MAG_CATALOGUE_FAI`
- output: `MAG_CRAM`
- log: `MAG_MAPPING / f"{LIBRARY}.log"`
- benchmark: `MAG_MAPPING / f"benchmark/{LIBRARY}.tsv"`
- params: `sort_memory=params["quantify"]["samtools"]["mem"]`
- params: `rg_id=compose_rg_id`
- params: `rg_extra=compose_rg_extra`
- container: `bowtie2` (see `config/.docker.yml`)
- escalation: medium

`quantify__bowtie2`

Map every library against the MAG catalogue

Target rule: it runs nothing itself and only asks for these files.

- input: `[MAG_MAPPING / f"{sample_id}.{library_id}.cram" for sample_id, library_id in SAMPLE_LIBRARY]`

A.6.3. `quantify/coverm.smk`

`quantify__coverm__genome__run`

CoverM values of the MAGs in one library, for one method

- input: `alignment=MAG_CRAM`
- input: `index=MAG_CRAI`
- input: `catalogue=MAG_CATALOGUE`
- input: `fai=MAG_CATALOGUE_FAI`
- output: `COVERM / "genome" / "{method}" / f"{LIBRARY}.tsv"`
- log: `COVERM / "genome" / "{method}" / f"{LIBRARY}.log"`
- benchmark: `COVERM / "genome" / "{method}" / "benchmark" / f"{LIBRARY}.tsv"`
- params: `settings=params["quantify"]["coverm"]["genome"]`
- container: `quantify` (see `config/.docker.yml`)
- escalation: small

`quantify__coverm__genome_aggregate`

Join the genome-level tables of all libraries for one method

- input: `coverm_library_tables("genome")`
- output: `COVERM / "genome.{method}.tsv"`
- log: `COVERM / "genome.{method}.log"`
- benchmark: `COVERM / "benchmark_genome.{method}.tsv"`
- params: `script_folder=SCRIPT_FOLDER`
- container: `quantify` (see `config/.docker.yml`)
- escalation: small

`quantify__coverm__genome`

Run genome-level CoverM for every configured method

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(COVERM / "genome.{method}.tsv", method=params["quantify"]["coverm"]["genome"]["methods"])`

`quantify__coverm__contig_one`

CoverM values of the catalogue's contigs in one library, for one method

- input: `alignment=MAG_CRAM`
- input: `index=MAG_CRAI`
- input: `catalogue=MAG_CATALOGUE`
- input: `fai=MAG_CATALOGUE_FAI`
- output: `COVERM / "contig" / "{method}" / f"{LIBRARY}.tsv"`

- log: `COVERM / "contig" / "{method}" / f"{LIBRARY}.log"`
- benchmark: `COVERM / "contig" / "{method}" / "benchmark" / f"{LIBRARY}.tsv"`
- container: `quantify` (see `config/.docker.yml`)
- escalation: small

`quantify__coverm__contig_aggregate`

Join the contig-level tables of all libraries for one method

- input: `coverm_library_tables("contig")`
- output: `COVERM / "contig.{method}.tsv"`
- log: `COVERM / "contig.{method}.log"`
- benchmark: `COVERM / "benchmark_contig.{method}.tsv"`
- params: `script_folder=SCRIPT_FOLDER`
- container: `quantify` (see `config/.docker.yml`)
- escalation: small

`quantify__coverm__contig`

Run contig-level CoverM for every configured method

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(COVERM / "contig.{method}.tsv", method=params["quantify"]["coverm"]["contig"]["methods"])`

`quantify__coverm`

Aggregate contig- and genome-level CoverM results

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.quantify__coverm__contig.input`
- input: `rules.quantify__coverm__genome.input`

A.6.4. `quantify/samtools.smk`

`quantify__samtools__stats_cram`

Mapping statistics of one library on the MAG catalogue with samtools stats

- input: `alignment=MAG_CRAM`
- input: `index=MAG_CRAI`
- input: `catalogue=MAG_CATALOGUE`
- input: `fai=MAG_CATALOGUE_FAI`
- output: `MAG_MAPPING / f"{LIBRARY}.stats.txt"`
- log: `MAG_MAPPING / f"{LIBRARY}.stats.log"`

- benchmark: `MAG_MAPPING / f"benchmark/{LIBRARY}.stats.tsv"`
- container: `quantify` (see `config/.docker.yml`)
- escalation: small

`quantify__samtools`

Collect samtools stats/flagstats for every MAG-mapping CRAM

Target rule: it runs nothing itself and only asks for these files.

- input: `[MAG_MAPPING / f"{sample_id}.{library_id}.{ext}" for sample_id, library_id in SAMPLE_LIBRARY for ext in ("stats.txt", "flagstats.txt")]`

A.7. Module `mag_annotate`

A.7.1. `mag_annotate/__main__.smk`

`mag_annotate`

Annotate the dereplicated MAG catalogue with every configured tool

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.mag_annotate__bakta.output`
- input: `rules.mag_annotate__bakta_mags.input`
- input: `rules.mag_annotate__camper.input`
- input: `rules.mag_annotate__eggnog.output`
- input: `rules.mag_annotate__quast.output`
- input: `rules.mag_annotate__checkm2__predict.output`
- input: `rules.mag_annotate__gtdbtk__classify.output`
- input: `rules.mag_annotate__dram.input`
- input: `rules.mag_annotate__dram_mags.input`
- input: `rules.mag_annotate__proteinortho.output`
- input: `rules.mag_annotate__phylophlan.output`

A.7.2. `mag_annotate/bakta.smk`

`mag_annotate__bakta`

Run Bakta over the dereplicated mags

- input: `contigs=MAG_CATALOGUE`
- output: `tsv=BAKTA / "bakta.tsv"`
- output: `faa=BAKTA / "bakta.faa"`

- log: `BAKTA / "bakta.log"`
- benchmark: `BAKTA / "benchmark.tsv"`
- params: `out_dir=BAKTA`
- params: `db=features["databases"]["bakta"]`
- params: `options=params["mag_annotate"]["bakta"]["additional_options"]`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium_nvme

`mag_annotate__bakta_mags__run`

Run Bakta over the dereplicated mags

- input: `contigs=MAGSCOT / "{assembly_id}.fa.gz"`
- output: `tsv=BAKTAMAG / "bakta_{assembly_id}.tsv"`
- output: `faa=BAKTAMAG / "bakta_{assembly_id}.faa"`
- log: `BAKTAMAG / "bakta_{assembly_id}.log"`
- benchmark: `BAKTAMAG / "benchmark_{assembly_id}.tsv"`
- params: `out_dir=BAKTAMAG`
- params: `db=features["databases"]["bakta"]`
- params: `options=params["mag_annotate"]["bakta"]["additional_options"]`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium_nvme

`mag_annotate__bakta_mags`

Run Bakta over the dereplicated mags

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(BAKTAMAG / "bakta_{assembly_id}.faa", assembly_id=ASSEMBLIES)`

A.7.3. `mag_annotate/camper.smk`

`mag_annotate__camper__annotate`

Annotate dereplicate genomes with CAMPER

- input: `genes=DRAM / "annotate" / "genes.faa"`
- input: `annotation=DRAM / "annotate" / "annotations.tsv"`
- output: `annotation=CAMPER / "annotations.tsv"`
- output: `distill=CAMPER / "distill.tsv"`
- log: `CAMPER / "camper.log"`
- benchmark: `CAMPER / "benchmark.tsv"`
- params: `camper_dir=CAMPER`
- params: `camper_db=features["databases"]["camper"]`

- params: `nvme=config["nvme_storage"]`
- container: `camper` (see `config/.docker.yml`)
- escalation: medium_nvme

`mag_annotate__camper`

Run CAMPER on dereplicated genomes.

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.mag_annotate__camper__annotate.output`

A.7.4. `mag_annotate/checkm2.smk`

`mag_annotate__checkm2__predict`

Assess genome completeness/contamination of the dereplicated MAGs with CheckM2

- input: `genomes=MAG_GENOMES`
- input: `reference_data=features["databases"]["checkm2"]`
- output: `CHECKM2 / "quality_report.tsv"`
- log: `CHECKM2 / "quality_report.log"`
- benchmark: `CHECKM2 / "benchmark.tsv"`
- container: `checkm2` (see `config/.docker.yml`)
- escalation: medium_nvme

`mag_annotate__checkm2`

Assess all MAGs with CheckM2

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.mag_annotate__checkm2__predict.output`

A.7.5. `mag_annotate/dram.smk`

`mag_annotate__dram__annotate`

Annotate the dereplicated genome catalogue with DRAM

- input: `catalogue=MAG_CATALOGUE`
- input: `taxonomy=GTDBTK / "gtdbtk.summary.tsv"`
- input: `reference_data=features["databases"]["dram"]`
- output: `_DRAM_TABLES / "annotations.tsv"`
- output: `_DRAM_TABLES / "trnas.tsv"`
- output: `_DRAM_TABLES / "rrnas.tsv"`
- output: `_DRAM_TABLES / "genes.faa"`

- log: `DRAM / "annotate.log"`
- benchmark: `DRAM / "benchmark_annotate.tsv"`
- params: `dram_config=config["dram-config"]`
- container: `dram` (see `config/.docker.yml`)
- escalation: large_nvme

`mag_annotate__fix_dram_annotations_scaffold`

Patch up DRAM's annotation table (see fix_annotations.py) before distillation

- input: `_DRAM_TABLES / "annotations.tsv"`
- output: `_DRAM_TABLES / "annotations.fixed.tsv"`
- log: `DRAM / "fix_annotation.log"`
- params: `script_folder=SCRIPT_FOLDER`
- container: `assemble` (see `config/.docker.yml`)
- escalation: default

`mag_annotate__dram__distill`

Summarize the annotated genomes into DRAM's genome/metabolism reports

- input: `annotations=_DRAM_TABLES / "annotations.fixed.tsv"`
- input: `trnas=_DRAM_TABLES / "trnas.tsv"`
- input: `rrnas=_DRAM_TABLES / "rrnas.tsv"`
- input: `reference_data=features["databases"]["dram"]`
- output: `DRAM / "genome_stats.tsv"`
- output: `DRAM / "metabolism_summary.xlsx"`
- output: `DRAM / "product.html"`
- output: `DRAM / "product.tsv"`
- log: `DRAM / "distill.log2"`
- benchmark: `DRAM / "benchmark_distill.tsv"`
- params: `dram_config=config["dram-config"]`
- container: `dram` (see `config/.docker.yml`)
- escalation: small

`mag_annotate__dram`

Run DRAM annotation and distillation over the dereplicated genome catalogue

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.mag_annotate__dram__distill.output`

A.7.6. mag_annotate/dram_mag.smk

mag_annotate__dram_mag__annotate

Annotate the binned contigs of one assembly with DRAM

- input: `contigs=MAGSCOT / "{assembly_id}.fa.gz"`
- input: `reference_data=features["databases"]["dram"]`
- output: `_DRAM_ASSEMBLY_TABLES / "annotations.tsv"`
- output: `_DRAM_ASSEMBLY_TABLES / "trnas.tsv"`
- output: `_DRAM_ASSEMBLY_TABLES / "rrnas.tsv"`
- log: `_DRAM_ASSEMBLY / "annotate_{assembly_id}.log"`
- benchmark: `_DRAM_ASSEMBLY / "benchmark_annotate.tsv"`
- params: `dram_config=config["dram-config"]`
- params: `tables=lambda wildcards: DRAMMAG / wildcards.assembly_id / "annotate"`
- container: `dram` (see `config/.docker.yml`)
- escalation: large_nvme

mag_annotate__fix_dram_mag_annotations_scaffold

Patch up DRAM's annotation table (see fix_annotations.py) before distillation

- input: `_DRAM_ASSEMBLY_TABLES / "annotations.tsv"`
- output: `_DRAM_ASSEMBLY_TABLES / "annotations.fixed.tsv"`
- log: `DRAMMAG / "{assembly_id}_fix_annotation.log"`
- params: `script_folder=SCRIPT_FOLDER`
- container: `assemble` (see `config/.docker.yml`)
- escalation: small

mag_annotate__dram_mag__distill

Summarize the annotated contigs of one assembly into DRAM's genome/metabolism reports

- input: `annotations=_DRAM_ASSEMBLY_TABLES / "annotations.fixed.tsv"`
- input: `trnas=_DRAM_ASSEMBLY_TABLES / "trnas.tsv"`
- input: `rrnas=_DRAM_ASSEMBLY_TABLES / "rrnas.tsv"`
- input: `reference_data=features["databases"]["dram"]`
- output: `_DRAM_ASSEMBLY / "genome_stats.tsv"`
- output: `_DRAM_ASSEMBLY / "metabolism_summary.xlsx"`
- output: `_DRAM_ASSEMBLY / "product.tsv"`
- log: `_DRAM_ASSEMBLY / "distill.log2"`
- benchmark: `_DRAM_ASSEMBLY / "benchmark_distill.tsv"`
- params: `dram_config=config["dram-config"]`
- params: `assembly_folder=lambda wildcards: DRAMMAG / wildcards.assembly_id`

- container: `dram` (see `config/.docker.yml`)
- escalation: medium

`mag_annotate__dram_mags`

Run DRAM over the sample-wise mags

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(_DRAM_ASSEMBLY / "genome_stats.tsv", assembly_id=ASSEMBLIES)`

A.7.7. `mag_annotate/eggnog.smk`

`mag_annotate__eggnog`

Run eggnog over the dereplicated mags

- input: `contigs=MAG_CATALOGUE`
- output: `directory(EGGNOG)`
- log: `protected(EGGNOG / "eggnog.log")`
- benchmark: `EGGNOG / "benchmark.tsv"`
- params: `out_dir=EGGNOG`
- params: `db=features["databases"]["eggnog"]`
- params: `prefix="eggnog"`
- params: `tmp_dir=lambda wc: f"{config['tmp_storage']}/mag_annotate__eggnog"`
- container: `eggnog` (see `config/.docker.yml`)
- escalation: large_nvme

A.7.8. `mag_annotate/gtdbtk.smk`

`mag_annotate__gtdbtk__classify`

Place the MAGs in the GTDB taxonomy with the classify workflow of GTDB-Tk

- input: `genomes=MAG_GENOMES`
- input: `reference_data=features["databases"]["gtdbtk"]`
- output: `merged_summary=GTDBTK / "gtdbtk.summary.tsv"`
- output: `archives=expand(GTDBTK / "{part}.tar.gz", part=("align", "classify", "identify"))`
- log: `GTDBTK / "gtdbtk_classify.log"`
- benchmark: `GTDBTK / "benchmark.tsv"`
- container: `gtdbtk` (see `config/.docker.yml`)
- escalation: large

mag_annotate__gtdbtk

Classify the MAGs with GTDB-Tk

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.mag_annotate__gtdbtk__classify.output`

A.7.9. mag_annotate/phylophlan.smk

mag_annotate__phylophlan

Run phylophlan for the dereplicated genomes

- input: `MAG_GENOMES`
- output: `folder=directory(PHYLOPHLAN / "SGB")`
- output: `result=PHYLOPHLAN / "SGB" / "SGB.tsv"`
- log: `PHYLOPHLAN / "phylophlan_sgb.log"`
- benchmark: `PHYLOPHLAN / "benchmark/phylophlan_sgb.tsv"`
- params: `result=PHYLOPHLAN / "SGB" / "SGB"`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium_nvme

A.7.10. mag_annotate/proteinortho.smk

mag_annotate__proteinortho

Run Proteinortho over all bakta output FAA files together

- input: `faa=expand(BAKTAMAG / "bakta_{assembly_id}.faa", assembly_id=ASSEMBLIES)`
- output: `tsv=PROTEINORTHO / "proteinortho.proteinortho.tsv"`
- log: `PROTEINORTHO / "proteinortho.log"`
- benchmark: `PROTEINORTHO / "benchmark.tsv"`
- params: `outdir=PROTEINORTHO`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium

A.7.11. mag_annotate/quast.smk

mag_annotate__quast

Assembly statistics of the MAG catalogue with QUAST

- input: `MAG_CATALOGUE`
- output: `directory(QUAST)`
- log: `QUAST / "quast.log"`
- benchmark: `QUAST / "benchmark.tsv"`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium

A.8. Module contig_annotate

A.8.1. contig_annotate/__main__.smk

contig_annotate

Annotate on contig level

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.contig_annotate__camper.input`
- input: `[CONTIG_EGGNOG / f"{assembly_id}/eggnog_output.emapper.annotations" for assembly_id in ASSEMBLIES]`
- input: `lambda wildcards: [CONTIG_FEATURECOUNTS / f"{assembly_id}/{assembly_id}_{sample_id}.{library_id}.prodigal_fc.txt" for assembly_id, sample_id, library_id in ASSEMBLY_LIBRARIES]`
- input: `rules.contig_annotate__diamond.input`
- input: `rules.contig_annotate__hmmer.input`
- input: `rules.contig_annotate__eggnog7.input`

contig_annotate_extra

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.contig_annotate__ncyc.input`

A.8.2. contig_annotate/camper.smk

contig_annotate__camper__annotate

Annotate dereplicate genomes with CAMPER

- input: fa=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa"
- output: annotation=CONTIG_CAMPER / "{assembly_id}" / "annotations.tsv"
- output: distillate=CONTIG_CAMPER / "{assembly_id}" / "distillate.tsv"
- log: CONTIG_CAMPER / "{assembly_id}.log"
- benchmark: CONTIG_CAMPER / "benchmark/{assembly_id}.tsv"
- params: camper_dir=CONTIG_CAMPER
- params: out_dir=lambda wildcards: f"{CONTIG_CAMPER}/{wildcards.assembly_id}"
- params: camper_db=features["databases"]["camper"]
- params: nvme=config["nvme_storage"]
- params: sample=lambda wildcards: f"{wildcards.assembly_id}"
- container: camper (see config/.docker.yml)
- escalation: medium_nvme

contig_annotate__camper

Run CAMPER on contig gene sequences.

Target rule: it runs nothing itself and only asks for these files.

- input: [CONTIG_CAMPER / f"{assembly_id}" / "annotations.tsv" for assembly_id in ASSEMBLIES]

A.8.3. contig_annotate/diamond.smk

contig_annotate__diamond__assign

Run Diamond with SHM/NVME staging if possible.

- input: fa=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa"
- input: database=lambda w: features["databases"]["diamond_contig_protein"][w.diamond_d
- output: CONTIG_DIAMOND / "{diamond_db}" / "{assembly_id}.out"
- log: CONTIG_DIAMOND / "{diamond_db}_{assembly_id}.log"
- benchmark: CONTIG_DIAMOND / "benchmark/{diamond_db}_{assembly_id}.tsv"
- params: max_attempt=len(get_escalation_order("contig_annotate__diamond__assign"))
- params: in_folder=FASTP
- params: out_folder=lambda w: CONTIG_DIAMOND / w.diamond_db
- params: run_in_shm=config["diamond_shm"]
- params: copy_dbs=config["copy_dbs"]
- params: diamond_shm=DIAMONDSHM

- params: `diamond_nvme=DIAMONDNVME`
- params: `diamond_db_shm=lambda w: os.path.join(DIAMONDSHM, w.diamond_db)`
- params: `diamond_path=lambda w: os.path.basename(features["databases"]["diamond_contig`
- params: `diamond_db_nvme=lambda w: os.path.join(DIAMONDNVME, w.diamond_db)`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium_nvme

`contig_annotate__diamond__summarise`

Run R script to summarise Diamond results

- input: `lambda w: [CONTIG_DIAMOND / w.diamond_db / f"{assembly_id}.out" for assembly_id in ASSEMBLIES]`
- output: `CONTIG_DIAMOND / "{diamond_db}" / "summary_a.tsv"`
- output: `CONTIG_DIAMOND / "{diamond_db}" / "summary_b.tsv"`
- log: `CONTIG_DIAMOND / "{diamond_db}" / "contig_annotate__diamond__summarise.log"`
- benchmark: `CONTIG_DIAMOND / "{diamond_db}" / "benchmark_summarise.tsv"`
- params: `script=lambda w: config["pipeline_folder"] + "workflow/scripts/downstream_scripts/create_featuretable_diamond_" + w.diamond_db + ".R"`
- params: `project_folder=WD`
- params: `database=lambda w: w.diamond_db`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

`contig_annotate__diamond`

Run all Diamond steps

Target rule: it runs nothing itself and only asks for these files.

- input: `[CONTIG_DIAMOND / diamond_db / f"{assembly_id}.out" for assembly_id in ASSEMBLIES for diamond_db in features["databases"]["diamond_contig_protein"]]`

A.8.4. `contig_annotate/eggnog.smk`

`contig_annotate__eggnog_find_homology`

Find homolog genes in data (EGGNOG).

- input: `folder=CONTIG_PRODIGAL / "{assembly_id}/Chunks/"`
- input: `files=CONTIG_PRODIGAL / "{assembly_id}/Chunks/prodigal.chunk.{i}"`

- output: file=CONTIG_EGGNOG / "{assembly_id}/Chunks/prodigal.chunk.{i}.emapper.seed_or
- log: CONTIG_EGGNOG / "{assembly_id}/Chunks/prodigal.chunk.{i}.log"
- benchmark: CONTIG_EGGNOG / "{assembly_id}/Chunks/benchmark_homology.{i}.tsv"
- params: folder=lambda wildcards: CONTIG_EGGNOG / f"{wildcards.assembly_id}/Chunks/"
- params: tmp=config["nvme_storage"]
- params: out="prodigal.chunk.{i}"
- params: fa=features["databases"]["eggnog"]
- container: mag_annotate (see config/.docker.yml)
- escalation: medium_nvme

contig_annotate__eggnog_orthology_chunk

Annotate eggnog hits table per chunk (EGGNOG) using /dev/shm for speed, with usage counter to ensure DB is only deleted once all jobs are finished.

- input: seed=CONTIG_EGGNOG / "{assembly_id}/Chunks/prodigal.chunk.{i}.emapper.seed_ort
- output: annotation=CONTIG_EGGNOG / "{assembly_id}/Chunks/prodigal.chunk.{i}.emapper.a
- output: done=CONTIG_EGGNOG / "{assembly_id}/Chunks/prodigal.chunk.{i}.emapper.annotat
- log: CONTIG_EGGNOG / "{assembly_id}/Chunks/prodigal.chunk.{i}.emapper.annotations.log
- benchmark: CONTIG_EGGNOG / "{assembly_id}/Chunks/benchmark_orthology.{i}.tsv"
- params: fa=features["databases"]["eggnog"]
- params: out=lambda wc: f"prodigal.chunk.{wc.i}"
- params: outdir=lambda wc: CONTIG_EGGNOG / f"{wc.assembly_id}/Chunks"
- params: run_in_shm=config["eggnog_shm"]
- params: copy_dbs=config["copy_dbs"]
- params: shm=config["shm_storage"]
- params: nvme=config["nvme_storage"]
- container: mag_annotate (see config/.docker.yml)
- escalation: large_nvme

contig_annotate__eggnog_merge_annotations

- input: contig_annotate_aggregate_annotations_eggnog_search
- output: CONTIG_EGGNOG / "{assembly_id}/eggnog_output.emapper.annotations"
- log: CONTIG_EGGNOG / "{assembly_id}/eggnog_output.emapper.annotations.log"
- benchmark: CONTIG_EGGNOG / "{assembly_id}/benchmark_merge_annotations.tsv"
- escalation: default

contig_annotate__eggnog

Run eggnog on all assemblies

Target rule: it runs nothing itself and only asks for these files.

- input: [CONTIG_EGGNOG / f"{assembly_id}/eggnog_output.emapper.annotations" for assembly_id in ASSEMBLIES]

A.8.5. contig_annotate/eggnog7.smk

contig_annotate__eggnog7__run

Run the eggnog annotator for version 7

- input: fa=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa"
- output: CONTIG_EGGNOG7 / "{assembly_id}/{assembly_id}.eggnog.tsv.gz"
- log: CONTIG_EGGNOG7 / "logs/{assembly_id}_eggnog.log"
- benchmark: CONTIG_EGGNOG7 / "benchmark/{assembly_id}.tsv"
- params: sample=lambda wildcards: f"{wildcards.assembly_id}"
- params: out=CONTIG_EGGNOG7
- params: db=features["databases"]["eggnog7"]
- params: folder=config["pipeline_folder"]
- container: eggnog (see config/.docker.yml)
- escalation: medium

contig_annotate__eggnog7

Run eggnog on all assemblies

Target rule: it runs nothing itself and only asks for these files.

- input: expand(CONTIG_EGGNOG7 / "{assembly_id}/{assembly_id}.eggnog.tsv.gz", assembly_id=ASSEMBLIES)

A.8.6. contig_annotate/featurecounts.smk

contig_annotate__cram_to_bam

Create temporary bam-files for quantification

- input: ASSEMBLY_MAPPING / "{assembly_id}.{sample_id}.{library_id}.cram"
- output: temp(ASSEMBLY_MAPPING / "{assembly_id}.{sample_id}.{library_id}.bam")
- log: log=ASSEMBLY_MAPPING / "{assembly_id}.{sample_id}.{library_id}.cramToBam.log"
- container: assemble (see config/.docker.yml)
- escalation: small

contig_annotate__featurecounts__run

Quantify the mapped assembly reads in predicted genes (featureCounts).

- input: bam=ASSEMBLY_MAPPING / "{assembly_id}.{sample_id}.{library_id}.bam"
- input: gtf=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.gtf"
- output: file=CONTIG_FEATURECOUNTS / "{assembly_id}/{assembly_id}_{sample_id}.{library
- log: CONTIG_FEATURECOUNTS / "{assembly_id}/{assembly_id}_{sample_id}.{library_id}.pro
- container: subread (see config/.docker.yml)

- escalation: medium

contig_annotate__featurecounts

Quantify all samples for all the assemblies that they belong to

Target rule: it runs nothing itself and only asks for these files.

- input: `[CONTIG_FEATURECOUNTS / f"{assembly_id}/{assembly_id}_{sample_id}.{library_id}` `for assembly_id, sample_id, library_id in ASSEMBLY_LIBRARIES]`

A.8.7. contig_annotate/hmmer.smk

contig_annotate__hmmer__assign

Run HMMER.

- input: `fa=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa"`
- input: `database=lambda w: features["databases"]["hmmer"][w.hmmer_db]`
- output: `out=CONTIG_HMMER / "{hmmer_db}" / "{assembly_id}.out"`
- output: `aln=CONTIG_HMMER / "{hmmer_db}" / "{assembly_id}.aln"`
- output: `tblout=CONTIG_HMMER / "{hmmer_db}" /` `"{assembly_id}.tblout.tsv"`
- output: `domtblout=CONTIG_HMMER / "{hmmer_db}" /` `"{assembly_id}.domains-tblout.tsv"`
- output: `pfam=CONTIG_HMMER / "{hmmer_db}" / "{assembly_id}.pfam"`
- log: `CONTIG_HMMER / "{hmmer_db}_{assembly_id}.log"`
- benchmark: `CONTIG_HMMER / "benchmark/{hmmer_db}_{assembly_id}.tsv"`
- params: `out_folder=lambda w: CONTIG_HMMER / w.hmmer_db`
- params: `domt=lambda w: params["contig_annotate"]["hmmer"]["domt"].get(w.hmmer_db,` `params["contig_annotate"]["hmmer"]["domt"]["fallback"])`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium_nvme

contig_annotate__hmmer

Run all HMMER steps

Target rule: it runs nothing itself and only asks for these files.

- input: `[CONTIG_HMMER / hmmer_db / f"{assembly_id}.out"` `for assembly_id in ASSEMBLIES for hmmer_db in` `features["databases"]["hmmer"]]`

A.8.8. contig_annotate/ncyc.smk

contig_annotate__ncyc__prepare_input

Create sample sheet for NCycDB Perl wrapper (runs all samples at once)

- input: `[CONTIG_PRODIGAL / f"{assembly_id}/{assembly_id}.prodigal.fa" for assembly_id in ASSEMBLIES]`
- output: `CONTIG_NCYC / "ncyc_samples.tsv"`
- log: `CONTIG_NCYC / "log" / "ncyc_samples.log"`
- benchmark: `CONTIG_NCYC / "benchmark" / "ncyc_samples.tsv"`
- params: `folder=config["pipeline_folder"]`
- params: `input_dir=CONTIG_NCYC / "input"`
- params: `outdir=NCYC / "results"`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: small

contig_annotate__ncyc__run

Run NCycDB Perl wrapper on all samples from prodigal

- input: `CONTIG_NCYC / "ncyc_samples.tsv"`
- output: `CONTIG_NCYC / "ncyc_results.tsv"`
- log: `CONTIG_NCYC / "log" / "ncyc_run.log"`
- benchmark: `CONTIG_NCYC / "benchmark" / "ncyc_run.tsv"`
- params: `folder=config["pipeline_folder"]`
- params: `outdir=CONTIG_NCYC`
- params: `input_dir=CONTIG_NCYC / "input"`
- params: `faa=features["databases"]["ncyc"]`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium

contig_annotate__ncyc

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.contig_annotate__ncyc__run.output`

A.8.9. contig_annotate/prodigal.smk

contig_annotate__prodigal__run

Predict genes from assemblies (PRODIGAL).

- input: `assembly=CONTIGS`
- output: `fa=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa"`
- output: `gtf=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.gtf"`

- output: `gtfplain=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal_plain.gtf"`
- log: `CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.log"`
- benchmark: `CONTIG_PRODIGAL / "benchmark/{assembly_id}.tsv"`
- container: `assemble` (see `config/.docker.yml`)
- escalation: medium

`contig_annotate__cut_prodigal`

Cut prodigal output into smaller contigs (BASH).

- input: `fa=CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa"`
- output: `directory(CONTIG_PRODIGAL / "{assembly_id}/Chunks/")`
- log: `CONTIG_PRODIGAL / "logs/{assembly_id}_cut_prodigal.log"`
- benchmark: `CONTIG_PRODIGAL / "benchmark/{assembly_id}_cut_prodigal.tsv"`
- params: `out=lambda wildcards: CONTIG_PRODIGAL / f"{wildcards.assembly_id}/Chunks/prodigal.chunk"`
- params: `folder=config["pipeline_folder"]`
- params: `split=params["contig_annotate"]["prodigal"]["split"]`
- container: `mag_annotate` (see `config/.docker.yml`)
- escalation: medium

`contig_annotate__prodigal`

Run prodigal on all assemblies

Target rule: it runs nothing itself and only asks for these files.

- input: `expand(CONTIG_PRODIGAL / "{assembly_id}/{assembly_id}.prodigal.fa", assembly_id=ASSEMBLIES)`

A.9. Module `eukaryote`

A.9.1. `eukaryote/4cac.smk`

`eukaryote__4cac__extract_graph`

Pull one assembly's contigs, contig paths and assembly graph out of the metaSPAdes tarball 4CAC reads scaffolds.fasta and scaffolds.paths. The pipeline's contigs are metaSPAdes' contigs.fasta, and contigs.paths has the same format as scaffolds.paths, so both are extracted under the scaffold names: 4CAC then classifies exactly the contigs the rest of the pipeline uses.

- input: `tarball=METASPADES / "{assembly_id}.tar.gz"`
- output: `contigs=temp(EUK_4CAC / "{assembly_id}" / "scaffolds.fasta")`
- output: `paths=temp(EUK_4CAC / "{assembly_id}" / "scaffolds.paths")`

- output: graph=temp(EUK_4CAC / "{assembly_id}" / "assembly_graph_with_scaffolds.gfa")
- log: EUK_4CAC / "{assembly_id}.extract_graph.log"
- benchmark: EUK_4CAC / "benchmark/{assembly_id}.extract_graph.tsv"
- params: tmp_dir=lambda w: EUK_4CAC / w.assembly_id / "extract_tmp"
- container: assemble (see config/.docker.yml)
- escalation: small, medium

eukaryote__4cac__run

Classify one metaSPAdes assembly's contigs with 4CAC (XGBoost, refined on the assembly graph) Columns: contig_id (pipeline name), node_id (metaSPAdes name), class (phage, plasmid, prokarya, eukarya, uncertain), then the four XGBoost scores the graph step started from.

- input: contigs=EUK_4CAC / "{assembly_id}" / "scaffolds.fasta"
- input: paths=EUK_4CAC / "{assembly_id}" / "scaffolds.paths"
- input: graph=EUK_4CAC / "{assembly_id}" / "assembly_graph_with_scaffolds.gfa"
- output: tsv=EUK_4CAC / "{assembly_id}.tsv"
- log: EUK_4CAC / "{assembly_id}.log"
- benchmark: EUK_4CAC / "benchmark/{assembly_id}.tsv"
- params: workdir=lambda w: EUK_4CAC / w.assembly_id
- params: install_dir=params["eukaryote"]["4cac"]["install_dir"]
- params: phage=params["eukaryote"]["4cac"]["phage"]
- params: plasmid=params["eukaryote"]["4cac"]["plasmid"]
- params: script_folder=SCRIPT_FOLDER
- container: 4cac (see config/.docker.yml)
- escalation: medium, large, highmem

eukaryote__4cac

Run 4CAC on every assembly (metaSPAdes assemblies only, see FOURCAC_SUPPORTED)

Target rule: it runs nothing itself and only asks for these files.

- input: [EUK_4CAC / f"{assembly_id}.tsv" for assembly_id in ASSEMBLIES] if FOURCAC_SUPPORTED else []

A.9.2. eukaryote/__main__.smk

eukaryote

Classify contigs as eukaryotic or prokaryotic (Tiara, 4CAC, REMAG) and bin the eukaryotic ones (REMAG)

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.eukaryote__tiara.input`
- input: `rules.eukaryote__4cac.input`
- input: `rules.eukaryote__remag.input`

A.9.3. `eukaryote/remag.smk`

`eukaryote__remag__filter`

Classify one assembly's contigs with REMAG's HyenaDNA eukaryote filter Columns: contig_id, length, prediction (eukaryote / non_eukaryote), eukaryote_prob, confidence, num_windows, resampled. Contigs shorter than min_contig_length are not in it. REMAG names its files after the input ({assembly_id}.fa.gz -> {assembly_id}.fa_*). The classification table is copied next to the other tools' tables; the REMAG folder is kept as is, because eukaryote___remag___bin resumes from it. If the classifier can't be loaded, or a batch fails, REMAG keeps those contigs as eukaryotic and only logs it; both are turned into failures here.

- input: `assembly=CONTIGS`
- output: `tsv=EUK_REMAG / "{assembly_id}.tsv"`
- output: `filtered=EUK_REMAG / "{assembly_id}" / "{assembly_id}.fa_eukaryotic_filtered.fasta"`
- log: `EUK_REMAG / "{assembly_id}.filter.log"`
- benchmark: `EUK_REMAG / "benchmark/{assembly_id}.filter.tsv"`
- params: `out_dir=lambda w: EUK_REMAG / w.assembly_id`
- params: `min_contig_length=params["eukaryote"]["remag"]["min_contig_length"]`
- params: `hyenadna_batch_size=params["eukaryote"]["remag"]["hyenadna_batch_size"]`
- container: `remag` (see `config/.docker.yml`)
- escalation: medium, large, longrun

`eukaryote__remag__bin`

Bin one assembly's eukaryotic contigs with REMAG Runs in eukaryote___remag___filter's folder, where REMAG finds its own filtered FASTA and skips classifying again. Skipped, with an empty bins folder and a header-only bins.csv, when the contigs called eukaryotic add up to less than min_bin_size: no bin could pass the size cutoff then, and REMAG exits with an error when it has (nearly) nothing to bin.

- input: `assembly=CONTIGS`
- input: `tsv=EUK_REMAG / "{assembly_id}.tsv"`
- input: `filtered=EUK_REMAG / "{assembly_id}" / "{assembly_id}.fa_eukaryotic_filtered.fasta"`
- input: `alignments=assembly_alignments`
- input: `indexes=assembly_alignment_indexes`
- output: `bins=directory(EUK_REMAG / "{assembly_id}" / "bins")`

- output: assignments=EUK_REMAG / "{assembly_id}" / "bins.csv"
- log: EUK_REMAG / "{assembly_id}.bin.log"
- benchmark: EUK_REMAG / "benchmark/{assembly_id}.bin.tsv"
- params: out_dir=lambda w: EUK_REMAG / w.assembly_id
- params: coverage=compose_coverage_args_for_remag
- params: min_contig_length=params["eukaryote"]["remag"]["min_contig_length"]
- params: min_bin_size=params["eukaryote"]["remag"]["min_bin_size"]
- params: hyenadna_batch_size=params["eukaryote"]["remag"]["hyenadna_batch_size"]
- params: extra=params["eukaryote"]["remag"]["extra"]
- container: remag (see config/.docker.yml)
- escalation: large, longrun, highmem_longrun

eukaryote__remag

Run REMAG's eukaryote filter and binning on every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: [EUK_REMAG / f"{assembly_id}.tsv" for assembly_id in ASSEMBLIES]
- input: [EUK_REMAG / f"{assembly_id}" / "bins.csv" for assembly_id in ASSEMBLIES]

A.9.4. eukaryote/tiara.smk

eukaryote__tiara__run

Classify one assembly's contigs with Tiara (prokaryote / eukaryote / organelle) Columns: sequence_id, class_fst_stage (archaea, bacteria, prokarya, eukarya, organelle, unknown), class_snd_stage (mitochondrion, plastid, unknown for organelles, n/a otherwise), then the per-class probabilities. Contigs shorter than min_len are not in the table. Tiara writes its class counts next to the table as log_

.

- input: assembly=CONTIGS
- output: tsv=EUK_TIARA / "{assembly_id}.tsv"
- output: summary=EUK_TIARA / "log_{assembly_id}.tsv"
- log: EUK_TIARA / "{assembly_id}.log"
- benchmark: EUK_TIARA / "benchmark/{assembly_id}.tsv"
- params: min_len=params["eukaryote"]["tiara"]["min_len"]
- params: prob_cutoff=params["eukaryote"]["tiara"]["prob_cutoff"]
- container: tiara (see config/.docker.yml)
- escalation: medium, large

eukaryote__tiara

Run Tiara on every assembly

Target rule: it runs nothing itself and only asks for these files.

- input: `[EUK_TIARA / f"{assembly_id}.tsv" for assembly_id in ASSEMBLIES]`

A.10. Module `report`

A.10.1. `report/__main__.smk`

report

Generate the report for every module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__assemble.output`
- input: `rules.report__contig_annotate.output`
- input: `rules.report__mag_annotate.output`
- input: `rules.report__preprocess.output`
- input: `rules.report__quantify.output`
- input: `rules.report__read_annotate.output`
- input: `rules.report__reads.output`
- input: `rules.report__reference.output`

report_assemble

Report assemble module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__assemble.output`

report_contig_annotate

Report contig_annotate module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__contig_annotate.output`

report_mag_annotate

Report mag_annotate module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__mag_annotate.output`

`report_preprocess`

Report preprocess module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__preprocess.output`

`report_quantify`

Report preprocess module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__quantify.output`

`report_read_annotate`

Report read_annotate module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__read_annotate.output`

`report_reads`

Report reads module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__reads.output`

`report_reference`

Report reference module

Target rule: it runs nothing itself and only asks for these files.

- input: `rules.report__reference.output`

A.10.2. report/assemble.smk

report__assemble

Render the assemble module PDF report: assembly/binning/dereplication summary, runtime

- input: rules.assemble.input
- output: pdf=PIPELINE_REPORT / "assemble.pdf"
- log: PIPELINE_REPORT / "assemble.log"
- benchmark: PIPELINE_REPORT / "assemble_benchmark.tsv"
- params: script=ASSEMBLE_R
- params: pipeline_folder=config["pipeline_folder"]
- params: features=config["features-file"]
- params: sample_file=config["sample-file"]
- params: project_folder=WD
- container: r_report (see config/.docker.yml)
- escalation: small

A.10.3. report/contig_annotate.smk

report__contig_annotate

Render the contig_annotate module PDF report: gene prediction/annotation summary, runtime

- input: rules.contig_annotate.input
- output: pdf=PIPELINE_REPORT / "contig_annotate.pdf"
- log: PIPELINE_REPORT / "contig_annotate.log"
- benchmark: PIPELINE_REPORT / "contig_annotate_benchmark.tsv"
- params: script=CONTIG_ANNOTATE_R
- params: pipeline_folder=config["pipeline_folder"]
- params: features=config["features-file"]
- params: sample_file=config["sample-file"]
- params: project_folder=WD
- container: r_report (see config/.docker.yml)
- escalation: small

A.10.4. report/mag_annotate.smk

report__mag_annotate

Render the mag_annotate module PDF report: CheckM2/GTDB-Tk/QUAST/DRAM summaries, runtime

- input: `rules.mag_annotate.input`
- output: `pdf=PIPELINE_REPORT / "mag_annotate.pdf"`
- log: `PIPELINE_REPORT / "mag_annotate.log"`
- benchmark: `PIPELINE_REPORT / "mag_annotate_benchmark.tsv"`
- params: `script=MAG_ANNOTATE_R`
- params: `pipeline_folder=config["pipeline_folder"]`
- params: `features=config["features-file"]`
- params: `sample_file=config["sample-file"]`
- params: `project_folder=WD`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

A.10.5. `report/preprocess.smk`

`report__preprocess`

Render the preprocess module PDF report: fastp + host-decontamination stats, runtime

- input: `rules.preprocess.input`
- output: `pdf=PIPELINE_REPORT / "preprocess.pdf"`
- log: `PIPELINE_REPORT / "preprocess.log"`
- benchmark: `PIPELINE_REPORT / "preprocess_benchmark.tsv"`
- params: `script=PREPROCESS_R`
- params: `pipeline_folder=config["pipeline_folder"]`
- params: `features=config["features-file"]`
- params: `sample_file=config["sample-file"]`
- params: `project_folder=WD`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

A.10.6. `report/quantify.smk`

`report__quantify`

Render the quantify module PDF report: CoverM genome/contig abundance summary, runtime

- input: `rules.quantify.input`
- output: `pdf=PIPELINE_REPORT / "quantify.pdf"`
- log: `PIPELINE_REPORT / "quantify.log"`
- benchmark: `PIPELINE_REPORT / "quantify_benchmark.tsv"`
- params: `script=QUANTIFY_R`
- params: `pipeline_folder=config["pipeline_folder"]`

- params: `features=config["features-file"]`
- params: `sample_file=config["sample-file"]`
- params: `project_folder=WD`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

A.10.7. `report/read_annotate.smk`

`report__read_annotate`

Render the read_annotate module PDF report: kraken2/nonpareil/singlem summaries, runtime

- input: `rules.read_annotate.input`
- output: `pdf=PIPELINE_REPORT / "read_annotate.pdf"`
- log: `PIPELINE_REPORT / "read_annotate.log"`
- benchmark: `PIPELINE_REPORT / "read_annotate_benchmark.tsv"`
- params: `script=READ_ANNOTATE_R`
- params: `pipeline_folder=config["pipeline_folder"]`
- params: `features=config["features-file"]`
- params: `sample_file=config["sample-file"]`
- params: `project_folder=WD`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

A.10.8. `report/reads.smk`

`report__reads`

Render the reads module PDF report: sample sheet summary, FastQC overview, runtime stats

- input: `rules.reads.input`
- output: `pdf=PIPELINE_REPORT / "reads.pdf"`
- log: `PIPELINE_REPORT / "reads.log"`
- benchmark: `PIPELINE_REPORT / "reads_benchmark.tsv"`
- params: `script=READS_R`
- params: `pipeline_folder=config["pipeline_folder"]`
- params: `features=config["features-file"]`
- params: `sample_file=config["sample-file"]`
- params: `project_folder=WD`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

A.10.9. `report/reference.smk`

`report__reference`

Render the reference module PDF report: configured hosts, recompression runtime

- input: `rules.reference.input`
- output: `pdf=PIPELINE_REPORT / "reference.pdf"`
- log: `PIPELINE_REPORT / "reference.log"`
- benchmark: `PIPELINE_REPORT / "reference_benchmark.tsv"`
- params: `script=REFERENCE_R`
- params: `pipeline_folder=config["pipeline_folder"]`
- params: `features=config["features-file"]`
- params: `sample_file=config["sample-file"]`
- params: `project_folder=WD`
- container: `r_report` (see `config/.docker.yml`)
- escalation: small

A.11. Module `helpers`

A.11.1. `helpers/fastqc.smk`

`helpers__fastqc`

Generic FastQC rule: report for any {prefix}.fq.gz, reused across modules

- input: `"{prefix}.fq.gz"`
- output: `html="{prefix}_fastqc.html"`
- output: `zip="{prefix}_fastqc.zip"`
- log: `"{prefix}_fastqc.log"`
- benchmark: `"{prefix}_fastqc_benchmark.tsv"`
- container: `helpers` (see `config/.docker.yml`)
- escalation: small

A.11.2. `helpers/samtools.smk`

`helpers__samtools__index_bam`

samtools index of a BAM

- input: `"{prefix}.bam"`
- output: `"{prefix}.bam.bai"`
- log: `"{prefix}.bam.bai.log"`
- benchmark: `"benchmark/{prefix}.bam.bai.tsv"`

- container: `helpers` (see `config/.docker.yml`)
- escalation: medium

`helpers__samtools__index_cram`

samtools index of a CRAM

- input: `"{prefix}.cram"`
- output: `"{prefix}.cram.crai"`
- log: `"{prefix}.cram.crai.log"`
- benchmark: `"benchmark/{prefix}.cram.crai.tsv"`
- container: `helpers` (see `config/.docker.yml`)
- escalation: small

`helpers__samtools__faidx_fa`

samtools faidx of a plain FASTA

- input: `"{prefix}.fa"`
- output: `"{prefix}.fa.fai"`
- log: `"{prefix}.fa.fai.log"`
- benchmark: `"benchmark/{prefix}.fa.fai.tsv"`
- container: `helpers` (see `config/.docker.yml`)
- escalation: small

`helpers__samtools__faidx_fagz`

samtools faidx of a bgzipped FASTA

- input: `"{prefix}.fa.gz"`
- output: `fai="{prefix}.fa.gz.fai"`
- output: `gzi="{prefix}.fa.gz.gzi"`
- log: `"{prefix}.fa.gz.log"`
- benchmark: `"benchmark/{prefix}.fa.gz.tsv"`
- container: `helpers` (see `config/.docker.yml`)
- escalation: small

`helpers__samtools__idxstats_cram`

samtools idxstats of a CRAM

- input: `cram="{prefix}.cram"`
- input: `crai="{prefix}.cram.crai"`
- output: `tsv="{prefix}.idxstats.tsv"`
- log: `"{prefix}.idxstats.log"`
- benchmark: `"benchmark/{prefix}.idxstats.tsv"`
- container: `helpers` (see `config/.docker.yml`)
- escalation: small

`helpers__samtools__flagstats_cram`

samtools flagstats of a CRAM

- input: `cram="{prefix}.cram"`
- input: `crai="{prefix}.cram.crai"`
- output: `txt="{prefix}.flagstats.txt"`
- log: `"{prefix}.flagstats.log"`
- container: `helpers` (see `config/.docker.yml`)
- escalation: small

A.12. Module `devel`

A.12.1. `devel/throw_error.smk`

`test_error`

- output: `"done.txt"`
- log: `"log_file.txt"`
- params: `fail_at_sec=20`
- escalation: small

This reference lists 171 rules.